Principia Softwarica: The Plan 9 Windowing System `rio`

version 0.1

Yoann Padioleau
yoann.padioleau@gmail.com

with code from
Rob Pike

February 4, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1 Introduction **9**
1.1 Motivations . . . 9
1.2 The Plan 9 windowing system: `rio` . . . 10
1.3 Other windowing systems . . . 11
1.4 Getting started . . . 12
1.5 Requirements . . . 13
1.6 About this document . . . 15
1.7 Copyright . . . 15
1.8 Acknowledgments . . . 15

2 Overview **16**
2.1 Windowing system principles . . . 16
2.1.1 The window . . . 18
2.1.2 Display server . . . 19
2.1.3 Window compositor . . . 19
2.1.4 Window manager . . . 19
2.1.5 Terminal emulator . . . 20
2.1.6 Windowing system API . . . 21
2.1.7 Window applications versus graphical applications . . . 23
2.2 `rio` interfaces . . . 24
2.2.1 Command-line interface . . . 25
2.2.2 Graphical user interface . . . 25
2.2.3 Filesystem interface . . . 27
2.3 `hellorio.c` . . . 31
2.3.1 Skeleton and output code . . . 32
2.3.2 Input code . . . 34
2.4 Code organization . . . 36
2.5 Software architecture . . . 36
2.5.1 Processes, procs, and threads relationships . . . 38
2.5.2 Trace of a new window creation . . . 40
2.5.3 Trace of a mouse click . . . 44
2.5.4 Trace of a key press . . . 45
2.5.5 Trace of a drawing operation . . . 45
2.6 Book structure . . . 49

3 Core Data Structures **50**
3.1 Device handlers . . . 50
3.1.1 Output device: `display` and `view` . . . 50
3.1.2 Input devices: `mousectl` and `keyboardctl` . . . 50
3.2 Desktop, `desktop` . . . 50

3.3 Windows . . . 51
3.3.1 `Window` . . . 51
3.3.2 `windows` . . . 52
3.3.3 `current` . . . 52
3.3.4 Graphical windows . . . 52
3.3.5 Textual windows . . . 53
3.4 Filesystem server . . . 53
3.4.1 `FilSys` and `filsys` . . . 53
3.4.2 File state: `Fid` . . . 54
3.4.3 Workers and jobs: `Xfid` . . . 55
3.4.4 9P callbacks: `fcall` . . . 55

4 `main()` 56
4.1 Graphics initialization . . . 56
4.2 Mouse initialization . . . 57
4.3 Keyboard initialization . . . 57
4.4 Channels creation . . . 57
4.5 Threads creation . . . 57
4.6 Filesystem server initialization . . . 58
4.6.1 `filsysinit()` . . . 58
4.6.2 Worker allocator: `xfidinit()` . . . 59

5 Procs and Threads 60
5.1 Keyboard thread . . . 60
5.2 Mouse thread . . . 60
5.2.1 Application mouse events . . . 62
5.2.2 Windowing system mouse events . . . 62
5.3 Window threads . . . 63
5.3.1 Keyboard events listening . . . 64
5.3.2 Mouse events listening . . . 65
5.3.3 Control events listening . . . 66
5.4 Filesystem server proc . . . 66
5.4.1 `filsysproc()` . . . 66
5.4.2 `filsysversion()` . . . 67
5.5 `Xfid` allocator thread . . . 68
5.6 `Xfid` threads . . . 69

6 Cursors 71
6.1 Cursor graphics . . . 71
6.1.1 Classic cursors . . . 71
6.1.2 Border and corner cursors . . . 72
6.2 Setting the cursor . . . 74
6.2.1 `riosetcursor()` . . . 74
6.2.2 `cornercursor()` . . . 74
6.2.3 `wsetcursor()` . . . 75

7 Window Manager 76
7.1 Overview . . . 76
7.2 Right-click system menu . . . 76
7.3 Window borders click . . . 77

7.4 Wctlmesg . . . 78
7.5 Window creation . . . 79
7.5.1 Window thread creation: new() . . . 79
7.5.2 Window allocation: wmk() . . . 80
7.5.3 Window process creation: winshell() . . . 81
7.5.4 Namespace adjustments: filsysmount() . . . 83
7.5.5 Public layer: wsetname() . . . 84
7.5.6 Mouse action sweep() . . . 85
7.6 Window focus . . . 87
7.7 Window deletion . . . 88
7.7.1 delete() . . . 88
7.7.2 deletetimeoutproc() and deletethread() . . . 89
7.7.3 Deleted and Exited . . . 89
7.7.4 Mouse action pointto() . . . 91
7.8 Window move . . . 91
7.8.1 move() . . . 91
7.8.2 Mouse action drag() . . . 92
7.9 Window resize . . . 93
7.9.1 resize() . . . 93
7.9.2 Mouse action bandsize() . . . 95
7.10 Window visibility . . . 97
7.10.1 hide() . . . 97
7.10.2 hiddden . . . 97
7.10.3 unhide() . . . 98

8 Filesystem Server **100**
8.1 Additional data structures . . . 100
8.1.1 Qxxx . . . 100
8.1.2 dirtab . . . 100
8.2 filsysrespond() . . . 101
8.3 Attach . . . 101
8.3.1 filsysattach() . . . 101
8.3.2 xfidattach() . . . 102
8.4 Walk . . . 103
8.4.1 filsyswalk() . . . 103
8.4.2 Cloning fid . . . 104
8.4.3 105
8.4.4 Error management . . . 105
8.5 Open . . . 105
8.5.1 filsysopen() . . . 105
8.5.2 xfidopen() . . . 106
8.6 Clunk/Close . . . 107
8.6.1 filsysclunk() . . . 107
8.6.2 xfidclose() . . . 107
8.7 Read . . . 107
8.7.1 filsysread() . . . 107
8.7.2 Reading a directory . . . 108
8.7.3 xfidread() . . . 109
8.8 Write . . . 109
8.8.1 filsyswrite() . . . 109

8.8.2 `xfidwrite()` . . . 109
8.9 Stats . . . 110
8.9.1 `filsysstat()` . . . 110
8.10 Forbidden operations . . . 111

9 Virtual Devices **112**
9.1 `/mnt/wsys/mouse` . . . 112
9.1.1 Reading part1 . . . 112
9.1.2 Writing . . . 113
9.2 `/mnt/wsys/cons` . . . 114
9.2.1 Reading part1 . . . 114
9.2.2 Writing part1 . . . 115
9.2.3 Bytes versus runes, partial runes . . . 116
9.3 `/mnt/wsys/consctl` . . . 117
9.4 `/mnt/wsys/cursor` . . . 117
9.5 `/dev/draw/` and `/mnt/wsys/winname` . . . 118

10 Graphical Windows **119**
10.1 Graphical window setup . . . 119
10.1.1 `initdraw()` . . . 119
10.1.2 `initmouse()`, mouse-open mode . . . 119
10.1.3 `initkeyboard()`, raw-access mode . . . 119
10.2 Mouse events . . . 120
10.2.1 Mouse state queue . . . 120
10.2.2 `/mnt/wsys/mouse` reading part2 . . . 121
10.3 Keyboard events . . . 122
10.3.1 Raw keys queue . . . 122
10.3.2 `/mnt/wsys/cons` reading part2 . . . 122
10.4 Resize events . . . 124
10.5 `/mnt/wsys/window` . . . 124

11 Textual Windows **126**
11.1 Textual window creation . . . 126
11.1.1 Scrollbar . . . 126
11.1.2 Frame . . . 126
11.2 Frame widget . . . 126
11.2.1 `Frame` . . . 126
11.2.2 `frinit()` . . . 127
11.2.3 Frame colors . . . 128
11.2.4 Frame tick . . . 128
11.2.5 Frame boxes . . . 130
11.2.6 Frame strings . . . 132
11.2.7 Frame rune position, point, and box number . . . 133
11.2.8 Frame selection . . . 136
11.3 Content modification . . . 136
11.3.1 `winsert()` . . . 137
11.3.2 Growing array . . . 137
11.4 Content rendering . . . 138
11.4.1 `frinsert()` . . . 138
11.4.2 `frdelete()` . . . 147

11.4.3 wshow() . . . 149
11.4.4 Drawing the scrollbar: wscrdraw() . . . 149
11.4.5 Drawing the tick: frtick() . . . 151
11.4.6 Drawing the text and the selection: frdrawsel() . . . 152
11.4.7 Moving the frame origin . . . 153
11.4.8 Selecting . . . 155
11.4.9 Repainting . . . 158
11.5 Keyboard events . . . 159
11.5.1 Text input queue . . . 159
11.5.2 /mnt/wsys/cons reading part3 . . . 159
11.5.3 Navigation keys . . . 160
11.5.4 Special keys . . . 163
11.6 Application output events . . . 165
11.6.1 /mnt/wsys/cons writing part2 . . . 165
11.7 Mouse events . . . 167
11.7.1 Middle click menu . . . 167
11.7.2 Other clicks . . . 167
11.8 Automatic scrolling mode . . . 168
11.9 Scroll bar interaction . . . 168
11.10 Resize . . . 171
11.11 /mnt/wsys/text . . . 172

12 Windowing System Files **173**
12.1 /mnt/wsys/winid . . . 173
12.2 /mnt/wsys/label . . . 173
12.3 /mnt/wsys/screen . . . 174
12.4 /mnt/wsys/wsys/ . . . 174
12.5 /mnt/wsys/wctl . . . 176
12.5.1 Reading . . . 176
12.5.2 Writing (controlling windows) . . . 178

13 Advanced Topics TODO **181**
13.1 External mount: /srv/rio.user.pid . . . 181
13.2 Command-line control: /srv/riowctl.user.pid . . . 182
13.3 Recursive rio . . . 190
13.4 Advanced terminal editing . . . 191
13.4.1 Snarf . . . 191
13.4.2 Plumb . . . 195
13.4.3 Auto complete . . . 206
13.4.4 Word selection . . . 211
13.5 Automatic scrolling: rio -s . . . 214
13.6 Initial command: rio -i . . . 215
13.7 Fake keyboard input: rio -k . . . 215
13.7.1 rio -k . . . 215
13.7.2 wkeyboard . . . 216
13.7.3 /mnt/wsys/kdbin . . . 216
13.7.4 Keyboard hide . . . 217
13.8 Font selection: rio -f . . . 217
13.9 Holding mode . . . 218
13.10 Signals, notes . . . 219

13.11 Timer . . . 220
13.12 Flushing . . . 222
13.13 Security . . . 225
13.14 TODO `Wakeup` . . . 225
13.15 TODO `Refresh` . . . 226

14 Conclusion **227**

A Debugging **228**

B Error Management **229**
B.1 Error codes . . . 229
B.2 `error()`, `derror()` . . . 230

C Utilities **231**

D Examples of Windowing System Applications TODO **234**

E Extra Code **235**
E.1 `include/` . . . 235
E.1.1 `include/complete.h` . . . 235
E.1.2 `include/frame.h` . . . 235
E.1.3 `include/plumb.h` . . . 236
E.2 `windows/rio/` . . . 237
E.2.1 `windows/rio/dat.h` . . . 237
E.2.2 `windows/rio/fns.h` . . . 239
E.2.3 `windows/rio/globals.c` . . . 242
E.2.4 `windows/rio/rio.c` . . . 243
E.2.5 `windows/rio/thread_mouse.c` . . . 243
E.2.6 `windows/rio/thread_keyboard.c` . . . 244
E.2.7 `windows/rio/threads_window.c` . . . 244
E.2.8 `windows/rio/threads_worker.c` . . . 245
E.2.9 `windows/rio/threads_misc.c` . . . 245
E.2.10 `windows/rio/wm.c` . . . 246
E.2.11 `windows/rio/data.c` . . . 247
E.2.12 `windows/rio/cursor.c` . . . 248
E.2.13 `windows/rio/wind.c` . . . 249
E.2.14 `windows/rio/processes_winshell.c` . . . 250
E.2.15 `windows/rio/terminal.c` . . . 250
E.2.16 `windows/rio/snarf.c` . . . 253
E.2.17 `windows/rio/graphical_window.c` . . . 253
E.2.18 `windows/rio/9p.c` . . . 254
E.2.19 `windows/rio/proc_fileserver.c` . . . 254
E.2.20 `windows/rio/fsys.c` . . . 255
E.2.21 `windows/rio/xfid.c` . . . 256
E.2.22 `windows/rio/scrl.c` . . . 257
E.2.23 `windows/rio/time.c` . . . 258
E.2.24 `windows/rio/wctl.c` . . . 258
E.2.25 `windows/rio/error.c` . . . 259
E.2.26 `windows/rio/util.c` . . . 259
E.3 `windows/apps/` . . . 260

E.3.1 windows/apps/lens.c . . . 260
E.3.2 windows/apps/statusbar.c . . . 266
E.3.3 windows/apps/winwatch.c . . . 273
E.4 windows/libcomplete/ . . . 279
E.4.1 windows/libcomplete/complete.c . . . 279
E.5 windows/libframe/ . . . 279
E.5.1 windows/libframe/frbox.c . . . 279
E.5.2 windows/libframe/frdelete.c . . . 280
E.5.3 windows/libframe/frdraw.c . . . 280
E.5.4 windows/libframe/frinit.c . . . 281
E.5.5 windows/libframe/frinsert.c . . . 281
E.5.6 windows/libframe/frptofchar.c . . . 281
E.5.7 windows/libframe/frselect.c . . . 282
E.5.8 windows/libframe/frstr.c . . . 282
E.5.9 windows/libframe/frutil.c . . . 282
E.6 windows/libplumb/ . . . 283
E.6.1 windows/libplumb/event.c . . . 283
E.6.2 windows/libplumb/mesg.c . . . 283
E.6.3 windows/libplumb/plumbsendtext.c . . . 284

Glossary **285**

Index **286**

References **286**

Chapter 1

Introduction

The goal of this book is to explain with full details the source code of a windowing system.

1.1 Motivations

Why a windowing system? Because I think you are a better programmer if you fully understand how things work under the hood, and one of the first thing you should see on your screen is a set of windows. The windowing system is the program allowing you to create and manipulate those windows.

Windowing systems are usually coupled with a graphics system to form a graphical user interface (GUI). GUIs, introduced in the 1970's with the Xerox Alto [TML+79], were a vast improvement over text-based user interfaces, to the point where every mainstream operating systems now come with a GUI (e.g., Microsoft Windows, macOS, or Linux with X Window).

A windowing system relies on a graphics system to render the graphics of a window on a specific rectangular surface of the screen. However, a window is not just a surface; it is also a process. Thus, a windowing system manages not only a set of surfaces, but also a set of processes. This is similar to what a kernel does. Moreover, just like the kernel manages the CPU and memory and virtualizes those resources shared among multiple processes, a windowing system manages the screen and input devices (e.g., the mouse, the keyboard) and virtualizes those resources shared among multiple windows. The windowing system is a natural extension of the kernel. In fact, the need for multiple processes and a multi-tasking kernel is less obvious without a windowing system. Linux offers virtual consoles where the user can launch independent commands, but those consoles are a poor's man windowing system.

Surprisingly, there is almost no book explaining how a windowing system works, even though there is a myriad of books on kernels. I can cite *The NeWS book: An Introduction to the Network/Extensible Window System* [GRA89], or one chapter of *Computer Graphics, Principles and Practice* [FDFH90] dedicated to user interface software. Books on operating systems usually do not even include a chapter on windowing systems. This is a pity because the windowing system is as important as the kernel for the user.

Here are a few questions I hope this book will answer:

- What is the software architecture of a windowing system? Is the windowing system a regular program? How does it have access to the mouse and the screen? Does it need special privileges from the kernel? How does it cooperate with the kernel?

- How does the windowing system manage multiple windows/processes? How does it communicate with those processes?

- How does the windowing system control access to the screen, a resource used by multiple windows at the same time? How does it cooperate with the graphics system?

- How does the windowing system intercept the drawing operations done by windows to make sure they can not draw in other windows? How is the screen virtualized?
- How does the windowing system handle the mouse device? When are mouse events dispatched to the windows? How does the windowing system decide which window should receive the mouse event?
- How does the introduction of the mouse, graphics, and windows changes the programming model of an application? How can an application react to a mouse event? How can the windowing system itself react to a mouse event?
- How does the windowing system handle the keyboard device? How does it decide which window should receive a keyboard event? How does it deliver a keyboard event to this window?
- How are overlapping windows managed? Where are stored the pixels of a window overlapped by another window? How are those pixels restored on the screen when an overlapped window is exposed back?
- What are the differences among a windowing system, a window manager, a window compositor, a window server, and a desktop system?
- How does a terminal emulator (e.g., `xterm`) work? What are the standard input and output of traditional command-line applications when running under an emulator? How does the emulator offer a backward-compatible environment for those applications?
- What happens when you type `ls` in a terminal emulator? What are the set of programs involved in such a command? What is the trace of such a command through the different layers of the software stack, from the keyboard interrupt to the display of text glyphs on the screen in the appropriate window?

1.2 The Plan 9 windowing system: `rio`

I will explain in this book the code of the Plan 9 windowing system `rio` [Pik00b][1], which contains about 10 000 lines of code (LOC). `rio` is written entirely in C.

In most operating systems (e.g., macOS, Microsoft Windows), the windowing system is *strongly* coupled with the graphics system. In Plan 9, the windowing system `rio`, and the graphics system, called `draw`, are clearly separated; `rio` is a user-space program that relies on `draw`, which is implemented as a device in the kernel (for more information on `draw`, read the GRAPHICS book [Pad16b]). In Plan 9, you can run graphical applications with or without `rio`. In fact, `rio` itself is just a graphical application.

Like for most books in Principia Softwarica, I chose a Plan 9 program because those programs are simple, small, elegant, open source, and they form together a coherent set. `rio` comes from a series of windowing systems designed by Rob Pike: `8-1/2` [Pik91], the direct ancestor of `rio`, running also under Plan 9; the "Concurrent Window System" [Pik89], programmed in the Newsqueak language; and `mpx`, the windowing system of the Blit [Pik83a] machine. `mpx` was the first windowing system for Unix, and one of the first windowing system back in 1982[2]. It was created even before the Macintosh, X Window, and Microsoft Windows.

Like many other services in Plan 9, some of the `rio` services are accessible through files. Indeed, `rio` is a graphical application *and* a filesystem. To understand why a part of `rio` is implemented as a filesystem, you need (1) to have a general idea on how to implement a windowing system, and (2) be familiar with some of the advanced features of the Plan 9 kernel.

Regarding the first point, at a high level, a windowing system is a program that uses the mouse (via `/dev/mouse` under Plan 9), the keyboard (via `/dev/cons`), and the screen (via `/dev/draw`). However, an

[1] See http://plan9.bell-labs.com/magic/man2html/4/rio for its manual page.

[2] See https://www.youtube.com/watch?v=emh22gT5e9k for an historical demo of the Blit and `mpx`, which has a user interface almost identical to `rio`.

application running in a window is not different; such an application also wants to use the mouse, the keyboard, and the screen. Thus, a windowing system can be implemented simply as a *multiplexer*; the windowing system can use the *physical devices* (managed by the kernel) and serve *virtual devices* to the multiple windows running under it[3].

Regarding the second point, the Plan 9 kernel has a few original features that makes it easy to implement virtual devices served by programs in user space. Those features are the *per-process namespace*, the *union-mount*, and the *file-server protocol 9P* (see the KERNEL book [Pad14] for more information on those features, or the two Plan 9 articles [PPD+95, PPT+93], which contain both good introductions to those features). With `rio`, the virtual devices are accessible under `/mnt/wsys/` (e.g., `/mnt/wsys/mouse`, `/mnt/wsys/cons`), but also under `/dev/` (e.g., `/dev/mouse`, `/dev/cons`), thanks to the union-mount. Thanks to the per-process namespace, the applications running under `rio` see a different `/dev/mouse`, `/dev/cons`, and could see a different `/dev/draw`[4]. Finally, thanks to 9P, all those virtual device files can be served by a single user-space program: `rio`.

A nice side effect of the multiplexer approach used by `rio` is that `rio` can run under itself (see Section 13.3). This is useful for development and debugging purposes. Moreover, because you can export filesystems through the network in Plan 9, `rio` is also a networked windowing system, similar to X Window (even though the code of `rio` does not include a single line of code related to networking). Thus, in Plan 9, programs running on one machine can have their window displayed on another machine.

1.3 Other windowing systems

Here are a few windowing systems that I considered for this book, but which I ultimately discarded:

- Xorg[5], which I mentioned already in the GRAPHICS book [Pad16b], is the most popular open-source implementation of the X Window System [SG86], a windowing system (and a graphics system) designed in the 1980's at MIT. However, its codebase is enormous. In fact, the whole system is divided in hundreds of repositories[6] to better handle its complexity. One of this repository, `xserver`[7], which contains the code of the display server, has already more than 500 000 LOC. This does not even include the code of the window manager (e.g., `twm` with 17 000 LOC), the terminal emulator (e.g., `xterm` with 80 000 LOC), or the libraries required by clients to communicate with the display server (e.g., `Xlib` with 150 000 LOC). Xorg, in total, contains more than two orders of magnitude more code than `rio`.

  Part of the reason for the enormous size of Xorg is that Xorg supports many graphic cards, many monitors, many input devices, and many extensions (e.g., 3D operations). Another reason is that X Window is an old program; programmers extended X Window for more than 30 years now. Programmers added many extensions while still being forced to remain backward compatible with applications designed in the 1980's.

  X Window defines a communication protocol, X11, for a networked client/server architecture. Client applications must use *sockets* to connect to the display server. Unfortunately, the set of mechanisms used by clients to interact with the screen, mouse, or keyboard is quite different from the one offered by the kernel, for instance, the simple opening of files in `/dev/` such as `/dev/mouse`. In some sense, X Window *masks* the features of the underlying kernel. On the opposite, `rio` is *transparent* [Pik88] and instead generalizes the services offered by the kernel, for instance, with the virtual device file `/dev/mouse`. Of course, the use of sockets in X Window allows client applications to display their result on another machine on the network. However, this is also possible with `rio`, for free, thanks to the generic 9P protocol.

[3] For more information, see Section 2.1.6 and especially Figure 2.3.

[4] For `/dev/draw`, the `draw` device can already multiplex the screen among multiple clients (in `/dev/draw/1/`, `/dev/draw/2`, etc). There is no need for a virtual `/dev/draw`. However, the ancestor of `rio`, `8-1/2` [Pik91], was serving a virtual `/dev/draw` device file, which was more elegant but also more inefficient. For more information, see Section 2.2.3.

[5] http://xorg.freedesktop.org

[6] https://cgit.freedesktop.org/xorg

[7] https://cgit.freedesktop.org/xorg/xserver/

Finally, the graphics and windowing system parts of Xorg are strongly coupled; this coupling makes the whole system harder to understand than `rio` and `draw`, which we can study separately.

- Wayland[8] is a protocol, similar to X11, specifying the communication between a display server, called a Wayland *compositor*, and a set of local clients. Weston[9] is a reference implementation of a Wayland compositor. Wayland and Weston grew out of the frustation of some developers of Xorg with the complexity of X Window, as well as the difficulty for X Window to support the modern needs of a windowing system: transluscent windows, drop shadows on the window's border as in macOS Aqua, fancy window-switcher such as macOS Expose, etc.

  Fortunately, during the last ten years, lots of the code of Xorg got gradually moved out of the display server and put either in the Linux kernel (e.g., the resolution setting of the screen, called KMS for kernel mode setting, or the ability to interact directly with the graphics hardware, called DRM for direct rendering manager), or in external libraries (e.g., Cairo for an advanced drawing API, or `libinput` for a generic interface to the input devices). What remains in Xorg is an old drawing API, the ability to have remote applications, and a display server that is backward compatible with old applications. The developers of Wayland used this opportunity to redesign from scratch a modern windowing system, while reusing lots of the code that was now outside Xorg.

  There are many differences between Wayland and X11. For instance, Wayland does not specify any drawing API. Instead, it assumes the clients do their own graphics rendering by using libraries such as Cairo on locally-shared image buffers. Weston then just uses those shared buffers and composes them together (hence the use of the word "compositor"), while possibly applying effects during the image composition such as transluscence. The use of locally-shared buffers means that Wayland does not support remote applications. Fortunately, most users now run and display their applications on the same machine.

  The code of Wayland and Weston is far smaller than Xorg: 120 000 LOC (not including the tests). However, this is still one order of magnitude more code than `rio`. Moreover, Weston relies on many libraries (e.g., Cairo, `libinput`), as well as lots of code and subsystems of the Linux graphics stack (e.g., KMS, DRM, GEM, fbdev, evdev); this would add lots of code to explain.

- Nano-X[10] (previously known as MicroWindows) is a windowing system and graphics system designed originally for small devices such as PDAs. It started as a fork of Mini-X, a graphics system for MINIX. Both Mini-X and Nano-X are modeled after X Window, and offer an API similar to Xlib. Nano-X added a client/server architecture to Mini-X, as well as a window manager, to become a full windowing system.

  Nano-X is highly portable, with support for many machines (e.g., x86 desktops, MIPS machines, ARM embedded devices). Moreover, Nano-X does not require any external graphics library; it just requires an access to the framebuffer from the Linux kernel. It is far smaller than Xorg: 80 000 LOC (not including the tests, application demos, the Win32 API, and the fonts). However, this is still bigger than the code of `draw` and `rio` combined.

Figure 1.1 presents a timeline of major windowing systems. I think `rio` represents the best compromise for this book: it implements the essential features of a windowing system while still having a small and understandable codebase (10 000 LOC).

1.4 Getting started

To play with `rio`, you will first need to install the Plan 9 fork used in Principia Softwarica (see http://www.principia-softwarica.org/wiki/Install). Once installed, you can test `rio` under Plan 9 by executing the

[8] https://wayland.freedesktop.org/
[9] https://cgit.freedesktop.org/wayland/weston/
[10] http://www.microwindows.org/

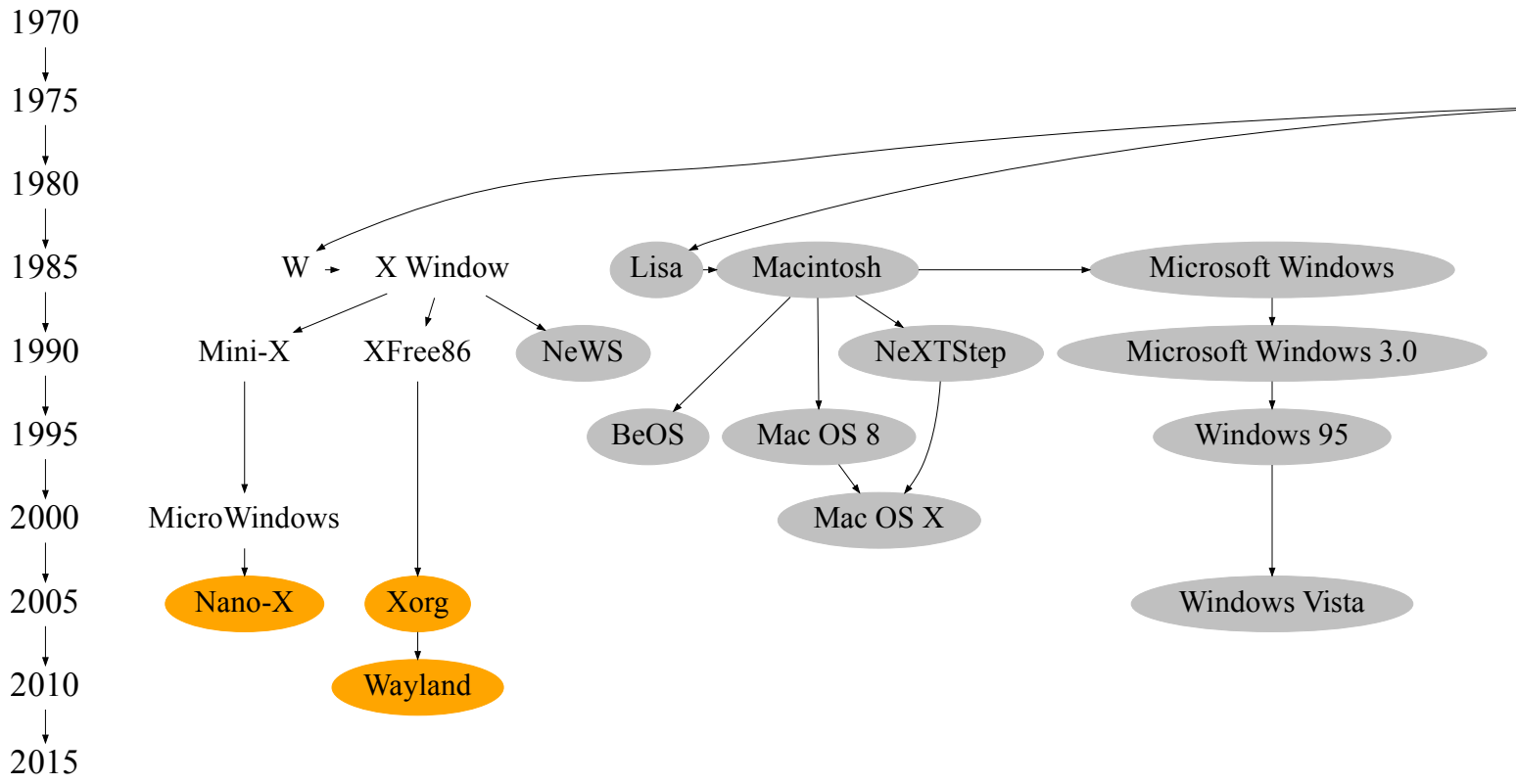

Figure 1.1: Windowing systems timeline

following commands:

```
1   $ bind -a '#v' /dev
2   $ vga -l 640x480x8
# screen should change layout
3   $ bind -a '#i' /dev

4   $ rio
```

Then, if you right-click with the mouse somewhere on the screen, you should see graphics similar to the one in Figure 1.2.

Line 1 through 3, above, install the graphics system of Plan 9 (`draw`) and configure it to run at the `640x480x8` resolution (See the GRAPHICS book [Pad16b] for more information on `draw`). Line 4 then executes `rio`, which should take over the screen to create the graphics shown in Figure 1.2.

1.5 Requirements

Because most of this book is made of C source code, you will need a good knowledge of the C programming language [KR88] to understand it. Moreover, because `rio` relies on many advanced features of the Plan 9 kernel, and builds upon the Plan 9 graphics system `draw`, I strongly suggest you to read the KERNEL book [Pad14] and GRAPHICS book [Pad16b] before reading this book. Note that `rio` is implemented as a filesystem in user space, and uses the protocol 9P to communicate with the kernel. Thus, it can also be useful to read the NETWORK book [Pad16c], which describes 9P. In the same way, I also suggest you to read the LIBCORE book [Pad16a], which explains the thread library, which is heavily used by `rio`.

Figure 1.2: The screen, just after `rio` started and the user right-clicked.

Book	Concepts	Device Files	Codes	Headers

GRAPHICS book	display server, drawing API, shared image, overlapping layers	`/dev/draw`
		`/dev/winname`
		`/dev/vgactl`
			`#i #v`	`draw.h window.h`
				`mouse.h`
				`keyboard.h`

KERNEL book	filesystem, device, pipe, console, per-process namespace, union-mount, shared memory	`/dev/cons`
		`/dev/consctl`
		`/dev/mouse`
		`/dev/pipes/`
			`#c #m #|`	`syscall.h`

LIBCORE book	channel, proc, thread, message, message queue			`libc.h thread.h`
NETWORK book	remote procedure call (RPC), filesystem in user space, 9P protocol	`/srv`	`#s`	`fcall.h`

Table 1.1: Principia Softwarica books related to the WINDOWS book.

Table 1.1 presents the list of related Principia Softwarica books, as well as the concepts, devices, and header files used by `rio` and introduced by those books. The most important book in Table 1.1 is the GRAPHICS book. Regarding the three other books, you can probably understand most of the code in the following chapters without reading those books if you read at least *Plan 9 from Bell Labs* [PPD+95], as well as *The Use of Name Spaces in Plan 9* [PPT+93]; those two articles introduce many of the concepts listed in Table 1.1.

As I said in Section 1.1, there are very few books explaining the concepts, theories, and algorithms used in windowing systems. I can cite *The NeWS book* [GRA89], *Methodology of Window Management* [HDF+86], and one chapter of *Computer Graphics, Principles and Practice* [FDFH90]. Those books are useful, but they are not mandatory to understand this book.

If, while reading this book, you have specific questions on the interfaces of `rio`, or on the API used by `rio`, you can find answers in certain manual pages. Those pages are located under `docs/man/` in my Plan 9 repository. Here is a list of pages relevant to `rio` and a short description of their content:

- `1/rio`: the command-line and graphical user interface of `rio`
- `4/rio`: the filesystem interface of `rio`
- `2/draw`: the `draw.h` API

- `2/window`: the `window.h` API
- `2/keyboard`: the `keyboard.h` API
- `2/mouse`: the `mouse.h` API
- `2/thread`: the thread and channel library
- `5/0intro`: the 9P protocol

Finally, the `windows/docs/` directory in my Plan 9 repository contains documents describing either `rio` [Pik00b] or ancestors of `rio` [Pik91, Pik89, Pik88, Pik83a]. All those documents are useful to understand some of the design decisions presented in this book.

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

1.8 Acknowledgments

I would like to acknowledge of course the author of `rio`, Rob Pike, who wrote in some sense most of this book.

Chapter 2

Overview

Before showing the source code of `rio` in the following chapters, I first give in this chapter an overview of the general principles of a windowing system. I also describe quickly the graphical user interface of `rio`, as well as its filesystem interface in `/mnt/wsys/`. I also show the code of a toy application running under `rio`. Finally, I define terms, explain how the code is organized, and more generally give the background necessary to understand the code I will show later.

2.1 Windowing system principles

A *windowing system* is a program (or a set of programs) with a graphical user interface (GUI) allowing the user to create and manipulate windows. A *window* is a usually rectangular and resizeable surface of the screen containing the GUI of another program. Just like the kernel is a *meta-program*, that is a program allowing the user to run other programs, the windowing system is a *meta-GUI*, that is a graphical user interface allowing the user to run other graphical user interfaces.

In addition to windows, a windowing system traditionally uses *icons*, *menus*, and a *pointer*, or *WIMP* for short. Note that windowing systems are not the only kind of meta-GUIs. For example, the user interface of phones running under iOS or Android do not have any windows. Moreover, the user can use multiple pointers at the same time through his multiple fingers. Those interfaces are called *post-WIMP* interfaces.

A *desktop system* is a kind of windowing system promoting the following metaphor: a windowing system is like a physical desk in an office. A desktop system usually includes applications and icons mimicking the real-life objects of an office: a garbage can, folders, a clock, a rolodex, an alarm, a calendar, etc. The window is then like a paper on a desk; it can be moved around, or stacked on top of other papers. Each window represents a separate activity.

The Xerox Star [SIKH82] was the first desktop system. Desktop systems are the most popular windowing systems (e.g., Microsoft Windows, macOS), because they offer a familiar interface to the user. On Linux, the desktop systems KDE[1] and GNOME[2] are implemented as a set of applications on top of X Window. Plan 9 does not have a desktop system; `rio` does not use any icon and does not promote an office metaphor. `rio` is not a WIMP, just a WMP.

A windowing system has many components, which sometimes can even be separate programs. I mentioned them already in Section 1.3: the display server, the window compositor, the window manager, and the terminal emulator. Figure 2.1 presents the relationships between those components. Figure 2.1 shows also four important layers: the hardware at the very bottom, the kernel and windowing system in the middle, and the applications at the top. Moreover, in this book, I divide applications in three different categories:

- *Textual applications*: those are command-line programs, which just read and output text (e.g., `grep`).

[1] https://www.kde.org/
[2] https://www.gnome.org/

```
+-------------+ +-------------+ +-----------+
|   Window    | |   Window    | |  Textual  |
|application 1| |application 2| |application|
+------+--^---+ +-+---^-------+ +----^------+
       |  |       |   |              |
+------+--+-------+---+--------------+------+
|      |  |       |   |          +---v----+ |
|      |  +-------+---+----+------>Terminal| |
|   +--v----------v+       |     |emulator| |
|   |    Window    <-------+-----+        | |
|   |  compositor  |       |     +--------+ |
|   +--------------+     +-+------------+   |
|   |   Display    |     |    Window    |   |
|   |    server    <-----+   manager    |   |
|   +----+---------+     +----^-----^---+   |
|        |    Windowing system|     |       |
+--------+--------------------+-----+-------+
+--------v--------------------+-----+-------+
|        |         Kernel     |     |       |
+--------+--------------------^-----^-------+
+--------v---+          +-----+-+ +-+-------+
|  Display   |          | Mouse | |Keyboard |
|   device   |          |device | | device  |
+------------+          +-------+ +---------+
```

Figure 2.1: Components of a windowing system.

```
+-----------------------------------+
|+--------------------+----icons----+|
||                    |     +--+ \ /||
||       Title Bar    |     |  |  X ||
||                    | --- +--+ / \||
|+--------------------+-------------+|
||                                   ||
||                                   ||
||                                   ||
||          Window Content           ||
||                                   ||
||                                   ||
||                                   ||
||                                   ||
|+-----------------------------------+|
+-----------Window border-------------+
```

Figure 2.2: Window elements.

- *Graphical applications*: those are programs drawing on the whole screen and using the mouse (e.g., the Doom video game)
- *Window applications*: those are programs drawing and using the mouse inside a window (e.g., the Microsoft minesweeper video game)

A windowing system is mostly concerned with the last category, but it usually offers a way to run in a special environment the other kinds of applications. For instance, the terminal emulator in Figure 2.1 allows to run textual applications.

The following sections will explain the relations between the different components in Figure 2.1 as well as the role of each of those components. I will also quickly describe how those components are implemented in X Window and `rio`. But before, I need to better characterize what is a window.

2.1.1 The window

A window is multiple things at the same time. First, it is a delimited surface of the screen containing the visual *output* of a running program[3]. It is also an interactive region of the screen responding to *input* from the mouse; when the mouse hovers inside a window, the underlying program can react. Finally, it is a container that can be manipulated from the outside; a window can be moved, resized, closed, etc.

A window surface is made of multiple elements, as shown in Figure 2.2. Here is the list of those elements as well as their functions:

- *Window content*: contains the GUI of the underlying running program
- *Window border*: allows the user to move or resize the window
- *Title bar*: contains usually the name of the program
- *Windowing system icons*: allows the user to close, expand, or hide the window

[3]The term "window" is actually a misnomer [Pik83b]. In graphics terminology, such an output is called instead the *viewport*.

Only the first element is mandatory; the other elements, forming the *window decoration*, are all optionals. For instance, in `rio`, windows have a border but they have neither a title bar nor icons (see Figure 2.6).

In `rio`, as well as in many other windowing systems, windows can overlap each other. Thus, windows can be *stacked* on top of each other, hiding the pixels of the windows below. In other windowing systems, windows instead are *tiled* automatically next to each other (or hidden completely). Finally, in recent windowing systems, windows can be *composited* with each other; the windowing system composes the images representing the different windows together and can apply special effects, for instance, transluscence or drop shadows as in macOS[4].

2.1.2 Display server

The first component of a windowing system is the display server. A *display server* is a graphics system that accepts drawing commands from multiple *clients* via a *communication protocol*, and then translates those commands into instructions to the graphics card. The display server is responsible for all the visual output of all the applications, as well as the visual output of the windowing system itself (e.g., the window decorations, the background image). This is why in Figure 2.1, the applications, window manager, and terminal emulator are all connected to the display server (through the window compositor, which I will explain in Section 2.1.3). A display server uses a client/server architecture because it needs to serve many clients: the multiple processes corresponding to the multiple windows on the screen.

In X Window (as well as in most windowing systems), the display server is an integral part of the windowing system. Moreover, the communication protocol of X Window, X11, is used not only to carry drawing commands from the clients, but also to relay input events from the devices to the clients. In that case, the display server acts also as a *window server* as it manages all the communications with the windows.

In Plan 9, the display server `draw` is outside `rio`, in the kernel, and serves only the drawing commands from the clients. The communication protocol of `draw` is described in the Graphics book [Pad16b] (and involves the `/dev/draw/x/data` files).

2.1.3 Window compositor

A *window compositor* is an optional component of a windowing system (found in modern windowing systems) allowing windows to be *composited* with each other. Each window application draws first in an *off-screen image*. Then, the compositor composes all those images together while applying possibly advanced effects such as transluscence or drop shadows. Finally, the composition is sent to the display server, which outputs the result on the screen. This is why in Figure 2.1, the window applications are all connected first to the compositor. The compositor and display server are usually tightly coupled, hence the direct contact between the two respective boxes in Figure 2.1.

In `rio`, which favors a minimalist approach, there is no compositor; each window application is connected directly to the display server (`draw`). That means `rio` does not offer special effects such as transluscence, but it would not be difficult to extend `rio` to include a compositor to support those effects.

2.1.4 Window manager

The *window manager* is the component responsible for all the user inputs to the windowing system: inputs from the mouse, the keyboard, or other devices. Those inputs can lead to the moving, resizing, opening, closing, or hiding of windows, hence the term "window manager". Indeed, the action of the mouse over the window decorations can trigger the changes to the windows listed above. In fact, the window manager is also responsible for the display of those window decorations. This is why in Figure 2.1, the window manager is connected to the display server,

[4] Another nice effect is to zoom out and tile automatically all the images of the windows, as in macOS Expose, to get a bird's eye view of all the windows.

When the mouse is over the window content, the role of the window manager is then to relay the input events to the application. This is why in Figure 2.1, the mouse and keyboard devices are connected to the window manager, which is connected itself to all the window applications (and the terminal emulator) in order to *dispatch* the input to the appropriate window.

In X Window, the window manager is a separate program. There are more than 50 different window managers available for X Window, each with different window decorations, different input policies, different menus, etc. The window manager communicates also with the display server, like other clients. However, the window manager is assigned a special role by the display server.

In Weston and Wayland, the display server, compositor, and window manager are all parts of the same program.

With `rio`, there is only one window manager, which is an integral part of the windowing system. There is only one style of window decoration, kept to a minimum: no title bar, no icon, just a thin blue window border (see Figure 2.6).

2.1.5 Terminal emulator

The last component of a windowing system is the terminal emulator. A *terminal emulator* provides a backward-compatible environment for command-line applications to run inside a window, without having to rewrite and recompile those applications. The emulator has also usually some basic line-editing capabilities such as handling the backspace key or copy-pasting.

The terminal emulator is an optional component of the windowing system that most users never use, but that most programmers can not live without. Indeed, thanks to the emulator, the programmer can run all his classic tools (e.g., shells, compilers, linkers, `grep`) under the windowing system, and even run multiple tools at the same time in different windows. An alternative is to use an integrated development environment (IDE), but IDEs rarely integrate all the tools used by a programmer.

The environment needed by a command-line application under UNIX or Plan 9 is minimal: three opened files, in the first three file descriptors of the process, corresponding to the *standard input*, *standard output*, and *standard error*. Those file descriptors correspond usually to the teletype device (`/dev/tty`) under UNIX, or one of the virtual console under Linux (`/dev/tty1`, `/dev/tty2`, etc.), or finally the console device under Plan 9 (`/dev/cons`). Those file descriptors can also correspond to regular files when the user uses redirections in the shell (e.g., `ls > list.txt`).

Under a windowing system, those input/output descriptors must be connected to the emulator. This is why in Figure 2.1, the terminal emulator is connected in both directions to the textual application; the terminal emulator relays from the window manager the keyboard input to the application, and relays from the application the output text to the display server (by drawing this text with a special font in the window).

In X Window, terminal emulators (e.g., `xterm`, `rxvt`) are separate programs. The standard input and output of command-line applications running under those terminals are connected to a *pseudo-tty* (PTY), which is a pair of *pseudo-devices* (the master and the slave) managed by the kernel. Those pseudo-devices and connected themselves to the terminal program in user space, as well as to the command-line application. I must admit I do not fully understand how it works. The code of `xterm` is very complex with more than 80 000 LOC (eight times more code than the code of `rio`, which is the whole windowing system). `rxvt` is smaller, but still has 37 000 LOC.

With `rio`, the terminal emulator is an integral part of the windowing system and accounts for only 2600 LOC (including the code to support scrollbars, filename completion, copy-pasting, etc). This is mainly because the pseudo-tty is replaced by a more general approach: *virtual devices* and *filesystems in user space*. Indeed, under `rio`, a command-line application still opens the console device by opening in read and write modes `/dev/cons`. However, this file, thanks to the per-process namespace feature of Plan 9, is not the device file managed by the kernel but a virtual device managed by `rio`; every file request on `/dev/cons` is redirected to a 9P request to `rio` (which itself uses the "real" `/dev/cons` managed by the kernel).

```
                                      +---+
                  +------+ /dev/draw   |   |
                  |      +------------->   |
                  |      | /dev/mouse  |   | +--------+
                  |App 1 <-------------+   | |Display |
                  |      | /dev/cons   |   | | device |
                  |      <-------------+   | +--------+
                  +------+             |   |
+------+ /dev/draw   +---+             | K |
|      +------------> w |             | e | +--------+
|      | /dev/mouse  | i |             | r | | Mouse  |
|App 2 <-------------+ n |             | n | | device |
|      | /dev/cons   | d |             | e | +--------+
|      <-------------+ o | /dev/draw   | l |
+------+             | w +------------->   |
                     |   | /dev/mouse  |   |
+------+ /dev/draw   | s <-------------+   | +--------+
|      +------------> y | /dev/cons   |   | |Keyboard|
|      | /dev/mouse  | s <-------------+   | | device |
|App 3 <-------------+ t |             |   | +--------+
|      | /dev/cons   | e |             |   |
|      <-------------+ m |             |   |
+------+             +---+             +---+
```

Figure 2.3: Windowing system as a multiplexer.

2.1.6 Windowing system API

I just described the *internal* structure of a windowing system, as well as the environment needed by command-line applications when running under a terminal emulator. But, what about the *external* interface of a windowing system, as well as the needs of a window application? What API should a windowing system offer to its clients?

Obviously, a window application wants to access the screen, the mouse, and optionaly the keyboard. Note that there is already an API to access those devices under Plan 9. Indeed, a Plan 9 graphical application (e.g., Doom) can simply read or write in the `/dev/draw`, `/dev/mouse`, and `/dev/cons` devices files, which are managed by the Plan 9 kernel (see the GRAPHICS book [Pad16b] and KERNEL book [Pad14]). The top of Figure 2.3 illustrates one such graphical application called `App 1`. This program can also use functions from the `draw.h`, `mouse.h`, and `keyboard.h` header files instead of using directly the device files, but those functions are just thin wrappers that ultimately read and write in the corresponding device files.

Thus, under Plan 9, one way to define the external interface of a windowing system is to just mimic the interface defined by the kernel, as shown at the bottom of Figure 2.3. As mentioned in Section 1.2, `rio` is implemented as a *multiplexer*; it accesses the physical devices (managed by the kernel) at the bottom right of Figure 2.3, and provides virtual devices to its clients (`App 2` and `App 3`) with similar interfaces at the bottom left of Figure 2.3.

This multiplexer approach makes `rio` a *transparent* [Pik88] windowing system. Indeed, `rio` does not need to introduce a new API. In fact, this transparence enables the same graphical application to run with or without `rio`, as shown respectively in Figure 2.4 and Figure 2.5 for the `clock` application. Under Plan 9, there is almost no difference between a graphical application and a window application. This transparence enables also `rio` to run recursively under itself (see Section 13.3).

X Window, on the opposite, is not a transparent windowing system. It introduces a special protocol and special APIs to access the screen, the mouse, and the keyboard. You can not run a window application without

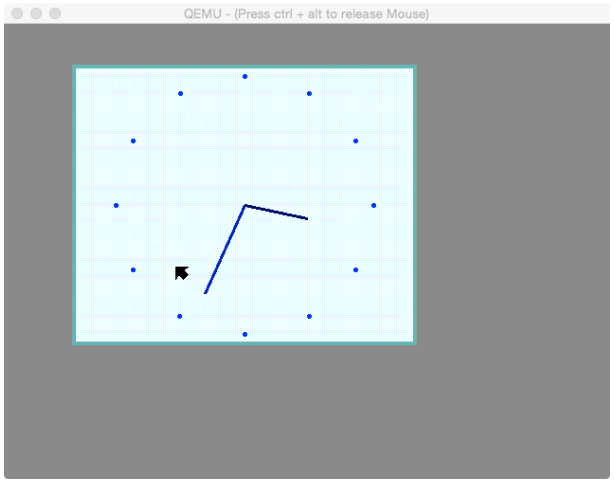

Figure 2.4: `clock` running inside `rio`.

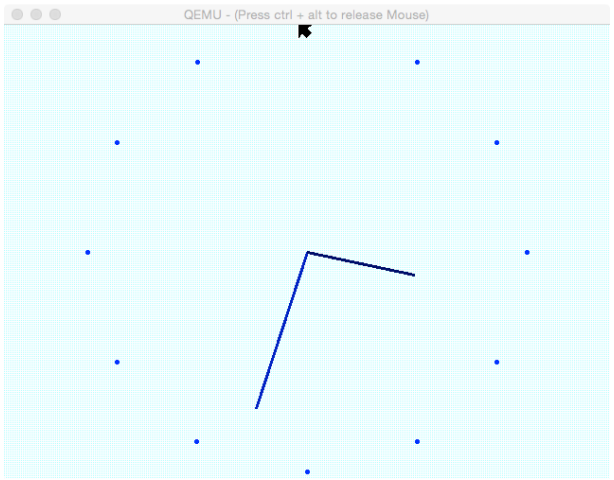

Figure 2.5: `clock` running without `rio`.

X Window, and you can not run a graphical application (e.g., Doom) inside a window in X Window (unless you rewrite Doom to use the X Window API).

2.1.7 Window applications versus graphical applications

Window applications and graphical applications have a lot in common: they both draw on the screen, use the mouse, and possibly use the keyboard. As I said above, in Plan 9, the difference between a graphical application and a window application is fuzzy since you can also run graphical applications inside a window. However, this is possible only because of some careful design decisions in `rio` and `draw`. There are a few differences between running inside or outside a windowing system, as explained in the following sections.

A virtual screen

One of the role of a windowing system is to offer abstractions hiding complexity, just like the kernel does. Indeed, thanks to *virtual memory*, a process thinks it is alone and has access to the whole memory, starting at address 0. This simplifies greatly programming; the programmer does not have to care about the physical memory layout and which memory is used by other processes. In the same way, thanks to preemptive scheduling, a process thinks it is alone and has exclusive access to the CPU. Again, the programmer does not have to care about the other processes and how they use the processor (or processors). Each process uses a *virtual CPU*.

When running without `rio`, a graphical application has access to the whole screen. The origin point (`Pt(0,0)`) corresponds to the top left corner of the screen (see the GRAPHICS book [Pad16b]). This should be similar when the graphical application is running under `rio`. This is why `rio` offers a *virtual screen* to each window, where the origin point corresponds to the top left corner of the window content.

Thanks to the virtual screen, a graphical application running in a window does not even know it is running inside a windowing system. Moreover, the programmer does not have to care about the other windows, or where is located the window on the screen; whatever the location, drawing a line from `Pt(0,0)` will always start from the top left corner of the window, even though the window itself is located at the very right of the screen. The programmer can use *virtual coordinates* (a.k.a. *logical coordinates*); those coordinates are then translated in *physical coordinates* on the screen by the windowing system.

A virtual mouse

What is true for the screen should also be true for the mouse. This is why `rio` offers also a *virtual mouse* to each window; the location of the mouse read by the window application is relative to the window, not the whole screen. The programmer can use virtual coordinates again.

With `rio`, the programmer does not have to check if the mouse is on top of its window, or if the mouse is used concurrently by another program; all of this is handled automatically by the windowing system, which hides complexity by offering a virtual mouse. When a window is at the top, and the mouse cursor over this window, `rio` then *dispatches* the mouse event to the corresponding process.

Overlapping windows

In addition to a reduced drawing surface, an important difference for a graphical application running inside a window is that windows can overlap each other, and so hide each other. Where are stored the hidden pixels? How are those hidden pixels restored when the window is exposed back?

In X Window, because the early graphics machines did not have much memory, the hidden pixels are not saved anywhere. Instead, the display server *notifies* the client by an *expose event* when a part of its window is exposed back. It is the responsibility of the client, and so of the programmer, to draw back what was hidden.

With `rio`, the programmer does not have to care about overlapping windows. The windowing system hides complexity by storing the hidden pixels in *off-screen images*. When a window is exposed back, the windowing

system then copies the pixels from those off-screen images back to the screen. This is consistent with the idea of the virtual screen: the programmer does not have to care about the other windows.

To manage overlapping windows, `rio` relies on a special data structure of `draw`: the image layer (see the GRAPHICS book [Pad16b] and [Pik83b]). A *layer* is an image that overlaps a rectangular sub-area of another image called the *base layer*. With `draw`, the programmer can use multiple layers stacked on top of each other and on top of the base layer. When a program draws in a layer, the pixels overlapped by another layer are automatically saved in an off-screen image. All the drawing functions of `draw.h` have special code to handle the case where the image passed as a parameter is a layer. The programmer can also use additional functions from `window.h` that are valid only for layers, for instance, moving a layer at the top with `topwindow()`. This function possibly copies the hidden pixels saved in an off-screen image (if the layer was overlapped) back to the base layer (e.g., the screen).

A layer is similar to a window, but the layer does not have any associated process; it is just a graphic construct. It is one of the building block of `rio`. Indeed, as you will see later in Section 2.5.2, with `rio` *each window is associated to a layer*; each window will draw in its layer. Moreover, when the user clicks on a window, `rio` internally calls `topwindow()` with the appropriate layer as a parameter.

Creating windows

A windowing system should offer an API to create windows, not just to draw things in a window. In Plan 9, the toplevel windows, whose dimensions are specified by the user (see Section 2.2.2), are created by `rio`, but each graphical application can also create *sub-windows* inside its window. Just like `rio` internally uses layers to represent toplevel windows, a graphical application can also use layers to represent sub-windows. Thanks again to `window.h`, an application such as the editor `acme` can use multiple layers to represent different files in multiple columns. In the same way, a dialog box, a menu, or any *widget* can be represented internally as a layer. By using a layer, the programmer of the widget does not have to care about the pixels overlapped by the widget (or the pixels of the widget overlapped by another widget).

Note that layers do not have a border. To implement the window decorations, `rio` draws a blue border rectangle inside the layer. Note also that sub-windows have to be inside the parent window. It is not possible to create a layer that extends above the boundaries of the window. However, `rio` offers another API to create toplevel windows. This API requires advanced features of `rio` and so is explained later in Chapter 13.

Resizing windows

The last difference between a window application and a graphical application is that windows can be resized. Unfortunately, as opposed to overlapping windows, this complexity can not be hidden to the programmer.

In most windowing systems, the window application is notified of a *resize event* when its window is resized. It is then the responsibility of the programmer to provide a *callback* for such an event that redraws everything.

In X Window, this event is communicated to the client application through the general *event mechanism* of X11. At startup time, the application communicates to the display server an *event mask* specifying the set of events the application is interested in. If the resize event matches the event mask, then X Window will notify the client of a resize event.

With `rio`, the event is communicated to the application through the `/dev/mouse` virtual device file. A graphical application usually reads this file to keep track of the changes to the mouse location and the states of its buttons. During a resize event, `/dev/mouse` contains the character `'r'` (for resize), instead of the character `'m'` (for mouse).

2.2 `rio` interfaces

I just described the general principles of a windowing system, and illustrated those principles with examples from X Window and `rio`. I will now focus exclusively on `rio` and give more details about its interfaces.

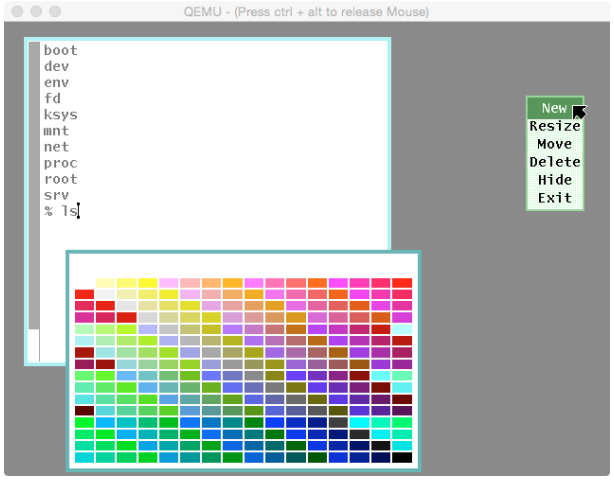

Figure 2.6: Graphical user interface of `rio` after a right-click.

`rio` is first a program you run through the command line, with optional command-line arguments. Then, it takes over the screen and becomes a graphical application. Finally, it internally spawns a new process acting as a filesystem. Thus, `rio` has three different interfaces, which I will describe in the following sections.

2.2.1 Command-line interface

The command-line interface of `rio` is pretty simple:

⟨*function* `usage` 25⟩≡ (243a)

```
void
usage(void)
{
    fprint(STDERR, "usage: rio [-f font] [-i initcmd] [-k kbdcmd] [-s]\n");
    exits("usage");
}
```

Most of the time you will run `rio` without any argument, as shown in Section 1.4. The arguments are all optionals and correspond to advanced features of `rio` I will explain in Chapter 13.

2.2.2 Graphical user interface

The most important interface of `rio` is of course its graphical user interface. Once launched from the command-line, `rio` takes over the whole screen[5] and displays a grey background image, as shown in Figure 1.2. Then, using the mouse, you can create new windows. Figure 2.6 illustrates the main elements of `rio`'s GUI.

As mentioned in Section 2.1, `rio` has a minimalist interface: no icon, no title bar, just a thin blue border around windows. You can left-click on a window to put it at the top if it was overlapped. Moreover, by doing so, the window gets the *focus*; this means every keyboard input will be sent to this window. When a window gets the focus, its border changes from a light blue to a darker blue, as shown at the bottom of Figure 2.6. Moreover, for window terminals, if the window loses the focus, the text inside the window changes from black to a light grey, as shown in the left of Figure 2.6.

By right-clicking outside any window, on the grey background image of `rio`, you trigger a *system menu* allowing you to create, resize, move, delete, or hide a window, as shown in the right of Figure 2.6. Note that `rio` requires a mouse with three buttons (left, middle, and right). To create a new window, activate the system menu, then hold the right-click, choose `New`, and finally release the right-click. The cursor will then change

[5] Unless it is run recursively inside one of its window, as shown in Section 13.3.

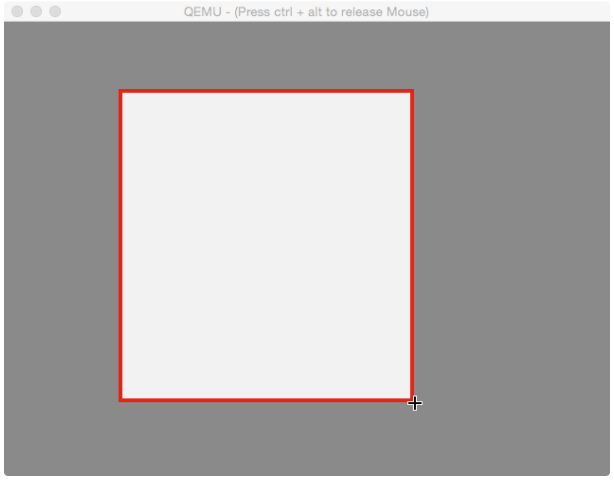

Figure 2.7: Creating a new window.

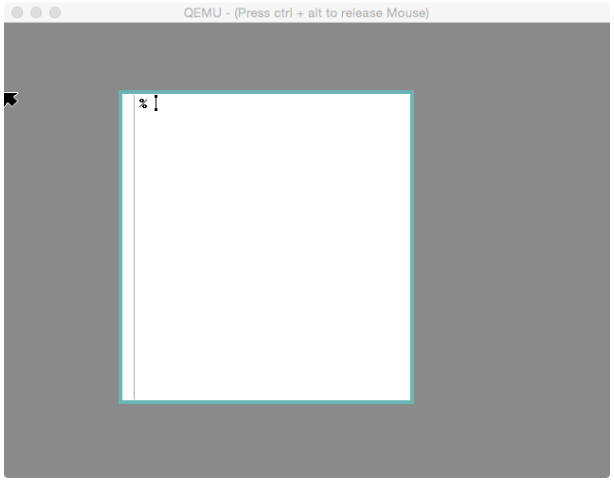

Figure 2.8: Built-in terminal running under `rio`.

from a big arrow to a plus sign, to indicate a different *mode* of operation. You must now specify a rectangle by right-clicking again and hold while drawing a rectangle on the screen, as shown in Figure 2.7. Once you release the right-click, a new window terminal will appear, as shown in Figure 2.8, with a shell prompt at the top.

Because there is no icon in `rio`, to launch a graphical application you need first to create a window terminal. Once created, you can type in the terminal window the name of the graphical application you want to launch, for instance, `colors`. This application should then take over the virtual screen of the window terminal[6], as shown in Figure 2.9. Once you quit this graphical application, the terminal will be back.

You can also use the border of the window to resize or move the window. When the mouse is over the border, the cursor changes again to indicate a possible action. By left-clicking or middle-clicking on the border, you can respectively resize or move the window.

For a full description of the GUI of `rio`, I refer you to its manual page in `docs/man/1/rio` in my Plan 9 repository. You can also watch the historical demo of the Blit and `mpx` at https://www.youtube.com/watch?v=emh22gT5e9k; the user interface of `mpx` is almost identical to the one in `rio`.

[6] Just like it takes over the whole screen when launched outside `rio`.

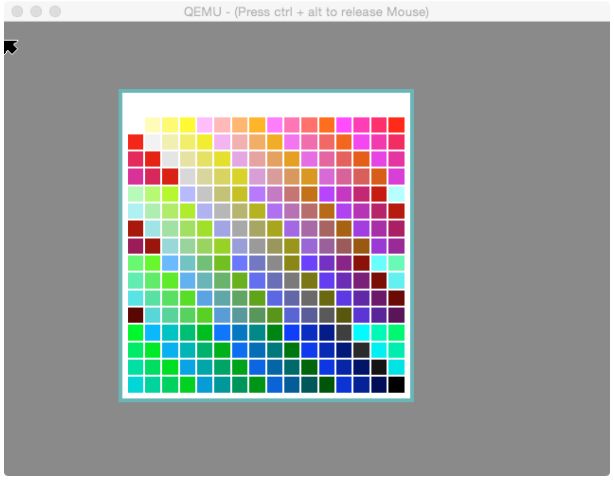

Figure 2.9: `colors` graphical application running under `rio`.

2.2.3 Filesystem interface

As opposed to the two previous interfaces, the last interface of `rio` is almost invisible to the user. It is the set of files served by `rio` to all its window processes: the filesystem interface. It is mostly invisible because it is mainly an interface for programs, not users. Moreover, as explained in Section 2.1.6, `rio` is a transparent windowing system; the files served by `rio` mimic and replace existing device files managed by the kernel (e.g., `/dev/mouse`), making the interface harder to notice.

In the following sections, I will describe the files served by `rio`. Those files act as interfaces to access the screen, the mouse, the keyboard, but also the windowing system itself. I will prefix those files with `/mnt/wsys/` because that is where the `rio` filesystem is originally mounted by the window processes. However, as mentioned in Section 1.2, those files are also available in `/dev`, thanks to the union-mount feature of the Plan 9 kernel. It is by being accessible in `/dev/` that `rio` can become a multiplexer, as explained in Section 2.1.6. However, by using the `/mnt/wsys` prefix, it is clearer that the file involved is the file served by `rio` (e.g., `/mnt/wsys/mouse`) and not the real device served by the kernel (e.g., `/dev/mouse`).

In the following sections, I will also repeat partly explanations found in the Graphics book [Pad16b] and Kernel book [Pad14], as `rio` emulates interfaces provided by the graphics system and the kernel. However, the interfaces have a few differences because the context in which the graphical application runs is different.

The screen, `/dev/draw` and `/mnt/wsys/winname`

The first interface `rio` needs to mimic is the interface to the screen, which under Plan 9 involves the `/dev/draw/` directory and the `draw.h` header file.

Figure 2.3 showed a windowing system multiplexing `/dev/draw`, `/dev/mouse`, and `/dev/cons`. This was how `8-1/2` [Pik91], the ancestor of `rio`, was implemented. Under `8-1/2`, each window was seeing a `/mnt/wsys/draw` virtual device file in which the process could write drawing commands. `8-1/2` would then convert the logical coordinates in the drawing command to physical coordinates, and send the resulting command to the real `/dev/draw` device file managed by the kernel. This was elegant. However, each drawing operation involved two communications: one between the application and the windowing system, and the other between the windowing system and the kernel. Those two round-trips were too inefficient for certain applications.

To remove one round-trip, `rio` does not multiplex `/dev/draw`. Instead, each window application connects directly to the `draw` device, which is more efficient. However, this forced the `draw` device to serve multiple processes in addition to `rio` (in `/dev/draw/2/`, `/dev/draw/3/`, etc), and to become a display server. Moreover, this required a new mechanism for `rio` to communicate to `draw` which part of the screen is allocated to each client of `draw`. This mechanism is complex and involves multiple features of `draw` and `rio`:

1. Instead of `/mnt/wsys/draw`, `rio` serves a file named `/mnt/wsys/winname`, which contains a different string for each window (e.g., `"window.3"`).

2. This string corresponds to the name of a layer allocated by `rio` for the window (see Section 2.1.7 for an introduction to layers). This layer has the dimension specified by the user during the creation of the window (see Figure 2.7).

3. This layer, which is also an image, is made *public* by `rio`. Thanks to an inter-process communication (IPC) feature of `draw`, this public image can be accessed by multiple processes (see the GRAPHICS book [Pad16b]).

4. On the client side, remember that each graphical application must first call `initdraw()`. `initdraw()` calls itself `gengetwindow()`, which reads `/dev/winname` and grabs a reference to the corresponding public image. This reference is then stored in the global `view`. By using `view` instead of `display->image` (see the GRAPHICS book [Pad16b]) for the arguments of the drawing functions of `draw.h`, the graphical application can become also a window application, for free.

Note that `initdraw()` calls `gengetwindow()` even when the graphical application runs outside `rio`. To *bootstrap*, the `draw` device serves also a `/dev/winname` file, which contains the string `noborder.screen.1`. This string corresponds also to an image: the whole screen. In that case, the global `view` and `display->image` references both the screen.

For more information, Section 2.5.5 explains the full trace of a drawing operation through the windowing system, the graphics system, and the kernel.

The mouse, `/mnt/wsys/mouse` and `/mnt/wsys/cursor`

The second interface `rio` needs to mimic is the interface to the mouse, which under Plan 9 involves the `/dev/mouse` and `/dev/cursor` files, as well as the `mouse.h` header file.

The `/dev/mouse` interface defined by the mouse device driver in the kernel is simple. As mentioned in Section 2.1.7, a graphical application can keep track of changes to the mouse location or the states of its buttons by reading (with `read()`) `/dev/mouse`. When the user does an action with the mouse, the `read()` system call returns, and the buffer parameter of `read()` is modified to contain the character `'m'` (for mouse), followed by integers encoding the location and the button states of the mouse. The graphical application can then inspect those integers and modify the GUI, or do nothing.

As opposed to `/dev/draw`, `rio` does multiplex `/dev/mouse`[7] as shown in Figure 2.3. The main job of `rio` regarding the mouse interface is to relay a `read()` by the window application on `/mnt/wsys/mouse` to a `read()` on the real `/dev/mouse`, and to convert the physical coordinates of the mouse to logical coordinates. Moreover, as mentioned in Section 2.1.7, `rio` abuses `/mnt/wsys/mouse` to also transmit the resize events to the graphical application. The buffer contains then the character `'r'` (for resize).

After a window application reads the `'r'` character in `/mnt/wsys/mouse`, it is the responsibility of the programmer to call `getwindow()`, which is a function similar to the `gengetwindow()` function I mentioned before. `getwindow()` also reads `/dev/winname`. This file should contain a new string corresponding to a new public image with the new dimension of the window. Then, `getwindow()` grabs the reference to this new public image in which the graphical application should now draw in. `getwindow()` also updates the global `view`.

Another job of `rio` is to multiplex `/dev/cursor` and to offer a *virtual cursor* to each application. Thanks to `/mnt/wsys/cursor`, each application can use a different cursor and can change the cursor. When the user

[7] Why the difference? Why is there not a mouse server, just like there is a display server? Because there is no need to optimise the access to `/dev/mouse`. Indeed, the two round trips underlying the access to `/mnt/wsys/mouse` are not as critical as the access to an hypothetical `/mnt/wsys/draw`. In the context of a video game, a game application may have to generate 30 to 60 images per second; this may require hundreds or more calls to drawing functions of `draw.h`, and many accesses to `/dev/draw`. However, user actions are slower and the application does not need to react to a mouse event so frequently. Moreover, the size of the data involved with `/dev/mouse` is small: just a few bytes to represent a mouse location and button states.

hovers a window, the cursor changes according to what the window application wrote in his `/mnt/wsys/cursor` file; if nothing was written, a default cursor is used.

The fact that a program running in a window opened or not its `/mnt/wsys/mouse` file is an important information for `rio`. Depending on this opened status, `rio` will behave differently. For instance, if an application opens `/mnt/wsys/mouse`, many features of the terminal emulator are automatically disabled. Indeed, using the mouse is a strong hint for `rio` that the application is a graphical application, not a command-line application. Another hint is the opening of `/dev/draw/new` by the application, but this file is not managed by `rio`, and so out of reach of `rio`. In the rest of the book, I will use the term *graphical window* for a window in which the underlying program opened `/mnt/wsys/mouse`, and *textual window* otherwise.

The last important information regarding the mouse interface concerns the `mouse.h` header file. A programming alternative to using directly `/dev/mouse` is to use the functions from `mouse.h` such as `initmouse()` (see the GRAPHICS book [Pad16b]). `initmouse()` internally uses `/dev/mouse`, but wraps the device file in a data structure (`Mousectl`) allowing the use of *threads* and *channels* (see the LIBCORE book [Pad16a]). Using channels gives more flexibility to the programmer. For instance, the program can *react* simultaneously to changes in `/dev/mouse` and `/dev/cons`, and so can handle mouse events as well as keyboard events. Without `mouse.h`, the programmer can either read synchronously `/dev/mouse`, or read synchronously `/dev/cons`, but not both at the same time[8]. Note that `rio` itself calls `initmouse()` at startup time, and uses heavily channels and threads, as you will see in the rest of the book.

The keyboard, `/mnt/wsys/cons` and `/mnt/wsys/consctl`

The last interface `rio` needs to mimic is the interface to the keyboard, which under Plan 9 involves the `/dev/cons` and `/dev/consctl` files, as well as the `keyboard.h` header file.

`/dev/cons` stands for *console device*; it is the device representing the terminal. The `/dev/cons` interface defined by the kernel is simple: a program reading from `/dev/cons` will read characters from the keyboard; a program writing to `/dev/cons` will output text on the screen. At boot time, the first Plan 9 process opens `/dev/cons` two times: one in read-mode, and the other in write-mode. Those two first file descriptors correspond to the *standard input* and *standard output* of the program (see the KERNEL book [Pad14]). Those file descriptors and then inherited through `fork()` and `exec()` by the shell, as well as by the command-line applications launched from the shell, unless the user used redirections (see the SHELL book [Pad18]). Remember that the `printf()` function from the C library internally does some `write(1, ...)`m and the `scanf()` function internally does some `read(0, ...)`.

The kernel offers also a few convenient features by default regarding the input of characters. For instance, when a program reads `/dev/cons`, the user can interactively edit the characters to sent to the program by using the *backspace* or *delete* keys to correct typing mistakes. He can also use the *cursor* keys to move in the line. Moreover, the characters typed are automatically *echoed* on the screen, making it easy to see the typing mistakes. Finally, the characters are sent only when the user types the *newline* character. By putting those features in the kernel, all command-line applications do not have to care about typing mistakes.

Note that those line-editing features can also be disabled by the application. Indeed, in certain contexts, the application may not want the input to be buffered. For instance, a video game wants to respond as soon as possible to the key typed by the user; it does not make sense to force each time the user to also type the newline character. This is why the kernel provides also the `/dev/consctl` (for console control) device file. By writing `rawon` (for *raw access* on) in `/dev/consctl`, the application indicates to the kernel that the default line-editing features of the kernel should be disabled; the application wants raw access to the keyboard (note that `rio` itself writes `rawon` in `/dev/consctl` at startup time, so it can handle the keyboard itself). A call to `read()` on `/dev/cons` will then return after each key is typed. Moreover, no character will be echoed by default on the screen.

[8] There is no `select()` system call in Plan 9. Threads, channels, and the Plan 9 function `alt()` are the Plan 9's way to do things done usually with `select()` under UNIX.

The job of `rio` regarding the keyboard interface depends also on whether the application in the window wants or not raw access to the keyboard. This is why `rio` multiplex not only `/dev/cons`, but also `/dev/consctl`. The behavior of `/mnt/wsys/cons` depends on the opening and content of `/mnt/wsys/consctl`. This is usually correlated with whether or not the window is a graphical window (raw access on), or textual window (line-editing on).

For graphical windows, writing on `/mnt/wsys/cons` should be considered an error. Indeed, the graphical application should instead use the `string()` function from `draw.h` to output text on the screen at a specific location via `/dev/draw` (see the GRAPHICS book [Pad16b]). Only reads on `/mnt/wsys/cons` should be supported by `rio`. Moreover, each key typed should be sent directly to the graphical application if its window has currently the focus.

For textual windows, the job of `rio` and its terminal emulator is to imitate what the kernel does by default, including the line-editing features. In fact, under `rio`, the user can also use *copy-pasting* with the mouse to edit a line before it is sent to the program. Both reads and writes on `/mnt/wsys/cons` should be supported by `rio` for textual windows. Reads to `/mnt/wsys/cons` by a command-line application should return only when the user typed the newline character in the terminal emulator. Writes to `/mnt/wsys/cons` should be converted to calls to `string()` by the terminal emulator, in order to output text at the right place in the window (possibly applying line wrapping).

Similar to the mouse, a programming alternative to using directly `/dev/cons` is to use the functions from `keyboard.h` such as `initkeyboard()` (see the GRAPHICS book [Pad16b]). `initkeyboard()` internally enables raw access to the keyboard by writing `rawon` in `/dev/consctl`. `initkeyboard()` internally uses `/dev/cons`, but wraps the device file in a data structure (`Keyboardctl`) allowing also the use of threads and channels. Again, as you will see in Chapter 4, `rio` itself calls `initkeyboard()` at startup time.

Other `/mnt/wsys/` files

I just presented the main files served by `rio` to its window processes. Those files are virtual versions of device files managed by the kernel. Thanks to those virtual devices, `rio` is a transparent windowing system enabling graphical applications to run inside windows.

`rio` serves also files that are interfaces to advanced features of the windowing system itself. Here is the list of those files and a short description of their content (Chapter 12 and Chapter 13 will give more details about those files):

- `/mnt/wsys/winid`: This read-only file contains a number unique to each window, the *window identifier*. This identifier is useful in conjunction with `/mnt/wsys/wsys/` presented below.
- `/mnt/wsys/label`: `rio` does not use title bars, but each window can write a string called a *window label* in its `/mnt/wsys/label` file, to describe the window. This label is then used by the system menu; the menu lists all the hidden windows by their labels (see Section 7.10)
- `/mnt/wsys/screen`: This read-only file contains an image (in the Plan 9 image format) representing the content of the screen at the moment `/mnt/wsys/screen` was read. Thanks to this file, it is very easy to take screenshots in Plan 9.
- `/mnt/wsys/window`: This read-only file contains also an image, but representing only the content of the window.
- `/mnt/wsys/text`: This read-only file is useful only for textual windows. It contains a full dump of the text displayed in the terminal.
- `/mnt/wsys/snarf`: This file is used for copy-pasting (see Section 13.4.1).
- `/mnt/wsys/wdir`: This file is used for filename completion (see Section 13.4.3).

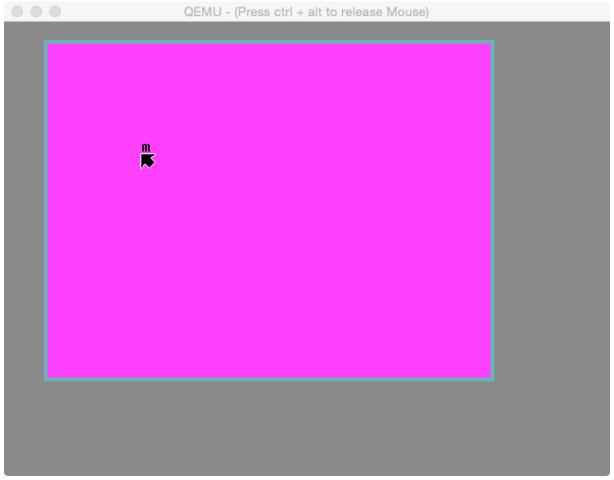

Figure 2.10: `hellorio` running in a window, after the user typed `'m'`.

- `/mnt/wsys/wsys/`: This directory allows to explore the set of windows. `/mnt/wsys/wsys/` is to windows what `/proc/` is to processes (see the KERNEL book [Pad14]). The *key* used for `/mnt/wsys/wsys/` is not the process identifier, as in `/proc`, but the window identifier presented above. Just like `/mnt/wsys/` contains files representing information about the window of the program, `/mnt/wsys/2/` contains the same files but representing the information about the window with the window identifier `2`.

- `/mnt/wsys/wctl`: This file is used to control programmatically a window. Just like a user can use the mouse to act on a window, for instance, by clicking on the border of a window to resize it, a program can use `/mnt/wsys/wctl` to do similar things. By writing *control commands* in `/mnt/wsys/wctl`, a program can control its own window. In fact, a program can control also another window, for instance, by writing in `/mnt/wsys/wsys/2/wctl` (see Section 12.5 for more information).

For more information on the files served by `rio` and their format, I refer you to the documentation of `rio` in `docs/man/4/rio` in my Plan 9 repository.

2.3 `hellorio.c`

Now that you know the principles and the interfaces of `rio`, I can present the code of a toy window application: `hellorio`.

`hellorio` is a simple application written in C displaying initially `Hello Rio` at the location of the mouse. Then, when the user types a key, the `Hello Rio` message is replaced by the typed character, as illustrated in Figure 2.10. Finally, you can quit the application by typing the letter `'q'` (for quit).

The goal of this section is to illustate some of the concepts I introduced before with concrete code. Moreover, the code of `hellorio` in `hellorio.c`, although simplistic, is a good introduction to the code of `rio` itself. Indeed, `rio` also needs to use the mouse, the keyboard, or draw things on the screen. `hellorio.c` will introduce also the use of channels and threads, which are heavily used by `rio`.

I already presented in the GRAPHICS book [Pad16b] the code of a toy graphical application: `hellodraw`. As I said in Section 2.1.6, a graphical application can also be a window application under Plan 9. However, this requires a few changes in the code of the graphical application, for instance, the use of the global `view` as mentioned in Section 2.2.3. `hellodraw` was not using this global, and so could not run inside a window. Moreover, `hellodraw` was just using the graphics facilities of `draw`, with functions from `draw.h`. `hellorio` is an interactive graphical application that also uses the mouse and the keyboard, with functions from `mouse.h` and `keyboard.h`. Finally, by handling the resize event, `hellorio` can also become a window application.

2.3.1 Skeleton and output code

Here is the skeleton of `hellorio.c`:

```
⟨windows/rio/tests/hellorio.c 32⟩≡
  #include <u.h>
  #include <libc.h>

  #include <draw.h>
  #include <mouse.h>
  #include <keyboard.h>

  #include <thread.h>

  Image *bgcolor;
  Point mouseloc;
  Rune str[20];

  ⟨type EventType (hellorio.c) 34a⟩

  void redraw(void);

  void threadmain(int argc, char* argv[]) {
    int result;
    Keyboardctl* keyboardctl;
    Mousectl*    mousectl;
    ⟨threadmain() other locals (hellorio.c) 34b⟩

    result = initdraw(nil, nil, "Hello Rio");
    ⟨threadmain() sanity check result (hellorio.c) 33a⟩
    mousectl = initmouse(nil, view);
    ⟨threadmain() sanity check mousectl (hellorio.c) 33b⟩
    keyboardctl = initkeyboard(nil);
    ⟨threadmain() sanity check keyboardctl (hellorio.c) 33c⟩

    bgcolor = allocimage(display, Rect(0,0,1,1), RGBA32, true, DMagenta);
    runestrcpy(str, L"Hello Rio");
    mouseloc = Pt(200, 200);

    ⟨threadmain() alts setup (hellorio.c) 34c⟩
    redraw();
    ⟨threadmain() event loop (hellorio.c) 35d⟩
  }

  void redraw(void)
  {
    draw(view, view->r, bgcolor, nil, ZP);
    runestring(view, mouseloc, display->black, ZP, font, str);
    flushimage(display, true);
  }
```

Here are a few important things to note about the skeleton of `hellorio.c`, from top to bottom, as well as how the code compares to the code of `hellodraw.c` in the Graphics book [Pad16b]:

- In addition to `draw.h`, `hellorio.c` also includes `mouse.h` and `keyboard.h`, as it uses the mouse and the keyboard.
- `hellorio.c` also includes `thread.h`, a header file containing functions related to threads, the `Channel` structure, and the declarations of primitives operating on channels (e.g., `send()`, `recv()`). A *channel* is essentially a queue of *messages*. Threads communicate and synchronize with each other by exchanging

messages through channels (see the LIBCORE book [Pad16a]). Even though the code in `hellorio.c` does not create threads explicitely, some of the functions called from `hellorio.c` (e.g., `initmouse()`, `initkeyboard()`) do create threads internally (see the GRAPHICS book [Pad16b]).

The Plan 9 thread library defines two separate constructs to carry computations: threads and procs. A *proc* is a Plan 9 process containing cooperatively-scheduled *threads*. Remember that in Plan 9, processes can share memory with each other (via the `RFMEM` flag to `rfork()`, see the KERNEL book [Pad14]). Thus, just like multiple threads in the same proc share memory, multiple procs can also share memory, and can communicate with each other through channels declared in this shared address space. In other operating systems, a Plan 9 proc is similar to a *system thread*, and a Plan 9 thread is similar to a *light-weight thread* (or a *coroutine*).

- As I said in the GRAPHICS book [Pad16b], with `draw` *everything is an image*, including colors, as shown by the type of the global `bgcolor`.

- The key typed by the user is stored in an array of `Rune`s. A *rune* is the name given to a *unicode character* in Plan 9 (see the LIBCORE book [Pad16a]). The type of a key entered in the keyboard is not a `char` but a `Rune` in Plan 9, which is convenient. Indeed, the code of `hellorio.c` will work also if the user enters chinese characters, or letters with european accents. Even special combinations such as the `Control` key and the `d` key will return a single rune, whose representation `^d` will be correctly displayed on the screen (thanks to `runestring()` called in `redraw()`).

- Most of the graphics-related code is not in `main()`, as in `hellodraw.c`, but in a separate function: `redraw()`. This is because this code will be called multiple times, after each input event, as you will see soon.

- The entry point of `hellorio.c` is not `main()` but `threadmain()`. Indeed, the thread library provides already a `main()` that sets up a proc with a single thread executing `threadmain()`.

- Similar to `hellodraw.c`, the first call of `hellorio.c` is to `initdraw()`, to connect to the display server. `hellorio.c` also calls `initmouse()` and `initkeyboard()` to connect respectively to the mouse and the keyboard device. Those two functions internally create a proc reading synchronously on respectively `/dev/mouse` and `/dev/cons` (see the GRAPHICS book [Pad16b]). Those two functions uses a proc, and not a thread, because a thread should not block. Indeed, if for instance a thread is blocked on a read on `/dev/mouse`, the other threads in the same proc would also be blocked. Threads are cooperatively-scheduled; they need to cooperate with each other.

- Regarding the graphics-related code, `hellorio.c` is very similar to `hellodraw.c`, with the use of functions of `draw.h` such as `allocimage()`, `draw()`, `runestring()`, or `flushimage()`. The main difference is the use of the global `view`, set by `initdraw()` (see Section 2.2.3), instead of `display->image`, for the arguments of the drawing functions.

The skeleton of `hellorio.c`, above, omits the error management code shown below:

⟨`threadmain()` *sanity check result (hellorio.c)* 33a⟩≡ (32)
```
  if (result < 0)
    exits("Error in initdraw");
```

⟨`threadmain()` *sanity check mousectl (hellorio.c)* 33b⟩≡ (32)
```
  if(mousectl == nil)
    exits("can't find mouse");
```

⟨`threadmain()` *sanity check keyboardctl (hellorio.c)* 33c⟩≡ (32)
```
  if(keyboardctl == nil)
    exits("can't find keyboard");
```

In the rest of this book, I will usually not comment the error-management code. Such code is often trivial (but necessary).

2.3.2 Input code

I have shown the code responsible for the visual output of `hellorio`. I can now present the code dealing with the inputs to `hellorio`, making `hellorio` an interactive program.

`hellorio` must handle three different kinds of inputs: inputs from the mouse, from the keyboard, and from the windowing system itself with its resize event. The type below defines those different *event types* for `hellorio`:

⟨*type EventType (hellorio.c)* 34a⟩≡ (32)

```
enum EventType {
  EMouse,
  EKey,
  EResize,

  NALT
};
```

`hellorio` needs to deal *simultaneously* with the mouse, keyboard, and windowing system. Indeed, the program can not just synchronously read `/dev/mouse`; maybe the next event will be a keyboard event, not a mouse event. To do so, the `thread.h` header file defines the `Alt` (for alternative) structure. As mentioned in Section 2.2.3, there is no `select()` system call in Plan 9. Instead, Plan 9 provides the `alt()` function (see the LIBCORE book [Pad16a]) that takes as an argument a map of event types to `Alt`. This map is stored first in a local variable:

⟨`threadmain()` *other locals (hellorio.c)* 34b⟩≡ (32) 35a ▷

```
// map<EventType, Alt>
Alt alts[NALT+1];
```

Each value of this map (each "alternative") contains essentially a channel. A call to `alt()` will then return if anything is exchanged (received or sent) on *one of the channels* of the map. In fact, `alt()` will also return the event type associated with the channel in which a message was exchanged. With `alt()`, it is possible to listen to multiple channels at the same time (or to emit to multiple channels at the same time).

For each event type, the local variable `alts` is initialized with the channel to receive from (or send to) in the `Alt.c` field:

⟨`threadmain()` *alts setup (hellorio.c)* 34c⟩≡ (32) 35c ▷

```
alts[EMouse].c = mousectl->c;
alts[EMouse].v = &mouse;
alts[EMouse].op = CHANRCV;

alts[EKey].c = keyboardctl->c;
alts[EKey].v = keys;
alts[EKey].op = CHANRCV;

alts[EResize].c = mousectl->resizec;
alts[EResize].v = nil;
alts[EResize].op = CHANRCV;
```

The `Alt.op` field contains the type of *channel operation*, Here, `CHANRCV` because the program is listening (receiving) on all channels. It is also possible to emit simultaneously on multiple channels by using instead `CHANSND`.

`mousectl` and `keyboardctl`, above, are two local variables containing the return value of respectively `initmouse()` and `initkeyboard()` (see the skeleton of `hellorio.c`). The types of those variables are respectively the structure `Mousectl` and the structure `Keyboardctl`, defined respectively in `mouse.h` and `keyboard.h`. Both structures are explained in the GRAPHICS book [Pad16b]. The most important fields in those structures

are `Mousectl.c` and `Keyboardctl.c` which are the channels used to communicate with the procs created by `initmouse()` and `initkeyboard()`. For instance, the proc created by `initmouse()` reads synchronously on `/dev/mouse`, and when `read()` returns data, this data is sent on `Mousectl.c`. This data can then be read by any thread receiving on `Mousectl.c`. Thanks to `alts`, `hellorio` can listen simultaneously on changes in `/dev/mouse` or `/dev/cons`.

The field `Alt.v` must contain a pointer to an area where to store the data received on a channel (or the data to send, if the channel operation was `CHANSND`). The size of this area depends on the *channel type* (see the LIBCORE book [Pad16a]). For the keyboard, `Keyboardctl.c` is a channel containing up to 20 buffered keys (so even if the user types really fast, no data is lost).

⟨`threadmain()` *other locals (hellorio.c)* 35a⟩+≡ (32) ◁34b 35b▷

```
Rune keys[20];
```

For the mouse, `Mousectl.c` is a channel containing just one `Mouse`:

⟨`threadmain()` *other locals (hellorio.c)* 35b⟩+≡ (32) ◁35a ??▷

```
Mouse mouse;
```

Each array of `Alt`s must finish with a special *end marker*: `CHANEND`.

⟨`threadmain()` *alts setup (hellorio.c)* 35c⟩+≡ (32) ◁34c

```
alts[NALT].op = CHANEND;
```

I can finally show the last piece of code of `hellorio.c`: the *event loop* and the call to `alt()`:

⟨`threadmain()` *event loop (hellorio.c)* 35d⟩≡ (32)

```
for(;;) {
  switch(alt(alts)){
  ⟨threadmain() event loop cases (hellorio.c) 35e⟩
  }
  redraw();
}
```

When one of the channel in `alts` receives a message, `alt()` returns the corresponding event type. The program can then switch on the event type, inspect the message referenced from `Alt.v`, and modify globals.

For the mouse, the program modifies the global `mouseloc` with the last coordinate of the mouse:

⟨`threadmain()` *event loop cases (hellorio.c)* 35e⟩≡ (35d) 35f▷

```
case EMouse:
  mouseloc = mouse.xy;
  break;
```

This global is then used in `redraw()` as a parameter to `draw()` to display some text at the right location.

For the keyboard, the program modifies the global `keys`. As mentioned above, the channel type of `keyboardctl->c` is an array of 20 runes. A channel is a bufferred or unbuffered queue of messages. The call to `alt()` stores only one message in the area pointed by `Alt.v`. It is the responsibility of the programmer to call `nbrecv()` to transfer the remaining messages in the queue.

⟨`threadmain()` *event loop cases (hellorio.c)* 35f⟩+≡ (35d) ◁35e 36▷

```
case EKey:
    for(i=1; i<nelem(keys)-1; i++)
        if(nbrecv(keyboardctl->c, keys+i) <= 0)
            break;
    keys[i] = L'\0';
    runestrcpy(str, keys);
    if(keys[0] == L'q')
      exits("done");
    break;
```

For the resize event, the program just needs to call `getwindow()`, which will update the global `view`, as explained in Section 2.2.3:

⟨`threadmain()` *event loop cases (hellorio.c)* 36⟩+≡ (35d) ◁35f

```
case EResize:
  if(getwindow(display, Refnone) < 0)
    exits("failed to re-attach window");
  break;
```

This concludes the code of `hellorio.c`.

With command-line programs, the program is in control; the program ask questions to the user. With graphical applications (and window applications), the user is in control; the application *reacts* to external events. This is a new programming model. The application is more than an interactive program, it is a reactive program.

2.4 Code organization

The code of `rio` in my Plan 9 repository is split in multiple directories, but the most important one is `windows/rio/`. Table 2.1 presents short descriptions of the source files in `windows/rio/`, as well as the main entities (e.g., structures, functions, globals) the file defines, and the corresponding chapter in this document in which the code contained in the file is primarily discussed. The other directories of `rio` correspond to libraries used by code in `windows/rio`:

- `windows/libframe/`: This library contains the generic code of a *text widget*. This widget consists of a scrollbar on the left and an editable text area on the right. This library is used by the terminal emulator of `rio`, and is discussed in Section 11.2. This library is also used by the Plan 9 editor Acme.
- `windows/libplumb/`: This library is used to interact with the Plan 9 *plumb* mechanism [Pik00a], which is triggered in `rio` by the middle-click menu of the terminal emulator (see Section 13.4.2). Plumbing is a generalization of the cut-and-paste, drag-and-drop, and Multipurpose Internet Mail Extensions (MIME) mechanisms found in maintream operating systems. It allows applications to cooperate with each other without having to know each other.
- `windows/libcomplete/`: This library is used to provide filename completion in the terminal emulator (see Section 13.4.3).

The code of `rio` depends also on code in `lib_graphics/` and `lib_core/`, but the code in those directories is explained respectively in the Graphics book [Pad16b] and Libcore book [Pad16a].

2.5 Software architecture

`rio` is first a Plan 9 graphical application. It is not a special program; it relies, like all graphical applications, on devices managed by the Plan 9 kernel and its graphics stack:

- `/dev/draw/` to access the display device
- `/dev/mouse` and `/dev/cursor` to access the mouse device
- `/dev/cons` and `/dev/consctl` to access the keyboard device

`rio` communicates with the kernel through system calls (e.g., `open()`, `read()`) on those devices (see the Kernel book [Pad14]).

In fact, `rio` does not use directly those devices. Instead, it uses functions from `draw.h`, `mouse.h`, and `keyboard.h`. Those header files offer data structures and functions that are convenient wrappers around those

Function	Ch.	File	Entities	LOC
data structures	3	dat.h	Window[51] Filsys[53i] Fid[54c] Xfid[55c]	0
globals	3	globals.c	mousectl[50b] windowsX input[52l] filsys[53j]	0
entry point	4	rio.c	threadmain()[56a]	0
prototypes	4	fns.h		0
keyboard thread	5	thread_keyboard.c	keyboardthread()[60a]	0
mouse thread	5	thread_mouse.c	mousethread()[61b]	0
window threads	5	threads_window.c	winctl()[64a]	0
fileserver proc	5	proc_fileserver.c	filsysproc()[66f] filsysmount()[83e]	0
master and worker threads	5	threads_worker.c	xfidctl()[69c]	0
cursor graphics	6	data.c	crosscursorX cornersX	0
cursor operations	6	cursor.c	riosetcursor()[74c]	0
window manager	7	wm.c	button3menu()[76b] new()[79b] drag()[92]	0
window process creation	7	processes_winshell.c	winshell()[82d]	0
window methods	7	wind.c	wtop()[87b] wclose()[90c]	0
closing threads	7	threads_misc.c	deletethread()[89e] winclosethread()[90b]	0
fileserver utilities	8	9p.c	filsysrespond()[101b] filsyscancel()[223f]	0
fileserver methods	8	fsys.c	filsysattach()[101f] filsysopen()[105g]	0
virtual device methods	9	xfid.c	xfidopen()[106f] xfidread()[109a]	0
graphical window	10	graphical_window.c	waddraw()[122e]	0
terminal emulator	11	terminal.c	winsert()[137a] wshow()[149a] wkeyctl()[65d]	0
terminal scrolling	11	scrl.c	wscrdraw()[149c] wscroll()[169d]	0
external window control	13	wctl.c	wctlthread()[183b] parsewctl()[187]	0
copy/paste clipboard	13	snarf.c	putsnarf()[194g] getsnarf()[194h]	0
timer	13	timer.c	timerstart()[222f]	0
error management	B	error.c	error()[230j] derror()[230k]	0
utilities	C	util.c	min()[231a] emalloc()[231d] strrune()[233a]	0
Total				8142

Table 2.1: Chapters and source files of `windows/rio/`.

device files. Thus, `rio` depends mainly on `lib_graphics/libdraw/`, the library behind `draw.h`, `mouse.h`, and `keyboard.h` (see the GRAPHICS book [Pad16b]).

`rio` uses many functions from `draw.h`, but also many functions from `window.h`. Indeed, `rio` makes heavy use of `draw`'s layers (see Section 2.1.7 for an introduction to layers). Many of the window management tasks are actually done by `draw`, not `rio`. For instance, it is the `draw` function `topwindow()` which does most of the heavy lifting when you click on a window to put it at the top; it is `topwindow()` that restores the pixels overlapped previously by other windows.

`rio` is also a Plan 9 filesystem that communicates with the kernel using the 9P protocol (see the NETWORK book [Pad16c]). `rio` is a file server that answers to file requests on virtual devices accessed by its windows. Thus, `rio` must be two different things at once: a GUI and a server. Moreover, `rio` must coordinate many *independent* activities: user actions with the keyboard, user actions with the mouse, computations in the window

processes, and actions from those window processes such as file requests on virtual devices. What is a good software architecture to deal with all those activities? A natural architecture is to use *threads*. Indeed, each independent activity can be represented by an independent thread. Then, *channels* can be used to communicate messages between threads. Both channel and thread functions are declared in `thread.h`. Thus, in addition to `lib_graphics/libdraw/`, `rio` depends also on `lib_core/libthread/`, the library behind `thread.h` (see the LIBCORE book [Pad16a]).

I will present in the next section the thread (and process) architecture of `rio`. A good way to understand how those threads work together is to *trace* a user action through those different threads. This is why in the following sections, I will also explain the trace resulting from the creation of a new window by the user, the trace of a mouse click and key press in this window, and the trace of a call to a drawing function by the program running in this window.

2.5.1 Processes, procs, and threads relationships

Figure 2.11 presents the threads and processes resulting from the execution of `rio` and two other programs in two different windows, as displayed in Figure 2.12. As mentioned in Section 2.3, the Plan 9 thread library provides two separate constructs to represent computation: procs and threads. A proc is a Plan 9 process containing one or more cooperatively-scheduled threads. Procs and processes are represented by enclosing rectangles in Figure 2.11, while threads are represented by enclosed rectangles. Moreover, multiple procs sharing the same address space are enclosed by a dashed rectangle.

Figure 2.11 can be decomposed in four different layers:

- The hardware layer, at the very bottom
- The kernel layer, providing abstractions of the hardware
- The filesystem layer (and its many device files), one of the main abstractions offered by the kernel
- The user processes layer

The user processes can themselves be divided in three different groups:

- A set of procs created by `rio` and sharing the same address space, at the top of Figure 2.11
- The `hellorio` process (see Section 2.3), running inside a window on the right in Figure 2.12
- A shell process, running in a terminal emulator on the left in Figure 2.12.

Of course, the most important group is the first group, which contains the procs of `rio`. `rio` is made of four different procs:

- `IO-proc-keyboard`: A proc containing a single thread reading `/dev/cons`, at the top left in Figure 2.11
- `IO-proc-mouse`: A proc containing a single thread reading `/dev/mouse`, at the middle left in Figure 2.11
- `fileserver`: A proc containing a single thread communicating with the kernel using the 9P protocol, at the top right in Figure 2.11
- `main-proc`: A proc containing many threads writing in `/dev/draw/1/`, in the middle of Figure 2.11

```
+ - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - +
     rio        +---------------------------------------------+
|               |  main proc       +----------+  +----------+  | +----+ |
   +----------+ |                  |threadmain|  |  Worker  |  | |+--+|
|  |+--------+| |  +--------+      |          |  |allocator <--+->|  || |
   ||        || |  |Keyboard|      +----------+  +----------+  | ||  ||
|  ||IO proc |+-+--> thread |  +----------+  +--------------+  | ||f || |
   ||keyboard|| |  |        +-->          <-->   Worker 1   |  | ||i ||
|  ||        || |  +---+----+  | Window 1 |  |(r wsys/cons) <--+->|l || |
   |+--------+| |      |       |(terminal)|  +--------------+  | ||e ||
|  +----^-----+ |      |  +---->          <-->   Worker 2   |  | ||s || |
  +-----+       |      |  |    +----------+  |(w wsys/cons) <--+->|e ||
| |             |      |  |                  +--------------+  | ||r || |
  | +---------+ |      |  |    +----------+  +--------------+  | ||v ||
| | |+-------+| |      +--+---->          <-->   Worker 3   <--+->|e || |
  | ||       || |  +------+-+  | Window 2 |  |(r wsys/mouse)|  | ||r ||
| | ||IO proc|| |  | Mouse  |  |(hellorio)|  +--------------+  | ||  || |
  | || mouse |+-+--> thread +-->          <-->   Worker 4   <--+->|  ||
| | ||       || |  |        |  +----------+  |(r wsys/cons) |  | |+--+| |
  | |+-------+| |  +--------+                +--------------+  | +-^--+
| | +---^-----+ |                                              |   |    |
  |     |       +----------+-----------------------------------+   |
+ + - - + - - - - - - - - - -|- - - - - - - - - - - - - - - - - - - -|- - +
  |     |                  |         +-----------+    +---------+  |
  |     +-------+          |         | hellorio  |    |  shell  |  |
  |             |          |         |  process  |    | process |  |
  |             |          v         +-----------+    +---------+  |
 /dev/cons  /dev/mouse /dev/draw/1  /mnt/wsys/mouse  /mnt/wsys/cons|
     ^          ^          |        /mnt/wsys/cons^         | ^    |
     |          |          |        /dev/draw/2 ^ |         | |    |
     |          |          |             |      | |         | |    |
+----+----------+----------v-------------v------+-+---------v-+----v--+
|    |          |          |    Kernel   |       <---------------->   |
+----^----------^----------+-------------+---------------------------+
     |          |          v             |
 +--------+ +--------+ +---------+       |
 |Keyboard| | Mouse  | | Display |       |
 | device | | device | | device  |<------+
 +--------+ +--------+ +---------+
```

Figure 2.11: Processes, procs, and threads in `rio`.

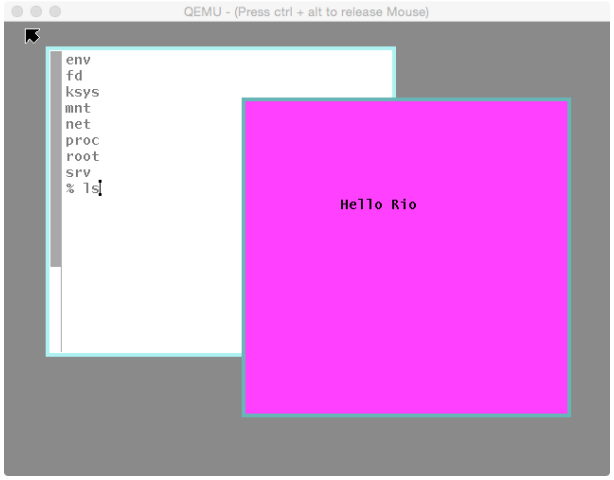

Figure 2.12: `hellorio` and a shell running in two different windows.

I mentioned already the first two procs in Section 2.3 in the context of the `hellorio` program[9]. They are the results of calls to respectively `initkeyboard()` and `initmouse()` by `rio` during startup. Both procs are usually blocked in a `read()` on a device file. Once the user performs an action, with the keyboard or the mouse, one of the `read()` returns and the corresponding proc (actually the single thread in the proc) sends a message on the `keyboardctl->c` or `mousectl->c` channels. Two threads of `main-proc`, the keyboard thread and the mouse thread, are listening on those channels.

The use of multiple procs and threads to read one device file is due to the design of threads in Plan 9. As mentioned in Section 2.3, in Plan 9, threads are scheduled cooperatively. They should avoid to block on a system call doing IO. Indeed, if one of the thread in the proc is blocked, all the threads in the proc are blocked (see the Libcore book [Pad16a]).

However, one advantage of this design is that threads do not need to use locks, an error-prone concurrency construct, for mutual exclusion with each other. By construction, only one thread of a proc is running at one time (but multiple procs can run in parallel).

The third proc, `fileserver`, creates a *pipe* and reads in a loop one side of the pipe. On the other side of the pipe are clients of `rio`'s filesystem (`hellorio` and the shell process in Figure 2.11). Section 2.5.2 will give more details about this pipe and how `rio` and its clients communicate.

The last proc, `main-proc`, is the most important proc of `rio`. It contains many threads. I mentioned already the keyboard and mouse threads above. Both are created by `threadmain`, the first thread of `main-proc`. Moreover, *each window is represented by a thread*. Each window, in addition to being associated with an external process (e.g., the `hellorio` process in Figure 2.11), is also associated with a thread (e.g., `Window-2` in Figure 2.11). Finally, *each opened file of* `rio`*'s filesystem is represented as a (worker) thread*. The next sections will clarify how those threads work together.

2.5.2 Trace of a new window creation

In Figure 2.11, `threadmain`, the first thread of `main-proc`, creates the two IO procs on the left, the `fileserver` proc on the right, and the keyboard and mouse threads in the middle. In this section, I will explain the trace leading to the creation of the shell and `hellorio` processes in Figure 2.11, as well as the creation of the associated window threads. In the next section, I will explain the creation of the worker threads.

As I said in Section 2.2.2, you create a new window by first activating the system menu with a right-click, then choosing `New` in this menu, and finally by drawing a rectangle with the mouse specifying the dimension of

[9]In fact, `hellorio` in Figure 2.11 creates also two IO procs. They are not shown in the figure to simplify things. Those procs would read `/mnt/wsys/cons` and `/mnt/wsys/mouse`.

a window. The trace for those mouse actions will be explained soon in Section 2.5.3, but for now, the important thing is that eventually the mouse thread of `main-proc` will be in control, with the information about the new window dimension stored in a local variable. It is the mouse thread that creates windows. Each window creation involves the creation of a new process, a new namespace, a new `Window` structure, a new window thread, and a new image layer, as explained in the following sections.

A new process

When you create a new window, `rio` internally (1) forks a new process (with `proccreate()`), (2) adjusts the environment in the child process, and finally (3) executes (with `procexec()`) a command (the Plan 9 shell `/bin/rc`) from this child process. This is why the creation of the two windows in Figure 2.12 leads to the creation of two processes in Figure 2.11: `hellorio` and the shell process.

The sequence of steps above is a classic idiom in UNIX programming. Indeed, this is for instance the way the shell implements redirections, as in `ls > list.txt`. In that example, the shell adjusts the environment by changing the standard output file descriptor in the child process to point to a text file instead of the `/dev/cons` console device file (see the SHELL book [Pad18]). For `rio`, the adjust-the-environment part consists in changing the namespace in the child process by using the `mount()` and `bind()` system calls.

A new namespace

A *namespace* is the Plan 9 equivalent of a mount-table in UNIX (and its derivatives such as Linux or macOS). A *mount-table* allows to plug together multiple filesystems in a single uniform tree. However, in Plan 9, each process has its own mount-table, its own namespace; there is not a global mount-table as in UNIX. After the call to `mount()` mentioned above, the child process of `rio` will see special files in `/mnt/wsys/`, thanks to the per-process namespace feature of the Plan 9 kernel. Moreover, because of the `bind()` call, the same files will also be accessible under `/dev/`, and will take precedence over any existing files there, thanks to the union-mount feature of the Plan 9 kernel.

Before showing the actual calls to `bind()` and `mount()` in `rio`, here are the slightly simplified signatures of `bind()` and `mount()` from `syscall.h`:

⟨*simplified signatures of namespace functions* `syscall.h` 41⟩≡
```
error bind(char *str,   char *path, int flag);
error mount(fdt fd, ..., char *path, int flag, char *extra);
```

`bind()` takes as a parameter a string (`str`) and a path (`path`). If the string starts with a `#`, as in `bind("#m", "/dev/", ...)`, the string must correspond to the code of a device in the Plan 9 kernel, for instance, `#m` is the code of the `mouse` device. In that case, every access from the process to files under path will be performed with the methods of the corresponding device, for instance, `mousewalk()` if the process was listing files in `/dev/`. Remember that in Plan 9, devices are filesystems (see the KERNEL book [Pad14]). The mouse device, for instance, manages more than one file (e.g., `/dev/mouse`, `/dev/cursor`). Some devices such as `draw` manage even multiple directories (e.g., `/dev/draw/1/`, `/dev/draw/2/`). If the string did not start with a `#`, every access to files under path are redirected to the path denoted by the string. In effect, it is like `path` becomes an *alias* for the path denoted by `str`.

With `mount()`, the kernel delegates the management of a directory neither to a device in the kernel, nor to another directory, but to a userspace program. Indeed, `mount()` takes as a parameter a file descriptor (`fd`) and a path (`path`), in order to tell the kernel that every access to files under path should be handled by the process behind the file descriptor (see the KERNEL book [Pad14] and the NETWORK book [Pad16c]). This allows to implement filesystems in user space, a powerful and convenient feature. Note that the file descriptor argument can be anything; it can correspond to a pipe to a local process, but also to an opened connection on the network. Thus, filesystems can also be imported (and exported) through the network in Plan 9.

Both `bind()` and `mount()` operate on an existing path and alter the filesystem tree at this path. Moreover, in Plan 9, you can not only plug a filesystem on top of another filesystem at a specific directory, you can also

join filesystems in a directory. Indeed, a directory can be the union of two or more filesystems. Such a directory is called a *union-mount*. Both `bind()` and `mount()` take a flag parameter specifying how to handle the union of the old content of the directory with the new content of the `bind()`'ed device (or directory) or the new `mount()`'ed user-space filesystem. For instance, the flag argument `MREPL` (for replace) indicates that the old content should be ignored and replaced by the new content. The flag argument `MAFTER` indicates that the old content should be kept and should take precedence over the new content in case of name conflicts. Indeed, two filesystems can contain files with identical names. Thanks to the union-mount and a series of calls to `bind()`, the `/dev` directory can contain many files and directories managed by multiple devices.

Here are the slightly simplified calls to `mount()` and `bind()` in the function `filsysmount()` called in the child process of `rio` when a new window is created:

⟨`filsysmount()` *simplified code* 42⟩≡

```
err = mount(fs->cfd, ..., "/mnt/wsys", MREPL, windowid);
...
err = bind("/mnt/wsys", "/dev", MBEFORE);
```

With the call to `bind()`, `/dev/` becomes the union of the old `/dev/` directory and the `/mnt/wsys` directory. The flag argument `MBEFORE` indicates that files under `/mnt/wsys` takes precedence over the old files under `/dev/`. This means than an access to `/dev/mouse` by the process who executed the code above (or the children of this process) will be redirected to an access to `/mnt/wsys/mouse`, and not to the old `/dev/mouse` file that was managed by the kernel (and which was there because of a previous call to `bind("#m", "/dev", ...)`). This is why in Figure 2.11, the `hellorio` and shell processes, at the bottom right, are connected to files starting with `/mnt/wsys/`, while `rio` itself is connected to files starting with `/dev/`, at the bottom left.

Regarding `mount()`, the file descriptor argument corresponds to one side of a pipe created by the `fileserver` proc (see Section 2.5.1) and stored in the `cfd` (for client file descriptor) field of the global `fs` (for filesystem). The other side of the pipe is stored in the `sfd` (for server file descriptor) field of `fs`. Note that file descriptors and namespaces are inherited by default through `rfork()` (or `proccreate()`), and maintained through `exec()` (or `procexec()`). Thus, after the child process of `rio` has called `filsysmount()` and `exec()`'ed the shell `/bin/rc`, this shell process will still have in its file descriptor table in the kernel a link to the pipe created by the `fileserver` proc, and in its mount-table the information about `/mnt/wsys/`. This means that an access from this shell process to `/mnt/wsys/mouse` will be converted by the kernel to a 9P request written in the pipe (see the NETWORK book [Pad16c]). On the other side of the pipe, the `fileserver` proc will read the pipe, decode the 9P request, and return a result to the kernel through the pipe (e.g., an error code).

Thus, it is through a pipe that `rio` and its clients are communicating. This communication is indirect. Indeed, the clients are not writing directly into the pipe; they are just performing system calls on (virtual device) files, as indicated for example by the arrows between the `hellorio` process and the kernel in Figure 2.11. However, the kernel redirects those system calls to 9P requests on the pipe created initially by the `fileserver` proc and inherited by those clients. The arrow connecting the `fileserver` proc to the kernel on the right of Figure 2.11 is the pipe.

Note that `mount()` can take an extra argument (called `extra` in the signature of `mount()` above). For `rio`, this argument contains the window identifier (in the `windowid` local variable in the code above) of the newly created window. This extra argument is stored in the process mount-table and passed by the kernel through the 9P request. Thus, the `fileserver` proc can know from which window (and so from which process) a file request comes from. This is important because the `fileserver` proc reads from a pipe that is shared by many processes.

A new `Window`

Now that the process for the new window has been created, and its namespace adjusted, `rio` needs to keep track of the new window (and the new process). To do so, `rio` allocates a new `Window` structure, which gathers information about a window. `rio` also adds this structure in the global `windows`, which contains the list of all `Windows` (see Section 3.3 for more information). The `Window` structure has many fields. For instance, `Window.id`

contains the window identifier, while `Window.pid` contains the process identifier of the process associated with the window. The other fields will be presented gradually in this document.

A new thread

Each window must deal with inputs from the mouse or the keyboard. Those inputs, which correspond to two independent activities of the user, are managed in `rio` by two independent threads in the left of Figure 2.11: the mouse and keyboard threads (and associated IO procs). Those inputs must then find a way to the right of Figure 2.11 until the `fileserver` proc, which is connected to the window processes (via a pipe). To separate concerns, the mouse and keyboard threads do not communicate directly with the `fileserver` proc. Instead, they communicate with *window threads* (e.g., `Window 1` and `Window 2` in the middle of Figure 2.11). Those threads handle the independent input activities to the independent windows. *Each window is represented by a thread.*

Thus, when you create a new window, `rio` creates internally a new window thread. Moreover, `rio` creates multiple channels to communicate with this new thread. For instance, `Window.mch` contains a channel used to exchange messages between the mouse thread and the window thread.

When you move the mouse, the event is transmitted at some point to the mouse thread of `rio`, which gets the information by listening on `mousectl->c` (this is similar to what `hellorio` did in Section 2.3.2). The mouse thread then looks for the window with the focus, which is stored in the global `current` (see Section 3.3.3), a pointer to a `Window` in `windows`. It then writes information about the mouse event in `current->mch`. On the other side of the channel, a single window thread is listening on this channel and is ready to receive the mouse input for further processing (for instance, by relaying the event to a worker thread, as explained soon in Section 2.5.3).

Because the window with the focus can change, the mouse and keyboard threads can communicate with different window threads. This is why in Figure 2.11 the mouse and keyboard threads are connected to all the window threads.

A new layer

The last thing `rio` internally creates when you create a new window is a new image layer with the dimension of the window. This image is stored in the `Window.img` field of the window. *Each window is associated to one layer.* As I said in Section 2.1.7, an image layer is a convenient graphics construct that `rio` uses to implement overlapping windows. What we need now is a way to communicate this layer to the window application so that this application can draw in this new layer.

As mentioned in Section 2.2.3, each window application is connected directly to the `draw` device, as shown in Figure 2.11 with the `hellorio process` connected to `draw` via `/dev/draw/2/`. The drawing operations by the window application do not go through a virtual `/mnt/wsys/draw` file that would be managed by `rio` (and its `fileserver` proc). However, it is `rio` that creates the layer and so knows where the window application should draw. Fortunately, `draw` has an IPC feature allowing multiple processes to share an image or an image layer. By giving a public name to the new image layer (with `nameimage()`), `rio` allows another process to retrieve a handle to the image (with `namedimage()`) if this process knows the name of the image. This unique name is stored in the `Window.name` field of the window. However, we still need a way to communicate this name to the window application. To do so, `rio` uses the same mechanism it uses to transmit any kind of information to its window: a file under `/mnt/wsys/`, in that case `/mnt/wsys/winname`, which contains the string in `Window.name`.

Note that each window application will read a different string from his `/mnt/wsys/winname` file. Indeed, each window process has a different namespace since a unique window identifier is passed to the `mount()` command (see Section 2.5.2). When a window application reads `/mnt/wsys/winname`, the request goes to the `fileserver` proc through a pipe, and `fileserver` knows which `Window.name` field to read since the window identifier is passed also in the request.

This concludes the trace of a new window creation. Now that `rio` has created all the necessary entities to support the new window, I can explain the trace of a mouse click in this new window in the next section.

2.5.3 Trace of a mouse click

When you click or move the mouse, the information is first transmitted from the mouse device to the kernel, at the bottom left in Figure 2.11. This information must then find a way to `rio` and possibly to the window with the focus, for instance, `hellorio` at the bottom right in Figure 2.11. The goal of this section is to describe the full trace of a mouse click in the window of `hellorio`. This trace will go through many elements of Figure 2.11. Moreover, in this section, I will also explain the creation of the last remaining threads of `main-proc`: the worker threads.

Mouse initialization

Before you click on the mouse, the window application `hellorio` must first become ready to receive such a mouse event by calling `initmouse()`. The trace resulting from such a call is explained below:

1. When `hellorio` starts, it opens `/dev/mouse` (via `initmouse()`) and reads `/dev/mouse` (via its own IO proc, see Section 2.3).

2. The kernel resolves the access to `/dev/mouse` to the file `/mnt/wsys/mouse`, because of a `bind()` call in the parent process of `hellorio` (see Section 2.5.2).

3. The kernel relays a read on `/mnt/wsys/mouse` to a 9P request written in the pipe to `fileserver` proc, because of a call to `mount()` in the parent process of `hellorio` (see Section 2.5.2). At this point, `hellorio` is blocked (actually its mouse IO proc) waiting for his `read()` system call to return.

4. The kernel scheduler eventually schedules the `fileserver` proc, which will read from the pipe, decode the 9P request, and start to process the request.

Worker initialization

The question now is what should the `fileserver` proc do with the read request on `/mnt/wsys/mouse` from `hellorio`? It can not block until you move the mouse. Indeed, another process may also need to communicate with the `fileserver` proc in the mean time, for instance, a process in a terminal window may want to display some text by writing in its `/mnt/wsys/cons` file. To manage independent file requests, the `fileserver` proc offloads work to independent worker threads. *Each worker thread represents an opened file of* `rio`*'s filesystem.*

In the case of `hellorio` reading `/mnt/wsys/mouse`, the worker thread is called `Worker 3` in Figure 2.11. This thread will then wait for mouse events by listening on a special channel, as explained soon. Once `Worker 3` receives a mouse event, it can communicate back (by using another channel) the event to the `fileserver` proc, which can write back the appropriate 9P answers into the pipe. At this point, the kernel will unblock the `hellorio` process (actually its mouse IO proc), which can modify the GUI or do nothing depending on the mouse event returned from `/mnt/wsys/mouse`.

Mouse event

The remaining question is how does the mouse event arrive to the `Worker 3` thread? Now that `hellorio` is ready to receive mouse events, and `Worker 3` has been created, I can finally explain the trace resulting from a mouse click in the window of `hellorio`:

1. A click on the mouse device, at the bottom left in Figure 2.11, generates an hardware interrupt, which is managed by the kernel (see the KERNEL book [Pad14]).

2. The kernel looks for a process waiting on `/dev/mouse` and so unblocks the `IO-proc-mouse` proc of `rio`, at the left in Figure 2.11.

3. This proc contains a single thread that was reading on `/dev/mouse`. This thread parses the mouse event contained in the data read from `/dev/mouse`, and sends it to `mousectl->c`.

4. The message is received by the mouse thread, which was listening on `mousectl->c` (see Section 2.5.2). The mouse thread then looks for the window with the focus and relays the message on `wcurrent->mch`.

5. The `Window 2` thread, at the middle of Figure 2.11, which was listening on `Window.mch` (as well as other channels via `alt()`) receives the message on `Window.mch`. It needs then to transmit this message to `Worker 3`. When the `fileserver` proc created the `Worker 3` thread, it also created a channel to communicate with this thread. This channel is stored in the `Window.mouseread` field of the `Window` structured allocated for `hellorio`. Thus, `Window 2` can communicate with `Worker 3` by writing the mouse event through this channel.

At this point, The `Worker 3` thread, which was listening on `Window.mouseread`, receives the mouse event and relays it to the `fileserver` proc as explained above. This concludes the trace of mouse click.

Note that if you click with the mouse outside any window, the mouse event does not go to any window thread. Instead, the event stops at the mouse thread, which contains code to process the event to possibly move, resize, hide, or create a new window. Indeed, as mentioned in Section 2.5.2, it is the mouse thread that creates windows and all their associated entities (processes, namespaces, layers, threads, and `Windows`).

Worker allocator

2.5.4 Trace of a key press

The trace of a key press in the window of `hellorio` is similar to the trace of a mouse click. The trace starts also at the bottom left in Figure 2.11, but this time with the keyboard device. The key event is transmitted from the kernel to the `IO-proc-keyboard` through `/dev/cons`, then relayed to the keyboard thread through `keyboardctl->c`, then to the window thread `Window 2` through a `Window.ck` channel, then to the worker thread `Worker 4` through a `Window.consread` channel, then to the `fileserver` proc, and finally back to the kernel and `hellorio` through a pipe. `hellorio` finally reads the key event through his `/mnt/wsys/cons` file that was opened (in raw access mode) at startup via a call to `initkeyboard()`.

For textual windows, such as the shell process in Figure 2.11 (represented by the `Window 1` thread), the trace of a key press is more complicated. Indeed, `rio` provides advanced line-editing features that complicates the flow of data to `/mnt/wsys/cons`. Moreover, this file is opened both in read and write modes, resulting from two worker threads in Figure 2.11: `Worker 1` and `Worker 2`. This is why the trace of a key press in a textual window will be explained later in Chapter 11.

2.5.5 Trace of a drawing operation

The final trace illustrating Figure 2.11 is the trace of a drawing operation by `hellorio`. As I said in Section 2.2.3, the ancestor of `rio`, `8-1/2` [Pik91], was offering a virtual `/mnt/wsys/draw` device file; the trace of a drawing operation was then similar to the trace of a mouse click or a key press in Section 2.5.3 and Section 2.5.4. However, for efficiency reasons, with `rio` the window applications are connected directly to the `draw` device, as shown in Figure 2.11 with the `hellorio` process connected to `draw` via `/dev/draw/2/`. This simplifies the trace of a drawing operation, but complicates the initialization of graphics for the window process. Indeed, as I explained partly already in different sections, `rio` and `draw` needs to cooperate to enable window applications to draw on the screen:

- `rio` is using `draw`'s layers to implement overlapping windows (see Section 2.1.7)

- `draw` is using a client/server architecture to support the multiple clients of `rio` (see Section 2.2.3)
- `draw` supports the naming and sharing of layers across multiple clients (see Section 2.2.3)
- `hellorio` uses in his code the global `view`, which is a reference to a layer, to draw things in its window (see Section 2.3)
- `rio` is using `/mnt/wsys/winname` to communicate the name of a layer to the window (see Section 2.5.2)

The goal of this section is to put together all those pieces of information to better understand how a window application draws on the screen.

Graphics initialization

Before being able to call drawing functions from `draw.h`, `hellorio` must first connect to the `draw` device with `initdraw()` (see the GRAPHICS book [Pad16b]). This is similar to what `hellorio` did in the trace of a mouse click with `initmouse()`, and in the trace of a key press with `initkeyboard()`. In fact, `rio` itself also calls at startup those `initxxx()` functions. In the case of `rio`, all those functions connect to real `/dev/` devices, not virtual devices under `/mnt/wsys/`. The graphics initialization of `hellorio` relies actually on two calls done previously by `rio`:

1. The call to `initdraw()`, which sets the global `view`, during the initialization of `rio`
2. The call to `allocwindow()` to create a new layer, using `view` as the base layer, during the creation by `rio` of the new terminal window from where `hellorio` will be eventially launched

Thanks to those two previous calls, the call to `initdraw()` in `hellorio` can fetch a reference to the layer created by `rio`.

The following paragraphs will give more details about those steps by describing the traces of those steps. However, those traces will be partial. Indeed, I will omit certain explanations that are contained in the GRAPHICS book [Pad16b].

`rio` and `initdraw()` The trace of `initdraw()` in `rio` is as follows:

1. When `rio` starts, it calls `initdraw()`, which opens `/dev/draw/new` to open a new connexion to the display server (see the GRAPHICS book [Pad16b]).
2. The kernel resolves the access to `/dev/draw/new` to a method of the `draw` device in the kernel, because of a call to `bind("#i", "/dev/")` in the parent process of `rio` (see Section 1.4). Since `rio` is the first application to open `/dev/draw/new`, the kernel returns a file handler to `/dev/draw/1/ctl`. This is why in Figure 2.11, the `rio` process is connected to the `/dev/draw/1` directory.
3. `initdraw()` reads the information about the screen contained in `/dev/draw/1/ctl` and sets the global `display`.
4. `initdraw()` calls `gengetwindow()`, which opens and reads `/dev/winname`.
5. The kernel resolves the access to `/dev/winname` to a method of the `draw` device. The content returned for this file is `"noborder.screen.1"`, the name given by `draw` to the framebuffer image.
6. `initdraw()` then calls `namedimage()` to grab a reference to the image named `"noborder.screen.1"`. Remember that API calls to `draw` such as `namedimage()` are translated by `libdraw` in written commands in `/dev/draw/1/data` (see the GRAPHICS book [Pad16b]).

7. The kernel resolves the access to `/dev/draw/1/data` to a method of the `draw` device. This method parses the `namedimage()` command, creates a new image identifier, find the image corresponding to `"noborder.screen.1"` (the framebuffer), internally links the new image identifier to this image, and finally returns the new image identifier.

8. `initdraw()` reads information about this new image identifier in `/dev/draw/1/ctl` and stores the information in the global `view`.

Once `rio` called `initdraw()`, it can draw things on the screen by passing the global `view` as an argument to drawing functions. Moreover, after `initmouse()`, `rio` enables also the user to use the mouse.

`rio` and `allocwindow()` The trace leading to the creation of a new layer is as follows:

1. To create a new window, you must right-click on the mouse while dragging the mouse to specify a rectangle on the screen (see Section 2.2.2). All those actions are handled by code in the mouse thread of `rio`. This code also uses the global `view` as an argument to functions in `draw.h`, for instance, to draw on the screen the white rectangle specified with the mouse, as well as its red border (see Figure 2.7).

2. Once you release the right-click, `rio` creates internally many new entities: a process, a namespace, a `Window` (stored in the global array `windows`), and a thread (see Section 2.5.2). `rio` also calls `allocwindow()` from `window.h` to create a new layer. It passes `view` as the base-layer argument to `allocwindow()`, and the rectangle specified with the mouse for the dimension of the layer. The call to `allocwindow()` is translated by `libdraw` in a command written in `/dev/draw/1/data`. The `view` argument is reduced to its image identifier.

3. The kernel parses the `allocwindow()` command written in `/dev/draw/1/data` and allocates a new layer with a new image identifier. It uses the framebuffer as the base layer for this layer, since the image identifier written in `/dev/draw/1/data` was previously linked by the kernel to the framebuffer. The kernel also records in a list the set of layers associated with the framebuffer, and put this new layer at the top of the list. If a program draws in this layer, it will draw directly in the framebuffer since the layer is at the top. Finally, the kernel returns the new image identifier to `rio`.

4. `rio` stores the new image identifier in the `Window.img` field of the `Window` structure allocated for the new window. `rio` creates also a new name for this layer, e.g. `"window.1"`. and stores it in the `Window.name` field. `rio` then calls `nameimage()` with `Window.img` and `Window.name` as arguments, to name and share the layer. This call is translated again by `libdraw` in a command written in `/dev/draw/1/data`.

5. The kernel parses the `nameimage()` command and adds in a global hash table in the kernel an association between `"window.1"` and the layer created previously.

Remember that when you create a new window, this window is always first a textual window managed by the terminal emulator. This window is initially connected to a shell process, as shown at the bottom right in Figure 2.11. Because the terminal emulator is an integral part of `rio`, it has access to the globals of `rio` such as `view` or `windows`. Thus, the terminal emulator can draw in the layer stored previously in `Window.img`. It can translate text output from the shell process through `/mnt/wsys/cons` to calls to drawing functions taking `Window.img` as an argument (e.g., `string()`).

At some point, you can launch `hellorio` from this terminal window by typing the name of the command in the shell.

`hellorio` and `initdraw()` Here is finally the trace of `initdraw()` in `hellorio`:

1. When `hellorio` starts, it also calls `initdraw()` (like `rio` did), which opens `/dev/draw/new`. Since `hellorio` is the second application to open this file, the kernel returns a file handler to `/dev/draw/2/ctl`. This is why in Figure 2.11, the `hellorio` process is connected to the `/dev/draw/2` directory.

2. `initdraw()` calls `gengetwindow()`, which opens and reads `/dev/winname`.

3. The kernel resolves this time the access to `/dev/winname` to `/mnt/wsys/winname`, and communicates the read request on this file to the `fileserver` proc. This is different from the trace of `initdraw()` in `rio`. Indeed, the namespace set in the parent process of `hellorio` by `rio` is different from the namespace of `rio` itself.

4. The `fileserver` proc allocates a new worker thread to serve `/mnt/wsys/winname` for `hellorio`. Note that this thread is not shown in Figure 2.11. Indeed, this worker thread exists only during the call to `initdraw()`. Before returning, `initdraw()` closes `/dev/winname`, and so `fileserver` releases the worker thread.

5. The worker thread looks for the name of the layer for `hellorio` in the `Window.name` field of the `Window` associated with `hellorio`. This `Window` is stored in the global array `windows`. To find the appropriate `Window`, the worker thread compares the window identifier stored in `Window.id` and the window identifier in the 9P request (which was originally passed to `mount()` in the parent process of `hellorio`, see Section 2.5.2). Finally, the worker thread returns `"window.1"`, the content of `Window.name` for the window from where `hellorio` was launched.

6. `initdraw()` then calls `namedimage()` to grab a reference to the image named `"window.1"`. The kernel creates a new image identifier, find the image corresponding to `"window.1"` by looking in a global hash table (this image is the layer created by `rio` above), internally links the new image identifier to this image, and finally returns the new image identifier. `initdraw()` then stores the new image identifier in the global `view` of the `hellorio` process.

This concludes the graphics initialization for `hellorio`.

Graphics operation

Once `hellorio` called `initdraw()`, it can draw things in its window by passing the global `view` as an argument to drawing functions. This time, `view` corresponds to an image layer in the kernel, not the framebuffer. However, this layer is linked to the framebuffer, as well as the other layers created for the other windows. Here is the trace of the call to `draw()` in the `redraw()` function in `hellorio.c` (see Section 2.3.1):

1. When `hellorio` calls `draw()`, the call is translated by `libdraw` in a command written in `/dev/draw/2/data`. The `view` argument is reduced to its image identifier.

2. The kernel resolves the access to `/dev/draw/2/data` to a method of the `draw` device. This method parses the `draw()` command and fetches the image associated to the image identifier written in `/dev/draw/2/data`. This image is actually a layer, which is associated to the framebuffer.

3. The kernel then calls `memdraw()`, a function from `libmemlayer` that contains special code to handle image layers. `memdraw()` then goes through the list of layers associated to the framebuffer to check if the layer of `hellorio` is the top layer. If it is, `memdraw()` calls `memimagedraw()` from `libmemdraw` to draw pixels directly in the framebuffer. If it is not the top layer, `memdraw()` allocates a new off-screen image and calls `memimagedraw()` to draw in this image instead for all the rectangles overlapped by other layers.

Later, if you click on the window of `hellorio` to make it the top window, `rio` will call the `draw` function `topwindow()`. This command will be parsed by the kernel. The kernel will readjust the list of layers associated to the framebuffer, and copy back the pixels from the off-screen image associated to the layer of `hellorio` to the framebuffer.

This concludes the trace of a drawing operation, as well as the presentation of the software architecture of `rio`. The traces above are simplified, but the essential elements are there. For a more precise trace of a drawing operation, see the GRAPHICS book [Pad16b]. For more details about the other traces, read the following chapters.

2.6 Book structure

You now have enough background to understand the source code of `rio`. The rest of the book is organized as follows. I will start by describing the core data structures of `rio` in Chapter 3. Then, I will use a top-down approach, starting with Chapter 4 with the description of `main()` and the core initializations in `rio`. I will continue in Chapter 5 by presenting the main functions called by the threads and procs of `rio`. I will describe also the messages and channels used by those threads. Then, I will describe the code to manage cursors in Chapter 6, before presenting the code of the window manager in Chapter 7. Starting from a mouse click, I will describe the main functions to create, focus, delete, move, resize, and hide windows. In Chapter **??** I will switch to a bottom-up approach, and switch the focus from `main-proc` to `fileserver`, by presenting the filesystem methods of the `fileserver` proc. Those methods are entry points that offload the work to worker threads and specific functions to handle the different virtual devices of `rio` under `/mnt/wsys/` (e.g., `/mnt/wsys/cons`, `/mnt/wsys/mouse`). I will present the code to serve those virtual devices in Chapter 9. Some virtual devices behave differently depending if the window is a graphical or textual window. This is why I will describe the specifics of graphical windows in Chapter 10, and of textual windows in Chapter 11, including the code of the terminal emulator. Chapter **??** presents the code to serve the other files under `/mnt/wsys` (e.g., `/mnt/wsys/ctl`). Chapter 13 presents advanced functionalities of `rio` that I did not present before to simplify the explanations. Finally, Chapter 14 concludes and gives pointers to other books in the Principia Softwarica series.

Some appendices present the code of non-functional properties: code to help debug `rio` in Appendix A, and code to manage errors in Appendix B. Appendix C contains the code of utility functions used by `rio`, but which are not specific to `rio`. Finally, Appendix D contains examples of graphical applications that extends the window-manager component of `rio`, for instance, a window-switching bar.

Chapter 3

Core Data Structures

Show me your code and conceal your data structures, and I shall continue to be mystified. Show me your data structures, and I won't usually need your code; it'll be obvious.

Fred Brooks

In this chapter, I will present the core data structures of `rio`, which are essentially: TODO

3.1 Device handlers

3.1.1 Output device: `display` and `view`

⟨*global* `viewr` 50a⟩≡ (243b)
```
Rectangle viewr;
```

3.1.2 Input devices: `mousectl` and `keyboardctl`

⟨*global* `mousectl` 50b⟩≡ (242)
```
Mousectl *mousectl;
```

⟨*global* `keyboardctl` 50c⟩≡ (242)
```
Keyboardctl *keyboardctl;
```

⟨*global* `mouse` 50d⟩≡ (242)
```
// alias for &mousectl->Mouse
Mouse *mouse;
```

3.2 Desktop, `desktop`

⟨*global* `desktop` 50e⟩≡ (242)
```
Screen *desktop;
```

⟨*global* `background` 50f⟩≡ (242)
```
Image *background;
```

⟨*global* `red` 50g⟩≡ (242)
```
Image *red;
```

3.3 Windows

3.3.1 Window

⟨*struct* Window 51⟩≡ (237b)

```
struct Window
{
    //-----------------------------------------------------------------
    // ID
    //-----------------------------------------------------------------
    ⟨Window id fields 52a⟩

    //-----------------------------------------------------------------
    // Graphics
    //-----------------------------------------------------------------
    ⟨Window graphics fields 52c⟩

    //-----------------------------------------------------------------
    // Mouse
    //-----------------------------------------------------------------
    ⟨Window mouse fields 65g⟩

    //-----------------------------------------------------------------
    // Keyboard
    //-----------------------------------------------------------------
    ⟨Window keyboard fields 64b⟩

    //-----------------------------------------------------------------
    // Control
    //-----------------------------------------------------------------
    ⟨Window control fields 66a⟩

    //-----------------------------------------------------------------
    // Process
    //-----------------------------------------------------------------
    ⟨Window process fields 83a⟩

    //-----------------------------------------------------------------
    // Config
    //-----------------------------------------------------------------
    ⟨Window config fields 53h⟩

    //-----------------------------------------------------------------
    // Textual Window
    //-----------------------------------------------------------------
    ⟨Window textual window fields 53a⟩

    //-----------------------------------------------------------------
    // Graphical Window
    //-----------------------------------------------------------------
    ⟨Window graphical window fields 52m⟩

    //-----------------------------------------------------------------
    // Misc
    //-----------------------------------------------------------------
    ⟨Window other fields 52k⟩

    //-----------------------------------------------------------------
    // Extra
    //-----------------------------------------------------------------
```

```
    ⟨Window extra fields 52f⟩
  };
```

⟨Window *id fields* 52a⟩≡ (51) 52e▷

```
  int     id;       // visible through /mnt/wsys/winid
  char    name[32]; // visible through /mnt/wsys/winname
```

⟨*global* id 52b⟩≡ (249)

```
  static int id;
```

⟨Window *graphics fields* 52c⟩≡ (51) 52d▷

```
  // ref_own<Image>, public image for the window (name in /dev/winname)
  Image *i;
```

⟨Window *graphics fields* 52d⟩+≡ (51) ◁52c

```
  /*
   * Rio once used originwindow, so screenr could be different from i->r.
   * Now they're always the same but the code doesn't assume so.
  */
  Rectangle screenr; /* screen coordinates of window */
```

⟨Window *id fields* 52e⟩+≡ (51) ◁52a 84g▷

```
  char    *label;   // writable through /mnt/wsys/label
```

⟨Window *extra fields* 52f⟩≡ (51) 52g▷

```
  Ref;
```

⟨Window *extra fields* 52g⟩+≡ (51) ◁52f

```
  QLock;
```

3.3.2 windows

⟨*global* window 52h⟩≡ (242)

```
  // growing_array<ref_own<Window>> (size = nwindow+1)
  Window **windows;
```

⟨*global* nwindow 52i⟩≡ (242)

```
  int nwindow;
```

⟨*global* topped 52j⟩≡ (249)

```
  static int  topped;
```

⟨Window *other fields* 52k⟩≡ (51) 81h▷

```
  int   topped;
```

3.3.3 current

⟨*global* input 52l⟩≡ (242)

```
  //option<ref<Window>>, the window with the focus! the window to send input to
  Window *input;
```

3.3.4 Graphical windows

⟨Window *graphical window fields* 52m⟩≡ (51) 122c▷

```
  bool mouseopen;
```

3.3.5 Textual windows

⟨Window *textual window fields* 53a⟩≡ (51)
  ⟨Window *textual window fields, text data* 53b⟩
  ⟨Window *textual window fields, text cursors* 53c⟩
  ⟨Window *textual window fields, visible text* 53d⟩
  ⟨Window *textual window fields, graphics* 53f⟩

⟨Window *textual window fields, text data* 53b⟩≡ (53a)
```
// growing_array<Rune> (size = Window.maxr)
Rune  *r;
uint  nr; /* number of runes in window */
uint  maxr; /* number of runes allocated in r */
```

⟨Window *textual window fields, text cursors* 53c⟩≡ (53a) 53e ▷
```
// index in Window.r
uint  q0; // cursor, where entered text go (and selection start)
// index in Window.r
uint  q1; // selection end or same value than q0 when no selection
```

⟨Window *textual window fields, visible text* 53d⟩≡ (53a)
```
uint  org;
```

⟨Window *textual window fields, text cursors* 53e⟩+≡ (53a) ◁ 53c
```
// index in Window.r
uint  qh; // output point
```

⟨Window *textual window fields, graphics* 53f⟩≡ (53a) 53g ▷
```
Frame frm;
```

⟨Window *textual window fields, graphics* 53g⟩+≡ (53a) ◁ 53f
```
Rectangle scrollr;
```

⟨Window *config fields* 53h⟩≡ (51) 88d ▷
```
bool scrolling;
```

3.4 Filesystem server

3.4.1 FilSys and filsys

⟨*struct* Filsys 53i⟩≡ (237b)
```
struct Filsys
{
    // client
    fdt  cfd;
    // server
    fdt  sfd;

    // ref_own<string>
    char *user;

    // map<fid, Fid> (next in bucket = Fid.next)
    Fid  *fids[Nhash];

    ⟨Filsys other fields 55b⟩
};
```

⟨*global* filsys 53j⟩≡ (242)
```
Filsys *filsys;
```

⟨*constant* Nhash 54a⟩≡ (237b)

```
#define Nhash 16
```

⟨Fid *extra fields* 54b⟩≡ (54c) 54e ▷

```
// list<Fid> (head = Filsys.fids[i])
Fid  *next;
```

3.4.2 File state: Fid

⟨*struct* Fid 54c⟩≡ (237b)

```
struct Fid
{
    // the key
    int  fid;

    // the state
    bool open;
    int  mode;

    ⟨Fid other fields 54d⟩

    // Extra
    ⟨Fid extra fields 54b⟩
};
```

⟨Fid *other fields* 54d⟩≡ (54c) 55a ▷

```
Qid  qid;
```

⟨Fid *extra fields* 54e⟩+≡ (54c) ◁ 54b

```
bool busy;
```

⟨*function* newfid 54f⟩≡ (255)

```
Fid*
newfid(Filsys *fs, int fid)
{
    Fid *f, *ff, **fh;

    ff = nil; // free fid
    fh = &fs->fids[fid&(Nhash-1)];

    // lookup_hash(fid, fs->fids)
    for(f=*fh; f; f=f->next) {
        if(f->fid == fid)
            // found!
            return f;
        else if(ff==nil && !f->busy)
            ff = f;
    }
    // else
    if(ff){
        ff->fid = fid;
        return ff;
    }
    // else

    f = emalloc(sizeof(Fid));
    f->fid = fid;

    // insert_hash(f, fs->fids)
```

```
    f->next = *fh;
    *fh = f;

    return f;
}
```

⟨Fid *other fields* 55a⟩+≡ (54c) ◁54d 101a▷

```
Window *w;
```

3.4.3 Workers and jobs: Xfid

⟨Filsys *other fields* 55b⟩≡ (53i)

```
// chan<ref<Xfid>> (listener = filsysproc, sender = xfidallocthread)
Channel *cxfidalloc; /* chan(Xfid*) */
```

⟨*struct* Xfid 55c⟩≡ (237b)

```
struct Xfid
{
        // incoming parsed request
        Fcall req;
        // answer buffer
        byte *buf;

        // handler to worker thread
        // chan<void(*)(Xfid*)> (listener = xfidctl, senders = filsysxxx)
        Channel *c; /* chan(void(*)(Xfid*)) */

        Fid *f;

        Filsys *fs;

        ⟨Xfid flushing fields 222g⟩
        ⟨Xfid other fields 223b⟩

        // Extra
        Ref;
        ⟨Xfid extra fields 68f⟩
};
```

3.4.4 9P callbacks: fcall

⟨*global* fcall 55d⟩≡ (255)

```
Xfid*  (*fcall[Tmax])(Filsys*, Xfid*, Fid*) =
{
    [Tattach]  = filsysattach,

    [Twalk]    = filsyswalk,

    [Topen]    = filsysopen,
    [Tclunk]   = filsysclunk,
    [Tread]    = filsysread,
    [Twrite]   = filsyswrite,
    [Tstat]    = filsysstat,

    ⟨fcall other methods 67e⟩
};
```

Chapter 4

main()

⟨*function* threadmain 56a⟩≡ (243a)

```
void threadmain(int argc, char *argv[])
{
    ⟨main() locals 81j⟩

    ARGBEGIN{
    ⟨main() command line processing 215a⟩
    }ARGEND

    ⟨main() set some globals 81k⟩

    // Rio, a graphical application

    ⟨main() graphics initializations 56c⟩

    // Rio, a concurrent application

    ⟨main() communication channels creation 57f⟩
    ⟨main() threads creation 58a⟩

    // Rio, a filesystem server

    filsys = filsysinit(xfidinit());
    ⟨main() if filsys is nil 56b⟩
    else{
        ⟨main() error management after everything setup 230i⟩

        // blocks until get exit message on exitchan
        recv(exitchan, nil);
    }
    killprocs();
    threadexitsall(nil);
}
```

⟨main() *if filsys is nil* 56b⟩≡ (56a)

```
if(filsys == nil)
    fprint(STDERR, "rio: can't create file system server: %r\n");
```

4.1 Graphics initialization

⟨main() *graphics initializations* 56c⟩≡ (56a)

```
if(geninitdraw(nil, derror, nil, "rio", nil, Refnone) < 0){
    fprint(STDERR, "rio: can't open display: %r\n");
```

```
    exits("display open");
}
viewr = view->r;

iconinit(); // allocate background and red images

⟨main() mouse initialisation 57c⟩
⟨main() keyboard initialisation 57d⟩

desktop = allocscreen(view, background, false);
⟨main() sanity check desktop 57a⟩

draw(view, viewr, background, nil, ZP);
flushimage(display, true);
```

⟨main() *sanity check desktop* 57a⟩≡ (56c)

```
if(desktop == nil)
    error("can't allocate desktop");
```

⟨*function* iconinit 57b⟩≡ (247)

```
void
iconinit(void)
{
    background = allocimage(display, Rect(0,0,1,1), RGB24, true, 0x777777FF);
    red        = allocimage(display, Rect(0,0,1,1), RGB24, true, 0xDD0000FF);
}
```

4.2 Mouse initialization

⟨main() *mouse initialisation* 57c⟩≡ (56c)

```
mousectl = initmouse(nil, view);
if(mousectl == nil)
    error("can't find mouse");
mouse = mousectl;
```

4.3 Keyboard initialization

⟨main() *keyboard initialisation* 57d⟩≡ (56c)

```
keyboardctl = initkeyboard(nil);
if(keyboardctl == nil)
    error("can't find keyboard");
```

4.4 Channels creation

⟨*global* exitchan 57e⟩≡ (242)

```
// chan<unit> (listener = threadmain, sender = mousethread(Exit) | ?)
Channel *exitchan; /* chan(int) */
```

⟨main() *communication channels creation* 57f⟩≡ (56a) 89c ▷

```
exitchan     = chancreate(sizeof(int), 0);
```

4.5 Threads creation

⟨*constant* STACK 57g⟩≡ (237b)

```
#define STACK 8192
```

⟨main() *threads creation* 58a⟩≡ (56a) 89d ▷

```
threadcreate(keyboardthread, nil, STACK);
threadcreate(mousethread, nil, STACK);
```

4.6 Filesystem server initialization

4.6.1 filsysinit()

⟨*function* filsysinit 58b⟩≡ (254b)

```
Filsys*
filsysinit(Channel *cxfidalloc)
{
    int pid;
    Filsys *fs;
    ⟨filsysinit() other locals 182d⟩

    ⟨filsysinit() install dumper 228b⟩

    fs = emalloc(sizeof(Filsys));

    if(cexecpipe(&fs->cfd, &fs->sfd) < 0)
        goto Rescue;

    ⟨filsysinit() set clockfd 220c⟩
    ⟨filsysinit() set fs user 225e⟩
    pid = getpid();

    fs->cxfidalloc = cxfidalloc;

    ⟨filsysinit() wctl pipe, process, and thread creation 182c⟩

    proccreate(filsysproc, fs, 10000);

    ⟨filsysinit() srv pipe 181b⟩

    return fs;

Rescue:
    free(fs);
    return nil;
}
```

⟨*function* cexecpipe 58c⟩≡ (254b)

```
/*
 * Build pipe with OCEXEC set on second fd.
 * Can't put it on both because we want to post one in /srv.
 */
errorneg1
cexecpipe(fdt *p0, fdt *p1)
{
    /* pipe the hard way to get close on exec */
    if(bind("#|", "/mnt/temp", MREPL) < 0)
        return ERROR_NEG1;
    *p0 = open("/mnt/temp/data", ORDWR);
    *p1 = open("/mnt/temp/data1", ORDWR|OCEXEC);
    unmount(nil, "/mnt/temp");
    if(*p0<0 || *p1<0)
        return ERROR_NEG1;
```

```
    return OK_0;
}
```

4.6.2 Worker allocator: `xfidinit()`

⟨*global* `cxfidalloc` 59a⟩≡ (245a)

```
// chan<ref<Xfid>> (listener = filsysproc, sender = xfidallocthread)
static Channel *cxfidalloc; /* chan(Xfid*) */
```

⟨*global* `cxfidfree` 59b⟩≡ (245a)

```
// chan<ref<Xfid>> (listner = ??, sender = ??)
static Channel *cxfidfree; /* chan(Xfid*) */
```

⟨*function* `xfidinit` 59c⟩≡ (245a)

```
Channel*
xfidinit(void)
{
    cxfidalloc = chancreate(sizeof(Xfid*), 0);
    cxfidfree = chancreate(sizeof(Xfid*), 0);
    threadcreate(xfidallocthread, nil, STACK);
    return cxfidalloc;
}
```

Chapter 5

Procs and Threads

5.1 Keyboard thread

⟨*function* keyboardthread 60a⟩≡ (244a)

```
void
keyboardthread(void*)
{
    Rune buf[2][20];
    // points to buf[0] or buf[1]
    Rune *rp;
    int n, i;

    threadsetname("keyboardthread");

    n = 0;
    for(;;){
        rp = buf[n];
        n = 1-n;

        // Listen
        recv(keyboardctl->c, rp);

        for(i=1; i<nelem(buf[0])-1; i++)
            if(nbrecv(keyboardctl->c, rp+i) <= 0)
                break;
        rp[i] = L'\0';

        if(input != nil)
            // Dispatch, to current window thread!
            sendp(input->ck, rp);
    }
}
```

5.2 Mouse thread

⟨*enum* Mxxx 60b⟩≡ (243b)

```
enum {
    MMouse,
    ⟨Mxxx cases 190a⟩
    NALT
};
```

⟨mousethread() *locals* 61a⟩≡ (61b) 61d ▷

```
// map<enum<Mxxx>, Alt>
static Alt alts[NALT+1];
```

⟨*function* mousethread 61b⟩≡ (243b)

```
void
mousethread(void*)
{
    ⟨mousethread() locals 61a⟩

    threadsetname("mousethread");

    ⟨mousethread() alts setup 61c⟩
    alts[NALT].op = CHANEND;

    for(;;)
        // message loop
        switch(alt(alts)){
        ⟨mousethread() event loop cases 61f⟩
        }
}
```

⟨mousethread() *alts setup* 61c⟩≡ (61b) 190b ▷

```
// listen
alts[MMouse].c = mousectl->c;
alts[MMouse].v = &mousectl->Mouse;
alts[MMouse].op = CHANRCV;
```

⟨mousethread() *locals* 61d⟩+≡ (61b) ◁ 61a 61e ▷

```
Window *winput;
Point xy; // logical coord
```

⟨mousethread() *locals* 61e⟩+≡ (61b) ◁ 61d 62c ▷

```
bool sending = false;
```

⟨mousethread() *event loop cases* 61f⟩≡ (61b) 190c ▷

```
case MMouse:
    ⟨mousethread() if wkeyboard and button 6 216c⟩
Again:
    winput = input;
    ⟨mousethread() if wkeyboard and ptinrect 216d⟩

    if(winput != nil && winput->i != nil){
        /* convert to logical coordinates */
        xy.x = mouse->xy.x + (winput->i->r.min.x - winput->screenr.min.x);
        xy.y = mouse->xy.y + (winput->i->r.min.y - winput->screenr.min.y);

        ⟨mousethread() goto Sending if scroll buttons 169b⟩

        inside = ptinrect(mouse->xy, insetrect(winput->screenr, Selborder));

        ⟨mousethread() set scrolling 169a⟩
        ⟨mousethread() set moving to true for some conditions 77d⟩
        else
           ⟨mousethread() set sending to true for some conditions 62b⟩
    }else
        sending = false;

    ⟨mousethread() if sending 62d⟩
    ⟨mousethread() if not sending 62g⟩

⟨mousethread() Drain label 63d⟩
```

⟨*constant* Selborder 62a⟩≡ (237a)

```
Selborder  = 4, /* border of selected window */
```

5.2.1 Application mouse events

⟨mousethread() *set sending to true for some conditions* 62b⟩≡ (61f)

```
if(inside &&
   ((mouse->buttons&1) || winput->mouseopen || scrolling))
     sending = true;
```

⟨mousethread() *locals* 62c⟩+≡ (61b) ◁61e 62f▷

```
Mouse tmp;
```

⟨mousethread() *if sending* 62d⟩≡ (61f)

```
if(sending){
Sending:
    ⟨mousethread() when sending mouse message to window, set the cursor 62e⟩

    tmp = mousectl->Mouse;
    tmp.xy = xy; // logical coordinates

    // Dispatch, to current window thread!
    send(winput->mc.c, &tmp);
    continue;
}
```

⟨mousethread() *when sending mouse message to window, set the cursor* 62e⟩≡ (62d)

```
if(mouse->buttons == 0){
    // cornercursor will call wsetcursor if cursor not on the border
    cornercursor(winput, mouse->xy, false);
    sending = false;
}else
    wsetcursor(winput, false);
```

5.2.2 Windowing system mouse events

⟨mousethread() *locals* 62f⟩+≡ (61b) ◁62c 77c▷

```
Window *w;
```

⟨mousethread() *if not sending* 62g⟩≡ (61f)

```
w = wpointto(mouse->xy);

⟨mousethread() when not sending, set cursor part1 63a⟩
⟨mousethread() if moving and buttons 77f⟩
⟨mousethread() when not sending, set cursor part2 63b⟩

⟨mousethread() when not sending, if buttons 63c⟩
moving = false;
break;
```

⟨*function* wpointto 62h⟩≡ (249)

```
Window*
wpointto(Point pt)
{
    int i;
    Window *v, *w;

    w = nil;
```

```
    for(i=0; i<nwindow; i++){
        v = windows[i];
        if(ptinrect(pt, v->screenr))
         if(!v->deleted)
          if(w==nil || v->topped > w->topped)
            w = v;
    }
    return w;
}
```

⟨mousethread() *when not sending, set cursor part1* 63a⟩≡ (62g)

```
/* change cursor if over anyone's border */
if(w != nil)
    cornercursor(w, mouse->xy, false);
else
    riosetcursor(nil, false);
```

⟨mousethread() *when not sending, set cursor part2* 63b⟩≡ (62g)

```
if(w != nil)
    cornercursor(w, mouse->xy, false);
```

⟨mousethread() *when not sending, if buttons* 63c⟩≡ (62g)

```
/* we're not sending the event, but if button is down maybe we should */
if(mouse->buttons){
    /* w->topped will be zero or less if window has been bottomed */
    if(w==nil || (w==winput && w->topped > 0)){
        if(mouse->buttons & 1){
            ;
        }else if(mouse->buttons & 2){
            if(winput && !winput->mouseopen)
                ⟨mousethread() middle click under certain conditions 167a⟩
        }else if(mouse->buttons & 4)
                ⟨mousethread() right click under certain conditions 76a⟩
    }else{
        /* if button 1 event in the window, top the window and wait for button up. */
        /* otherwise, top the window and pass the event on */
        ⟨mousethread() click on unfocused window, set w 87a⟩
        if(w && (mouse->buttons!=1 || winborder(w, mouse->xy)))
            // input changed
            goto Again;

        goto Drain;
    }
}
```

⟨mousethread() *Drain label* 63d⟩≡ (61f)

```
Drain:
    do {
        readmouse(mousectl);
    } while(mousectl->buttons);
    moving = false;
    goto Again; /* recalculate mouse position, cursor */
```

5.3 Window threads

⟨*enum* Wxxx 63e⟩≡ (244b)

```
enum {
    WKey,
```

```
    WMouse,
    WCtl,
    ⟨Wxxx cases 121d⟩

    NWALT
};
```

⟨*function* winctl 64a⟩≡ (244b)

```
void
winctl(void *arg)
{
    Window *w = arg;
    // map<enum<Wxxx>, Alt>
    Alt alts[NWALT+1];
    int event;
    ⟨winctl() other locals 65a⟩

    snprint(buf, sizeof buf, "winctl-id%d", w->id);
    threadsetname(buf);

    ⟨winctl() channels creation 121f⟩

    ⟨winctl() alts setup 65b⟩
    alts[NWALT].op = CHANEND;

    for(;;){
        ⟨winctl() alts adjustments 121i⟩

        // to isolate messaging bug
        //alts[WKey].op = CHANNOP;
        //alts[WMouse].op = CHANNOP;
        ////alts[WCtl].op = CHANNOP;
        //alts[WMouseread].op = CHANNOP;
        //alts[WCread].op = CHANNOP;
        //alts[WCwrite].op = CHANNOP;
        //alts[WWread].op = CHANNOP;

        // event loop
        event = alt(alts);
        if(DEBUG) fprint(STDERR, "winctl: win=%d, event=%d\n", w->id, event);
        if(event == -1) {
          fprint(STDERR, "winctl: interrupted %r");
          exits("interrupted");
        }
        switch(event){
        ⟨winctl() event loop cases 65c⟩
        }

        if(!w->deleted)
            flushimage(display, true);
    }
}
```

5.3.1 Keyboard events listening

⟨Window *keyboard fields* 64b⟩≡ (51)

```
// chan<Rune, 20> (listener = winctl, sender = keyboardthread)
Channel *ck;  /* chan(Rune[10]) */
```

⟨winctl() *other locals* 65a⟩≡ (64a) 65i ▷
```
Rune *kbdr;
```

⟨winctl() *alts setup* 65b⟩≡ (64a) 65h ▷
```
alts[WKey].c = w->ck;
alts[WKey].v = &kbdr;
alts[WKey].op = CHANRCV;
```

⟨winctl() *event loop cases* 65c⟩≡ (64a) 65j ▷
```
case WKey:
    for(i=0; kbdr[i] != L'\0'; i++)
        wkeyctl(w, kbdr[i]);
    break;
```

⟨*function* wkeyctl 65d⟩≡ (250b)
```
void
wkeyctl(Window *w, Rune r)
{
    ⟨wkeyctl() locals 159c⟩

    ⟨wkeyctl() sanity check rune 65e⟩
    ⟨wkeyctl() return if window was deleted 65f⟩

    /* navigation keys work only when mouse is not open */
    ⟨wkeyctl() when mouse not opened and navigation keys 160d⟩

    ⟨wkeyctl() if rawing 122d⟩
    ⟨wkeyctl() if holding 219c⟩

    ⟨wkeyctl() when not rawing 159b⟩
}
```

⟨wkeyctl() *sanity check rune* 65e⟩≡ (65d)
```
if(r == 0)
    return;
```

⟨wkeyctl() *return if window was deleted* 65f⟩≡ (65d)
```
if(w->deleted)
    return;
```

5.3.2 Mouse events listening

⟨Window *mouse fields* 65g⟩≡ (51) 75c ▷
```
// mc.c = chan<Mouse> (listener = winctl, sender = mousethread)
Mousectl mc;
```

⟨winctl() *alts setup* 65h⟩+≡ (64a) ◁ 65b 66c ▷
```
alts[WMouse].c = w->mc.c;
alts[WMouse].v = &w->mc.Mouse;
alts[WMouse].op = CHANRCV;
```

⟨winctl() *other locals* 65i⟩+≡ (64a) ◁ 65a 66b ▷
```
char buf[128]; // /dev/mouse interface
```

⟨winctl() *event loop cases* 65j⟩+≡ (64a) ◁ 65c 66d ▷
```
case WMouse:
    ⟨winctl() WMouse case if mouseopen 121c⟩
    else
    ⟨winctl() WMouse case if not mouseopen 167g⟩
    break;
```

5.3.3 Control events listening

⟨Window *control fields* 66a⟩≡ (51)
```
// chan<Wctlmesg, 20> (listener = winctl, sender = mousethread | ...)
Channel *cctl;  /* chan(Wctlmesg)[20] */
```

⟨winctl() *other locals* 66b⟩+≡ (64a) ◁65i 121b▷
```
Wctlmesg wcm;
```

⟨winctl() *alts setup* 66c⟩+≡ (64a) ◁65h 121h▷
```
alts[WCtl].c = w->cctl;
alts[WCtl].v = &wcm;
alts[WCtl].op = CHANRCV;
```

⟨winctl() *event loop cases* 66d⟩+≡ (64a) ◁65j 122b▷
```
case WCtl:
    if(wctlmesg(w, wcm.type, wcm.r, wcm.image) == Exited){
        ⟨winctl() Wctl case, free channels if wctlmesg is Excited 121g⟩
        threadexits(nil);
    }
    continue;
```

5.4 Filesystem server proc

⟨*global* messagesize 66e⟩≡ (254a)
```
int messagesize = 8192+IOHDRSZ; /* good start */
```

5.4.1 filsysproc()

⟨*function* filsysproc 66f⟩≡ (254b)
```
static
void
filsysproc(void *arg)
{
    Filsys *fs = arg;
    int n;
    byte *buf;
    Xfid *x = nil;
    Fid *f;
    ⟨filsysproc() other locals 67c⟩

    threadsetname("FILSYSPROC");

    for(;;){
        buf = emalloc(messagesize+UTFmax); /* UTFmax for appending partial rune in xfidwrite */

        n = read9pmsg(fs->sfd, buf, messagesize);
        ⟨filsysproc() sanity check n 67a⟩
        if(x == nil){
            send(fs->cxfidalloc, nil);
            recv(fs->cxfidalloc, &x);
            x->fs = fs;
        }
        x->buf = buf;

        if(convM2S(buf, n, &x->req) != n)
            error("convert error in convM2S");
        ⟨filsysproc() dump Fcall if debug 228c⟩
```

```
        ⟨filsysproc() sanity check x type 67d⟩
        else{
            ⟨filsysproc() if x type is Tversion or Tauth 67b⟩
            else
                f = newfid(fs, x->req.fid);
            x->f = f;

            // Dispatch
            x  = (*fcall[x->req.type])(fs, x, f);
        }
        ⟨filsysproc() end of loop 68a⟩
    }
}
```

⟨filsysproc() *sanity check n* 67a⟩≡ (66f)

```
if(n <= 0){
    yield(); /* if threadexitsall'ing, will not return */
    fprint(STDERR, "rio: %d: read9pmsg: %d %r\n", getpid(), n);
    errorshouldabort = false;
    error("eof or i/o error on server channel");
}
```

⟨filsysproc() *if x type is Tversion or Tauth* 67b⟩≡ (66f)

```
if(x->req.type==Tversion || x->req.type==Tauth)
    f = nil;
```

⟨filsysproc() *other locals* 67c⟩≡ (66f)

```
Fcall fc;
```

⟨filsysproc() *sanity check x type* 67d⟩≡ (66f)

```
if(fcall[x->req.type] == nil)
    x = filsysrespond(fs, x, &fc, Ebadfcall);
```

5.4.2 filsysversion()

⟨fcall *other methods* 67e⟩≡ (55d) 111a ▷

```
[Tversion] = filsysversion,
```

⟨*function* filsysversion 67f⟩≡ (255)

```
static
Xfid*
filsysversion(Filsys *fs, Xfid *x, Fid*)
{
    Fcall fc;

    ⟨filsysversion() sanity checks 67h⟩
    // else
    messagesize = x->req.msize;
    fc.msize = messagesize;
    fc.version = "9P2000";
    return filsysrespond(fs, x, &fc, nil);
}
```

⟨*global* firstmessage 67g⟩≡ (255)

```
bool firstmessage = true;
```

⟨filsysversion() *sanity checks* 67h⟩≡ (67f) 68b ▷

```
if(!firstmessage)
    return filsysrespond(x->fs, x, &fc, "version request not first message");
```

⟨filsysproc() *end of loop* 68a⟩≡ (66f)

```
firstmessage = false;
```

⟨filsysversion() *sanity checks* 68b⟩+≡ (67f) ◁67h

```
if(x->req.msize < 256)
    return filsysrespond(x->fs, x, &fc, "version: message size too small");
if(strncmp(x->req.version, "9P2000", 6) != 0)
    return filsysrespond(x->fs, x, &fc, "unrecognized 9P version");
```

5.5 Xfid allocator thread

⟨*enum* Xxxx 68c⟩≡ (245a)

```
enum {
    Alloc,
    Free,

    N
};
```

⟨*global* xfid 68d⟩≡ (245a)

```
// list<ref_own<Xfid>> (next = Xfid.next)
static Xfid *xfid;
```

⟨*global* xfidfree 68e⟩≡ (245a)

```
// list<ref_own<Xfid>> (next = Xfid.free)
static Xfid *xfidfree;
```

⟨Xfid *extra fields* 68f⟩≡ (55c)

```
Xfid *next;
Xfid *free;
```

⟨*function* xfidallocthread 68g⟩≡ (245a)

```
void
xfidallocthread(void*)
{
    Xfid *x;
    static Alt alts[N+1];

    alts[Alloc].c = cxfidalloc;
    alts[Alloc].v = nil;
    alts[Alloc].op = CHANRCV;
    alts[Free].c = cxfidfree;
    alts[Free].v = &x;
    alts[Free].op = CHANRCV;
    alts[N].op = CHANEND;

    for(;;){
        // event loop
        switch(alt(alts)){
        case Alloc:
            x = xfidfree;
            if(x)
                xfidfree = x->free;
            else{
                x = emalloc(sizeof(Xfid));
                x->c = chancreate(sizeof(void(*)(Xfid*)), 0);
                ⟨xfidallocthread() create flushc channel 223a⟩
```

```
                // insert_list(x, xfid)
                x->next = xfid;
                xfid = x;

                // new Xfid threads!
                threadcreate(xfidctl, x, 16384);
            }
            ⟨xfidallocthread() sanity check x when Alloc 69a⟩
            incref(x);

            sendp(cxfidalloc, x);
            break;

        case Free:
            ⟨xfidallocthread() sanity check x when Free 69b⟩
            // insert_list(x, xfidfree)
            x->free = xfidfree;
            xfidfree = x;
            break;
        }
    }
}
```

⟨xfidallocthread() *sanity check x when Alloc* 69a⟩≡ (68g)

```
if(x->ref != 0){
    fprint(STDERR, "%p incref %ld\n", x, x->ref);
    error("incref");
}
if(x->flushtag != -1)
    error("flushtag in allocate");
```

⟨xfidallocthread() *sanity check x when Free* 69b⟩≡ (68g)

```
if(x->ref != 0){
    fprint(STDERR, "%p decref %ld\n", x, x->ref);
    error("decref");
}
if(x->flushtag != -1)
    error("flushtag in free");
```

5.6 Xfid threads

⟨*function* xfidctl 69c⟩≡ (245a)

```
void
xfidctl(void *arg)
{
    Xfid *x = arg;
    void (*f)(Xfid*);
    char buf[64];

    snprint(buf, sizeof buf, "xfid.%p", x);

    threadsetname(buf);

    for(;;){
        f = recvp(x->c);

        // Executing a xfidxxx()
        (*f)(x);
```

```
        if(decref(x) == 0)
            sendp(cxfidfree, x);
    }
}
```

Chapter 6

Cursors

6.1 Cursor graphics

6.1.1 Classic cursors

⟨*global crosscursor (windows/rio/data.c)* 71a⟩≡ (247)

```
Cursor crosscursor = {
    {-7, -7},
    {0x03, 0xC0, 0x03, 0xC0, 0x03, 0xC0, 0x03, 0xC0,
     0x03, 0xC0, 0x03, 0xC0, 0xFF, 0xFF, 0xFF, 0xFF,
     0xFF, 0xFF, 0xFF, 0xFF, 0x03, 0xC0, 0x03, 0xC0,
     0x03, 0xC0, 0x03, 0xC0, 0x03, 0xC0, 0x03, 0xC0, },
    {0x00, 0x00, 0x01, 0x80, 0x01, 0x80, 0x01, 0x80,
     0x01, 0x80, 0x01, 0x80, 0x01, 0x80, 0x7F, 0xFE,
     0x7F, 0xFE, 0x01, 0x80, 0x01, 0x80, 0x01, 0x80,
     0x01, 0x80, 0x01, 0x80, 0x01, 0x80, 0x00, 0x00, }
};
```

⟨*global boxcursor (windows/rio/data.c)* 71b⟩≡ (247)

```
Cursor boxcursor = {
    {-7, -7},
    {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
     0xFF, 0xFF, 0xF8, 0x1F, 0xF8, 0x1F, 0xF8, 0x1F,
     0xF8, 0x1F, 0xF8, 0x1F, 0xF8, 0x1F, 0xFF, 0xFF,
     0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, },
    {0x00, 0x00, 0x7F, 0xFE, 0x7F, 0xFE, 0x7F, 0xFE,
     0x70, 0x0E, 0x70, 0x0E, 0x70, 0x0E, 0x70, 0x0E,
     0x70, 0x0E, 0x70, 0x0E, 0x70, 0x0E, 0x70, 0x0E,
     0x7F, 0xFE, 0x7F, 0xFE, 0x7F, 0xFE, 0x00, 0x00, }
};
```

⟨*global sightcursor (windows/rio/data.c)* 71c⟩≡ (247)

```
Cursor sightcursor = {
    {-7, -7},
    {0x1F, 0xF8, 0x3F, 0xFC, 0x7F, 0xFE, 0xFB, 0xDF,
     0xF3, 0xCF, 0xE3, 0xC7, 0xFF, 0xFF, 0xFF, 0xFF,
     0xFF, 0xFF, 0xFF, 0xFF, 0xE3, 0xC7, 0xF3, 0xCF,
     0x7B, 0xDF, 0x7F, 0xFE, 0x3F, 0xFC, 0x1F, 0xF8, },
    {0x00, 0x00, 0x0F, 0xF0, 0x31, 0x8C, 0x21, 0x84,
     0x41, 0x82, 0x41, 0x82, 0x41, 0x82, 0x7F, 0xFE,
     0x7F, 0xFE, 0x41, 0x82, 0x41, 0x82, 0x41, 0x82,
     0x21, 0x84, 0x31, 0x8C, 0x0F, 0xF0, 0x00, 0x00, }
};
```

⟨*global whitearrow (windows/rio/data.c)* 72a⟩≡ (247)

```
Cursor whitearrow = {
    {0, 0},
    {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFE, 0xFF, 0xFC,
     0xFF, 0xF0, 0xFF, 0xF0, 0xFF, 0xF8, 0xFF, 0xFC,
     0xFF, 0xFE, 0xFF, 0xFF, 0xFF, 0xFE, 0xFF, 0xFC,
     0xF3, 0xF8, 0xF1, 0xF0, 0xE0, 0xE0, 0xC0, 0x40, },
    {0xFF, 0xFF, 0xFF, 0xFF, 0xC0, 0x06, 0xC0, 0x1C,
     0xC0, 0x30, 0xC0, 0x30, 0xC0, 0x38, 0xC0, 0x1C,
     0xC0, 0x0E, 0xC0, 0x07, 0xCE, 0x0E, 0xDF, 0x1C,
     0xD3, 0xB8, 0xF1, 0xF0, 0xE0, 0xE0, 0xC0, 0x40, }
};
```

⟨*global query (windows/rio/data.c)* 72b⟩≡ (247)

```
Cursor query = {
    {-7,-7},
    {0x0f, 0xf0, 0x1f, 0xf8, 0x3f, 0xfc, 0x7f, 0xfe,
     0x7c, 0x7e, 0x78, 0x7e, 0x00, 0xfc, 0x01, 0xf8,
     0x03, 0xf0, 0x07, 0xe0, 0x07, 0xc0, 0x07, 0xc0,
     0x07, 0xc0, 0x07, 0xc0, 0x07, 0xc0, 0x07, 0xc0, },
    {0x00, 0x00, 0x0f, 0xf0, 0x1f, 0xf8, 0x3c, 0x3c,
     0x38, 0x1c, 0x00, 0x3c, 0x00, 0x78, 0x00, 0xf0,
     0x01, 0xe0, 0x03, 0xc0, 0x03, 0x80, 0x03, 0x80,
     0x00, 0x00, 0x03, 0x80, 0x03, 0x80, 0x00, 0x00, }
};
```

6.1.2 Border and corner cursors

⟨*global corners (windows/rio/data.c)* 72c⟩≡ (247)

```
Cursor *corners[9] = {
    &tl,    &t,   &tr,
    &l,    nil,   &r,
    &bl,    &b,   &br,
};
```

⟨*global* `tl` 72d⟩≡ (247)

```
Cursor tl = {
    {-4, -4},
    {0xfe, 0x00, 0x82, 0x00, 0x8c, 0x00, 0x87, 0xff,
     0xa0, 0x01, 0xb0, 0x01, 0xd0, 0x01, 0x11, 0xff,
     0x11, 0x00, 0x11, 0x00, 0x11, 0x00, 0x11, 0x00,
     0x11, 0x00, 0x11, 0x00, 0x11, 0x00, 0x1f, 0x00, },
    {0x00, 0x00, 0x7c, 0x00, 0x70, 0x00, 0x78, 0x00,
     0x5f, 0xfe, 0x4f, 0xfe, 0x0f, 0xfe, 0x0e, 0x00,
     0x0e, 0x00, 0x0e, 0x00, 0x0e, 0x00, 0x0e, 0x00,
     0x0e, 0x00, 0x0e, 0x00, 0x0e, 0x00, 0x00, 0x00, }
};
```

⟨*global* `t` 72e⟩≡ (247)

```
static Cursor t = {
    {-7, -8},
    {0x00, 0x00, 0x00, 0x00, 0x03, 0x80, 0x06, 0xc0,
     0x1c, 0x70, 0x10, 0x10, 0x0c, 0x60, 0xfc, 0x7f,
     0x80, 0x01, 0x80, 0x01, 0x80, 0x01, 0xff, 0xff,
     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, },
    {0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0x00,
     0x03, 0x80, 0x0f, 0xe0, 0x03, 0x80, 0x03, 0x80,
     0x7f, 0xfe, 0x7f, 0xfe, 0x7f, 0xfe, 0x00, 0x00,
     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, }
};
```

⟨*global* tr 73a⟩≡ (247)

```
Cursor tr = {
    {-11, -4},
    {0x00, 0x7f, 0x00, 0x41, 0x00, 0x31, 0xff, 0xe1,
     0x80, 0x05, 0x80, 0x0d, 0x80, 0x0b, 0xff, 0x88,
     0x00, 0x88, 0x0, 0x88, 0x00, 0x88, 0x00, 0x88,
     0x00, 0x88, 0x00, 0x88, 0x00, 0x88, 0x00, 0xf8, },
    {0x00, 0x00, 0x00, 0x3e, 0x00, 0x0e, 0x00, 0x1e,
     0x7f, 0xfa, 0x7f, 0xf2, 0x7f, 0xf0, 0x00, 0x70,
     0x00, 0x70, 0x00, 0x70, 0x00, 0x70, 0x00, 0x70,
     0x00, 0x70, 0x00, 0x70, 0x00, 0x70, 0x00, 0x00, }
};
```

⟨*global* r 73b⟩≡ (247)

```
Cursor r = {
    {-8, -7},
    {0x07, 0xc0, 0x04, 0x40, 0x04, 0x40, 0x04, 0x58,
     0x04, 0x68, 0x04, 0x6c, 0x04, 0x06, 0x04, 0x02,
     0x04, 0x06, 0x04, 0x6c, 0x04, 0x68, 0x04, 0x58,
     0x04, 0x40, 0x04, 0x40, 0x04, 0x40, 0x07, 0xc0, },
    {0x00, 0x00, 0x03, 0x80, 0x03, 0x80, 0x03, 0x80,
     0x03, 0x90, 0x03, 0x90, 0x03, 0xf8, 0x03, 0xfc,
     0x03, 0xf8, 0x03, 0x90, 0x03, 0x90, 0x03, 0x80,
     0x03, 0x80, 0x03, 0x80, 0x03, 0x80, 0x00, 0x00, }
};
```

⟨*global* br 73c⟩≡ (247)

```
Cursor br = {
    {-11, -11},
    {0x00, 0xf8, 0x00, 0x88, 0x00, 0x88, 0x00, 0x88,
     0x00, 0x88, 0x00, 0x88, 0x00, 0x88, 0x00, 0x88,
     0xff, 0x88, 0x80, 0x0b, 0x80, 0x0d, 0x80, 0x05,
     0xff, 0xe1, 0x00, 0x31, 0x00, 0x41, 0x00, 0x7f, },
    {0x00, 0x00, 0x00, 0x70, 0x00, 0x70, 0x00, 0x70,
     0x0, 0x70, 0x00, 0x70, 0x00, 0x70, 0x00, 0x70,
     0x00, 0x70, 0x7f, 0xf0, 0x7f, 0xf2, 0x7f, 0xfa,
     0x00, 0x1e, 0x00, 0x0e, 0x00, 0x3e, 0x00, 0x00, }
};
```

⟨*global* b 73d⟩≡ (247)

```
Cursor b = {
    {-7, -7},
    {0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
     0xff, 0xff, 0x80, 0x01, 0x80, 0x01, 0x80, 0x01,
     0xfc, 0x7f, 0x0c, 0x60, 0x10, 0x10, 0x1c, 0x70,
     0x06, 0xc0, 0x03, 0x80, 0x00, 0x00, 0x00, 0x00, },
    {0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
     0x00, 0x00, 0x7f, 0xfe, 0x7f, 0xfe, 0x7f, 0xfe,
     0x03, 0x80, 0x03, 0x80, 0x0f, 0xe0, 0x03, 0x80,
     0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, }
};
```

⟨*global* bl 73e⟩≡ (247)

```
Cursor bl = {
    {-4, -11},
    {0x1f, 0x00, 0x11, 0x00, 0x11, 0x00, 0x11, 0x00,
     0x11, 0x00, 0x11, 0x00, 0x11, 0x00, 0x11, 0x00,
     0x11, 0xff, 0xd0, 0x01, 0xb0, 0x01, 0xa0, 0x01,
     0x87, 0xff, 0x8c, 0x00, 0x82, 0x00, 0xfe, 0x00, },
    {0x00, 0x00, 0x0e, 0x00, 0x0e, 0x00, 0x0e, 0x00,
```

```
     0x0e, 0x00, 0x0e, 0x00, 0x0e, 0x00, 0x0e, 0x00,
     0x0e, 0x00, 0x0f, 0xfe, 0x4f, 0xfe, 0x5f, 0xfe,
     0x78, 0x00, 0x70, 0x00, 0x7c, 0x00, 0x00, 0x0, }
};
```

⟨*global* l 74a⟩≡ (247)

```
Cursor l = {
    {-7, -7},
    {0x03, 0xe0, 0x02, 0x20, 0x02, 0x20, 0x1a, 0x20,
     0x16, 0x20, 0x36, 0x20, 0x60, 0x20, 0x40, 0x20,
     0x60, 0x20, 0x36, 0x20, 0x16, 0x20, 0x1a, 0x20,
     0x02, 0x20, 0x02, 0x20, 0x02, 0x20, 0x03, 0xe0, },
    {0x00, 0x00, 0x01, 0xc0, 0x01, 0xc0, 0x01, 0xc0,
     0x09, 0xc0, 0x09, 0xc0, 0x1f, 0xc0, 0x3f, 0xc0,
     0x1f, 0xc0, 0x09, 0xc0, 0x09, 0xc0, 0x01, 0xc0,
     0x01, 0xc0, 0x01, 0xc0, 0x01, 0xc0, 0x00, 0x00, }
};
```

6.2 Setting the cursor

6.2.1 riosetcursor()

⟨*global* lastcursor 74b⟩≡ (248)

```
Cursor *lastcursor;
```

⟨*function* riosetcursor 74c⟩≡ (248)

```
void
riosetcursor(Cursor *p, bool force)
{
    if(!force && p==lastcursor)
        return;
    setcursor(mousectl, p);
    lastcursor = p;
}
```

6.2.2 cornercursor()

⟨*function* cornercursor 74d⟩≡ (246)

```
void
cornercursor(Window *w, Point p, bool force)
{
    if(w != nil && winborder(w, p))
        riosetcursor(corners[whichcorner(w, p)], force);
    else
        wsetcursor(w, force);
}
```

⟨*function* winborder 74e⟩≡ (249)

```
bool
winborder(Window *w, Point xy)
{
    return ptinrect(xy, w->screenr) &&
           !ptinrect(xy, insetrect(w->screenr, Selborder));
}
```

⟨*function* whichcorner 75a⟩≡ (246)

```
int
whichcorner(Window *w, Point p)
{
    int i, j;

    i = portion(p.x, w->screenr.min.x, w->screenr.max.x);
    j = portion(p.y, w->screenr.min.y, w->screenr.max.y);
    return 3*j + i;
}
```

⟨*function* portion 75b⟩≡ (246)

```
int
portion(int x, int lo, int hi)
{
    x -= lo;
    hi -= lo;

    if(x < 20)
        return 0; // top
    if(x > hi-20)
        return 2; // below
    return 1; // middle
}
```

6.2.3 wsetcursor()

⟨Window *mouse fields* 75c⟩+≡ (51) ◁65g 120d▷

```
Cursor  cursor;
// option<ref<Cursor>> (to Window.cursor when not None)
Cursor  *cursorp;
```

⟨*function* wsetcursor 75d⟩≡ (249)

```
void
wsetcursor(Window *w, bool force)
{
    Cursor *p;

    if(w==nil || w->i==nil || Dx(w->screenr)<=0)
        p = nil;
    else if(wpointto(mouse->xy) == w){
        p = w->cursorp;
        ⟨wsetcursor() if holding 218c⟩
    }else
        p = nil;

    if(!menuing)
        riosetcursor(p, force);
}
```

Chapter 7

Window Manager

7.1 Overview

7.2 Right-click system menu

⟨`mousethread()` *right click under certain conditions* 76a⟩≡ (63c)

```
button3menu();
```

⟨*function* `button3menu` 76b⟩≡ (246)

```
void
button3menu(void)
{
    int i;

    ⟨button3menu() menu3str adjustments with hidden windows 98c⟩

    sweeping = true;
    switch(i = menuhit(3, mousectl, &menu3, desktop)){
    ⟨button3menu() cases 77b⟩
    case -1:
        break;
    }
    sweeping = false;
}
```

⟨*global* `sweeping` 76c⟩≡ (242)

```
bool sweeping;
```

⟨*global* `menu3` 76d⟩≡ (246)

```
Menu menu3 = { .item = menu3str };
```

⟨*global* `menu3str` 76e⟩≡ (246)

```
char*  menu3str[100] = {
 [New]  "New",
 [Reshape] "Resize",
 [Move]  "Move",
 [Delete] "Delete",
 [Hide]  "Hide",
 [Exit]  "Exit",
 nil
};
```

⟨*enum* _anon_ (windows/rio/rio.c) 77a⟩≡ (246)

```
enum RightMenuCommand
{
    New,

    Reshape,
    Move,
    Delete,
    Hide,

    Exit,

    Hidden,
};
```

⟨button3menu() *cases* 77b⟩≡ (76b) 79a ▷

```
case Exit:
    send(exitchan, nil);
    break;
```

7.3 Window borders click

⟨mousethread() *locals* 77c⟩+≡ (61b) ◁62f 77e ▷

```
bool moving = false;
```

⟨mousethread() *set moving to true for some conditions* 77d⟩≡ (61f)

```
/* topped will be zero or less if window has been bottomed */
if(!sending && !scrolling
   && winborder(winput, mouse->xy) && winput->topped > 0){
    moving = true;
}
```

⟨mousethread() *locals* 77e⟩+≡ (61b) ◁77c 168f ▷

```
bool inside, band;
Window *oin;
Image *i;
Rectangle r;
```

⟨mousethread() *if moving and buttons* 77f⟩≡ (62g)

```
if(moving && (mouse->buttons&7)){
    oin = winput;
    band = mouse->buttons & 3; // left or middle click

    sweeping = true;
    if(band)
        i = bandsize(winput);
    else
        i = drag(winput, &r);
    sweeping = false;

    if(i != nil){
        if(winput == oin){
            if(band)
                wsendctlmesg(winput, Reshaped, i->r, i);
            else
                wsendctlmesg(winput, Moved, r, i);
            cornercursor(winput, mouse->xy, true);
        }else
```

```
            freeimage(i);
        }
    }
```

7.4 Wctlmesg

⟨*enum* wctlmesgkind 78a⟩≡ (237b)

```
enum ControlMessage /* control messages */
{
    Reshaped, // Resized, Hide/Expose
    Moved,
    ⟨Wctlmesgkind cases 88c⟩
};
```

⟨*struct* Wctlmesg 78b⟩≡ (237b)

```
struct Wctlmesg
{
    // enum<Wctlmesgkind>
    int  type;

    Rectangle r;
    Image *image;
};
```

⟨*function* wsendctlmesg 78c⟩≡ (249)

```
void
wsendctlmesg(Window *w, int type, Rectangle r, Image *image)
{
    Wctlmesg wcm;

    wcm.type = type;
    wcm.r = r;
    wcm.image = image;

    if(DEBUG) fprint(STDERR, "wsendctlmesg: win=%d, type=%d\n", w->id, type);
    send(w->cctl, &wcm);
}
```

⟨*function* wctlmesg 78d⟩≡ (244b)

```
int
wctlmesg(Window *w, int m, Rectangle r, Image *i)
{
    char buf[64];

    if(DEBUG) fprint(STDERR, "wctlmesg: win=%d, type=%d\n", w->id, m);

    switch(m){
    ⟨wctlmesg() cases 88e⟩
    default:
        error("unknown control message");
        break;
    }
    return m;
}
```

7.5 Window creation

⟨button3menu() *cases* 79a⟩+≡ (76b) ◁77b 88a▷

```
case New:
    new(sweep(), false, scrolling, 0, nil, "/bin/rc", nil);
    break;
```

7.5.1 Window thread creation: new()

⟨*function* new 79b⟩≡ (246)

```
Window*
new(Image *i, bool hideit, bool scrollit, int pid, char *dir, char *cmd, char **argv)
{
    Channel *cm, *ck, *cctl;
    Channel *cpid;
    Mousectl *mc;
    Window *w;
    ⟨new() other locals 82a⟩

    ⟨new() sanity check i 80c⟩

    ⟨new() channels creation 80a⟩
    cpid = chancreate(sizeof(int), 0);
    ⟨new() sanity check channels 80d⟩

    ⟨new() mc allocation 80b⟩

    // create Window data structure
    w = wmk(i, mc, ck, cctl, scrollit);
    free(mc); /* wmk copies *mc */

    // growing array
    windows = erealloc(windows, ++nwindow * sizeof(Window*));
    windows[nwindow-1] = w;
    ⟨new() if hideit 98a⟩

    if(DEBUG) fprint(STDERR, "new: creating winctl thread for win=%d\n", w->id);

    // create a new thread! for this new window!
    threadcreate(winctl, w, 8192);

    if(!hideit)
        wcurrent(w);

    flushimage(display, true);

    // create a new process
    ⟨new() if pid == 0, create winshell process and set pid 82b⟩
    ⟨new() sanity check pid received from winshell 82c⟩
    wsetpid(w, pid, true);

    // create a new layer
    wsetname(w);

    if(dir)
        w->dir = estrdup(dir);

    chanfree(cpid);
    return w;
```

```
}
```

⟨new() *channels creation* 80a⟩≡ (79b)

```
cm = chancreate(sizeof(Mouse), 0);
ck = chancreate(sizeof(Rune*), 0);
cctl = chancreate(sizeof(Wctlmesg), 4);
```

⟨new() *mc allocation* 80b⟩≡ (79b)

```
mc = emalloc(sizeof(Mousectl));
*mc = *mousectl;
mc->image = i;
mc->c = cm;
```

⟨new() *sanity check i* 80c⟩≡ (79b)

```
if(i == nil)
    return nil;
```

⟨new() *sanity check channels* 80d⟩≡ (79b)

```
if(cm==nil || ck==nil || cctl==nil)
    error("new: channel alloc failed");
```

7.5.2 Window allocation: wmk()

⟨*function* wmk 80e⟩≡ (249)

```
Window*
wmk(Image *i, Mousectl *mc, Channel *ck, Channel *cctl, bool scrolling)
{
    Window *w;
    Rectangle r;

    ⟨wmk() colors initialisation 128e⟩

    w = emalloc(sizeof(Window));

    w->i = i;
    w->screenr = i->r;
    w->cursorp = nil;

    w->id = ++id;
    w->topped = ++topped;

    w->label = estrdup("<unnamed>");

    ⟨wmk() channels settings 80f⟩
    ⟨wmk() channels creation 112d⟩
    ⟨wmk() textual window settings 126a⟩
    ⟨wmk() process settings 81g⟩

    ⟨wmk() drawing border 81a⟩
    ⟨wmk() drawing scrollbar 149b⟩

    incref(w); /* ref will be removed after mounting; avoids delete before ready to be deleted */
    return w;
}
```

⟨wmk() *channels settings* 80f⟩≡ (80e)

```
w->mc = *mc;
w->ck = ck;
w->cctl = cctl;
```

⟨wmk() *drawing border* 81a⟩≡ (80e)
```
wborder(w, Selborder);
```

⟨*function* wborder 81b⟩≡ (249)
```
void
wborder(Window *w, int type)
{
    Image *col;

    ⟨wborder() sanity check w 81f⟩
    ⟨wborder() if holding 218d⟩
    else{
        if(type == Selborder)
            col = titlecol;
        else
            col = lighttitlecol;
    }

    border(w->i, w->i->r, Selborder, col, ZP);
}
```

⟨*global* titlecol 81c⟩≡ (249)
```
static Image *titlecol;
```

⟨*global* lighttitlecol 81d⟩≡ (249)
```
static Image *lighttitlecol;
```

⟨wmk() *extra colors initialisation* 81e⟩≡ (128e) 128i ▷
```
titlecol     = allocimage(display, Rect(0,0,1,1), CMAP8, true, DGreygreen);
lighttitlecol= allocimage(display, Rect(0,0,1,1), CMAP8, true, DPalegreygreen);
```

⟨wborder() *sanity check w* 81f⟩≡ (81b)
```
if(w->i == nil)
    return;
```

⟨wmk() *process settings* 81g⟩≡ (80e)
```
w->notefd = -1;
w->dir = estrdup(startdir);
```

⟨Window *other fields* 81h⟩+≡ (51) ◁ 52k 94b ▷
```
char  *dir; // /dev/wdir
```

⟨*global* startdir 81i⟩≡ (242)
```
char  *startdir;
```

⟨main() *locals* 81j⟩≡ (56a) 165f ▷
```
char buf[256];
```

⟨main() *set some globals* 81k⟩≡ (56a) 165g ▷
```
if(getwd(buf, sizeof buf) == nil)
    startdir = estrdup(".");
else
    startdir = estrdup(buf);
```

7.5.3 Window process creation: winshell()

⟨*global* rcargv 81l⟩≡ (246)
```
char *rcargv[] = { "rc", "-i", nil };
```

⟨new() *other locals* 82a⟩≡ (79b)

```
void **arg;
```

⟨new() *if pid == 0, create winshell process and set pid* 82b⟩≡ (79b)

```
if(pid == 0){
    arg = emalloc(5 * sizeof(void*));
    arg[0] = w;
    arg[1] = cpid;
    arg[2] = cmd;
    if(argv == nil)
        arg[3] = rcargv;
    else
        arg[3] = argv;
    arg[4] = dir;

    if(DEBUG) fprint(STDERR, "new: creating new winshell\n");
    proccreate(winshell, arg, 8192);

    pid = recvul(cpid);
    if(DEBUG) fprint(STDERR, "new: created winshell=%d\n", pid);

    free(arg);
}
```

⟨new() *sanity check pid received from winshell* 82c⟩≡ (79b)

```
if(pid == 0){
    /* window creation failed */
    wsendctlmesg(w, Deleted, ZR, nil);
    chanfree(cpid);
    return nil;
}
```

⟨*function* winshell 82d⟩≡ (250a)

```
void
winshell(void *args)
{
    Window *w;
    Channel *pidc;
    void **arg;
    char *cmd, *dir;
    char **argv;
    errorneg1 err;

    arg = args;

    w    = arg[0];
    pidc = arg[1];
    cmd  = arg[2];
    argv = arg[3];
    dir  = arg[4];

    if(DEBUG) fprint(STDERR, "winshell: cmd = %s\n", cmd);

    // copy namespace/file-descriptors/environment-variables (do not share)
    rfork(RFNAMEG|RFFDG|RFENVG);

    ⟨winshell() adjust namespace 83d⟩
    ⟨winshell() reassign STDIN/STDOUT after namespace adjustment 84a⟩

    if(wclose(w) == false){ /* remove extra ref hanging from creation */
```

```
        if(DEBUG) fprint(STDERR, "winshell: before procexec %s\n", cmd);
        notify(nil);
        dup(STDOUT, STDERR); // STDERR = STDOUT
        if(dir)
            chdir(dir);

        // Exec!!
        procexec(pidc, cmd, argv);
        _exits("exec failed"); // should never be reached
    }
}
```

⟨Window *process fields* 83a⟩≡ (51) 83b ▷

```
int  pid;
```

⟨Window *process fields* 83b⟩+≡ (51) ◁ 83a

```
// /proc/<pid>/notepg
fdt   notefd;
```

⟨*function* wsetpid 83c⟩≡ (249)

```
void
wsetpid(Window *w, int pid, bool dolabel)
{
    char buf[128];
    fdt fd;

    w->pid = pid;
    if(DEBUG) fprint(STDERR, "wsetpid: win=%d pid =%d\n", w->id, w->pid);

    if(dolabel){
        sprint(buf, "rc %d", pid);
        free(w->label);
        w->label = estrdup(buf);
    }

    sprint(buf, "/proc/%d/notepg", pid);
    fd = open(buf, OWRITE|OCEXEC);
    if(w->notefd > 0)
        close(w->notefd);
    w->notefd = fd;
}
```

7.5.4 Namespace adjustments: filsysmount()

⟨winshell() *adjust namespace* 83d⟩≡ (82d)

```
err = filsysmount(filsys, w->id);
⟨winshell() sanity check err filsysmount 84d⟩
```

⟨*function* filsysmount 83e⟩≡ (254b)

```
/*
 * Called only from a different FD group
 */
errorneg1
filsysmount(Filsys *fs, int id)
{
    char buf[32];
    errorneg1 err;
```

```
    if(DEBUG) fprint(STDERR, "filsysmount1\n");

    close(fs->sfd); /* close server end so mount won't hang if exiting */
    sprint(buf, "%d", id);
    err = mount(fs->cfd, -1, "/mnt/wsys", MREPL, buf);
    ⟨filsysmount() sanity check err mount 84e⟩
    err = bind("/mnt/wsys", "/dev", MBEFORE);
    ⟨filsysmount() sanity check err bind 84f⟩

    if(DEBUG) fprint(STDERR, "filsysmount2\n");
    return OK_0;
}
```

⟨winshell() *reassign STDIN/STDOUT after namespace adjustment* 84a⟩≡ (82d)

```
// reassign stdin/stdout to virtualized /dev/cons from filsysmount
close(STDIN);
err = open("/dev/cons", OREAD);
⟨winshell() sanity check err open cons stdin 84b⟩
close(STDOUT);
err = open("/dev/cons", OWRITE);
⟨winshell() sanity check err open cons stdout 84c⟩
```

⟨winshell() *sanity check err open cons stdin* 84b⟩≡ (84a)

```
if(err < 0){
    fprint(STDERR, "can't open /dev/cons: %r\n");
    sendul(pidc, 0);
    threadexits("/dev/cons");
}
```

⟨winshell() *sanity check err open cons stdout* 84c⟩≡ (84a)

```
if(err < 0){
    fprint(STDERR, "can't open /dev/cons: %r\n");
    sendul(pidc, 0);
    threadexits("open"); /* BUG? was terminate() */
}
```

⟨winshell() *sanity check err filsysmount* 84d⟩≡ (83d)

```
if(err < 0){
    fprint(STDERR, "mount failed: %r\n");
    sendul(pidc, 0);
    threadexits("mount failed");
}
```

⟨filsysmount() *sanity check err mount* 84e⟩≡ (83e)

```
if(err < 0){
    fprint(STDERR, "mount failed: %r\n");
    return ERROR_NEG1;
}
```

⟨filsysmount() *sanity check err bind* 84f⟩≡ (83e)

```
if(err < 0){
    fprint(STDERR, "bind failed: %r\n");
    return ERROR_NEG1;
}
```

7.5.5 Public layer: wsetname()

⟨Window *id fields* 84g⟩+≡ (51) ◁52e

```
uint  namecount;
```

⟨*function* wsetname 85a⟩≡ (249)

```
void
wsetname(Window *w)
{
    int i, n;
    char err[ERRMAX];

    n = sprint(w->name, "window.%d.%d", w->id, w->namecount++);

    if(nameimage(w->i, w->name, true) > 0)
        return;
    // else
    ⟨wsetname() if image name already in use, try another name 85b⟩
}
```

⟨wsetname() *if image name already in use, try another name* 85b⟩≡ (85a)

```
for(i='A'; i<='Z'; i++){
    // ok try again
    if(nameimage(w->i, w->name, true) > 0)
        return;
    // else, retry

    errstr(err, sizeof err);
    if(strcmp(err, "image name in use") != 0)
        break;
    w->name[n] = i;
    w->name[n+1] = '\0';
}
// else
w->name[0] = '\0';
fprint(STDERR, "rio: setname failed: %s\n", err);
```

7.5.6 Mouse action sweep()

⟨*global* menuing 85c⟩≡ (242)

```
bool menuing;/* menu action is pending; waiting for window to be indicated */
```

⟨*function* sweep 85d⟩≡ (246)

```
Image*
sweep(void)
{
    Point p0, p;
    Rectangle r;
    Image *i, *oi;

    i = nil;

    menuing = true;
    riosetcursor(&crosscursor, true);
    while(mouse->buttons == 0)
        readmouse(mousectl);

    p0 = onscreen(mouse->xy);
    p = p0;
    r = Rpt(p0, p);
    oi = nil;

    while(mouse->buttons == 4){ // right click
        readmouse(mousectl);
```

```
        if(mouse->buttons != 4 && mouse->buttons != 0)
            break;
        if(!eqpt(mouse->xy, p)){
            p = onscreen(mouse->xy);
            r = canonrect(Rpt(p0, p));

            if(Dx(r)>5 && Dy(r)>5){
                i = allocwindow(desktop, r, Refnone, 0xEEEEEEFF); /* grey */
                freeimage(oi);
                ⟨sweep() sanity check i 86b⟩
                oi = i;
                border(i, r, Selborder, red, ZP);
                flushimage(display, true);
            }
        }
    }
    ⟨sweep() sanity check mouse buttons, i, and rectangle size 86c⟩
    oi = i;
    i = allocwindow(desktop, oi->r, Refbackup, DWhite);
    freeimage(oi);
    ⟨sweep() sanity check i 86b⟩
    border(i, r, Selborder, red, ZP);
    cornercursor(input, mouse->xy, true);
    goto Return;
⟨sweep() Rescue handler 86d⟩

 Return:
    moveto(mousectl, mouse->xy); /* force cursor update; ugly */
    menuing = false;
    return i;
}
```

⟨*function* onscreen 86a⟩≡ (246)

```
Point
onscreen(Point p)
{
    p.x = max(view->clipr.min.x, p.x);
    p.x = min(view->clipr.max.x, p.x);
    p.y = max(view->clipr.min.y, p.y);
    p.y = min(view->clipr.max.y, p.y);
    return p;
}
```

⟨sweep() *sanity check i* 86b⟩≡ (85)

```
if(i == nil)
    goto Rescue;
```

⟨sweep() *sanity check mouse buttons, i, and rectangle size* 86c⟩≡ (85d)

```
if(mouse->buttons != 0)
    goto Rescue;
if(i==nil || Dx(i->r) < 100 || Dy(i->r) < 3*font->height)
    goto Rescue;
```

⟨sweep() *Rescue handler* 86d⟩≡ (85d)

```
Rescue:
    freeimage(i);
    i = nil;
    cornercursor(input, mouse->xy, true);
    while(mouse->buttons)
        readmouse(mousectl);
```

7.6 Window focus

⟨mousethread() *click on unfocused window, set w* 87a⟩≡ (63c)

```
w = wtop(mouse->xy);
```

⟨*function* wtop 87b⟩≡ (249)

```
Window*
wtop(Point pt)
{
    Window *w;

    w = wpointto(pt);
    if(w){
        if(w->topped == topped)
            return nil;
        topwindow(w->i); // window.h (was in draw.h)
        wcurrent(w);
        flushimage(display, true);
        w->topped = ++topped;
    }
    return w;
}
```

⟨*function* wcurrent 87c⟩≡ (249)

```
void
wcurrent(Window *w)
{
    Window *oi;

    ⟨wcurrent() if wkeyboard 216e⟩
    oi = input;
    // updated input!
    input = w;

    if(oi && oi != w)
        wrepaint(oi);
    if(w){
        wrepaint(w);
        wsetcursor(w, false);
    }
    ⟨wcurrent() wakeup w and oi 176f⟩
}
```

⟨*function* wrepaint 87d⟩≡ (249)

```
void
wrepaint(Window *w)
{

    ⟨wrepaint() update cols 128f⟩
    ⟨wrepaint() after updated cols, redraw content if mouse not opened 158c⟩

    if(w == input){
        wborder(w, Selborder);
        wsetcursor(w, false);
    }else
        wborder(w, Unselborder);
}
```

⟨*constant* Unselborder 87e⟩≡ (237a)

```
Unselborder  = 1, /* border of unselected window */
```

7.7 Window deletion

⟨button3menu() *cases* 88a⟩+≡ (76b) ◁79a 91c▷

```
case Delete:
    delete();
    break;
```

7.7.1 delete()

⟨*function* delete 88b⟩≡ (246)

```
void
delete(void)
{
    Window *w;

    w = pointto(true);
    if(w)
        wsendctlmesg(w, Deleted, ZR, nil);
}
```

⟨Wctlmesgkind *cases* 88c⟩≡ (78a) 90f▷

```
Deleted,
```

⟨Window *config fields* 88d⟩+≡ (51) ◁53h 119c▷

```
bool deleted;
```

⟨wctlmesg() *cases* 88e⟩≡ (78d) 90g▷

```
case Deleted:
    ⟨wctlmesg() break if window was deleted 88f⟩
    write(w->notefd, "hangup", 6);
    proccreate(deletetimeoutproc, estrdup(w->name), 4096);
    wclosewin(w);
    break;
```

⟨wctlmesg() *break if window was deleted* 88f⟩≡ (219d 120c 88e)

```
if(w->deleted)
    break;
```

⟨*function* wclosewin 88g⟩≡ (249)

```
void
wclosewin(Window *w)
{
    int i;

    w->deleted = true;

    if(w == input){
        input = nil;
        wsetcursor(w, false);
    }
    ⟨wclosewin() if wkeyboard 216f⟩
    ⟨wclosewin() remove w from hidden 98b⟩

    for(i=0; i<nwindow; i++)
        if(windows[i] == w){
            --nwindow;
            memmove(windows+i, windows+i+1, (nwindow-i)*sizeof(Window*));

            freeimage(w->i);
```

```
            w->i = nil;
            return;
        }
    error("unknown window in closewin");
}
```

7.7.2 deletetimeoutproc() and deletethread()

⟨*function* deletetimeoutproc 89a⟩≡ (244b)

```
void
deletetimeoutproc(void *v)
{
    char *s = v;

    sleep(750); /* remove window from screen after 3/4 of a second */
    sendp(deletechan, s);
}
```

⟨*global* deletechan 89b⟩≡ (242)

```
// chan<string> (listener = deletethread, sender = deletetimeoutproc)
Channel* deletechan;
```

⟨main() *communication channels creation* 89c⟩+≡ (56a) ◁57f 89g▷

```
deletechan   = chancreate(sizeof(char*), 0);
```

⟨main() *threads creation* 89d⟩+≡ (56a) ◁58a 90a▷

```
threadcreate(deletethread, nil, STACK);
```

⟨*function* deletethread 89e⟩≡ (245b)

```
/* thread to make Deleted windows that the client still holds disappear offscreen after an interval */
void
deletethread(void*)
{
    char *s;
    Image *i;

    threadsetname("deletethread");

    for(;;){
        s = recvp(deletechan);

        i = namedimage(display, s);
        if(i != nil){
            /* move it off-screen to hide it, since client is slow in letting it go */
            originwindow(i, i->r.min, view->r.max);
        }
        freeimage(i);
        free(s);
    }
}
```

7.7.3 Deleted and Exited

⟨*global* winclosechan 89f⟩≡ (242)

```
// chan<ref<Window>> (listener = winclosethread, sender = filsyswalk | filsysclunk )
Channel *winclosechan; /* chan(Window*); */
```

⟨main() *communication channels creation* 89g⟩+≡ (56a) ◁89c

```
winclosechan = chancreate(sizeof(Window*), 0);
```

⟨main() *threads creation* 90a⟩+≡ (56a) ◁89d 220f▷

```
threadcreate(winclosethread, nil, STACK);
```

⟨*function* winclosethread 90b⟩≡ (245b)

```
/* thread to allow fsysproc to synchronize window closing with main proc */
void
winclosethread(void*)
{
    Window *w;

    threadsetname("winclosethread");

    for(;;){
        w = recvp(winclosechan);
        wclose(w);
    }
}
```

⟨*function* wclose 90c⟩≡ (249)

```
bool
wclose(Window *w)
{
    int i;

    i = decref(w);
    if(i > 0)
        return false;
    ⟨wclose() sanity check i 90d⟩
    ⟨wclose() sanity check w 90e⟩

    wsendctlmesg(w, Exited, ZR, nil);
    return true;
}
```

⟨wclose() *sanity check i* 90d⟩≡ (90c)

```
if(i < 0)
    error("negative ref count");
```

⟨wclose() *sanity check w* 90e⟩≡ (90c)

```
if(!w->deleted)
    wclosewin(w);
```

⟨Wctlmesgkind *cases* 90f⟩+≡ (78a) ◁88c 113c▷

```
Exited,
```

⟨wctlmesg() *cases* 90g⟩+≡ (78d) ◁88e 94a▷

```
case Exited:
    frclear(&w->frm, true);
    close(w->notefd);
    chanfree(w->mc.c);
    chanfree(w->ck);
    chanfree(w->cctl);
    chanfree(w->conswrite);
    chanfree(w->consread);
    chanfree(w->mouseread);
    chanfree(w->wctlread);
    free(w->raw);
    free(w->r);
    free(w->dir);
    free(w->label);
    free(w);
    break;
```

7.7.4 Mouse action pointto()

⟨*function* pointto 91a⟩≡ (246)

```
Window*
pointto(bool wait)
{
    Window *w;

    menuing = true;
    riosetcursor(&sightcursor, true);

    while(mouse->buttons == 0)
        readmouse(mousectl);

    if(mouse->buttons == 4)
        w = wpointto(mouse->xy);
    else
        w = nil;

    if(wait){
        while(mouse->buttons){
            ⟨pointto() cancel pointto if clicked another button 91b⟩
            readmouse(mousectl);
        }
        if(w != nil && wpointto(mouse->xy) != w)
            w = nil;
    }

    cornercursor(input, mouse->xy, false);
    moveto(mousectl, mouse->xy); /* force cursor update; ugly */
    menuing = false;
    return w;
}
```

⟨pointto() *cancel pointto if clicked another button* 91b⟩≡ (91a)

```
if(mouse->buttons!=4 && w != nil){ /* cancel */
    cornercursor(input, mouse->xy, false);
    w = nil;
}
```

7.8 Window move

⟨button3menu() *cases* 91c⟩+≡ (76b) ◁88a 93d▷

```
case Move:
    move();
    break;
```

7.8.1 move()

⟨*function* move 91d⟩≡ (246)

```
void
move(void)
{
    Window *w;
    Image *i;
    Rectangle r;
```

```
    w = pointto(false);
    if(w == nil)
        return;
    i = drag(w, &r);
    if(i)
        wsendctlmesg(w, Moved, r, i);
    cornercursor(input, mouse->xy, true);
}
```

7.8.2 Mouse action drag()

⟨*function* drag 92⟩≡ (246)

```
Image*
drag(Window *w, Rectangle *rp)
{
    Image *i;
    Image *ni = nil;
    Point p, op, d, dm, om;
    Rectangle r;

    i = w->i;

    menuing = true;
    om = mouse->xy;
    riosetcursor(&boxcursor, true);

    dm = subpt(mouse->xy, w->screenr.min);
    d = subpt(i->r.max, i->r.min);
    op = subpt(mouse->xy, dm);

    drawborder(Rect(op.x, op.y, op.x+d.x, op.y+d.y), true);
    flushimage(display, true);

    while(mouse->buttons == 4){
        p = subpt(mouse->xy, dm);
        if(!eqpt(p, op)){
            // will move previously drawn rectangle thx to originwindow
            drawborder(Rect(p.x, p.y, p.x+d.x, p.y+d.y), true);
            flushimage(display, true);
            op = p;
        }
        readmouse(mousectl);
    }

    r = Rect(op.x, op.y, op.x+d.x, op.y+d.y);
    drawborder(r, false);

    cornercursor(w, mouse->xy, true);
    moveto(mousectl, mouse->xy); /* force cursor update; ugly */
    menuing = false;

    flushimage(display, true);
    if(mouse->buttons == 0)
        ni = allocwindow(desktop, r, Refbackup, DWhite);
    ⟨drag() sanity check mouse buttons and ni 93a⟩
    draw(ni, ni->r, i, nil, i->r.min);
    *rp = r;
    return ni;
}
```

⟨drag() *sanity check mouse buttons and ni* 93a⟩≡ (92)

```
if(mouse->buttons!=0 || ni==nil){
    moveto(mousectl, om);
    while(mouse->buttons)
        readmouse(mousectl);
    *rp = Rect(0, 0, 0, 0);
    return nil;
}
```

⟨*function* drawborder 93b⟩≡ (246)

```
void
drawborder(Rectangle r, bool show)
{
    static Image *b[4];
    int i;
    if(!show){
        for(i = 0; i < 4; i++){
            freeimage(b[i]);
            b[i] = nil;
        }
    }else{
        r = canonrect(r);
        drawedge(&b[0], Rect(r.min.x, r.min.y, r.min.x+Borderwidth, r.max.y));
        drawedge(&b[1], Rect(r.min.x+Borderwidth, r.min.y, r.max.x-Borderwidth, r.min.y+Borderwidth));
        drawedge(&b[2], Rect(r.max.x-Borderwidth, r.min.y, r.max.x, r.max.y));
        drawedge(&b[3], Rect(r.min.x+Borderwidth, r.max.y-Borderwidth, r.max.x-Borderwidth, r.max.y));
    }
}
```

⟨*function* drawedge 93c⟩≡ (246)

```
void
drawedge(Image **bp, Rectangle r)
{
    Image *b = *bp;
    if(b != nil && Dx(b->r) == Dx(r) && Dy(b->r) == Dy(r))
        originwindow(b, r.min, r.min);
    else{
        freeimage(b);
        *bp = allocwindow(desktop, r, Refbackup, DRed);
    }
}
```

7.9 Window resize

⟨button3menu() *cases* 93d⟩+≡ (76b) ◁91c 97a▷

```
case Reshape:
    resize();
    break;
```

7.9.1 resize()

⟨*function* resize 93e⟩≡ (246)

```
void
resize(void)
{
    Window *w;
    Image *i;
```

```
    w = pointto(true);
    if(w == nil)
        return;
    i = sweep();
    if(i)
        wsendctlmesg(w, Reshaped, i->r, i);
}
```

⟨wctlmesg() *cases* 94a⟩+≡ (78d) ◁90g 113d▷

```
  case Moved:
  case Reshaped:
      if(w->deleted){
          freeimage(i);
          break;
      }
      w->screenr = r;
      strcpy(buf, w->name);
      wresize(w, i, m==Moved);
      w->wctlready = true;

      proccreate(deletetimeoutproc, estrdup(buf), 4096);

      if(Dx(r) > 0){
          if(w != input)
              wcurrent(w);
      }else if(w == input)
          wcurrent(nil);
      flushimage(display, true);
      break;
```

⟨Window *other fields* 94b⟩+≡ (51) ◁81h 112c▷

```
  bool   resized;
```

⟨*function* wresize 94c⟩≡ (249)

```
  void
  wresize(Window *w, Image *i, bool move)
  {
      Rectangle r, or;

      or = w->i->r;
      if(move || (Dx(or)==Dx(i->r) && Dy(or)==Dy(i->r)))
          draw(i, i->r, w->i, nil, w->i->r.min);
      freeimage(w->i);
      w->i = i;
      wsetname(w); // publish new window name by incrementing namecount
      w->mc.image = i;

      ⟨wresize() textual window updates 171c⟩

      wborder(w, Selborder);
      w->topped = ++topped;

      w->resized = true;
      w->mouse.counter++;
  }
```

7.9.2 Mouse action bandsize()

⟨*function* bandsize 95a⟩≡ (246)

```
Image*
bandsize(Window *w)
{
    Image *i;
    Rectangle r, or;
    Point p, startp;
    int which, but;

    p = mouse->xy;
    but = mouse->buttons;
    which = whichcorner(w, p);
    p = cornerpt(w->screenr, p, which);
    wmovemouse(w, p);

    readmouse(mousectl);
    r = whichrect(w->screenr, p, which);
    drawborder(r, true);

    or = r;
    startp = p;

    while(mouse->buttons == but){
        p = onscreen(mouse->xy);
        r = whichrect(w->screenr, p, which);
        if(!eqrect(r, or) && goodrect(r)){
            drawborder(r, true);
            flushimage(display, true);
            or = r;
        }
        readmouse(mousectl);
    }

    p = mouse->xy;
    drawborder(or, false);
    flushimage(display, true);

    wsetcursor(w, true);
    ⟨bandsize() sanity check mouse buttons, rectanglr or, point p 95b⟩
    i = allocwindow(desktop, or, Refbackup, DWhite);
    ⟨bandsize() sanity check i 95c⟩
    border(i, r, Selborder, red, ZP);
    return i;
}
```

⟨bandsize() *sanity check mouse buttons, rectanglr* or, *point* p 95b⟩≡ (95a)

```
if(mouse->buttons!=0 || Dx(or)<100 || Dy(or)<3*font->height){
    while(mouse->buttons)
        readmouse(mousectl);
    return nil;
}
if(abs(p.x - startp.x) + abs(p.y - startp.y) <= 1)
    return nil;
```

⟨bandsize() *sanity check i* 95c⟩≡ (95a)

```
if(i == nil)
    return nil;
```

⟨*function* wmovemouse 96a⟩≡ (249)

```
/*
 * Convert back to physical coordinates
 */
void
wmovemouse(Window *w, Point p)
{
    p.x += w->screenr.min.x - w->i->r.min.x;
    p.y += w->screenr.min.y - w->i->r.min.y;
    moveto(mousectl, p);
}
```

⟨*function* whichrect 96b⟩≡ (246)

```
Rectangle
whichrect(Rectangle r, Point p, int which)
{
    switch(which){
    case 0: /* top left */
        r = Rect(p.x, p.y, r.max.x, r.max.y);
        break;
    case 2: /* top right */
        r = Rect(r.min.x, p.y, p.x, r.max.y);
        break;
    case 6: /* bottom left */
        r = Rect(p.x, r.min.y, r.max.x, p.y);
        break;
    case 8: /* bottom right */
        r = Rect(r.min.x, r.min.y, p.x, p.y);
        break;
    case 1: /* top edge */
        r = Rect(r.min.x, p.y, r.max.x, r.max.y);
        break;
    case 5: /* right edge */
        r = Rect(r.min.x, r.min.y, p.x, r.max.y);
        break;
    case 7: /* bottom edge */
        r = Rect(r.min.x, r.min.y, r.max.x, p.y);
        break;
    case 3:  /* left edge */
        r = Rect(p.x, r.min.y, r.max.x, r.max.y);
        break;
    }
    return canonrect(r);
}
```

⟨*function* cornerpt 96c⟩≡ (246)

```
Point
cornerpt(Rectangle r, Point p, int which)
{
    switch(which){
    case 0: /* top left */
        p = Pt(r.min.x, r.min.y);
        break;
    case 2: /* top right */
        p = Pt(r.max.x,r.min.y);
        break;
    case 6: /* bottom left */
        p = Pt(r.min.x, r.max.y);
        break;
    case 8: /* bottom right */
```

```
        p = Pt(r.max.x, r.max.y);
        break;
    case 1: /* top edge */
        p = Pt(p.x,r.min.y);
        break;
    case 5: /* right edge */
        p = Pt(r.max.x, p.y);
        break;
    case 7: /* bottom edge */
        p = Pt(p.x, r.max.y);
        break;
    case 3:  /* left edge */
        p = Pt(r.min.x, p.y);
        break;
    }
    return p;
}
```

7.10 Window visibility

⟨button3menu() *cases* 97a⟩+≡ (76b) ◁93d 98d▷

```
case Hide:
    hide();
    break;
```

7.10.1 hide()

⟨*function* hide 97b⟩≡ (246)

```
void
hide(void)
{
    Window *w;

    w = pointto(true);
    if(w == nil)
        return;
    whide(w);
}
```

7.10.2 hiddden

⟨*global* hidden 97c⟩≡ (242)

```
// array<ref<Window>> (size valid = nhidden)
Window *hidden[100];
```

⟨*global* nhidden 97d⟩≡ (242)

```
int  nhidden;
```

⟨*function* whide 97e⟩≡ (246)

```
int
whide(Window *w)
{
    Image *i;
    int j;

    for(j=0; j<nhidden; j++)
```

```
        if(hidden[j] == w) /* already hidden */
            return -1;

    i = allocimage(display, w->screenr, w->i->chan, false, DWhite);
    if(i){
        hidden[nhidden++] = w;
        wsendctlmesg(w, Reshaped, ZR, i);
        return 1;
    }
    return 0;
}
```

⟨`new()` *if hideit* 98a⟩≡ (79b)

```
if(hideit){
    hidden[nhidden++] = w;
    w->screenr = ZR;
}
```

⟨`wclosewin()` *remove w from hidden* 98b⟩≡ (88g)

```
// delete_list(w, hidden)
for(i=0; i<nhidden; i++)
    if(hidden[i] == w){
        --nhidden;
        memmove(hidden+i, hidden+i+1, (nhidden-i)*sizeof(hidden[0]));
        hidden[nhidden] = nil;
        break;
    }
```

⟨`button3menu()` *menu3str adjustments with hidden windows* 98c⟩≡ (76b)

```
for(i=0; i<nhidden; i++)
    menu3str[i+Hidden] = hidden[i]->label;
menu3str[i+Hidden] = nil;
```

7.10.3 unhide()

⟨`button3menu()` *cases* 98d⟩+≡ (76b) ◁97a

```
default:
    unhide(i);
    break;
```

⟨*function* `unhide` 98e⟩≡ (246)

```
void
unhide(int h)
{
    Window *w;

    h -= Hidden;
    w = hidden[h];
    ⟨unhide() sanity check w 98f⟩
    wunhide(h);
}
```

⟨`unhide()` *sanity check w* 98f⟩≡ (98e)

```
if(w == nil)
    return;
```

⟨*function* wunhide 99⟩≡ (246)

```
int
wunhide(int h)
{
    Image *i;
    Window *w;

    w = hidden[h];
    i = allocwindow(desktop, w->i->r, Refbackup, DWhite);
    if(i){
        --nhidden;
        memmove(hidden+h, hidden+h+1, (nhidden-h)*sizeof(Window*));
        wsendctlmesg(w, Reshaped, w->i->r, i);
        return 1;
    }
    return 0;
}
```

Chapter 8

Filesystem Server

8.1 Additional data structures

8.1.1 Qxxx

⟨*enum* qid 100a⟩≡ (237b)

```
enum Qxxx
{
    Qdir,   /* /dev for this window */
    ⟨qid cases 112a⟩

    QMAX,
};
```

⟨*function* QID 100b⟩≡ (237b)

```
#define QID(winid,qxxx) ((winid<<8)|(qxxx))
```

⟨*function* WIN 100c⟩≡ (237b)

```
#define WIN(q) ((((ulong)(q).path)>>8) & 0xFFFFFF)
```

⟨*function* FILE 100d⟩≡ (237b)

```
#define FILE(q) (((ulong)(q).path) & 0xFF)
```

8.1.2 dirtab

⟨*struct* Dirtab 100e⟩≡ (237b)

```
struct Dirtab
{
    char *name;
    // bitset<enum<QTxxx>>
    byte type;
    // enum<Qxxx>
    uint qid;
    uint perm;
};
```

⟨*global* dirtab 100f⟩≡ (255)

```
Dirtab dirtab[]=
{
    { ".",  QTDIR, Qdir,  0500|DMDIR },
    ⟨dirtab array elements 112b⟩
    { nil, }
};
```

⟨Fid *other fields* 101a⟩+≡ (54c) ◁55a 116a▷

```
// ref<Dirtab>
Dirtab *dir;
```

8.2 filsysrespond()

⟨*function* filsysrespond 101b⟩≡ (254a)

```
Xfid*
filsysrespond(Filsys *fs, Xfid *x, Fcall *fc, char *err)
{
    int n;

    ⟨filsysrespond() if err 101e⟩
    else
        fc->type = x->req.type+1; // Reply type just after Transmit type

    fc->fid = x->req.fid;
    fc->tag = x->req.tag;

    ⟨filsysrespond() sanitize buf 101c⟩
    n = convS2M(fc, x->buf, messagesize);
    ⟨filsysrespond() sanity check n 101d⟩

    if(write(fs->sfd, x->buf, n) != n)
        error("write error in respond");
    ⟨filsysrespond() dump Fcall t if debug 228d⟩
    free(x->buf);
    x->buf = nil;
    return x;
}
```

⟨filsysrespond() *sanitize buf* 101c⟩≡ (101b)

```
if(x->buf == nil)
    x->buf = malloc(messagesize);
```

⟨filsysrespond() *sanity check n* 101d⟩≡ (101b)

```
if(n <= 0)
    error("convert error in convS2M");
```

⟨filsysrespond() *if err* 101e⟩≡ (101b)

```
if(err){
    fc->type = Rerror;
    fc->ename = err;
}
```

8.3 Attach

8.3.1 filsysattach()

⟨*function* filsysattach 101f⟩≡ (255)

```
static
Xfid*
filsysattach(Filsys *, Xfid *x, Fid *f)
{
    ⟨filsysattach() locals 102a⟩
```

```
    ⟨filsysattach() sanity check same user 102b⟩

    f->busy = true;
    f->open = false;

    f->qid.path = Qdir; // no window id, Qdir valid for all
    f->qid.type = QTDIR;
    f->qid.vers = 0;
    f->dir = dirtab; // entry for "."

    f->nrpart = 0;

    sendp(x->c, xfidattach);
    return nil;
}
```

⟨filsysattach() *locals* 102a⟩≡ (101f)

```
Fcall fc;
```

⟨filsysattach() *sanity check same user* 102b⟩≡ (101f)

```
if(strcmp(x->req.uname, x->fs->user) != 0)
    return filsysrespond(x->fs, x, &fc, Eperm);
```

8.3.2 xfidattach()

⟨*global* all 102c⟩≡ (242)

```
QLock all;   /* BUG */
```

⟨*function* xfidattach 102d⟩≡ (256)

```
void
xfidattach(Xfid *x)
{
    Fcall fc;
    int id;
    Window *w = nil;
    bool newlymade = false;
    ⟨xfidattach() other locals 103b⟩

    fc.qid = x->f->qid;
    qlock(&all);

    ⟨xfidattach() if mount "new ..." 181e⟩
    else{
        // mount(fs->cfd, ..., "/mnt/wsys", ..., "2"), winid
        id = atoi(x->req.aname);
        w = wlookid(id);
    }

    x->f->w = w;
    ⟨xfidattach() sanity check w 103c⟩
    if(!newlymade) /* counteract dec() in winshell() */
        incref(w);
    qunlock(&all);

    filsysrespond(x->fs, x, &fc, nil);
}
```

⟨*function* `wlookid` 103a⟩≡ (249)

```
Window*
wlookid(int id)
{
    int i;

    for(i=0; i<nwindow; i++)
        if(windows[i]->id == id)
            return windows[i];
    return nil;
}
```

⟨`xfidattach()` *other locals* 103b⟩≡ (102d) 181d ▷

```
char *err = Eunkid;
```

⟨`xfidattach()` *sanity check w* 103c⟩≡ (102d)

```
if(w == nil){
    qunlock(&all);
    x->f->busy = false;
    filsysrespond(x->fs, x, &fc, err);
    return;
}
```

8.4 Walk

8.4.1 filsyswalk()

⟨*function* `filsyswalk` 103d⟩≡ (255)

```
static
Xfid*
filsyswalk(Filsys *fs, Xfid *x, Fid *f)
{
    Fcall fc;
    Qid qid;
    Dirtab *dir;
    int i;
    int id;
    uchar type;
    ulong path;
    Dirtab *d;
    Window *w;
    char *err;
    ⟨filsyswalk() other locals 104b⟩

    ⟨filsyswalk() sanity check if opened 105b⟩
    ⟨filsyswalk() if clone walk message 104c⟩

    fc.nwqid = 0;
    err = nil;

    /* update f->qid, f->dir only if walk completes */
    qid = f->qid;
    dir = f->dir;

    if(x->req.nwname > 0){
        for(i=0; i < x->req.nwname; i++){
            ⟨filsyswalk() sanity check current qid is a directory 105e⟩
            ⟨filsyswalk() if dotdot 105a⟩
```

```
            ⟨filsyswalk() if Qwsys, then goto Accept 174h⟩
            ⟨filsyswalk() if snarf 192f⟩
            id = WIN(f->qid);
            d = dirtab;
            d++; /* skip '.' */
            for(; d->name; d++)
                if(strcmp(x->req.wname[i], d->name) == 0){
                    path = d->qid;
                    type = d->type;
                    dir = d;
                    goto Accept;
                }
            break; /* file not found */
        }
        ⟨filsyswalk() sanity check i and err 105c⟩
    }
    ⟨filsyswalk() sanity check err and nwqid 105d⟩
    else if(fc.nwqid == x->req.nwname){
        f->dir = dir;
        f->qid = qid;
    }

    return filsysrespond(fs, x, &fc, err);
}
```

⟨filsyswalk() *accept label and code* 104a⟩≡ (105a)

```
Accept:
⟨filsyswalk() sanity check path elements 105f⟩
qid.type = type;
qid.path = QID(id, path);
qid.vers = 0;
fc.wqid[fc.nwqid++] = qid;
continue;
```

8.4.2 Cloning fid

⟨filsyswalk() *other locals* 104b⟩≡ (103d)

```
Fid *nf = nil;
```

⟨filsyswalk() *if clone walk message* 104c⟩≡ (103d)

```
if(x->req.fid  != x->req.newfid){
    /* BUG: check exists */
    nf = newfid(fs, x->req.newfid);
    ⟨filsyswalk() when clone walk message, sanity check nf 104d⟩
    nf->busy = true;
    nf->open = false;
    nf->dir = f->dir;
    nf->qid = f->qid;
    nf->w = f->w;
    incref(f->w);
    nf->nrpart = 0; /* not open, so must be zero */
    f = nf; /* walk f */
}
```

⟨filsyswalk() *when clone walk message, sanity check nf* 104d⟩≡ (104c)

```
if(nf->busy)
    return filsysrespond(fs, x, &fc, "clone to busy fid");
```

8.4.3 ..

⟨filsyswalk() *if dotdot* 105a⟩≡ (103d)

```
if(strcmp(x->req.wname[i], "..") == 0){
    type = QTDIR;
    path = Qdir;
    dir = dirtab;
    ⟨filsyswalk() when in dotdot, if Qwsysdir adjust path 175d⟩
    id = 0;
    ⟨filsyswalk() accept label and code 104a⟩
}
```

8.4.4 Error management

⟨filsyswalk() *sanity check if opened* 105b⟩≡ (103d)

```
if(f->open)
    return filsysrespond(fs, x, &fc, "walk of open file");
```

⟨filsyswalk() *sanity check i and err* 105c⟩≡ (103d)

```
if(i==0 && err==nil)
    err = Eexist;
```

⟨filsyswalk() *sanity check err and nwqid* 105d⟩≡ (103d)

```
if(err!=nil || fc.nwqid < x->req.nwname){
    if(nf){
        if(nf->w)
            sendp(winclosechan, nf->w);
        nf->open = false;
        nf->busy = false;
    }
}
```

⟨filsyswalk() *sanity check current qid is a directory* 105e⟩≡ (103d)

```
if(!(qid.type & QTDIR)){
    err = Enotdir;
    break;
}
```

⟨filsyswalk() *sanity check path elements* 105f⟩≡ (104a)

```
if(i == MAXWELEM){
    err = "name too long";
    break;
}
```

8.5 Open

8.5.1 filsysopen()

⟨*function* filsysopen 105g⟩≡ (255)

```
static
Xfid*
filsysopen(Filsys *fs, Xfid *x, Fid *f)
{
    ⟨filsysopen() locals 106a⟩

    ⟨filsysopen() sanity check mode 106b⟩
    sendp(x->c, xfidopen);
    return nil;
⟨filsysopen() deny label 106e⟩
}
```

⟨filsysopen() *locals* 106a⟩≡ (105g) 106d ▷

```
int m;
```

⟨filsysopen() *sanity check mode* 106b⟩≡ (105g)

```
⟨filsysopen() sanitize mode 106c⟩
/* can't execute or remove anything */
if(x->req.mode==OEXEC || (x->req.mode & ORCLOSE))
    goto Deny;

switch(x->req.mode){
case OREAD:
    m = 0400;
    break;
case OWRITE:
    m = 0200;
    break;
case ORDWR:
    m = 0600;
    break;
default:
    goto Deny;
}
if(((f->dir->perm & ~(DMDIR|DMAPPEND)) & m) != m)
    goto Deny;
```

⟨filsysopen() *sanitize mode* 106c⟩≡ (106b)

```
/* can't truncate anything, so just disregard */
x->req.mode &= ~(OTRUNC|OCEXEC);
```

⟨filsysopen() *locals* 106d⟩+≡ (105g) ◁ 106a

```
Fcall fc;
```

⟨filsysopen() *deny label* 106e⟩≡ (105g)

```
Deny:
    return filsysrespond(fs, x, &fc, Eperm);
```

8.5.2 xfidopen()

⟨*function* xfidopen 106f⟩≡ (256)

```
void
xfidopen(Xfid *x)
{
    Fcall fc;
    Window *w = x->f->w;

    ⟨xfidxxx() respond error if window was deleted 107a⟩
    switch(FILE(x->f->qid)){
    ⟨xfidopen() cases 117d⟩
    }

    x->f->open = true;
    x->f->mode = x->req.mode;

    fc.qid = x->f->qid;
    fc.iounit = messagesize-IOHDRSZ;
    filsysrespond(x->fs, x, &fc, nil);
}
```

⟨xfidxxx() *respond error if window was deleted* 107a⟩≡ (109 106f)

```
if(w->deleted){
    filsysrespond(x->fs, x, &fc, Edeleted);
    return;
}
```

8.6 Clunk/Close

8.6.1 filsysclunk()

⟨*function* filsysclunk 107b⟩≡ (255)

```
static
Xfid*
filsysclunk(Filsys *fs, Xfid *x, Fid *f)
{
    Fcall fc;

    if(f->open){
        f->busy = false;
        f->open = false;
        sendp(x->c, xfidclose);
        return nil;
    }
    // else
    if(f->w)
        sendp(winclosechan, f->w);
    f->busy = false;
    f->open = false;
    return filsysrespond(fs, x, &fc, nil);
}
```

8.6.2 xfidclose()

⟨*function* xfidclose 107c⟩≡ (256)

```
void
xfidclose(Xfid *x)
{
    Fcall fc;
    Window *w = x->f->w;
    ⟨xfidclose() other locals 193b⟩

    switch(FILE(x->f->qid)){
    ⟨xfidclose() cases 117e⟩
    }
    wclose(w);
    filsysrespond(x->fs, x, &fc, nil);
}
```

8.7 Read

8.7.1 filsysread()

⟨*function* filsysread 107d⟩≡ (255)

```
static
Xfid*
```

```
filsysread(Filsys *fs, Xfid *x, Fid *f)
{
    int o, e;
    uint clock;
    byte *b;
    int n;
    Fcall fc;
    ⟨filsysread() other locals 108b⟩

    if(!(f->qid.type & QTDIR)){
        sendp(x->c, xfidread);
        return nil;
    }
    // else, a directory

    o = x->req.offset;
    e = x->req.offset + x->req.count;
    clock = getclock();
    b = malloc(messagesize-IOHDRSZ); /* avoid memset of emalloc */
    ⟨filsysread() sanity check b 108a⟩

    n = 0;
    switch(FILE(f->qid)){
    ⟨filsysread() cases 108c⟩
    }

    fc.data = (char*)b;
    fc.count = n;
    filsysrespond(fs, x, &fc, nil);
    free(b);
    return x;
}
```

⟨filsysread() *sanity check b* 108a⟩≡ (107d)

```
if(b == nil)
    return filsysrespond(fs, x, &fc, "out of memory");
```

8.7.2 Reading a directory

⟨filsysread() *other locals* 108b⟩≡ (107d) 174g ▷

```
Dirtab *d;
```

⟨filsysread() *cases* 108c⟩≡ (107d) 175b ▷

```
case Qdir:
case Qwsysdir:
    d = dirtab;
    d++; /* first entry is '.' */
    for(i=0; d->name != nil && i<e; i+=len){
        len = dostat(fs, WIN(x->f->qid), d, b+n, x->req.count - n, clock);
        if(len <= BIT16SZ)
            break;
        if(i >= o)
            n += len;
        d++;
    }
    break;
```

8.7.3 xfidread()

⟨*function* xfidread 109a⟩≡ (256)

```
void
xfidread(Xfid *x)
{
    uint qid;
    int off, cnt;
    Fcall fc;
    Fcall *req = &x->req;
    Window *w = x->f->w;
    ⟨xfidread() other locals 112g⟩

    ⟨xfidxxx() respond error if window was deleted 107a⟩
    qid = FILE(x->f->qid);
    off = req->offset;
    cnt = req->count;

    switch(qid){
    ⟨xfidread() cases 112h⟩
    default:
        fprint(STDERR, "unknown qid %d in read\n", qid);
        sprint(buf, "unknown qid in read");
        filsysrespond(x->fs, x, &fc, buf);
        break;
        }
}
```

8.8 Write

8.8.1 filsyswrite()

⟨*function* filsyswrite 109b⟩≡ (255)

```
static
Xfid*
filsyswrite(Filsys*, Xfid *x, Fid*)
{
    sendp(x->c, xfidwrite);
    return nil;
}
```

8.8.2 xfidwrite()

⟨*function* xfidwrite 109c⟩≡ (256)

```
void
xfidwrite(Xfid *x)
{
    Fcall fc;
    Fcall *req = &x->req;
    Window *w = x->f->w;
    uint qid;
    int off, cnt;
    char buf[256];
    ⟨xfidwrite() other locals 113a⟩

    ⟨xfidxxx() respond error if window was deleted 107a⟩
    qid = FILE(x->f->qid);
```

```
    cnt = req->count;
    off = req->offset;
    req->data[cnt] = '\0';

    switch(qid){
    ⟨xfidwrite() cases 113b⟩
    default:
        fprint(STDERR, buf, "unknown qid %d in write\n", qid);
        sprint(buf, "unknown qid in write");
        filsysrespond(x->fs, x, &fc, buf);
        return;
    }

    fc.count = cnt;
    filsysrespond(x->fs, x, &fc, nil);
}
```

8.9 Stats

8.9.1 filsysstat()

⟨*function* filsysstat 110a⟩≡ (255)

```
static
Xfid*
filsysstat(Filsys *fs, Xfid *x, Fid *f)
{
    Fcall fc;

    fc.stat = emalloc(messagesize-IOHDRSZ);
    fc.nstat = dostat(fs, WIN(x->f->qid), f->dir, fc.stat, messagesize-IOHDRSZ, getclock());
    x = filsysrespond(fs, x, &fc, nil);
    free(fc.stat);
    return x;
}
```

⟨*function* dostat 110b⟩≡ (255)

```
static
int
dostat(Filsys *fs, int id, Dirtab *dir, uchar *buf, int nbuf, uint clock)
{
    Dir d;

    d.qid.path = QID(id, dir->qid);
    ⟨dostat() adjust vers for snarf 193g⟩
    else
        d.qid.vers = 0;
    d.qid.type = dir->type;
    d.mode = dir->perm;
    d.length = 0; /* would be nice to do better */
    d.name = dir->name;
    d.uid = fs->user;
    d.gid = fs->user;
    d.muid = fs->user;
    d.atime = clock;
    d.mtime = clock;

    return convD2M(&d, buf, nbuf);
}
```

8.10 Forbidden operations

⟨fcall *other methods* 111a⟩+≡ (55d) ◁67e 223c▷

```
[Tcreate]  = filsyscreate,
[Tremove]  = filsysremove,
[Twstat]   = filsyswstat,
```

⟨*function* filsyscreate 111b⟩≡ (255)

```
static
Xfid*
filsyscreate(Filsys *fs, Xfid *x, Fid*)
{
    Fcall fc;

    return filsysrespond(fs, x, &fc, Eperm);
}
```

⟨*function* filsysremove 111c⟩≡ (255)

```
static
Xfid*
filsysremove(Filsys *fs, Xfid *x, Fid*)
{
    Fcall fc;

    return filsysrespond(fs, x, &fc, Eperm);
}
```

⟨*function* filsyswstat 111d⟩≡ (255)

```
static
Xfid*
filsyswstat(Filsys *fs, Xfid *x, Fid*)
{
    Fcall fc;

    return filsysrespond(fs, x, &fc, Eperm);
}
```

Chapter 9

Virtual Devices

9.1 /mnt/wsys/mouse

⟨qid *cases* 112a⟩≡ (100a) 114a ▷
```
Qmouse,
```

⟨*dirtab array elements* 112b⟩≡ (100f) 114b ▷
```
{ "mouse",  QTFILE, Qmouse,  0600 },
```

9.1.1 Reading part1

⟨Window *other fields* 112c⟩+≡ (51) ◁94b 114c ▷
```
// chan<chan<Mouse> > (listener = xfidread(Qmouse), sender = winctl)
Channel  *mouseread; /* chan(Mousereadmesg) */
```

⟨wmk() *channels creation* 112d⟩≡ (80e) 114d ▷
```
w->mouseread =  chancreate(sizeof(Mousereadmesg), 0);
```

⟨*struct* Mousereadmesg 112e⟩≡ (237b)
```
struct Mousereadmesg
{
    // chan<Mouse> (listener = xfidread(Qmouse), sender = winctl)
    Channel *cm;  /* chan(Mouse) */
};
```

⟨*enum* _anon_ (windows/rio/xfid.c)4 112f⟩≡ (256)
```
enum { MRdata, MRflush, NMR };
```

⟨xfidread() *other locals* 112g⟩≡ (109a) 114h ▷
```
Alt alts[NCR+1];
Mousereadmesg mrm;
Mouse ms;
int n, c;
```

⟨xfidread() *cases* 112h⟩≡ (109a) 114i ▷
```
case Qmouse:
    ⟨xfidxxx() set flushtag 223g⟩

    alts[MRdata].c = w->mouseread;
    alts[MRdata].v = &mrm;
    alts[MRdata].op = CHANRCV;
    ⟨xfidread() when Qmouse, set alts for flush 224b⟩
    alts[NMR].op = CHANEND;
```

```
    switch(alt(alts)){
    case MRdata:
        break;
    ⟨xfidread() when Qmouse, switch alt flush case 224c⟩
    }
    /* received data */
    ⟨xfidxxx() unset flushtag 224a⟩
    ⟨xfidread() when Qmouse, if flushing 224d⟩

    qlock(&x->active);
    recv(mrm.cm, &ms);
    c = 'm';
    ⟨xfidread() when Qmouse, adjust c for resize message if resized 124d⟩
    n = sprint(buf, "%c%11d %11d %11d %11ld ", c, ms.xy.x, ms.xy.y, ms.buttons, ms.msec);
    w->resized = false;

    fc.data = buf;
    fc.count = min(n, cnt);
    filsysrespond(x->fs, x, &fc, nil);
    qunlock(&x->active);
    break;
```

9.1.2 Writing

⟨xfidwrite() *other locals* 113a⟩≡ (109c) 115e ▷

```
  char *p;
  Point pt;
```

⟨xfidwrite() *cases* 113b⟩≡ (109c) 115g ▷

```
  case Qmouse:
      if(w!=input || Dx(w->screenr)<=0)
          break;
      if(req->data[0] != 'm'){
          filsysrespond(x->fs, x, &fc, Ebadmouse);
          return;
      }
      p = nil;
      pt.x = strtoul(req->data+1, &p, 0);
      if(p == nil){
          filsysrespond(x->fs, x, &fc, Eshort);
          return;
      }
      pt.y = strtoul(p, nil, 0);
      if(w==input && wpointto(mouse->xy)==w)
          wsendctlmesg(w, Movemouse, Rpt(pt, pt), nil);
      break;
```

⟨Wctlmesgkind *cases* 113c⟩+≡ (78a) ◁ 90f 120b ▷

```
  Movemouse,
```

⟨wctlmesg() *cases* 113d⟩+≡ (78d) ◁ 94a 120c ▷

```
  case Movemouse:
      if(sweeping || !ptinrect(r.min, w->i->r))
          break;
      wmovemouse(w, r.min);
      break;
```

9.2 /mnt/wsys/cons

⟨qid *cases* 114a⟩+≡ (100a) ◁112a 117a▷
```
Qcons,
```

⟨*dirtab array elements* 114b⟩+≡ (100f) ◁112b 117b▷
```
{ "cons",  QTFILE, Qcons,  0600 },
```

9.2.1 Reading part1

⟨Window *other fields* 114c⟩+≡ (51) ◁112c 115a▷
```
// chan<Consreadmesg> (listener = xfidread(Qcons), sender = winctl)
Channel  *consread; /* chan(Consreadmesg) */
```

⟨wmk() *channels creation* 114d⟩+≡ (80e) ◁112d 115b▷
```
w->consread =  chancreate(sizeof(Consreadmesg), 0);
```

⟨*struct* Consreadmesg 114e⟩≡ (237b)
```
struct Consreadmesg
{
    // chan<ref<array<Rune>> (listener = winctl, sender = xfidread(Qcons))
    Channel *c1;  /* chan(tuple(char*, int) == Stringpair) */
    // chan<ref<array<Rune>> (listener = xdidread(Qcons), sender = winctl)
    Channel *c2;  /* chan(tuple(char*, int) == Stringpair) */
};
```

⟨*struct* Stringpair 114f⟩≡ (237b)
```
struct Stringpair /* rune and nrune or byte and nbyte */
{
    void *s;
    int  ns;
};
```

⟨*enum* _anon_ (windows/rio/xfid.c)*3* 114g⟩≡ (256)
```
enum { CRdata, CRflush, NCR };
```

⟨xfidread() *other locals* 114h⟩+≡ (109a) ◁112g 124g▷
```
Consreadmesg crm;
Channel *c1, *c2; /* chan (tuple(char*, int)) */
char *t;
Stringpair pair;
```

⟨xfidread() *cases* 114i⟩+≡ (109a) ◁112h 117j▷
```
case Qcons:
    ⟨xfidxxx() set flushtag 223g⟩

    alts[CRdata].c = w->consread;
    alts[CRdata].v = &crm;
    alts[CRdata].op = CHANRCV;
    ⟨xfidread() when Qcons, set alts for flush 224e⟩
    alts[NCR].op = CHANEND;

    switch(alt(alts)){
    case CRdata:
        break;
    ⟨xfidread() when Qcons, switch alt flush case 224f⟩
    }
    /* received data */
    ⟨xfidxxx() unset flushtag 224a⟩
```

```
    c1 = crm.c1;
    c2 = crm.c2;
    t = malloc(cnt+UTFmax+1); /* room to unpack partial rune plus */
    pair.s = t;
    pair.ns = cnt;
    send(c1, &pair);

    ⟨xfidread() when Qcons, if flushing 224g⟩

    qlock(&x->active);
    recv(c2, &pair);
    fc.data = pair.s;
    fc.count = pair.ns;
    filsysrespond(x->fs, x, &fc, nil);
    free(t);
    qunlock(&x->active);
    break;
```

9.2.2 Writing part1

```
⟨Window other fields 115a⟩+≡                                    (51) ◁114c 117c▷
  // chan<Conswritemesg> (listener = xfidwrite(Qcons), sender = winctl)
  Channel  *conswrite; /* chan(Conswritemesg) */

⟨wmk() channels creation 115b⟩+≡                                (80e) ◁114d 177d▷
  w->conswrite = chancreate(sizeof(Conswritemesg), 0);

⟨struct Conswritemesg 115c⟩≡                                    (237b)
  struct Conswritemesg
  {
      // chan<ref<array<Rune>> (listener = winctl, sender = xfidwrite(Qcons))
      Channel *cw;  /* chan(Stringpair) */
  };

⟨enum _anon_ (windows/rio/xfid.c)2 115d⟩≡                       (256)
  enum { CWdata, CWflush, NCW };

⟨xfidwrite() other locals 115e⟩+≡                               (109c) ◁113a 115f▷
  Rune *r;
  int nr; // nb runes
  int nb; // nb bytes

⟨xfidwrite() other locals 115f⟩+≡                               (109c) ◁115e 116d▷
  Alt alts[NCW+1];
  Conswritemesg cwm;
  Stringpair pair;

⟨xfidwrite() cases 115g⟩+≡                                      (109c) ◁113b 117f▷
  case Qcons:

      ⟨xfidwrite() when Qcons, look for previous partial rune bytes 116c⟩
      r = runemalloc(cnt);
      cvttorunes(req->data, cnt-UTFmax, r, &nb, &nr, nil);
      ⟨xfidwrite() when Qcons, look if more full runes 116e⟩
      ⟨xfidwrite() when Qcons, store remaining partial rune bytes 116b⟩

      ⟨xfidxxx() set flushtag 223g⟩
```

```
    alts[CWdata].c = w->conswrite;
    alts[CWdata].v = &cwm;
    alts[CWdata].op = CHANRCV;
    ⟨xfidwrite() when Qcons, set alts for flush 224h⟩
    alts[NCW].op = CHANEND;

    switch(alt(alts)){
    case CWdata:
        break;
    ⟨xfidwrite() when Qcons, switch alt flush case 224i⟩
    }

    /* received data */
    ⟨xfidxxx() unset flushtag 224a⟩
    ⟨xfidwrite() when Qcons, if flushing 224j⟩

    qlock(&x->active);
    pair.s = r;
    pair.ns = nr;
    send(cwm.cw, &pair);
    fc.count = req->count;
    filsysrespond(x->fs, x, &fc, nil);
    qunlock(&x->active);
    return;
```

9.2.3 Bytes versus runes, partial runes

⟨Fid *other fields* 116a⟩+≡ (54c) ◁101a

```
uchar rpart[UTFmax];
int  nrpart;
```

⟨xfidwrite() *when Qcons, store remaining partial rune bytes* 116b⟩≡ (115g)

```
// assert(cnt-nb < UTFMAX);
if(nb < cnt){
    memmove(x->f->rpart, req->data + nb, cnt-nb);
    x->f->nrpart = cnt-nb;
}
```

⟨xfidwrite() *when Qcons, look for previous partial rune bytes* 116c⟩≡ (115g)

```
nr = x->f->nrpart;
if(nr > 0){
    memmove(req->data + nr, req->data, cnt); /* there's room: see malloc in filsysproc */
    memmove(req->data, x->f->rpart, nr);
    cnt += nr;
    x->f->nrpart = 0;
}
```

⟨xfidwrite() *other locals* 116d⟩+≡ (109c) ◁115f

```
int c;
```

⟨xfidwrite() *when Qcons, look if more full runes* 116e⟩≡ (115g)

```
/* approach end of buffer */
while(fullrune(req->data + nb, cnt-nb)){
    c = nb;
    nb += chartorune(&r[nr], req->data + c);
    if(r[nr])
        nr++;
}
```

9.3 /mnt/wsys/consctl

⟨qid *cases* 117a⟩+≡ (100a) ◁114a 117g▷
```
  Qconsctl,
```

⟨*dirtab array elements* 117b⟩+≡ (100f) ◁114b 117h▷
```
  { "consctl", QTFILE, Qconsctl, 0200 },
```

⟨Window *other fields* 117c⟩+≡ (51) ◁115a 150d▷
```
  bool ctlopen;
```

⟨xfidopen() *cases* 117d⟩≡ (106f) 119a▷
```
  case Qconsctl:
      if(w->ctlopen){
          filsysrespond(x->fs, x, &fc, Einuse);
          return;
      }
      w->ctlopen = true;
      break;
```

⟨xfidclose() *cases* 117e⟩≡ (107c) 117i▷
```
  case Qconsctl:
      ⟨xfidclose() Qconsctl case, if rawing 120a⟩
      ⟨xfidclose() Qconsctl case, if holding 219a⟩
      w->ctlopen = false;
      break;
```

⟨xfidwrite() *cases* 117f⟩+≡ (109c) ◁115g 118a▷
```
  case Qconsctl:
      ⟨xfidwrite() Qconsctl case 119d⟩
      // else
      filsysrespond(x->fs, x, &fc, "unknown control message");
      return;
```

9.4 /mnt/wsys/cursor

⟨qid *cases* 117g⟩+≡ (100a) ◁117a 118b▷
```
  Qcursor,
```

⟨*dirtab array elements* 117h⟩+≡ (100f) ◁117b 118c▷
```
  { "cursor",  QTFILE, Qcursor, 0600 },
```

⟨xfidclose() *cases* 117i⟩+≡ (107c) ◁117e 119b▷
```
  case Qcursor:
      w->cursorp = nil;
      wsetcursor(w, false);
      break;
```

⟨xfidread() *cases* 117j⟩+≡ (109a) ◁114i 118d▷
```
  case Qcursor:
      filsysrespond(x->fs, x, &fc, "cursor read not implemented");
      break;
```

⟨xfidwrite() *cases* 118a⟩+≡ (109c) ◁117f 173g▷

```
case Qcursor:
    if(cnt < 2*4+2*2*16)
        w->cursorp = nil;
    else{
        w->cursor.offset.x = BGLONG(req->data+0*4);
        w->cursor.offset.y = BGLONG(req->data+1*4);
        memmove(w->cursor.clr, req->data+2*4, 2*2*16);
        w->cursorp = &w->cursor;
    }
    wsetcursor(w, !sweeping);
    break;
```

9.5 /dev/draw/ and /mnt/wsys/winname

⟨qid *cases* 118b⟩+≡ (100a) ◁117g 124e▷

```
Qwinname,
```

⟨*dirtab array elements* 118c⟩+≡ (100f) ◁117h 124f▷

```
{ "winname", QTFILE, Qwinname, 0400 },
```

⟨xfidread() *cases* 118d⟩+≡ (109a) ◁117j 124h▷

```
case Qwinname:
    n = strlen(w->name);
    ⟨xfidread() when Qwinname case, sanity check n 118e⟩
    t = estrdup(w->name);
    goto Text;
```

⟨xfidread() *when Qwinname case, sanity check n* 118e⟩≡ (118d)

```
if(n == 0){
    filsysrespond(x->fs, x, &fc, "window has no name");
    break;
}
```

Chapter 10

Graphical Windows

10.1 Graphical window setup

10.1.1 initdraw()

10.1.2 initmouse(), mouse-open mode

⟨xfidopen() *cases* 119a⟩+≡ (106f) ◁117d 176d▷

```
  case Qmouse:
      if(w->mouseopen){
          filsysrespond(x->fs, x, &fc, Einuse);
          return;
      }
      w->mouseopen = true;

      /*
       * Reshaped: there's a race if the appl. opens the
       * window, is resized, and then opens the mouse,
       * but that's rare.  The alternative is to generate
       * a resized event every time a new program starts
       * up in a window that has been resized since the
       * dawn of time.  We choose the lesser evil.
       */
      w->resized = false;
      break;
```

⟨xfidclose() *cases* 119b⟩+≡ (107c) ◁117i 176e▷

```
  case Qmouse:
      w->mouseopen = false;

      w->resized = false;
      if(w->i != nil)
          wsendctlmesg(w, Refresh, w->i->r, nil);
      break;
```

10.1.3 initkeyboard(), raw-access mode

⟨Window *config fields* 119c⟩+≡ (51) ◁88d 218a▷

```
  bool rawing;
```

⟨xfidwrite() *Qconsctl case* 119d⟩≡ (117f) 218k▷

```
  if(strncmp(req->data, "rawon", 5)==0){
      ⟨xfidwrite() Qconsctl case, if rawon message and holding mode 219b⟩
      if(w->rawing++ == 0)
```

```
        wsendctlmesg(w, Rawon, ZR, nil);
    break;
}
if(strncmp(req->data, "rawoff", 6)==0 && w->rawing){
    if(--w->rawing == 0)
        wsendctlmesg(w, Rawoff, ZR, nil);
    break;
}
```

⟨xfidclose() *Qconsctl case, if rawing* 120a⟩≡ (117e)

```
if(w->rawing){
    w->rawing = false;
    wsendctlmesg(w, Rawoff, ZR, nil);
}
```

⟨Wctlmesgkind *cases* 120b⟩+≡ (78a) ◁113c 218j▷

```
Rawon,
Rawoff,
```

⟨wctlmesg() *cases* 120c⟩+≡ (78d) ◁113d 219d▷

```
case Rawon:
    // already setup w->rawing in xfidwrite, nothing else todo
    break;
case Rawoff:
    ⟨wctlmesg() break if window was deleted 88f⟩
    ⟨wctlmesg() When Rawoff, process raw keys in non rawing mode 122f⟩
    break;
```

10.2 Mouse events

10.2.1 Mouse state queue

⟨Window *mouse fields* 120d⟩+≡ (51) ◁75c

```
Mouseinfo mouse;
```

⟨*struct* Mouseinfo 120e⟩≡ (237b)

```
struct Mouseinfo
{
    // queue of mouse clicks and releases
    Mousestate queue[16];

    // consumer
    int ri; /* read index into queue */
    // producer
    int wi; /* write index */

    bool qfull;/* filled the queue; no more recording until client comes back */
    ⟨Mouseinfo other fields 120f⟩
};
```

⟨Mouseinfo *other fields* 120f⟩≡ (120e) 120g▷

```
ulong counter; /* serial no. of last mouse event we received */
ulong lastcounter;/* serial no. of last mouse event sent to client */
```

⟨Mouseinfo *other fields* 120g⟩+≡ (120e) ◁120f

```
int lastb; /* last button state we received */
```

⟨*struct* Mousestate 121a⟩≡ (237b)

```
struct Mousestate
{
    Mouse;
    ulong counter; /* serial no. of mouse event */
};
```

10.2.2 /mnt/wsys/mouse reading part2

Producer

⟨winctl() *other locals* 121b⟩+≡ (64a) ◁66b 121e▷

```
Mousestate *mp;
int lastb = -1;
```

⟨winctl() *WMouse case if mouseopen* 121c⟩≡ (65j)

```
if(w->mouseopen) {
    w->mouse.counter++;

    /* queue click events */
    if(!w->mouse.qfull && lastb != w->mc.buttons) { /* add to ring */

        //insert_queue(w->mc, w->mouse.queue)
        mp = &w->mouse.queue[w->mouse.wi];
        if(++w->mouse.wi == nelem(w->mouse.queue))
            w->mouse.wi = 0;
        if(w->mouse.wi == w->mouse.ri)
            w->mouse.qfull = true;
        mp->Mouse = w->mc;
        mp->counter = w->mouse.counter;

        lastb = w->mc.buttons;
    }
}
```

Consumer

⟨Wxxx *cases* 121d⟩≡ (63e) 123a▷

```
WMouseread,
```

⟨winctl() *other locals* 121e⟩+≡ (64a) ◁121b 122a▷

```
Mousereadmesg mrm;
```

⟨winctl() *channels creation* 121f⟩≡ (64a) 123c▷

```
mrm.cm = chancreate(sizeof(Mouse), 0);
```

⟨winctl() *Wctl case, free channels if wctlmesg is Excited* 121g⟩≡ (66d) 123d▷

```
chanfree(mrm.cm);
```

⟨winctl() *alts setup* 121h⟩+≡ (64a) ◁66c 123e▷

```
alts[WMouseread].c = w->mouseread;
alts[WMouseread].v = &mrm;
alts[WMouseread].op = CHANSND;
```

⟨winctl() *alts adjustments* 121i⟩≡ (64a) 123f▷

```
if(w->mouseopen && w->mouse.counter != w->mouse.lastcounter)
    alts[WMouseread].op = CHANSND;
else
    alts[WMouseread].op = CHANNOP;
```

⟨winctl() *other locals* 122a⟩+≡ (64a) ◁121e 123b▷

```
Mousestate m;
```

⟨winctl() *event loop cases* 122b⟩+≡ (64a) ◁66d 123h▷

```
case WMouseread:
    /* send a queued event or, if the queue is empty, the current state */
    /* if the queue has filled, we discard all the events it contained. */
    /* the intent is to discard frantic clicking by the user during long latencies. */
    w->mouse.qfull = false;
    // if produced more than read
    if(w->mouse.wi != w->mouse.ri) {
        m = w->mouse.queue[w->mouse.ri];
        if(++w->mouse.ri == nelem(w->mouse.queue))
            w->mouse.ri = 0;
    } else
        m = (Mousestate){w->mc.Mouse, w->mouse.counter};

    w->mouse.lastcounter = m.counter;
    // consumed and relay
    send(mrm.cm, &m.Mouse);
    continue;
```

10.3 Keyboard events

10.3.1 Raw keys queue

⟨Window *graphical window fields* 122c⟩+≡ (51) ◁52m

```
// growing_array<Rune> (size = Window.nraw)
Rune  *raw;
uint  nraw;
```

10.3.2 /mnt/wsys/cons reading part2

Producer

⟨wkeyctl() *if rawing* 122d⟩≡ (65d)

```
if(w->rawing && (w->mouseopen || w->q0 == w->nr)){
    waddraw(w, &r, 1);
    return;
}
```

⟨*function* waddraw 122e⟩≡ (253b)

```
void
waddraw(Window *w, Rune *r, int nr)
{
    w->raw = runerealloc(w->raw, w->nraw+nr);
    runemove(w->raw + w->nraw, r, nr);
    w->nraw += nr;
}
```

⟨wctlmesg() *When Rawoff, process raw keys in non rawing mode* 122f⟩≡ (120c)

```
while(w->nraw > 0){
    wkeyctl(w, w->raw[0]);
    --w->nraw;
    runemove(w->raw, w->raw+1, w->nraw);
}
```

Consumer

⟨Wxxx *cases* 123a⟩+≡ (63e) ◁121d 165h▷
```
WCread,
```

⟨winctl() *other locals* 123b⟩+≡ (64a) ◁122a 123g▷
```
Consreadmesg crm;
```

⟨winctl() *channels creation* 123c⟩+≡ (64a) ◁121f 165j▷
```
crm.c1 = chancreate(sizeof(Stringpair), 0);
crm.c2 = chancreate(sizeof(Stringpair), 0);
```

⟨winctl() *Wctl case, free channels if wctlmesg is Excited* 123d⟩+≡ (66d) ◁121g 165k▷
```
chanfree(crm.c1);
chanfree(crm.c2);
```

⟨winctl() *alts setup* 123e⟩+≡ (64a) ◁121h 165l▷
```
alts[WCread].c = w->consread;
alts[WCread].v = &crm;
alts[WCread].op = CHANSND;
```

⟨winctl() *alts adjustments* 123f⟩+≡ (64a) ◁121i 165m▷
```
⟨winctl() alts adjustments, if holding 218b⟩
else if((w->rawing && w->nraw>0) || npart)
    alts[WCread].op = CHANSND;
else{
    alts[WCread].op = CHANNOP;
    ⟨winctl() alts adjustments, revert to CHANSND if newline in queue 160a⟩
}
```

⟨winctl() *other locals* 123g⟩+≡ (64a) ◁123b 124a▷
```
Stringpair pair;
char *t;
int nb;
int i, c, wid;
```

⟨winctl() *event loop cases* 123h⟩+≡ (64a) ◁122b 166b▷
```
case WCread:
    recv(crm.c1, &pair);
    t = pair.s;
    nb = pair.ns;

    i = 0;
    ⟨winctl() when WCRead, copy in t previous partial rune bytes 124c⟩

    while(i<nb && (w->nraw > 0 || w->qh < w->nr)){

        // raw mode
        if(w->qh == w->nr){
            wid = runetochar(t+i, &w->raw[0]);
            w->nraw--;
            runemove(w->raw, w->raw+1, w->nraw);
        // buffered mode
        }else
            wid = runetochar(t+i, &w->r[w->qh++]);

        i += wid;
        ⟨winctl() when WCRead, break if newline and handle EOF character 160b⟩
        }
    }
    ⟨winctl() when WCRead, handle EOF character after while loop 160c⟩
```

⟨winctl() *when WCRead, store overflow bytes of partial rune* 124b⟩

```
pair.s = t;
pair.ns = i;
send(crm.c2, &pair);
continue;
```

Runes vs bytes, partial runes

⟨winctl() *other locals* 124a⟩+≡ (64a) ◁123g 165i▷

```
char part[3]; // UTFMAX-1
int npart = 0;
```

⟨winctl() *when WCRead, store overflow bytes of partial rune* 124b⟩≡ (123h)

```
if(i > nb){
    npart = i-nb;
    memmove(part, t+nb, npart);
    i = nb;
}
```

⟨winctl() *when WCRead, copy in t previous partial rune bytes* 124c⟩≡ (123h)

```
i = npart;
npart = 0;
if(i)
    memmove(t, part, i);
```

10.4 Resize events

⟨xfidread() *when Qmouse, adjust c for resize message if resized* 124d⟩≡ (112h)

```
if(w->resized)
    c = 'r';
```

10.5 /mnt/wsys/window

⟨qid *cases* 124e⟩+≡ (100a) ◁118b 172a▷

```
Qwindow,
```

⟨*dirtab array elements* 124f⟩+≡ (100f) ◁118c 172b▷

```
{ "window",  QTFILE, Qwindow, 0400 },
```

⟨xfidread() *other locals* 124g⟩+≡ (109a) ◁114h 177a▷

```
Image *i;
Rectangle r;
char buf[128];
char cbuf[30];
```

⟨xfidread() *cases* 124h⟩+≡ (109a) ◁118d 172c▷

```
case Qwindow:
    i = w->i;
    if(i == nil || Dx(w->screenr)<=0){
        filsysrespond(x->fs, x, &fc, Enowindow);
        return;
    }
    r = w->screenr;
    /* fall through */
```

```
caseImage:
    if(off < 5*12){
        n = sprint(buf, "%11s %11d %11d %11d %11d ",
            chantostr(cbuf, view->chan),
            i->r.min.x, i->r.min.y, i->r.max.x, i->r.max.y);
        t = estrdup(buf);
        goto Text;
    }
    t = malloc(cnt);
    fc.data = t;
    n = readwindow(i, t, r, off, cnt); /* careful; fc.count is unsigned */
    if(n < 0){
        buf[0] = '\0';
        errstr(buf, sizeof buf);
        filsysrespond(x->fs, x, &fc, buf);
    }else{
        fc.count = n;
        filsysrespond(x->fs, x, &fc, nil);
    }
    free(t);
    return;
```

⟨*function* readwindow 125⟩≡ (256)

```
int
readwindow(Image *i, char *t, Rectangle r, int offset, int n)
{
    int ww, y;

    offset -= 5*12;
    ww = bytesperline(r, view->depth);
    r.min.y += offset/ww;
    if(r.min.y >= r.max.y)
        return 0;
    y = r.min.y + n/ww;
    if(y < r.max.y)
        r.max.y = y;
    if(r.max.y <= r.min.y)
        return 0;
    return unloadimage(i, r, (uchar*)t, n);
}
```

Chapter 11

Textual Windows

11.1 Textual window creation

⟨`wmk()` *textual window settings* 126a⟩≡ (80e) 126d ▷
```
  ⟨wmk() textual window settings, set scrollbar 126b⟩
  ⟨wmk() textual window settings, set frame 126e⟩

  r = insetrect(w->i->r, Selborder);
  draw(w->i, r, cols[BACK], nil, w->frm.entire.min);
```

11.1.1 Scrollbar

⟨`wmk()` *textual window settings, set scrollbar* 126b⟩≡ (126a)
```
  r = insetrect(w->i->r, Selborder+1);
  w->scrollr = r;
  w->scrollr.max.x = r.min.x+Scrollwid;
```

⟨*constants Scrollxxx* 126c⟩≡ (237a) 126f ▷
```
  Scrollwid   = 12, /* width of scroll bar */
```

⟨`wmk()` *textual window settings* 126d⟩+≡ (80e) ◁ 126a
```
  w->scrolling = scrolling; // autoscroll
```

11.1.2 Frame

⟨`wmk()` *textual window settings, set frame* 126e⟩≡ (126a)
```
  r = insetrect(w->i->r, Selborder+1);
  r.min.x += Scrollwid+Scrollgap;
  frinit(&w->frm, r, font, i, cols);
  ⟨wmk() textual window settings, extra frame settings 150e⟩
```

⟨*constants Scrollxxx* 126f⟩+≡ (237a) ◁ 126c
```
  Scrollgap   = 4, /* gap right of scroll bar */
```

11.2 Frame widget

11.2.1 `Frame`

⟨*struct* `Frame` 126g⟩≡ (235b)
```
  struct Frame
  {
```

```
    Image *b;      /* on which frame appears */
    Rectangle r;  /* in which text appears */

    Font *font;  /* of chars in the frame */
    Display *display; /* on which frame appears */

    ⟨Frame colors 128a⟩
    ⟨Frame text fields 127a⟩
    ⟨Frame tick fields 128j⟩
    ⟨Frame box fields 130a⟩
    ⟨Frame scroll 168e⟩

    ⟨Frame other fields 127f⟩
};
```

⟨Frame *text fields* 127a⟩≡ (126g) 127b ▷

```
ushort nchars;  /* # runes in frame */
```

⟨Frame *text fields* 127b⟩+≡ (126g) ◁ 127a 127e ▷

```
ushort nlines;  /* # lines with text */
```

11.2.2 frinit()

⟨*function* frinit 127c⟩≡ (281a)

```
void
frinit(Frame *f, Rectangle r, Font *ft, Image *b, Image *cols[NCOL])
{
    f->font = ft;
    f->display = b->display;
    frsetrects(f, r, b);

    f->nchars = 0;
    f->nlines = 0;
    ⟨frinit() initialize other fields 128d⟩
}
```

⟨*function* frsetrects 127d⟩≡ (281a)

```
void
frsetrects(Frame *f, Rectangle r, Image *b)
{
    f->b = b;
    f->entire = r;
    f->r = r;
    f->r.max.y -= (r.max.y-r.min.y) % f->font->height;
    f->maxlines = (r.max.y-r.min.y) / f->font->height;
}
```

⟨Frame *text fields* 127e⟩+≡ (126g) ◁ 127b 136d ▷

```
ushort maxlines; /* total # lines in frame */
```

⟨Frame *other fields* 127f⟩≡ (126g)

```
Rectangle entire;  /* of full frame */
```

⟨*function* frclear 127g⟩≡ (281a)

```
void
frclear(Frame *f, bool freeall)
{
    ⟨frclear() free boxes 131d⟩
    ⟨frclear() free ticks 129i⟩
}
```

11.2.3 Frame colors

⟨Frame *colors* 128a⟩≡ (126g)
```
Image *cols[NCOL]; /* text and background colors */
```

⟨*enum* _anon_ (include/frame.h) 128b⟩≡ (235b)
```
enum FrameColors {
    BACK, // Background
    HIGH, // Background highlighted text
    BORD, // Border
    TEXT, // Text
    HTEXT, // Highlited text

    NCOL
};
```

⟨*global* cols 128c⟩≡ (249)
```
// map<Property, Color>
static Image *cols[NCOL];
```

⟨frinit() *initialize other fields* 128d⟩≡ (127c) 129b ▷
```
if(cols != nil)
    memmove(f->cols, cols, sizeof f->cols);
```

⟨wmk() *colors initialisation* 128e⟩≡ (80e)
```
if(cols[0] == nil){
    cols[BACK] = display->white;
    cols[HIGH] = allocimage(display, Rect(0,0,1,1), CMAP8, true, 0xCCCCCCFF);
    cols[BORD] = allocimage(display, Rect(0,0,1,1), CMAP8, true, 0x999999FF);
    cols[TEXT] = display->black;
    cols[HTEXT] = display->black;
    ⟨wmk() extra colors initialisation 81e⟩
);
}
```

⟨wrepaint() *update cols* 128f⟩≡ (87d)
```
wsetcols(w);
```

⟨*function* wsetcols 128g⟩≡ (249)
```
void
wsetcols(Window *w)
{
    ⟨wsetcols() if holding 218e⟩
    else
        if(w == input)
            w->frm.cols[TEXT] = w->frm.cols[HTEXT] = display->black;
        else
            w->frm.cols[TEXT] = w->frm.cols[HTEXT] = darkgrey;
}
```

⟨*global* darkgrey 128h⟩≡ (249)
```
static Image *darkgrey;
```

⟨wmk() *extra colors initialisation* 128i⟩+≡ (128e) ◁ 81e 218i ▷
```
/* greys are multiples of 0x11111100+0xFF, 14* being palest */
darkgrey = allocimage(display, Rect(0,0,1,1), CMAP8, true, 0x666666FF);
```

11.2.4 Frame tick

⟨Frame *tick fields* 128j⟩≡ (126g) 129a ▷
```
Image *tick; /* typing tick */
Image *tickback; /* saved image under tick */
```

⟨Frame *tick fields* 129a⟩+≡ (126g) ◁128j

```
bool ticked; /* flag: is tick onscreen? */
```

⟨frinit() *initialize other fields* 129b⟩+≡ (127c) ◁128d 130b▷

```
if(f->tick == nil && f->cols[BACK] != nil)
    frinittick(f);
```

⟨*constant* FRTICKW 129c⟩≡ (235b)

```
#define FRTICKW 3
```

⟨*function* frinittick 129d⟩≡ (281a)

```
void
frinittick(Frame *f)
{
    Image *b = f->display->screenimage;
    Font *ft = f->font;

    ⟨frinittick() free old tick 129e⟩
    f->tick = allocimage(f->display, Rect(0, 0, FRTICKW, ft->height), b->chan, 0, DWhite);
    ⟨frinittick() sanity check tick 129f⟩
    ⟨frinittick() free old tickback 129g⟩
    f->tickback = allocimage(f->display, f->tick->r, b->chan, false, DWhite);
    ⟨frinittick() sanity check tickback 129h⟩
    /* background color */
    draw(f->tick, f->tick->r, f->cols[BACK], nil, ZP);
    /* vertical line */
    draw(f->tick, Rect(FRTICKW/2, 0, FRTICKW/2+1, ft->height), f->cols[TEXT], nil, ZP);
    /* box on each end */
    draw(f->tick, Rect(0, 0, FRTICKW, FRTICKW), f->cols[TEXT], nil, ZP);
    draw(f->tick, Rect(0, ft->height-FRTICKW, FRTICKW, ft->height), f->cols[TEXT], nil, ZP);
}
```

⟨frinittick() *free old tick* 129e⟩≡ (129d)

```
if(f->tick)
    freeimage(f->tick);
```

⟨frinittick() *sanity check tick* 129f⟩≡ (129d)

```
if(f->tick == nil)
    return;
```

⟨frinittick() *free old tickback* 129g⟩≡ (129d)

```
if(f->tickback)
    freeimage(f->tickback);
```

⟨frinittick() *sanity check tickback* 129h⟩≡ (129d)

```
if(f->tickback == nil){
    freeimage(f->tick);
    f->tick = nil;
    return;
}
```

⟨frclear() *free ticks* 129i⟩≡ (127g)

```
if(freeall){
    freeimage(f->tick);
    freeimage(f->tickback);
    f->tick = nil;
    f->tickback = nil;
}
f->ticked = false;
```

11.2.5 Frame boxes

⟨Frame *box fields* 130a⟩≡ (126g)

```
// growing_array<Frbox> (size = nalloc, unused after nbox)
Frbox *box;
ushort nbox;
ushort nalloc;
```

⟨frinit() *initialize other fields* 130b⟩+≡ (127c) ◁129b 136e▷

```
f->box = nil;
f->nbox = 0;
f->nalloc = 0;
```

Frbox

⟨*struct* Frbox 130c⟩≡ (235b)

```
struct Frbox
{
    long  wid;  /* in pixels */

    long  nrune;  /* <0 ==> negate and treat as break char */
    union{
        // array<byte> UTF8?
        uchar *ptr;
        struct{
            short bc; /* break char */
            short minwid;
        };
    };
};
```

⟨*function* NRUNE 130d⟩≡ (235b)

```
#define NRUNE(b) ((b)->nrune < 0 ? 1 : (b)->nrune)
```

Adding boxes

⟨*constant* SLOP 130e⟩≡ (279b)

```
#define SLOP 25
```

⟨*function* _fraddbox 130f⟩≡ (279b)

```
void
_fraddbox(Frame *f, int bn, int n) /* add n boxes after bn, shift the rest up,
                 * box[bn+n]==box[bn] */
{
    int i;

    ⟨_fraddbox() sanity check bn 131b⟩
    if(f->nbox+n > f->nalloc)
        _frgrowbox(f, n+SLOP);
    for(i=f->nbox; --i>=bn; )
        f->box[i+n] = f->box[i];
    f->nbox+=n;
}
```

⟨*function* _frgrowbox 131a⟩≡ (279b)

```
void
_frgrowbox(Frame *f, int delta)
{
    f->nalloc += delta;
    f->box = realloc(f->box, f->nalloc*sizeof(Frbox));
    ⟨_frgrowbox() sanity check box 131c⟩
}
```

⟨_fraddbox() *sanity check bn* 131b⟩≡ (130f)

```
if(bn > f->nbox)
    drawerror(f->display, "_fraddbox");
```

⟨_frgrowbox() *sanity check box* 131c⟩≡ (131a)

```
if(f->box == nil)
    drawerror(f->display, "_frgrowbox");
```

Free boxes

⟨frclear() *free boxes* 131d⟩≡ (127g)

```
if(f->nbox)
    _frdelbox(f, 0, f->nbox-1);
if(f->box)
    free(f->box);
f->box = nil;
```

⟨*function* _frdelbox 131e⟩≡ (279b)

```
void
_frdelbox(Frame *f, int n0, int n1) /* inclusive */
{
    ⟨_frdelbox() sanity check n0 and n1 132a⟩
    _frfreebox(f, n0, n1);
    _frclosebox(f, n0, n1);
}
```

⟨*function* _frfreebox 131f⟩≡ (279b)

```
void
_frfreebox(Frame *f, int n0, int n1) /* inclusive */
{
    int i;

    ⟨_frfreebox() sanity check n0 and n1 132b⟩
    n1++;
    for(i=n0; i<n1; i++)
        if(f->box[i].nrune >= 0)
            free(f->box[i].ptr);
}
```

⟨*function* _frclosebox 131g⟩≡ (279b)

```
void
_frclosebox(Frame *f, int n0, int n1) /* inclusive */
{
    int i;

    ⟨_frclosebox() sanity check n0 and n1 132c⟩
    n1++;
    for(i=n1; i<f->nbox; i++)
        f->box[i-(n1-n0)] = f->box[i];
    f->nbox -= n1-n0;
}
```

⟨_frdelbox() *sanity check n0 and n1* 132a⟩≡ (131e)

```
if(n0>=f->nbox || n1>=f->nbox || n1<n0)
    drawerror(f->display, "_frdelbox");
```

⟨_frfreebox() *sanity check n0 and n1* 132b⟩≡ (131f)

```
if(n1<n0)
    return;
if(n0>=f->nbox || n1>=f->nbox)
    drawerror(f->display, "_frfreebox");
```

⟨_frclosebox() *sanity check n0 and n1* 132c⟩≡ (131g)

```
if(n0>=f->nbox || n1>=f->nbox || n1<n0)
    drawerror(f->display, "_frclosebox");
```

11.2.6 Frame strings

⟨*constant CHUNK (windows/libframe/frstr.c)* 132d⟩≡ (282b)

```
#define CHUNK 16
```

⟨*function* ROUNDUP 132e⟩≡ (282b)

```
#define ROUNDUP(n) ((n+CHUNK)&~(CHUNK-1))
```

⟨*function* _frallocstr 132f⟩≡ (282b)

```
uchar *
_frallocstr(Frame *f, unsigned n)
{
    uchar *p;

    p = malloc(ROUNDUP(n));
    if(p == nil)
        drawerror(f->display, "out of memory");
    return p;
}
```

⟨*function* _frinsure 132g⟩≡ (282b)

```
void
_frinsure(Frame *f, int bn, unsigned n)
{
    Frbox *b;
    uchar *p;

    b = &f->box[bn];
    if(b->nrune < 0)
        drawerror(f->display, "_frinsure");
    if(ROUNDUP(b->nrune) > n) /* > guarantees room for terminal NUL */
        return;
    p = _frallocstr(f, n);
    b = &f->box[bn];
    memmove(p, b->ptr, NBYTE(b)+1);
    free(b->ptr);
    b->ptr = p;
}
```

11.2.7 Frame rune position, point, and box number

frptofchar()

⟨*function* frptofchar 133a⟩≡ (281c)

```
Point
frptofchar(Frame *f, ulong p)
{
    return _frptofcharptb(f, p, f->r.min, 0);
}
```

⟨*function* _frptofcharptb 133b⟩≡ (281c)

```
Point
_frptofcharptb(Frame *f, ulong p, Point pt, int bn)
{
    Frbox *b;
    uchar *s;
    int w, l;
    Rune r;

    for(b = &f->box[bn]; bn<f->nbox; bn++,b++){
        _frcklinewrap(f, &pt, b);
        l=NRUNE(b);
        // p is in this box
        if(p < l){
            if(b->nrune > 0)
                for(s=b->ptr; p>0; s+=w, p--){
                    r = *s;
                    if(r < Runeself)
                        w = 1;
                    else
                        w = chartorune(&r, (char*)s);
                    pt.x += stringnwidth(f->font, (char*)s, 1);
                    ⟨_frptofcharptb() sanity check r and pt 133d⟩
                }
            break; // found it
        }
        // else
        p -= l;
        _fradvance(f, &pt, b);
    }
    return pt;
}
```

⟨*function* _fradvance 133c⟩≡ (282c)

```
void
_fradvance(Frame *f, Point *p, Frbox *b)
{
    if(b->nrune<0 && b->bc=='\n'){
        p->x = f->r.min.x;
        p->y += f->font->height;
    }else
        p->x += b->wid;
}
```

⟨_frptofcharptb() *sanity check r and pt* 133d⟩≡ (133b)

```
if(r==0 || pt.x > f->r.max.x)
    drawerror(f->display, "frptofchar");
```

⟨*function* _frcklinewrap 134a⟩≡ (282c)

```
void
_frcklinewrap(Frame *f, Point *p, Frbox *b)
{
    if((b->nrune<0? b->minwid : b->wid) > f->r.max.x - p->x){
        p->x = f->r.min.x;
        p->y += f->font->height;
    }
}
```

⟨*function* _frcklinewrap0 134b⟩≡ (282c)

```
void
_frcklinewrap0(Frame *f, Point *p, Frbox *b)
{
    if(_frcanfit(f, *p, b) == 0){
        p->x = f->r.min.x;
        p->y += f->font->height;
    }
}
```

⟨*function* _frcanfit 134c⟩≡ (282c)

```
int
_frcanfit(Frame *f, Point pt, Frbox *b)
{
    int left, w, nr;
    uchar *p;
    Rune r;

    left = f->r.max.x - pt.x;
    if(b->nrune < 0)
        return b->minwid <= left;
    if(left >= b->wid)
        return b->nrune;
    for(nr=0,p=b->ptr; *p; p+=w,nr++){
        r = *p;
        if(r < Runeself)
            w = 1;
        else
            w = chartorune(&r, (char*)p);
        left -= stringnwidth(f->font, (char*)p, 1);
        if(left < 0)
            return nr;
    }
    drawerror(f->display, "_frcanfit can't");
    return 0;
}
```

⟨*function* _frptofcharnb 134d⟩≡ (281c)

```
Point
_frptofcharnb(Frame *f, ulong p, int nb) /* doesn't do final _fradvance to next line */
{
    Point pt;
    int nbox;

    // save
    nbox = f->nbox;
    f->nbox = nb;
    pt = _frptofcharptb(f, p, f->r.min, 0);
    // restore
    f->nbox = nbox;
```

```
    return pt;
}
```

frcharofpt()

⟨*function* frcharofpt 135⟩≡ (281c)

```
ulong
frcharofpt(Frame *f, Point pt)
{
    Point qt;
    int w;
    uchar *s;
    Frbox *b;
    int bn;
    ulong p;
    Rune r;

    pt = _frgrid(f, pt);

    qt = f->r.min;
    // find the line
    for(b=f->box, bn=0, p=0; bn < f->nbox && qt.y < pt.y; bn++,b++){
        _frcklinewrap(f, &qt, b);
        if(qt.y >= pt.y)
            break;
        _fradvance(f, &qt, b);
        p += NRUNE(b);
    }
    // find the box in the line
    for(; bn<f->nbox && qt.x<=pt.x; bn++,b++){
        _frcklinewrap(f, &qt, b);
        if(qt.y > pt.y)
            break;
        if(qt.x + b->wid > pt.x){
            if(b->nrune < 0)
                _fradvance(f, &qt, b);
            else{
                s = b->ptr;
                for(;;){
                    if((r = *s) < Runeself)
                        w = 1;
                    else
                        w = chartorune(&r, (char*)s);
                    ⟨frcharofpt() sanity check r 136b⟩
                    qt.x += stringnwidth(f->font, (char*)s, 1);
                    s += w;
                    if(qt.x > pt.x)
                        break;
                    p++;
                }
            }
        }else{
            p += NRUNE(b);
            _fradvance(f, &qt, b);
        }
    }
    return p;
}
```

⟨*function* _frgrid 136a⟩≡ (281c)

```
static
Point
_frgrid(Frame *f, Point p)
{
    p.y -= f->r.min.y;
    p.y -= p.y%f->font->height;
    p.y += f->r.min.y;
    if(p.x > f->r.max.x)
        p.x = f->r.max.x;
    return p;
}
```

⟨frcharofpt() *sanity check r* 136b⟩≡ (135)

```
if(r == 0)
    drawerror(f->display, "end of string in frcharofpt");
```

_frdrawtext()

⟨*function* _frdrawtext 136c⟩≡ (280b)

```
void
_frdrawtext(Frame *f, Point pt, Image *text, Image *back)
{
    Frbox *b;
    int nb;
    static int x;

    for(nb=0,b=f->box; nb<f->nbox; nb++, b++){
        _frcklinewrap(f, &pt, b);
        if(b->nrune >= 0){
            stringbg(f->b, pt, text, ZP, f->font, (char*)b->ptr, back, ZP);
        }
        pt.x += b->wid;
    }
}
```

11.2.8 Frame selection

⟨Frame *text fields* 136d⟩+≡ (126g) ◁127e 142b▷

```
ulong p0, p1;  /* selection */
```

⟨frinit() *initialize other fields* 136e⟩+≡ (127c) ◁130b 155b▷

```
f->p0 = 0;
f->p1 = 0;
```

11.3 Content modification

⟨*simplified code when entered a single rune* r *in a terminal* 136f⟩≡

```
winsert(w, &r, 1, w->q0);
wshow(w, w->q0);
```

11.3.1 winsert()

⟨*function* winsert 137a⟩≡ (250b)

```
uint
winsert(Window *w, Rune *r, int n, uint q0)
{
    uint m;

    ⟨winsert() sanity check n 137c⟩
    ⟨winsert() if size of rune array is getting really big 138a⟩
    ⟨winsert() grow rune array if reach maxr 137e⟩

    // move to the right the runes after the cursor q0 to make some space
    runemove(w->r + q0 + n, w->r + q0, w->nr - q0);
    // fill the space
    runemove(w->r + q0, r, n);
    w->nr += n;

    ⟨winsert() adjust cursors 137b⟩

    return q0;
}
```

⟨winsert() *adjust cursors* 137b⟩≡ (137a)

```
/* if output touches, advance selection, not qh; works best for keyboard and output */
if(q0 <= w->q0)
    w->q0 += n; // move the q0 cursor
if(q0 <= w->q1)
    w->q1 += n;
if(q0 < w->qh)
    w->qh += n;

if(q0 < w->org)
    w->org += n;
else
   ⟨winsert() when q0 >= w->org, possibly update visible text 138b⟩
```

⟨winsert() *sanity check n* 137c⟩≡ (137a)

```
if(n == 0)
    return q0;
```

11.3.2 Growing array

⟨*enum* _anon_ (windows/rio/wind.c) 137d⟩≡ (250b)

```
enum
{
    HiWater = 640000, /* max size of history */
    LoWater = 400000, /* min size of history after max'ed */
    MinWater = 20000, /* room to leave available when reallocating */
};
```

⟨winsert() *grow rune array if reach maxr* 137e⟩≡ (137a)

```
if(w->nr+n > w->maxr){
    /*
     * Minimize realloc breakage:
     * Allocate at least MinWater
     *  Double allocation size each time
     * But don't go much above HiWater
     */
```

```
    m = max(min(2*(w->nr+n), HiWater), w->nr+n)+MinWater;
    if(m > HiWater)
        m = max(HiWater+MinWater, w->nr+n);
    if(m > w->maxr){
        w->r = runerealloc(w->r, m);
        w->maxr = m;
    }
}
```

⟨winsert() *if size of rune array is getting really big* 138a⟩≡ (137a)

```
if(w->nr + n > HiWater && q0 >= w->org && q0 >= w->qh){
    m = min(HiWater-LoWater, min(w->org, w->qh));
    w->org -= m;
    w->qh -= m;
    if(w->q0 > m)
        w->q0 -= m;
    else
        w->q0 = 0;
    if(w->q1 > m)
        w->q1 -= m;
    else
        w->q1 = 0;
    w->nr -= m;
    runemove(w->r, w->r+m, w->nr);
    q0 -= m;
}
```

11.4 Content rendering

11.4.1 frinsert()

⟨winsert() *when* q0 >= w->org, *possibly update visible text* 138b⟩≡ (137b)

```
if(q0 <= w->org + w->frm.nchars)
    frinsert(&w->frm, r, r+n, q0 - w->org); // echo back
```

⟨*global* frame 138c⟩≡ (281b)

```
static Frame  frame;
```

⟨*struct* points_frinsert 138d⟩≡ (281b)

```
struct points_frinsert {
    Point pt0, pt1;
};
```

⟨frinsert() *locals* 138e⟩≡ (139)

```
Point pt0, pt1, opt0, ppt0, ppt1, pt;
Frbox *b;
int n, n0, nn0, y;
ulong cn0;
Image *col;
Rectangle r;
static struct points_frinsert *pts;
static int nalloc=0;
int npts;
```

⟨*constant* DELTA 138f⟩≡ (281b)

```
#define DELTA 25
```

⟨*function* frinsert 139⟩≡ (281b)

```
void
frinsert(Frame *f, Rune *sp, Rune *ep, ulong p0)
{
    ⟨frinsert() locals 138e⟩

    if(p0>f->nchars || sp==ep || f->b==nil)
        return;

    n0 = _frfindbox(f, 0, 0, p0);
    cn0 = p0;
    nn0 = n0;
    pt0 = _frptofcharnb(f, p0, n0);
    ppt0 = pt0;
    opt0 = pt0;
    pt1 = bxscan(f, sp, ep, &ppt0);
    ppt1 = pt1;

    if(n0 < f->nbox){
        b = &f->box[n0];
        _frcklinewrap(f, &pt0, b); /* for frdrawsel() */
        _frcklinewrap0(f, &ppt1, b);
    }
    f->modified = true;
    /*
     * ppt0 and ppt1 are start and end of insertion as they will appear when
     * insertion is complete. pt0 is current location of insertion position
     * (p0); pt1 is terminal point (without line wrap) of insertion.
     */

    ⟨frinsert() remove tick 151f⟩

    /*
     * Find point where old and new x's line up
     * Invariants:
     * pt0 is where the next box (b, n0) is now
     * pt1 is where it will be after the insertion
     * If pt1 goes off the rectangle, we can toss everything from there on
     */
    for(b = &f->box[n0],npts=0;
         pt1.x!=pt0.x && pt1.y!=f->r.max.y && n0<f->nbox; b++,n0++,npts++){
        _frcklinewrap(f, &pt0, b);
        _frcklinewrap0(f, &pt1, b);
        if(b->nrune > 0){
            n = _frcanfit(f, pt1, b);
            ⟨frinsert() sanity check n canfit 142a⟩
            if(n != b->nrune){
                _frsplitbox(f, n0, n);
                b = &f->box[n0];
            }
        }
        if(npts == nalloc){
            pts = realloc(pts, (npts+DELTA)*sizeof(pts[0]));
            nalloc += DELTA;
            b = &f->box[n0];
        }
        pts[npts].pt0 = pt0;
        pts[npts].pt1 = pt1;
        /* has a text box overflowed off the frame? */
        if(pt1.y == f->r.max.y)
```

```
        break;
    _fradvance(f, &pt0, b);
    pt1.x += _frnewwid(f, pt1, b);
    cn0 += NRUNE(b);
}
if(pt1.y > f->r.max.y)
    drawerror(f->display, "frinsert pt1 too far");
if(pt1.y==f->r.max.y && n0<f->nbox){
    f->nchars -= _frstrlen(f, n0);
    _frdelbox(f, n0, f->nbox-1);
}
if(n0 == f->nbox)
    f->nlines = (pt1.y-f->r.min.y)/f->font->height+(pt1.x>f->r.min.x);
else if(pt1.y!=pt0.y){
    int q0, q1;

    y = f->r.max.y;
    q0 = pt0.y+f->font->height;
    q1 = pt1.y+f->font->height;
    f->nlines += (q1-q0)/f->font->height;
    if(f->nlines > f->maxlines)
        chopframe(f, ppt1, p0, nn0);
    if(pt1.y < y){
        r = f->r;
        r.min.y = q1;
        r.max.y = y;
        if(q1 < y)
            draw(f->b, r, f->b, nil, Pt(f->r.min.x, q0));
        r.min = pt1;
        r.max.x = pt1.x+(f->r.max.x-pt0.x);
        r.max.y = q1;
        draw(f->b, r, f->b, nil, pt0);
    }
}
/*
 * Move the old stuff down to make room.  The loop will move the stuff
 * between the insertion and the point where the x's lined up.
 * The draw()s above moved everything down after the point they lined up.
 */
for((y=pt1.y==f->r.max.y?pt1.y:0),b = &f->box[n0-1]; --npts>=0; --b){
    pt = pts[npts].pt1;
    if(b->nrune > 0){
        r.min = pt;
        r.max = r.min;
        r.max.x += b->wid;
        r.max.y += f->font->height;
        draw(f->b, r, f->b, nil, pts[npts].pt0);
        /* clear bit hanging off right */
        if(npts==0 && pt.y>pt0.y){
            /*
             * first new char is bigger than first char we're
             * displacing, causing line wrap. ugly special case.
             */
            r.min = opt0;
            r.max = opt0;
            r.max.x = f->r.max.x;
            r.max.y += f->font->height;
            if(f->p0<=cn0 && cn0<f->p1) /* b+1 is inside selection */
                col = f->cols[HIGH];
            else
```

```
                col = f->cols[BACK];
            draw(f->b, r, col, nil, r.min);
        }else if(pt.y < y){
            r.min = pt;
            r.max = pt;
            r.min.x += b->wid;
            r.max.x = f->r.max.x;
            r.max.y += f->font->height;
            if(f->p0<=cn0 && cn0<f->p1) /* b+1 is inside selection */
                col = f->cols[HIGH];
            else
                col = f->cols[BACK];
            draw(f->b, r, col, nil, r.min);
        }
        y = pt.y;
        cn0 -= b->nrune;
    }else{
        r.min = pt;
        r.max = pt;
        r.max.x += b->wid;
        r.max.y += f->font->height;
        if(r.max.x >= f->r.max.x)
            r.max.x = f->r.max.x;
        cn0--;
        if(f->p0<=cn0 && cn0<f->p1) /* b is inside selection */
            col = f->cols[HIGH];
        else
            col = f->cols[BACK];
        draw(f->b, r, col, nil, r.min);
        y = 0;
        if(pt.x == f->r.min.x)
            y = pt.y;
    }
}
/* insertion can extend the selection, so the condition here is different */
if(f->p0<p0 && p0<=f->p1)
    col = f->cols[HIGH];
else
    col = f->cols[BACK];

frselectpaint(f, ppt0, ppt1, col);

_frdrawtext(&frame, ppt0, f->cols[TEXT], col);

_fraddbox(f, nn0, frame.nbox);
for(n=0; n<frame.nbox; n++)
    f->box[nn0+n] = frame.box[n];
if(nn0>0 && f->box[nn0-1].nrune>=0 && ppt0.x-f->box[nn0-1].wid>=f->r.min.x){
    --nn0;
    ppt0.x -= f->box[nn0].wid;
}
n0 += frame.nbox;
_frclean(f, ppt0, nn0, n0<f->nbox-1? n0+1 : n0);

f->nchars += frame.nchars;
if(f->p0 >= p0)
    f->p0 += frame.nchars;
if(f->p0 > f->nchars)
    f->p0 = f->nchars;
if(f->p1 >= p0)
```

```
        f->p1 += frame.nchars;
    if(f->p1 > f->nchars)
        f->p1 = f->nchars;

    ⟨frinsert() draw tick 151g⟩
}
```

⟨frinsert() *sanity check n canfit* 142a⟩≡ (139)

```
if(n == 0)
    drawerror(f->display, "_frcanfit==0");
```

⟨Frame *text fields* 142b⟩+≡ (126g) ◁136d 155a▷

```
bool modified; /* changed since frselect() */
```

_frfindbox()

⟨*function* _frfindbox 142c⟩≡ (279b)

```
/* find box containing q and put q on a box boundary */
int
_frfindbox(Frame *f, int bn, ulong p, ulong q)
{
    Frbox *b;

    for(b = &f->box[bn]; bn < f->nbox && p+NRUNE(b) <= q; bn++, b++)
        p += NRUNE(b);
    if(p != q)
        _frsplitbox(f, bn++, (int)(q-p));
    return bn;
}
```

⟨*function* _frsplitbox 142d⟩≡ (279b)

```
void
_frsplitbox(Frame *f, int bn, int n)
{
    dupbox(f, bn);
    truncatebox(f, &f->box[bn], f->box[bn].nrune-n);
    chopbox(f, &f->box[bn+1], n);
}
```

⟨*function* dupbox 142e⟩≡ (279b)

```
static
void
dupbox(Frame *f, int bn)
{
    uchar *p;

    if(f->box[bn].nrune < 0)
        drawerror(f->display, "dupbox");
    _fraddbox(f, bn, 1);
    if(f->box[bn].nrune >= 0){
        p = _frallocstr(f, NBYTE(&f->box[bn])+1);
        strcpy((char*)p, (char*)f->box[bn].ptr);
        f->box[bn+1].ptr = p;
    }
}
```

⟨*function* NBYTE 142f⟩≡ (235b)

```
#define NBYTE(b) strlen((char*)(b)->ptr)
```

⟨*function* truncatebox 143a⟩≡ (279b)

```
static
void
truncatebox(Frame *f, Frbox *b, int n) /* drop last n chars; no allocation done */
{
    if(b->nrune<0 || b->nrune<n)
        drawerror(f->display, "truncatebox");
    b->nrune -= n;
    runeindex(b->ptr, b->nrune)[0] = 0;
    b->wid = stringwidth(f->font, (char *)b->ptr);
}
```

⟨*function* chopbox 143b⟩≡ (279b)

```
static
void
chopbox(Frame *f, Frbox *b, int n) /* drop first n chars; no allocation done */
{
    char *p;

    if(b->nrune<0 || b->nrune<n)
        drawerror(f->display, "chopbox");
    p = (char*)runeindex(b->ptr, n);
    memmove((char*)b->ptr, p, strlen(p)+1);
    b->nrune -= n;
    b->wid = stringwidth(f->font, (char *)b->ptr);
}
```

⟨*function* runeindex 143c⟩≡ (279b)

```
static
uchar*
runeindex(uchar *p, int n)
{
    int i, w;
    Rune rune;

    for(i=0; i<n; i++,p+=w)
        if(*p < Runeself)
            w = 1;
        else{
            w = chartorune(&rune, (char*)p);
            USED(rune);
        }
    return p;
}
```

bxscan()

⟨*constant* TMPSIZE 143d⟩≡ (281b)

```
#define TMPSIZE 256
```

⟨*function* bxscan 143e⟩≡ (281b)

```
static
Point
bxscan(Frame *f, Rune *sp, Rune *ep, Point *ppt)
{
    int w, c, nb, delta, nl, nr, rw;
    Frbox *b;
    char *s, tmp[TMPSIZE+3]; /* +3 for rune overflow */
    uchar *p;
```

```
    frame.r = f->r;
    frame.b = f->b;
    frame.font = f->font;
    frame.maxtab = f->maxtab;
    frame.nbox = 0;
    frame.nchars = 0;
    memmove(frame.cols, f->cols, sizeof frame.cols);
    delta = DELTA;
    nl = 0;
    for(nb=0; sp<ep && nl<=f->maxlines; nb++,frame.nbox++){
        if(nb == frame.nalloc){
            _frgrowbox(&frame, delta);
            if(delta < 10000)
                delta *= 2;
        }
        b = &frame.box[nb];
        c = *sp;
        if(c=='\t' || c=='\n'){
            b->bc = c;
            b->wid = 5000;
            b->minwid = (c=='\n')? 0 : stringwidth(frame.font, " ");
            b->nrune = -1;
            if(c=='\n')
                nl++;
            frame.nchars++;
            sp++;
        }else{
            s = tmp;
            nr = 0;
            w = 0;
            while(sp < ep){
                c = *sp;
                if(c=='\t' || c=='\n')
                    break;
                rw = runetochar(s, sp);
                if(s+rw >= tmp+TMPSIZE)
                    break;
                w += runestringnwidth(frame.font, sp, 1);
                sp++;
                s += rw;
                nr++;
            }
            *s++ = 0;
            p = _frallocstr(f, s-tmp);
            b = &frame.box[nb];
            b->ptr = p;
            memmove(p, tmp, s-tmp);
            b->wid = w;
            b->nrune = nr;
            frame.nchars += nr;
        }
    }
    _frcklinewrap0(f, ppt, &frame.box[0]);
    return _frdraw(&frame, *ppt);
}
```

⟨*function* _frdraw 144⟩≡ (280b)

```
Point
_frdraw(Frame *f, Point pt)
```

```
{
    Frbox *b;
    int nb, n;

    for(b=f->box,nb=0; nb<f->nbox; nb++, b++){
        _frcklinewrap0(f, &pt, b);
        if(pt.y == f->r.max.y){
            f->nchars -= _frstrlen(f, nb);
            _frdelbox(f, nb, f->nbox-1);
            break;
        }
        if(b->nrune > 0){
            n = _frcanfit(f, pt, b);
            if(n == 0)
                drawerror(f->display, "_frcanfit==0");
            if(n != b->nrune){
                _frsplitbox(f, nb, n);
                b = &f->box[nb];
            }
            pt.x += b->wid;
        }else{
            if(b->bc == '\n'){
                pt.x = f->r.min.x;
                pt.y+=f->font->height;
            }else
                pt.x += _frnewwid(f, pt, b);
        }
    }
    return pt;
}
```

⟨*function* _frnewwid 145a⟩≡ (282c)

```
int
_frnewwid(Frame *f, Point pt, Frbox *b)
{
    b->wid = _frnewwid0(f, pt, b);
    return b->wid;
}
```

⟨*function* _frnewwid0 145b⟩≡ (282c)

```
int
_frnewwid0(Frame *f, Point pt, Frbox *b)
{
    int c, x;

    c = f->r.max.x;
    x = pt.x;
    if(b->nrune>=0 || b->bc!='\t')
        return b->wid;
    if(x+b->minwid > c)
        x = pt.x = f->r.min.x;
    x += f->maxtab;
    x -= (x-f->r.min.x)%f->maxtab;
    if(x-pt.x<b->minwid || x>c)
        x = pt.x+b->minwid;
    return x-pt.x;
}
```

⟨*function* _frstrlen 145c⟩≡ (280b)

```
int
```

```
_frstrlen(Frame *f, int nb)
{
    int n;

    for(n=0; nb<f->nbox; nb++)
        n += NRUNE(&f->box[nb]);
    return n;
}
```

_chopframe()

⟨*function* chopframe 146a⟩≡ (281b)

```
static
void
chopframe(Frame *f, Point pt, ulong p, int bn)
{
    Frbox *b;

    for(b = &f->box[bn]; ; b++){
        if(b >= &f->box[f->nbox])
            drawerror(f->display, "endofframe");
        _frcklinewrap(f, &pt, b);
        if(pt.y >= f->r.max.y)
            break;
        p += NRUNE(b);
        _fradvance(f, &pt, b);
    }
    f->nchars = p;
    f->nlines = f->maxlines;
    if(b<&f->box[f->nbox])    /* BUG */
        _frdelbox(f, (int)(b-f->box), f->nbox-1);
}
```

_frclean()

⟨*function* _frclean 146b⟩≡ (282c)

```
void
_frclean(Frame *f, Point pt, int n0, int n1) /* look for mergeable boxes */
{
    Frbox *b;
    int nb, c;

    c = f->r.max.x;
    for(nb=n0; nb<n1-1; nb++){
        b = &f->box[nb];
        _frcklinewrap(f, &pt, b);
        while(b[0].nrune>=0 && nb<n1-1 && b[1].nrune>=0 && pt.x+b[0].wid+b[1].wid<c){
            _frmergebox(f, nb);
            n1--;
            b = &f->box[nb];
        }
        _fradvance(f, &pt, &f->box[nb]);
    }
    for(; nb<f->nbox; nb++){
        b = &f->box[nb];
        _frcklinewrap(f, &pt, b);
        _fradvance(f, &pt, &f->box[nb]);
    }
```

```
    f->lastlinefull = 0;
    if(pt.y >= f->r.max.y)
        f->lastlinefull = 1;
}
```

⟨*function* _frmergebox 147a⟩≡ (279b)

```
void
_frmergebox(Frame *f, int bn)  /* merge bn and bn+1 */
{
    Frbox *b;

    b = &f->box[bn];
    _frinsure(f, bn, NBYTE(&b[0])+NBYTE(&b[1])+1);
    strcpy((char*)runeindex(b[0].ptr, b[0].nrune), (char*)b[1].ptr);
    b[0].wid += b[1].wid;
    b[0].nrune += b[1].nrune;
    _frdelbox(f, bn+1, bn+1);
}
```

11.4.2 frdelete()

⟨*function* frdelete 147b⟩≡ (280a)

```
int
frdelete(Frame *f, ulong p0, ulong p1)
{
    Point pt0, pt1, ppt0;
    Frbox *b;
    int n0, n1, n;
    ulong cn1;
    Rectangle r;
    int nn0;
    Image *col;

    if(p0>=f->nchars || p0==p1 || f->b==nil)
        return 0;
    if(p1 > f->nchars)
        p1 = f->nchars;
    n0 = _frfindbox(f, 0, 0, p0);
    if(n0 == f->nbox)
        drawerror(f->display, "off end in frdelete");
    n1 = _frfindbox(f, n0, p0, p1);
    pt0 = _frptofcharnb(f, p0, n0);
    pt1 = frptofchar(f, p1);
    if(f->p0 == f->p1)
        frtick(f, frptofchar(f, f->p0), 0);
    nn0 = n0;
    ppt0 = pt0;
    _frfreebox(f, n0, n1-1);
    f->modified = 1;

    /*
     * Invariants:
     *  - pt0 points to beginning, pt1 points to end
     *  - n0 is box containing beginning of stuff being deleted
     *  - n1, b are box containing beginning of stuff to be kept after deletion
     *  - cn1 is char position of n1
     *  - f->p0 and f->p1 are not adjusted until after all deletion is done
     */
    b = &f->box[n1];
```

```
cn1 = p1;
while(pt1.x!=pt0.x && n1<f->nbox){
    _frcklinewrap0(f, &pt0, b);
    _frcklinewrap(f, &pt1, b);
    n = _frcanfit(f, pt0, b);
    if(n==0)
        drawerror(f->display, "_frcanfit==0");
    r.min = pt0;
    r.max = pt0;
    r.max.y += f->font->height;
    if(b->nrune > 0){
        if(n != b->nrune){
            _frsplitbox(f, n1, n);
            b = &f->box[n1];
        }
        r.max.x += b->wid;
        draw(f->b, r, f->b, nil, pt1);
        cn1 += b->nrune;
    }else{
        r.max.x += _frnewwid0(f, pt0, b);
        if(r.max.x > f->r.max.x)
            r.max.x = f->r.max.x;
        col = f->cols[BACK];
        if(f->p0<=cn1 && cn1<f->p1)
            col = f->cols[HIGH];
        draw(f->b, r, col, nil, pt0);
        cn1++;
    }
    _fradvance(f, &pt1, b);
    pt0.x += _frnewwid(f, pt0, b);
    f->box[n0++] = f->box[n1++];
    b++;
}
if(n1==f->nbox && pt0.x!=pt1.x) /* deleting last thing in window; must clean up */
    frselectpaint(f, pt0, pt1, f->cols[BACK]);
if(pt1.y != pt0.y){
    Point pt2;

    pt2 = _frptofcharptb(f, 32767, pt1, n1);
    if(pt2.y > f->r.max.y)
        drawerror(f->display, "frptofchar in frdelete");
    if(n1 < f->nbox){
        int q0, q1, q2;

        q0 = pt0.y+f->font->height;
        q1 = pt1.y+f->font->height;
        q2 = pt2.y+f->font->height;
        if(q2 > f->r.max.y)
            q2 = f->r.max.y;
        draw(f->b, Rect(pt0.x, pt0.y, pt0.x+(f->r.max.x-pt1.x), q0),
            f->b, nil, pt1);
        draw(f->b, Rect(f->r.min.x, q0, f->r.max.x, q0+(q2-q1)),
            f->b, nil, Pt(f->r.min.x, q1));
        frselectpaint(f, Pt(pt2.x, pt2.y-(pt1.y-pt0.y)), pt2, f->cols[BACK]);
    }else
        frselectpaint(f, pt0, pt2, f->cols[BACK]);
}
_frclosebox(f, n0, n1-1);
if(nn0>0 && f->box[nn0-1].nrune>=0 && ppt0.x-f->box[nn0-1].wid>=(int)f->r.min.x){
    --nn0;
```

```
        ppt0.x -= f->box[nn0].wid;
    }
    _frclean(f, ppt0, nn0, n0<f->nbox-1? n0+1 : n0);
    if(f->p1 > p1)
        f->p1 -= p1-p0;
    else if(f->p1 > p0)
        f->p1 = p0;
    if(f->p0 > p1)
        f->p0 -= p1-p0;
    else if(f->p0 > p0)
        f->p0 = p0;
    f->nchars -= p1-p0;
    if(f->p0 == f->p1)
        frtick(f, frptofchar(f, f->p0), 1);
    pt0 = frptofchar(f, f->nchars);
    n = f->nlines;
    f->nlines = (pt0.y-f->r.min.y)/f->font->height+(pt0.x>f->r.min.x);
    return n - f->nlines;
}
```

11.4.3 wshow()

⟨*function* wshow 149a⟩≡ (250b)

```
void
wshow(Window *w, uint q0)
{
    int qe;
    int nl;
    uint q;

    qe = w->org + w->frm.nchars;
    if(w->org <= q0 && (q0 < qe || (q0 == qe && qe == w->nr)))
        wscrdraw(w);
    ⟨wshow() else, when q0 is out of scope 153b⟩
}
```

11.4.4 Drawing the scrollbar: wscrdraw()

⟨wmk() *drawing scrollbar* 149b⟩≡ (80e)

```
wscrdraw(w);
```

⟨*function* wscrdraw 149c⟩≡ (257)

```
void
wscrdraw(Window *w)
{
    Rectangle r, r1, r2;
    Image *b;

    scrtemps();
    ⟨wscrdraw() sanity check the window image 151a⟩
    r = w->scrollr;
    b = scrtmp;
    // r1 is translation of r to (0,...)
    r1 = r;
    r1.min.x = 0;
    r1.max.x = Dx(r);
    r2 = scrpos(r1, w->org, w->org + w->frm.nchars, w->nr);
    if(!eqrect(r2, w->lastsr)){
```

```
        w->lastsr = r2;
        /* move r1, r2 to (0,0) to avoid clipping */
        r2 = rectsubpt(r2, r1.min);
        r1 = rectsubpt(r1, r1.min);
        draw(b, r1, w->frm.cols[BORD], nil, ZP);
        draw(b, r2, w->frm.cols[BACK], nil, ZP);
        // little separation line
        r2.min.x = r2.max.x-1;
        draw(b, r2, w->frm.cols[BORD], nil, ZP);

        // transfer back to main image
        draw(w->i, r, b, nil, Pt(0, r1.min.y));
    }
}
```

⟨*function* scrpos 150a⟩≡ (257)

```
static
Rectangle
scrpos(Rectangle r, uint p0, uint p1, uint tot)
{
    Rectangle q;
    int h;

    q = r;
    h = q.max.y - q.min.y; // Dy(r)
    if(tot == 0)
        return q;
    ⟨scrpos() adjust integers if total is big 150c⟩
    if(p0 > 0)
        q.min.y += h*p0 / tot;
    if(p1 < tot)
        q.max.y -= h*(tot-p1) / tot;

    ⟨scrpos() last adjustments 150b⟩
    return q;
}
```

⟨scrpos() *last adjustments* 150b⟩≡ (150a)

```
if(q.max.y < q.min.y+2){
    if(q.min.y+2 <= r.max.y)
        q.max.y = q.min.y+2;
    else
        q.min.y = q.max.y-2;
}
```

⟨scrpos() *adjust integers if total is big* 150c⟩≡ (150a)

```
if(tot > 1024*1024){
    tot>>=10;
    p0>>=10;
    p1>>=10;
}
```

⟨Window *other fields* 150d⟩+≡ (51) ◁117c 176a▷

```
Rectangle lastsr;
```

⟨wmk() *textual window settings, extra frame settings* 150e⟩≡ (126e) 165c▷

```
w->lastsr = ZR;
```

⟨wresize() *textual window updates, reset lastsr* 150f⟩≡ (171c)

```
w->lastsr = ZR;
```

⟨wscrdraw() *sanity check the window image* 151a⟩≡ (149c)

```
if(w->i == nil)
    error("scrdraw");
```

⟨*global* scrtmp 151b⟩≡ (257)

```
static Image *scrtmp;
```

⟨*constant* BIG 151c⟩≡ (237a)

```
BIG   = 3, /* factor by which window dimension can exceed screen */
```

⟨*function* scrtemps 151d⟩≡ (257)

```
static
void
scrtemps(void)
{
    int h;

    if(scrtmp)
        return;
    h = BIG * Dy(view->r);
    scrtmp = allocimage(display, Rect(0, 0, 32, h), view->chan, false, DWhite);
    ⟨scrtemps() sanity check scrtmp 151e⟩
}
```

⟨scrtemps() *sanity check scrtmp* 151e⟩≡ (151d)

```
if(scrtmp == nil)
    error("scrtemps");
```

11.4.5 Drawing the tick: frtick()

⟨frinsert() *remove tick* 151f⟩≡ (139)

```
if(f->p0 == f->p1)
    frtick(f, frptofchar(f, f->p0), false);
```

⟨frinsert() *draw tick* 151g⟩≡ (139)

```
if(f->p0 == f->p1)
    frtick(f, frptofchar(f, f->p0), true);
```

⟨*function* frtick 151h⟩≡ (280b)

```
void
frtick(Frame *f, Point pt, bool ticked)
{
    Rectangle r;

    if(f->ticked==ticked || f->tick==nil || !ptinrect(pt, f->r))
        return;
    pt.x--; /* looks best just left of where requested */
    r = Rect(pt.x, pt.y, pt.x + FRTICKW, pt.y + f->font->height);
    /* can go into left border but not right */
    if(r.max.x > f->r.max.x)
        r.max.x = f->r.max.x;
    if(ticked){
        draw(f->tickback, f->tickback->r, f->b, nil, pt);
        draw(f->b, r, f->tick, nil, ZP);
    }else
        draw(f->b, r, f->tickback, nil, ZP);
    f->ticked = ticked;
}
```

11.4.6 Drawing the text and the selection: `frdrawsel()`

⟨*function* `frdrawsel` 152a⟩≡ (280b)

```
void
frdrawsel(Frame *f, Point pt, ulong p0, ulong p1, bool issel)
{
    Image *back, *text;

    if(f->ticked)
        frtick(f, frptofchar(f, f->p0), false);

    if(p0 == p1){
        frtick(f, pt, issel);
        return;
    }
    // else

    if(issel){
        back = f->cols[HIGH];
        text = f->cols[HTEXT];
    }else{
        back = f->cols[BACK];
        text = f->cols[TEXT];
    }

    frdrawsel0(f, pt, p0, p1, back, text);
}
```

⟨*function* `frdrawsel0` 152b⟩≡ (280b)

```
Point
frdrawsel0(Frame *f, Point pt, ulong p0, ulong p1, Image *back, Image *text)
{
    Frbox *b;
    int nb, nr, w, x, trim;
    Point qt;
    uint p;
    char *ptr;

    p = 0;
    b = f->box;
    trim = 0;
    for(nb=0; nb<f->nbox && p<p1; nb++){
        //todo: nr = NRUNE(b);
        nr = b->nrune;
        if(nr < 0)
            nr = 1;

        if(p+nr <= p0)
            goto Continue;
        if(p >= p0){
            qt = pt;
            _frcklinewrap(f, &pt, b);
            /* fill in the end of a wrapped line */
            if(pt.y > qt.y)
                draw(f->b, Rect(qt.x, qt.y, f->r.max.x, pt.y), back, nil, qt);
        }
        ptr = (char*)b->ptr;
        if(p < p0){ /* beginning of region: advance into box */
            ptr += nbytes(ptr, p0-p);
            nr -= (p0-p);
```

```
            p = p0;
        }
        trim = 0;
        if(p+nr > p1){ /* end of region: trim box */
            nr -= (p+nr)-p1;
            trim = 1;
        }
        if(b->nrune<0 || nr==b->nrune)
            w = b->wid;
        else
            w = stringnwidth(f->font, ptr, nr);
        x = pt.x+w;
        if(x > f->r.max.x)
            x = f->r.max.x;
        draw(f->b, Rect(pt.x, pt.y, x, pt.y+f->font->height), back, nil, pt);
        if(b->nrune >= 0)
            stringnbg(f->b, pt, text, ZP, f->font, ptr, nr, back, ZP);
        pt.x += w;
        Continue:
        b++;
        p += nr;
    }
    /* if this is end of last plain text box on wrapped line, fill to end of line */
    if(p1>p0 &&  b>f->box && b<f->box+f->nbox && b[-1].nrune>0 && !trim){
        qt = pt;
        _frcklinewrap(f, &pt, b);
        if(pt.y > qt.y)
            draw(f->b, Rect(qt.x, qt.y, f->r.max.x, pt.y), back, nil, qt);
    }
    return pt;
}
```

⟨*function* nbytes 153a⟩≡ (280b)

```
static int
nbytes(char *s0, int nr)
{
    char *s;
    Rune r;

    s = s0;
    while(--nr >= 0)
        s += chartorune(&r, s);
    return s-s0;
}
```

11.4.7 Moving the frame origin

⟨wshow() *else, when q0 is out of scope* 153b⟩≡ (149a)

```
else{
    nl = 4 * w->frm.maxlines / 5;
    q = wbacknl(w, q0, nl);
    /* avoid going backwards if trying to go forwards - long lines! */
    if(!(q0 > w->org && q < w->org))
        wsetorigin(w, q, true);

    while(q0 > w->org + w->frm.nchars)
        wsetorigin(w, w->org+1, false);
}
```

⟨*function* wbacknl 154a⟩≡ (250b)

```
uint
wbacknl(Window *w, uint p, uint n)
{
    int i, j;

    /* look for start of this line if n==0 */
    if(n==0 && p>0 && w->r[p-1]!='\n')
        n = 1;

    i = n;
    while(i-->0 && p>0){
        --p; /* it's at a newline now; back over it */
        if(p == 0)
            break;
        /* at 128 chars, call it a line anyway */
        for(j=128; --j>0 && p>0; p--)
            if(w->r[p-1]=='\n')
                break;
    }
    return p;
}
```

⟨*function* wsetorigin 154b⟩≡ (250b)

```
void
wsetorigin(Window *w, uint org, bool exact)
{
    int i, a, fixup;
    Rune *r;
    uint n;
    Frame *frm = &w->frm;

    if(org>0 && !exact){
        /* org is an estimate of the char posn; find a newline */
        /* don't try harder than 256 chars */
        for(i=0; i<256 && org < w->nr; i++){
            if(w->r[org] == '\n'){
                org++;
                break;
            }
            org++;
        }
    }
    a = org - w->org;
    fixup = 0;
    if(a>=0 && a < frm->nchars){
        frdelete(frm, 0, a);
        fixup = 1; /* frdelete can leave end of last line in wrong selection mode; it doesn't know what follows
    }else if(a<0 && -a < frm->nchars){
        n = w->org - org;
        r = runemalloc(n);
        runemove(r, w->r+org, n);
        frinsert(frm, r, r+n, 0);
        free(r);
    }else
        frdelete(frm, 0, frm->nchars);
    w->org = org;
    wfill(w);
    wscrdraw(w);
    wsetselect(w, w->q0, w->q1);
```

```
    if(fixup && frm->p1 > frm->p0)
        frdrawsel(frm, frptofchar(frm, frm->p1-1), frm->p1-1, frm->p1, 1);
}
```

⟨Frame *text fields* 155a⟩+≡ (126g) ◁142b 165a▷

```
ushort lastlinefull; /* last line fills frame */
```

⟨frinit() *initialize other fields* 155b⟩+≡ (127c) ◁136e 165b▷

```
f->lastlinefull = 0;
```

⟨*function* wfill 155c⟩≡ (250b)

```
void
wfill(Window *w)
{
    Rune *rp;
    int i, n, m, nl;
    Frame *frm = &w->frm;

    if(frm->lastlinefull)
        return;
    rp = malloc(messagesize);
    do{
        n = w->nr - (w->org + frm->nchars);
        if(n == 0)
            break;
        if(n > 2000) /* educated guess at reasonable amount */
            n = 2000;
        runemove(rp, w->r + (w->org + frm->nchars), n);
        /*
         * it's expensive to frinsert more than we need, so
         * count newlines.
         */
        nl = frm->maxlines - frm->nlines;
        m = 0;
        for(i=0; i<n; ){
            if(rp[i++] == '\n'){
                m++;
                if(m >= nl)
                    break;
            }
        }
        frinsert(frm, rp, rp+i, frm->nchars);
    } while(frm->lastlinefull == false);
    free(rp);
}
```

11.4.8 Selecting

⟨*function* wsetselect 155d⟩≡ (250b)

```
void
wsetselect(Window *w, uint q0, uint q1)
{
    int p0, p1;
    Frame *frm = &w->frm;

    /* w->p0 and w->p1 are always right; w->q0 and w->q1 may be off */
    w->q0 = q0;
    w->q1 = q1;
    /* compute desired p0,p1 from q0,q1 */
```

```
    p0 = q0-w->org;
    p1 = q1-w->org;
    if(p0 < 0)
        p0 = 0;
    if(p1 < 0)
        p1 = 0;
    if(p0 > frm->nchars)
        p0 = frm->nchars;
    if(p1 > frm->nchars)
        p1 = frm->nchars;
    if(p0 == frm->p0 && p1 == frm->p1)
        return;

    /* screen disagrees with desired selection */
    if(frm->p1 <= p0 || p1 <= frm->p0 || p0==p1 || frm->p1 == frm->p0){
        /* no overlap or too easy to bother trying */
        frdrawsel(frm, frptofchar(frm, frm->p0), frm->p0, frm->p1, 0);
        frdrawsel(frm, frptofchar(frm, p0), p0, p1, 1);
        goto Return;
    }
    /* overlap; avoid unnecessary painting */
    if(p0 < frm->p0){
        /* extend selection backwards */
        frdrawsel(frm, frptofchar(frm, p0), p0, frm->p0, 1);
    }else if(p0 > frm->p0){
        /* trim first part of selection */
        frdrawsel(frm, frptofchar(frm, frm->p0), frm->p0, p0, 0);
    }
    if(p1 > frm->p1){
        /* extend selection forwards */
        frdrawsel(frm, frptofchar(frm, frm->p1), frm->p1, p1, 1);
    }else if(p1 < frm->p1){
        /* trim last part of selection */
        frdrawsel(frm, frptofchar(frm, p1), p1, frm->p1, 0);
    }

    Return:
    frm->p0 = p0;
    frm->p1 = p1;
}
```

⟨*function* frselect 156⟩≡ (282a)

```
void
frselect(Frame *f, Mousectl *mc) /* when called, button 1 is down */
{
    ulong p0, p1, q;
    Point mp, pt0, pt1, qt;
    int reg, b, scrled;

    mp = mc->xy;
    b = mc->buttons;

    f->modified = 0;
    frdrawsel(f, frptofchar(f, f->p0), f->p0, f->p1, 0);
    p0 = p1 = frcharofpt(f, mp);
    f->p0 = p0;
    f->p1 = p1;
    pt0 = frptofchar(f, p0);
    pt1 = frptofchar(f, p1);
    frdrawsel(f, pt0, p0, p1, 1);
```

```
reg = 0;
do{
    scrled = 0;
    if(f->scroll){
        if(mp.y < f->r.min.y){
            (*f->scroll)(f, -(f->r.min.y-mp.y)/(int)f->font->height-1);
            p0 = f->p1;
            p1 = f->p0;
            scrled = 1;
        }else if(mp.y > f->r.max.y){
            (*f->scroll)(f, (mp.y-f->r.max.y)/(int)f->font->height+1);
            p0 = f->p0;
            p1 = f->p1;
            scrled = 1;
        }
        if(scrled){
            if(reg != region(p1, p0))
                q = p0, p0 = p1, p1 = q; /* undo the swap that will happen below */
            pt0 = frptofchar(f, p0);
            pt1 = frptofchar(f, p1);
            reg = region(p1, p0);
        }
    }
    q = frcharofpt(f, mp);
    if(p1 != q){
        if(reg != region(q, p0)){ /* crossed starting point; reset */
            if(reg > 0)
                frdrawsel(f, pt0, p0, p1, 0);
            else if(reg < 0)
                frdrawsel(f, pt1, p1, p0, 0);
            p1 = p0;
            pt1 = pt0;
            reg = region(q, p0);
            if(reg == 0)
                frdrawsel(f, pt0, p0, p1, 1);
        }
        qt = frptofchar(f, q);
        if(reg > 0){
            if(q > p1)
                frdrawsel(f, pt1, p1, q, 1);
            else if(q < p1)
                frdrawsel(f, qt, q, p1, 0);
        }else if(reg < 0){
            if(q > p1)
                frdrawsel(f, pt1, p1, q, 0);
            else
                frdrawsel(f, qt, q, p1, 1);
        }
        p1 = q;
        pt1 = qt;
    }
    f->modified = 0;
    if(p0 < p1) {
        f->p0 = p0;
        f->p1 = p1;
    }
    else {
        f->p0 = p1;
        f->p1 = p0;
    }
```

```
        if(scrled)
            (*f->scroll)(f, 0);
        flushimage(f->display, 1);
        if(!scrled)
            readmouse(mc);
        mp = mc->xy;
    }while(mc->buttons == b);
}
```

⟨*function* `frselectpaint` 158a⟩≡ (282a)

```
void
frselectpaint(Frame *f, Point p0, Point p1, Image *col)
{
    int n;
    Point q0, q1;

    q0 = p0;
    q1 = p1;
    q0.y += f->font->height;
    q1.y += f->font->height;
    n = (p1.y-p0.y)/f->font->height;
    if(f->b == nil)
        drawerror(f->display, "frselectpaint b==0");
    if(p0.y == f->r.max.y)
        return;
    if(n == 0)
        draw(f->b, Rpt(p0, q1), col, nil, ZP);
    else{
        if(p0.x >= f->r.max.x)
            p0.x = f->r.max.x-1;
        draw(f->b, Rect(p0.x, p0.y, f->r.max.x, q0.y), col, nil, ZP);
        if(n > 1)
            draw(f->b, Rect(f->r.min.x, q0.y, f->r.max.x, p1.y),
                col, nil, ZP);
        draw(f->b, Rect(f->r.min.x, p1.y, q1.x, q1.y),
            col, nil, ZP);
    }
}
```

⟨*function* `region` 158b⟩≡ (282a)

```
static
int
region(int a, int b)
{
    if(a < b)
        return -1;
    if(a == b)
        return 0;
    return 1;
}
```

11.4.9 Repainting

⟨`wrepaint()` *after updated cols, redraw content if mouse not opened* 158c⟩≡ (87d)

```
if(!w->mouseopen)
    frredraw(&w->frm);
```

⟨*function* frredraw 159a⟩≡ (280b)

```
void
frredraw(Frame *f)
{
    bool ticked;
    Point pt;

    if(f->p0 == f->p1){
        ticked = f->ticked;
        if(ticked)
            frtick(f, frptofchar(f, f->p0), false);
        // redraw the text
        frdrawsel0(f, frptofchar(f, 0), 0, f->nchars, f->cols[BACK], f->cols[TEXT]);
        if(ticked)
            frtick(f, frptofchar(f, f->p0), true);
        return;
    }
    // else, redraw the selection and the text

    pt = frptofchar(f, 0);
    pt = frdrawsel0(f, pt, 0,     f->p0,     f->cols[BACK], f->cols[TEXT]);
    pt = frdrawsel0(f, pt, f->p0, f->p1,     f->cols[HIGH], f->cols[HTEXT]);
    pt = frdrawsel0(f, pt, f->p1, f->nchars, f->cols[BACK], f->cols[TEXT]);
}
```

11.5 Keyboard events

11.5.1 Text input queue

11.5.2 /mnt/wsys/cons reading part3

Producer

⟨wkeyctl() *when not rawing* 159b⟩≡ (65d)

```
// here when no navigation key, no rawing, no 0x1B holding

⟨wkeyctl() snarf and cut if not interrupt key 191c⟩
switch(r){
⟨wkeyctl() special key cases and no special mode 163a⟩
}
// else

/* otherwise ordinary character; just insert */
⟨wkeyctl() ordinary character 159d⟩
```

⟨wkeyctl() *locals* 159c⟩≡ (65d) 160e ▷

```
uint q0;
```

⟨wkeyctl() *ordinary character* 159d⟩≡ (159b)

```
q0 = w->q0;
q0 = winsert(w, &r, 1, q0);
wshow(w, q0+1);
```

Consumer

⟨winctl() *alts adjustments, revert to CHANSND if newline in queue* 160a⟩≡ (123f)

```
/* this code depends on NL and EOT fitting in a single byte */
/* kind of expensive for each loop; worth precomputing? */
for(i = w->qh; i < w->nr; i++){
     c = w->r[i];
     // buffering, until get a newline in which case we are ready to send
     if(c=='\n' || c=='\004'){
         alts[WCread].op = CHANSND;
         break;
     }
 }
```

⟨winctl() *when WCRead, break if newline and handle EOF character* 160b⟩≡ (123h)

```
c = t[i-wid]; /* knows break characters fit in a byte */
if(!w->rawing && (c == '\n' || c=='\004')){
    if(c == '\004')
        i--;
    break;
```

⟨winctl() *when WCRead, handle EOF character after while loop* 160c⟩≡ (123h)

```
if(i==nb && w->qh < w->nr && w->r[w->qh]=='\004')
    w->qh++;
```

11.5.3 Navigation keys

⟨wkeyctl() *when mouse not opened and navigation keys* 160d⟩≡ (65d)

```
if(!w->mouseopen)
switch(r){
⟨wkeyctl() when mouse not opened, switch key cases 160f⟩
default:
    ; // no return! fallthrough
}
```

⟨wkeyctl() *locals* 160e⟩+≡ (65d) ◁159c 164c▷

```
uint q1;
int n, nb;
```

Text boundaries (Home, end)

⟨wkeyctl() *when mouse not opened, switch key cases* 160f⟩≡ (160d) 160g▷

```
case Khome:
    wshow(w, 0);
    return;
```

⟨wkeyctl() *when mouse not opened, switch key cases* 160g⟩+≡ (160d) ◁160f 160h▷

```
case Kend:
    wshow(w, w->nr);
    return;
```

Down

⟨wkeyctl() *when mouse not opened, switch key cases* 160h⟩+≡ (160d) ◁160g 161a▷

```
case Kdown:
    n = w->frm.maxlines / 3;
    goto case_Down;
```

⟨wkeyctl() *when mouse not opened, switch key cases* 161a⟩+≡ (160d) ◁160h 161b▷

```
case Kscrollonedown:
    n = mousescrollsize(w->frm.maxlines);
    if(n <= 0)
        n = 1;
    goto case_Down;
```

⟨wkeyctl() *when mouse not opened, switch key cases* 161b⟩+≡ (160d) ◁161a 161c▷

```
case Kpgdown:
    n = 2 * w->frm.maxlines / 3;
    // Fallthrough
case_Down:
    q0 = w->org +
        frcharofpt(&w->frm, Pt(w->frm.r.min.x,
                               w->frm.r.min.y + n * w->frm.font->height));
    wsetorigin(w, q0, true);
    return;
```

Up

⟨wkeyctl() *when mouse not opened, switch key cases* 161c⟩+≡ (160d) ◁161b 161d▷

```
case Kup:
    n = w->frm.maxlines/3;
    goto case_Up;
```

⟨wkeyctl() *when mouse not opened, switch key cases* 161d⟩+≡ (160d) ◁161c 161e▷

```
case Kscrolloneup:
    n = mousescrollsize(w->frm.maxlines);
    if(n <= 0)
        n = 1;
    goto case_Up;
```

⟨wkeyctl() *when mouse not opened, switch key cases* 161e⟩+≡ (160d) ◁161d 161f▷

```
case Kpgup:
    n = 2*w->frm.maxlines/3;
    // Fallthrough
case_Up:
    q0 = wbacknl(w, w->org, n);
    wsetorigin(w, q0, true);
    return;
```

Left

⟨wkeyctl() *when mouse not opened, switch key cases* 161f⟩+≡ (160d) ◁161e 162a▷

```
case Kleft:
    if(w->q0 > 0){
        q0 = w->q0 - 1;
        wsetselect(w, q0, q0);
        wshow(w, q0);
    }
    return;
```

Right

⟨wkeyctl() *when mouse not opened, switch key cases* 162a⟩+≡ (160d) ◁161f 162b▷

```
case Kright:
    if(w->q1 < w->nr){
        q1 = w->q1+1;
        wsetselect(w, q1, q1);
        wshow(w, q1);
    }
    return;
```

Line boundaries

⟨wkeyctl() *when mouse not opened, switch key cases* 162b⟩+≡ (160d) ◁162a 162c▷

```
case 0x05: /* ^E: end of line */
    q0 = w->q0;
    while(q0 < w->nr && w->r[q0] != '\n')
        q0++;
    wsetselect(w, q0, q0);
    wshow(w, w->q0);
    return;
```

⟨wkeyctl() *when mouse not opened, switch key cases* 162c⟩+≡ (160d) ◁162b

```
case 0x01: /* ^A: beginning of line */
    if(w->q0==0 || w->q0 == w->qh || w->r[w->q0 - 1]=='\n')
        return;
    nb = wbswidth(w, 0x15 /* ^U */);
    wsetselect(w, w->q0 - nb, w->q0 - nb);
    wshow(w, w->q0);
    return;
```

⟨*function* wbswidth 162d⟩≡ (250b)

```
int
wbswidth(Window *w, Rune c)
{
    uint q, stop;
    Rune r;
    ⟨wbswidth() other locals 163c⟩

    ⟨wbswidth() return if erase character 163b⟩

    q = w->q0;
    stop = 0;
    if(q > w->qh)
        stop = w->qh;

    while(q > stop){
        r = w->r[q-1];
        if(r == '\n'){  /* eat at most one more character */
            if(q == w->q0) /* eat the newline */
                --q;
            break;
        }
        ⟨wbswidth() if c == 0x17 163d⟩
        --q;
    }
    return w->q0-q;
}
```

11.5.4 Special keys

Delete

⟨wkeyctl() *special key cases and no special mode* 163a⟩≡ (159b) 164d ▷

```
case 0x08: /* ^H: erase character */
case 0x15: /* ^U: erase line */
case 0x17: /* ^W: erase word */
    if(w->q0==0 || w->q0==w->qh)
        return;
    nb = wbswidth(w, r);
    q1 = w->q0;
    q0 = q1-nb;
    ⟨wkeyctl() when erase keys, adjust q0 and nb if before org 163e⟩
    if(nb > 0){
        wdelete(w, q0, q0+nb);
        wsetselect(w, q0, q0);
    }
    return;
```

⟨wbswidth() *return if erase character* 163b⟩≡ (162d)

```
/* there is known to be at least one character to erase */
if(c == 0x08) /* ^H: erase character */
    return 1;
```

⟨wbswidth() *other locals* 163c⟩≡ (162d)

```
bool skipping = true;
uint eq;
```

⟨wbswidth() *if* c == 0x17 163d⟩≡ (162d)

```
if(c == 0x17){
    eq = isalnum(r);
    if(eq && skipping) /* found one; stop skipping */
        skipping = false;
    else if(!eq && !skipping)
        break;
}
```

⟨wkeyctl() *when erase keys, adjust q0 and nb if before org* 163e⟩≡ (163a)

```
if(q0 < w->org){
    q0 = w->org;
    nb = q1-q0;
}
```

⟨*function* wdelete 163f⟩≡ (250b)

```
void
wdelete(Window *w, uint q0, uint q1)
{
    uint n, p0, p1;
    Frame *frm = &w->frm;

    n = q1-q0;
    ⟨wdelete() sanity check n 163g⟩
    runemove(w->r+q0, w->r+q1, w->nr-q1);
    w->nr -= n;

    ⟨wdelete() adjust cursors 164a⟩
}
```

⟨wdelete() *sanity check n* 163g⟩≡ (163f)

```
if(n == 0)
    return;
```

⟨wdelete() *adjust cursors* 164a⟩≡ (163f)

```
if(q0 < w->q0)
    w->q0 -= min(n, w->q0-q0);
if(q0 < w->q1)
    w->q1 -= min(n, w->q1-q0);

if(q1 < w->qh)
    w->qh -= n;
else if(q0 < w->qh)
    w->qh = q0;

if(q1 <= w->org)
    w->org -= n;
else
   ⟨wdelete() when q1 > w->org, possibly update visible text 164b⟩
```

⟨wdelete() *when* q1 > w->org, *possibly update visible text* 164b⟩≡ (164a)

```
if(q0 < w->org + frm->nchars){
    p1 = q1 - w->org;
    if(p1 > frm->nchars)
        p1 = frm->nchars;
    if(q0 < w->org){
        w->org = q0;
        p0 = 0;
    }else
        p0 = q0 - w->org;

    frdelete(frm, p0, p1);
    wfill(w);
}
```

Interrupt

⟨wkeyctl() *locals* 164c⟩+≡ (65d) ◁160e

```
int nr;
Rune *rp;
int *notefd;
```

⟨wkeyctl() *special key cases and no special mode* 164d⟩+≡ (159b) ◁163a 206a▷

```
case 0x7F:  /* send interrupt */
    w->qh = w->nr;
    wshow(w, w->qh);
    notefd = emalloc(sizeof(int));
    *notefd = w->notefd;
    proccreate(interruptproc, notefd, 4096);
    return;
```

⟨*function* interruptproc 164e⟩≡ (250b)

```
/*
 * Need to do this in a separate proc because if process we're interrupting
 * is dying and trying to print tombstone, kernel is blocked holding p->debug lock.
 */
void
interruptproc(void *v)
{
    int *notefd;

    notefd = v;
    write(*notefd, "interrupt", 9);
    free(notefd);
}
```

Tab

⟨Frame *text fields* 165a⟩+≡ (126g) ◁155a
```
  ushort maxtab;  /* max size of tab, in pixels */
```

⟨frinit() *initialize other fields* 165b⟩+≡ (127c) ◁155b
```
  f->maxtab = 8 * stringwidth(ft, "0");
```

⟨wmk() *textual window settings, extra frame settings* 165c⟩+≡ (126e) ◁150e
```
  w->frm.maxtab = maxtab * stringwidth(font, "0");
```

⟨wresize() *textual window updates, extra frame settings* 165d⟩≡ (171c)
```
  w->frm.maxtab = maxtab * stringwidth(w->frm.font, "0");
```

⟨*global* maxtab 165e⟩≡ (242)
```
  int  maxtab = 0;
```

⟨main() *locals* 165f⟩+≡ (56a) ◁81j 215b▷
```
  char *s;
```

⟨main() *set some globals* 165g⟩+≡ (56a) ◁81k 194c▷
```
  s = getenv("tabstop");
  if(s != nil)
      maxtab = strtol(s, nil, 0);
  if(maxtab == 0)
      maxtab = 4;
  free(s);
```

11.6 Application output events

11.6.1 /mnt/wsys/cons writing part2

Producer

Consumer

⟨Wxxx *cases* 165h⟩+≡ (63e) ◁123a 178a▷
```
  WCwrite,
```

⟨winctl() *other locals* 165i⟩+≡ (64a) ◁124a 166a▷
```
  Conswritemesg cwm;
```

⟨winctl() *channels creation* 165j⟩+≡ (64a) ◁123c 177f▷
```
  cwm.cw = chancreate(sizeof(Stringpair), 0);
```

⟨winctl() *Wctl case, free channels if wctlmesg is Excited* 165k⟩+≡ (66d) ◁123d 177g▷
```
  chanfree(cwm.cw);
```

⟨winctl() *alts setup* 165l⟩+≡ (64a) ◁123e 178b▷
```
  alts[WCwrite].c = w->conswrite;
  alts[WCwrite].v = &cwm;
  alts[WCwrite].op = CHANSND;
```

⟨winctl() *alts adjustments* 165m⟩+≡ (64a) ◁123f 178c▷
```
  if(!w->scrolling && !w->mouseopen && w->qh >  w->org + w->frm.nchars)
      alts[WCwrite].op = CHANNOP;
  else
      // scrolling || mouseopen || w->qh <= w->org + w->frm.nchars
      alts[WCwrite].op = CHANSND;
```

⟨winctl() *other locals* 166a⟩+≡ (64a) ◁165i 166d▷

```
Rune *rp;
int nr;
```

⟨winctl() *event loop cases* 166b⟩+≡ (64a) ◁123h 178e▷

```
case WCwrite:
    recv(cwm.cw, &pair);
    rp = pair.s;
    nr = pair.ns;

    ⟨winctl() when WCwrite, if runes contains backspace 166e⟩

    w->qh = winsert(w, rp, nr, w->qh) + nr;
    ⟨winctl() when WCwrite, if scrolling or mouseopen 166c⟩
    wsetselect(w, w->q0, w->q1);
    wscrdraw(w);

    free(rp);
    break;
```

⟨winctl() *when WCwrite, if scrolling or mouseopen* 166c⟩≡ (166b)

```
if(w->scrolling || w->mouseopen)
    wshow(w, w->qh);
```

⟨winctl() *other locals* 166d⟩+≡ (64a) ◁166a 177e▷

```
Rune *bp, *tp, *up;
int initial;
uint qh;
```

⟨winctl() *when WCwrite, if runes contains backspace* 166e⟩≡ (166b)

```
bp = rp;
for(i=0; i<nr; i++) {
    if(*bp++ == '\b'){
        --bp;
        initial = 0;
        tp = runemalloc(nr);
        runemove(tp, rp, i);
        up = tp+i;
        for(; i<nr; i++){
            *up = *bp++;
            if(*up == '\b')
                if(up == tp)
                    initial++;
                else
                    --up;
            else
                up++;
        }
        if(initial){
            if(initial > w->qh)
                initial = w->qh;
            qh = w->qh-initial;
            wdelete(w, qh, qh+initial);
            w->qh = qh;
        }
        free(rp);
        rp = tp;
        nr = up-tp;
        rp[nr] = 0;
        break;
    }
}
```

11.7 Mouse events

11.7.1 Middle click menu

⟨mousethread() *middle click under certain conditions* 167a⟩≡ (63c)
```
button2menu(winput);
```

⟨*function* button2menu 167b⟩≡ (250b)
```
void
button2menu(Window *w)
{
    ⟨button2menu() return if window was deleted 167c⟩

    incref(w);
    ⟨button2menu() menu2str adjustments for scrolling 168c⟩
    switch(menuhit(2, mousectl, &menu2, desktop)){
    ⟨button2menu() cases 168d⟩
    }
    wclose(w); // decref

    wsendctlmesg(w, Wakeup, ZR, nil);
    flushimage(display, true);
}
```

⟨button2menu() *return if window was deleted* 167c⟩≡ (167b)
```
if(w->deleted)
    return;
```

⟨*global* menu2 167d⟩≡ (250b)
```
Menu menu2 = { .item = menu2str };
```

⟨*global* menu2str 167e⟩≡ (250b)
```
char*  menu2str[] = {
 [Cut]  "cut",
 [Paste] "paste",
 [Snarf] "snarf",
 [Plumb] "plumb",
 [Send]  "send",
 [Scroll] "scroll",
 nil
};
```

⟨*enum* _anon_ (windows/rio/rio.c)*2* 167f⟩≡ (250b)
```
enum
{
    Cut,
    Paste,
    Snarf,
    Plumb,
    Send,
    Scroll,
};
```

11.7.2 Other clicks

⟨winctl() *WMouse case if not mouseopen* 167g⟩≡ (65j)
```
wmousectl(w);
```

⟨*function* wmousectl 168a⟩≡ (250b)

```
void
wmousectl(Window *w)
{
    int but;

    if(w->mc.buttons == 1)
        but = 1;
    else if(w->mc.buttons == 2)
        but = 2;
    else if(w->mc.buttons == 4)
        but = 3;
    else{
        if(w->mc.buttons == 8)
            wkeyctl(w, Kscrolloneup);
        if(w->mc.buttons == 16)
            wkeyctl(w, Kscrollonedown);
        return;
    }

    incref(w);  /* hold up window while we track */
    ⟨wmousectl() goto Return if window was deleted 168b⟩

    ⟨wmousectl() if pt in scrollbar 169c⟩
    if(but == 1)
        wselect(w);

    /* else all is handled by main process */
   Return:
    wclose(w);
}
```

⟨wmousectl() *goto Return if window was deleted* 168b⟩≡ (168a)

```
if(w->deleted)
    goto Return;
```

11.8 Automatic scrolling mode

⟨button2menu() *menu2str adjustments for scrolling* 168c⟩≡ (167b)

```
if(w->scrolling)
    menu2str[Scroll] = "noscroll";
else
    menu2str[Scroll] = "scroll";
```

⟨button2menu() *cases* 168d⟩≡ (167b) 191d ▷

```
case Scroll:
    if(w->scrolling ^= 1)
        wshow(w, w->nr);
    break;
```

11.9 Scroll bar interaction

⟨Frame *scroll* 168e⟩≡ (126g)

```
void (*scroll)(Frame*, int); /* scroll function provided by application */
```

⟨mousethread() *locals* 168f⟩+≡ (61b) ◁ 77e

```
bool scrolling = false;
```

⟨mousethread() *set scrolling* 169a⟩≡ (61f)

```
if(winput->mouseopen)
    scrolling = false;
else
  if(scrolling)
    scrolling = mouse->buttons;
  else
    scrolling = mouse->buttons && ptinrect(xy, winput->scrollr);
```

⟨mousethread() *goto Sending if scroll buttons* 169b⟩≡ (61f)

```
/* the up and down scroll buttons are not subject to the usual rules */
if((mouse->buttons&(8|16)) && !winput->mouseopen)
    goto Sending;
```

⟨wmousectl() *if pt in scrollbar* 169c⟩≡ (168a)

```
if(ptinrect(w->mc.xy, w->scrollr)){
    if(but)
        wscroll(w, but);
    goto Return;
}
```

⟨*function* wscroll 169d⟩≡ (257)

```
void
wscroll(Window *w, int but)
{
    uint p0, oldp0;
    Rectangle s;
    int x, y, my, h, first;

    s = insetrect(w->scrollr, 1);
    h = s.max.y-s.min.y;
    x = (s.min.x+s.max.x)/2;
    oldp0 = ~0;
    first = true;
    do{
        flushimage(display, 1);
        if(w->mc.xy.x<s.min.x || s.max.x<=w->mc.xy.x){
            readmouse(&w->mc);
        }else{
            my = w->mc.xy.y;
            if(my < s.min.y)
                my = s.min.y;
            if(my >= s.max.y)
                my = s.max.y;
            if(!eqpt(w->mc.xy, Pt(x, my))){
                wmovemouse(w, Pt(x, my));
                readmouse(&w->mc);  /* absorb event generated by moveto() */
            }
            if(but == 2){
                y = my;
                if(y > s.max.y-2)
                    y = s.max.y-2;
                if(w->nr > 1024*1024)
                    p0 = ((w->nr>>10)*(y-s.min.y)/h)<<10; // >>
                else
                    p0 = w->nr*(y-s.min.y)/h;
                if(oldp0 != p0)
                    wsetorigin(w, p0, false);
                oldp0 = p0;
                readmouse(&w->mc);
```

```
                continue;
            }
            if(but == 1)
                p0 = wbacknl(w, w->org, (my-s.min.y)/w->frm.font->height);
            else
                p0 = w->org + frcharofpt(&w->frm, Pt(s.max.x, my));
            if(oldp0 != p0)
                wsetorigin(w, p0, true);
            oldp0 = p0;
            /* debounce */
            if(first){
                flushimage(display, 1);
                sleep(200);
                nbrecv(w->mc.c, &w->mc.Mouse);
                first = false;
            }
            wscrsleep(w, 100);
        }
    }while(w->mc.buttons & (1<<(but-1))); // >>
    while(w->mc.buttons)
        readmouse(&w->mc);
}
```

⟨*enum* _anon_ (windows/rio/dat.h)*2* 170a⟩≡ (237b)

```
enum
{
    Kscrolloneup = KF|0x20,
    Kscrollonedown = KF|0x21,
};
```

⟨*function* freescrtemps 170b⟩≡ (257)

```
void
freescrtemps(void)
{
    freeimage(scrtmp);
    scrtmp = nil;
}
```

⟨*function* wscrsleep 170c⟩≡ (257)

```
void
wscrsleep(Window *w, uint dt)
{
    Timer *timer;
    int y, b;
    static Alt alts[3];

    timer = timerstart(dt);
    y = w->mc.xy.y;
    b = w->mc.buttons;
    alts[0].c = timer->c;
    alts[0].v = nil;
    alts[0].op = CHANRCV;
    alts[1].c = w->mc.c;
    alts[1].v = &w->mc.Mouse;
    alts[1].op = CHANRCV;
    alts[2].op = CHANEND;
    for(;;)
        switch(alt(alts)){
        case 0:
            timerstop(timer);
```

```
            return;
        case 1:
            if(abs(w->mc.xy.y-y)>2 || w->mc.buttons!=b){
                timercancel(timer);
                return;
            }
            break;
        }
}
```

⟨*function* `framescroll` 171a⟩≡ (250b)

```
/*
 * called from frame library
 */
void
framescroll(Frame *f, int dl)
{
    if(f != &selectwin->frm)
        error("frameselect not right frame");
    wframescroll(selectwin, dl);
}
```

⟨*function* `wframescroll` 171b⟩≡ (250b)

```
void
wframescroll(Window *w, int dl)
{
    uint q0;
    Frame *frm = &w->frm;

    if(dl == 0){
        wscrsleep(w, 100);
        return;
    }
    if(dl < 0){
        q0 = wbacknl(w, w->org, -dl);
        if(selectq > w->org + frm->p0)
            wsetselect(w, w->org + frm->p0, selectq);
        else
            wsetselect(w, selectq, w->org + frm->p0);
    }else{
        if(w->org + frm->nchars == w->nr)
            return;
        q0 = w->org + frcharofpt(frm, Pt(frm->r.min.x, frm->r.min.y + dl * frm->font->height));
        if(selectq >= w->org + frm->p1)
            wsetselect(w, w->org + frm->p1, selectq);
        else
            wsetselect(w, selectq, w->org + frm->p1);
    }
    wsetorigin(w, q0, true);
}
```

11.10 Resize

⟨`wresize()` *textual window updates* 171c⟩≡ (94c)

```
⟨wresize() textual window updates, reset lastsr 150f⟩
r = insetrect(i->r, Selborder+1);
w->scrollr = r;
w->scrollr.max.x = r.min.x+Scrollwid;
```

```
r.min.x += Scrollwid+Scrollgap;

if(move)
    frsetrects(&w->frm, r, w->i);
else{
    frclear(&w->frm, false);
    frinit(&w->frm, r, w->frm.font, w->i, cols);
    wsetcols(w);
    ⟨wresize() textual window updates, extra frame settings 165d⟩
    r = insetrect(w->i->r, Selborder);
    draw(w->i, r, cols[BACK], nil, w->frm.entire.min);

    wfill(w);
    wsetselect(w, w->q0, w->q1);
    wscrdraw(w);
}
```

11.11 /mnt/wsys/text

⟨qid *cases* 172a⟩+≡ (100a) ◁124e 173a▷

```
Qtext,
```

⟨*dirtab array elements* 172b⟩+≡ (100f) ◁124f 173b▷

```
{ "text",  QTFILE, Qtext,  0400 },
```

⟨xfidread() *cases* 172c⟩+≡ (109a) ◁124h 173c▷

```
case Qtext:
    t = wcontents(w, &n);
    goto Text;

Text:
    if(off > n){
        off = n;
        cnt = 0;
    }
    if(off+cnt > n)
        cnt = n-off;

    fc.data = t + off;
    fc.count = cnt;
    filsysrespond(x->fs, x, &fc, nil);
    free(t);
    break;
```

⟨*function* wcontents 172d⟩≡ (250b)

```
char*
wcontents(Window *w, int *ip)
{
    return runetobyte(w->r, w->nr, ip);
}
```

Chapter 12

Windowing System Files

12.1 /mnt/wsys/winid

⟨qid *cases* 173a⟩+≡ (100a) ◁172a 173d▷
```
  Qwinid,
```

⟨*dirtab array elements* 173b⟩+≡ (100f) ◁172b 173e▷
```
  { "winid",  QTFILE, Qwinid,  0400 },
```

⟨xfidread() *cases* 173c⟩+≡ (109a) ◁172c 173f▷
```
  case Qwinid:
      n = sprint(buf, "%11d ", w->id);
      t = estrdup(buf);
      goto Text;
```

12.2 /mnt/wsys/label

⟨qid *cases* 173d⟩+≡ (100a) ◁173a 174a▷
```
  Qlabel,
```

⟨*dirtab array elements* 173e⟩+≡ (100f) ◁173b 174b▷
```
  { "label",  QTFILE, Qlabel,  0600 },
```

⟨xfidread() *cases* 173f⟩+≡ (109a) ◁173c 174c▷
```
  case Qlabel:
      n = strlen(w->label);
      if(off > n)
          off = n;
      if(off+cnt > n)
          cnt = n - off;

      fc.data = w->label + off;
      fc.count = cnt;
      filsysrespond(x->fs, x, &fc, nil);
      break;
```

⟨xfidwrite() *cases* 173g⟩+≡ (109c) ◁118a 178f▷
```
  case Qlabel:
      if(off != 0){
          filsysrespond(x->fs, x, &fc, "non-zero offset writing label");
          return;
      }
      free(w->label);
```

```
    w->label = emalloc(cnt+1);
    memmove(w->label, req->data, cnt);
    w->label[cnt] = '\0';
    break;
```

12.3 /mnt/wsys/screen

⟨qid *cases* 174a⟩+≡ (100a) ◁173d 174d▷

```
  Qscreen,
```

⟨*dirtab array elements* 174b⟩+≡ (100f) ◁173e 174e▷

```
  { "screen",  QTFILE, Qscreen, 0400 },
```

⟨xfidread() *cases* 174c⟩+≡ (109a) ◁173f 177b▷

```
  case Qscreen:
      i = display->image;
      if(i == nil){
          filsysrespond(x->fs, x, &fc, "no top-level screen");
          break;
      }
      r = i->r;
      goto caseImage;
```

12.4 /mnt/wsys/wsys/

⟨qid *cases* 174d⟩+≡ (100a) ◁174a 175c▷

```
  Qwsys,  /* directory of window directories */
```

⟨*dirtab array elements* 174e⟩+≡ (100f) ◁174b 176c▷

```
  { "wsys",  QTDIR, Qwsys,  0500|DMDIR },
```

⟨*function* idcmp 174f⟩≡ (255)

```
  static
  int
  idcmp(void *a, void *b)
  {
      return *(int*)a - *(int*)b;
  }
```

⟨filsysread() *other locals* 174g⟩+≡ (107d) ◁108b

```
  int i, j, k;
  int len;
  int *ids;
  Dirtab dt;
  char buf[16];
```

⟨filsyswalk() *if Qwsys, then goto Accept* 174h⟩≡ (103d)

```
  if(qid.path == Qwsys){
      /* is it a numeric name? */
      if(!numeric(x->req.wname[i]))
          break;
      /* yes: it's a directory */
      id = atoi(x->req.wname[i]);
      qlock(&all);
      w = wlookid(id);
      if(w == nil){
          qunlock(&all);
```

```
        break;
    }
    path = Qwsysdir;
    type = QTDIR;
    qunlock(&all);
    incref(w);
    sendp(winclosechan, f->w);
    f->w = w;
    dir = dirtab;
    goto Accept;
}
```

⟨*function* numeric 175a⟩≡ (255)

```
static
int
numeric(char *s)
{
    for(; *s!='\0'; s++)
        if(*s<'0' || '9'<*s)
            return 0;
    return 1;
}
```

⟨filsysread() *cases* 175b⟩+≡ (107d) ◁108c

```
case Qwsys:

    qlock(&all);
    ids = emalloc(nwindow * sizeof(int));
    for(j=0; j<nwindow; j++)
        ids[j] = windows[j]->id;
    qunlock(&all);

    qsort(ids, nwindow, sizeof ids[0], idcmp);
    dt.name = buf;
    for(i=0, j=0; j<nwindow && i<e; i+=len){
        k = ids[j];
        sprint(dt.name, "%d", k);
        dt.qid = QID(k, Qdir);
        dt.type = QTDIR;
        dt.perm = DMDIR|0700;
        len = dostat(fs, k, &dt, b+n, x->req.count - n, clock);
        if(len == 0)
            break;
        if(i >= o)
            n += len;
        j++;
    }
    free(ids);
    break;
```

⟨qid *cases* 175c⟩+≡ (100a) ◁174d 176b▷

```
Qwsysdir,  /* window directory, child of wsys */
```

⟨filsyswalk() *when in dotdot, if Qwsysdir adjust path* 175d⟩≡ (105a)

```
if(FILE(qid) == Qwsysdir)
    path = Qwsys;
```

12.5 /mnt/wsys/wctl

⟨Window *other fields* 176a⟩+≡ (51) ◁150d 177c▷

```
bool wctlopen;
bool  wctlready;
```

⟨qid *cases* 176b⟩+≡ (100a) ◁175c 192d▷

```
Qwctl,
```

⟨*dirtab array elements* 176c⟩+≡ (100f) ◁174e 192e▷

```
{ "wctl",  QTFILE, Qwctl,  0600 },
```

⟨xfidopen() *cases* 176d⟩+≡ (106f) ◁119a 193a▷

```
case Qwctl:
    if(x->req.mode==OREAD || x->req.mode==ORDWR){
        /*
         * It would be much nicer to implement fan-out for wctl reads,
         * so multiple people can see the resizings, but rio just isn't
         * structured for that.  It's structured for /dev/cons, which gives
         * alternate data to alternate readers.  So to keep things sane for
         * wctl, we compromise and give an error if two people try to
         * open it.  Apologies.
         */
        if(w->wctlopen){
            filsysrespond(x->fs, x, &fc, Einuse);
            return;
        }
        w->wctlopen = true;
        w->wctlready = true;
        wsendctlmesg(w, Wakeup, ZR, nil);
    }
    break;
```

⟨xfidclose() *cases* 176e⟩+≡ (107c) ◁119b 193c▷

```
case Qwctl:
    if(x->f->mode==OREAD || x->f->mode==ORDWR)
        w->wctlopen = false;
break;
```

⟨wcurrent() *wakeup w and oi* 176f⟩≡ (87c)

```
if(w != oi){
    if(oi){
        oi->wctlready = true;
        wsendctlmesg(oi, Wakeup, ZR, nil);
    }
    if(w){
        w->wctlready = true;
        wsendctlmesg(w, Wakeup, ZR, nil);
    }
}
```

12.5.1 Reading

Producer

Consumer

⟨*enum* _anon_ (windows/rio/xfid.c)*5* 176g⟩≡ (256)

```
enum { WCRdata, WCRflush, NWCR };
```

⟨xfidread() *other locals* 177a⟩+≡ (109a) ◁124g

```
Consreadmesg cwrm;
```

⟨xfidread() *cases* 177b⟩+≡ (109a) ◁174c 193d▷

```
case Qwctl: /* read returns rectangle, hangs if not resized */
    if(cnt < 4*12){
        filsysrespond(x->fs, x, &fc, Etooshort);
        break;
    }
    ⟨xfidxxx() set flushtag 223g⟩

    alts[WCRdata].c = w->wctlread;
    alts[WCRdata].v = &cwrm;
    alts[WCRdata].op = CHANRCV;
    ⟨xfidread() when Qwctl, set alts for flush 225a⟩
    alts[NMR].op = CHANEND;

    switch(alt(alts)){
    case WCRdata:
        break;
    ⟨xfidread() when Qwctl, switch alt flush case 225b⟩
    }

    /* received data */
    ⟨xfidxxx() unset flushtag 224a⟩
    c1 = cwrm.c1;
    c2 = cwrm.c2;
    t = malloc(cnt+1); /* be sure to have room for NUL */
    pair.s = t;
    pair.ns = cnt+1;
    send(c1, &pair);
    ⟨xfidread() when Qwctl, if flushing 225c⟩

    qlock(&x->active);
    recv(c2, &pair);
    fc.data = pair.s;
    if(pair.ns > cnt)
        pair.ns = cnt;
    fc.count = pair.ns;
    filsysrespond(x->fs, x, &fc, nil);
    free(t);
    qunlock(&x->active);
    break;
```

⟨Window *other fields* 177c⟩+≡ (51) ◁176a

```
// chan<Consreadmesg> (listener = , sender = )
Channel  *wctlread; /* chan(Consreadmesg) */
```

⟨wmk() *channels creation* 177d⟩+≡ (80e) ◁115b

```
w->wctlread =  chancreate(sizeof(Consreadmesg), 0);
```

⟨winctl() *other locals* 177e⟩+≡ (64a) ◁166d 178d▷

```
Consreadmesg cwrm;
```

⟨winctl() *channels creation* 177f⟩+≡ (64a) ◁165j

```
cwrm.c1 = chancreate(sizeof(Stringpair), 0);
cwrm.c2 = chancreate(sizeof(Stringpair), 0);
```

⟨winctl() *Wctl case, free channels if wctlmesg is Excited* 177g⟩+≡ (66d) ◁165k

```
chanfree(cwrm.c1);
chanfree(cwrm.c2);
```

⟨Wxxx *cases* 178a⟩+≡ (63e) ◁165h

```
WWread,
```

⟨winctl() *alts setup* 178b⟩+≡ (64a) ◁165l

```
alts[WWread].c = w->wctlread;
alts[WWread].v = &cwrm;
alts[WWread].op = CHANSND;
```

⟨winctl() *alts adjustments* 178c⟩+≡ (64a) ◁165m

```
if(w->deleted || !w->wctlready)
    alts[WWread].op = CHANNOP;
else
    alts[WWread].op = CHANSND;
```

⟨winctl() *other locals* 178d⟩+≡ (64a) ◁177e

```
char *s;
```

⟨winctl() *event loop cases* 178e⟩+≡ (64a) ◁166b

```
case WWread:
    w->wctlready = false;
    recv(cwrm.c1, &pair);
    if(w->deleted || w->i==nil)
        pair.ns = sprint(pair.s, "");
    else{
        s = "visible";
        for(i=0; i<nhidden; i++)
            if(hidden[i] == w){
                s = "hidden";
                break;
            }
        t = "notcurrent";
        if(w == input)
            t = "current";
        pair.ns = snprint(pair.s, pair.ns, "%11d %11d %11d %11d %s %s ",
            w->i->r.min.x, w->i->r.min.y, w->i->r.max.x, w->i->r.max.y, t, s);
    }
    send(cwrm.c2, &pair);
    continue;
```

12.5.2 Writing (controlling windows)

⟨xfidwrite() *cases* 178f⟩+≡ (109c) ◁173g 193f▷

```
case Qwctl:
    if(writewctl(x, buf) < 0){
        filsysrespond(x->fs, x, &fc, buf);
        return;
    }
    flushimage(display, true);
    break;
```

⟨*function* writewctl 178g⟩≡ (258b)

```
int
writewctl(Xfid *x, char *err)
{
    int cnt, cmd, j, id, hideit, scrollit, pid;
    Image *i;
    char *arg, *dir;
    Rectangle rect;
    Window *w;
```

```
w = x->f->w;
cnt = x->req.count;
x->req.data[cnt] = '\0';
id = 0;

rect = rectsubpt(w->screenr, view->r.min);
cmd = parsewctl(&arg, rect, &rect, &pid, &id, &hideit, &scrollit, &dir, x->req.data, err);
if(cmd < 0)
    return -1;

if(mouse->buttons!=0 && cmd>=Top){
    strcpy(err, "action disallowed when mouse active");
    return -1;
}

if(id != 0){
    for(j=0; j<nwindow; j++)
        if(windows[j]->id == id)
            break;
    if(j == nwindow){
        strcpy(err, "no such window id");
        return -1;
    }
    w = windows[j];
    if(w->deleted || w->i==nil){
        strcpy(err, "window deleted");
        return -1;
    }
}

switch(cmd){
case New:
    return wctlnew(rect, arg, pid, hideit, scrollit, dir, err);
case Set:
    if(pid > 0)
        wsetpid(w, pid, 0);
    return 1;
case Move:
    rect = Rect(rect.min.x, rect.min.y, rect.min.x+Dx(w->screenr), rect.min.y+Dy(w->screenr));
    rect = rectonscreen(rect);
    /* fall through */
case Resize:
    if(!goodrect(rect)){
        strcpy(err, Ebadwr);
        return -1;
    }
    if(eqrect(rect, w->screenr))
        return 1;
    i = allocwindow(desktop, rect, Refbackup, DWhite);
    if(i == nil){
        strcpy(err, Ewalloc);
        return -1;
    }
    border(i, rect, Selborder, red, ZP);
    wsendctlmesg(w, Reshaped, i->r, i);
    return 1;
case Scroll:
    w->scrolling = 1;
    wshow(w, w->nr);
```

```
        wsendctlmesg(w, Wakeup, ZR, nil);
        return 1;
    case Noscroll:
        w->scrolling = 0;
        wsendctlmesg(w, Wakeup, ZR, nil);
        return 1;
    case Top:
        wtopme(w);
        return 1;
    case Bottom:
        wbottomme(w);
        return 1;
    case Current:
        wcurrent(w);
        return 1;
    case Hide:
        switch(whide(w)){
        case -1:
            strcpy(err, "window already hidden");
            return -1;
        case 0:
            strcpy(err, "hide failed");
            return -1;
        default:
            break;
        }
        return 1;
    case Unhide:
        for(j=0; j<nhidden; j++)
            if(hidden[j] == w)
                break;
        if(j == nhidden){
            strcpy(err, "window not hidden");
            return -1;
        }
        if(wunhide(j) == 0){
            strcpy(err, "hide failed");
            return -1;
        }
        return 1;
    case Delete:
        wsendctlmesg(w, Deleted, ZR, nil);
        return 1;
    }
    strcpy(err, "invalid wctl message");
    return -1;
}
```

Chapter 13

Advanced Topics TODO

13.1 External mount: /srv/rio.user.pid

⟨*global srvpipe (windows/rio/fsys.c)* 181a⟩≡ (254b)

```
char srvpipe[64];
```

⟨filsysinit() *srv pipe* 181b⟩≡ (58b)

```
/*
 * Post srv pipe
 */
sprint(srvpipe, "/srv/rio.%s.%d", fs->user, pid);
post(srvpipe, "wsys", fs->cfd);
```

⟨*function* post 181c⟩≡ (254b)

```
void
post(char *name, char *envname, fdt srvfd)
{
    fdt fd;
    char buf[32];

    fd = create(name, OWRITE|ORCLOSE|OCEXEC, 0600);
    if(fd < 0)
        error(name);
    sprint(buf, "%d", srvfd);
    if(write(fd, buf, strlen(buf)) != strlen(buf))
        error("srv write");

    putenv(envname, name);
}
```

⟨xfidattach() *other locals* 181d⟩+≡ (102d) ◁103b

```
Rectangle r;
int pid;
bool hideit = false;
bool scrollit;
char *dir, errbuf[ERRMAX];
Image *i;
```

⟨xfidattach() *if mount "new ..."* 181e⟩≡ (102d)

```
if(strncmp(x->req.aname, "new", 3) == 0){ /* new -dx -dy - new syntax, as in wctl */
    pid = 0;
    if(parsewctl(nil, ZR, &r, &pid, nil, &hideit, &scrollit, &dir, x->req.aname, errbuf) < 0)
        err = errbuf;
    else {
        if(!goodrect(r))
```

```
            err = Ebadrect;
        else{
            if(hideit)
                i = allocimage(display, r, view->chan, false, DWhite);
            else
                i = allocwindow(desktop, r, Refbackup, DWhite);
            if(i){
                border(i, r, Selborder, display->black, ZP);
                if(pid == 0)
                    pid = -1; /* make sure we don't pop a shell! - UGH */
                w = new(i, hideit, scrolling, pid, nil, nil, nil);
                flushimage(display, 1);
                newlymade = true;
            }else
                err = Ewindow;
        }
    }
}
```

13.2 Command-line control: /srv/riowctl.user.pid

⟨*global* wctlfd 182a⟩≡ (242)

```
int wctlfd;
```

⟨*global srvwctl (windows/rio/fsys.c)* 182b⟩≡ (254b)

```
char srvwctl[64];
```

⟨filsysinit() *wctl pipe, process, and thread creation* 182c⟩≡ (58b)

```
/*
 * Create and post wctl pipe
 */
⟨filsysinit() create wctl pipe 182e⟩

/*
 * Start server processes
 */
⟨filsysinit() create wctl process and thread 182f⟩
```

⟨filsysinit() *other locals* 182d⟩≡ (58b) 225d ▷

```
fdt p0;
// chan<??> (listener = ??, sender = ??)
Channel *c;
```

⟨filsysinit() *create wctl pipe* 182e⟩≡ (182c)

```
if(cexecpipe(&p0, &wctlfd) < 0)
    goto Rescue;
sprint(srvwctl, "/srv/riowctl.%s.%d", fs->user, pid);
post(srvwctl, "wctl", p0);
close(p0);
```

⟨filsysinit() *create wctl process and thread* 182f⟩≡ (182c)

```
c = chancreate(sizeof(char*), 0);
if(c == nil)
    error("wctl channel");

proccreate(wctlproc, c, 4096);
threadcreate(wctlthread, c, 4096);
```

⟨*function* wctlproc 183a⟩≡ (258b)

```
void
wctlproc(void *v)
{
    Channel *c = v;
    char *buf;
    int n, eofs;

    threadsetname("WCTLPROC");

    eofs = 0;
    for(;;){
        buf = emalloc(messagesize);
        // blocking call
        n = read(wctlfd, buf, messagesize-1); /* room for \0 */
        if(n < 0)
            break;
        if(n == 0){
            if(++eofs > 20)
                break;
            continue;
        }
        eofs = 0;

        buf[n] = '\0';
        sendp(c, buf);
    }
}
```

⟨*function* wctlthread 183b⟩≡ (258b)

```
void
wctlthread(void *v)
{
    Channel *c = v;
    char *buf, *arg, *dir;
    int cmd, id, pid, hideit, scrollit;
    Rectangle rect;
    char err[ERRMAX];

    threadsetname("WCTLTHREAD");

    for(;;){
        buf = recvp(c);

        cmd = parsewctl(&arg, ZR, &rect, &pid, &id, &hideit, &scrollit, &dir, buf, err);

        switch(cmd){
        case New:
            wctlnew(rect, arg, pid, hideit, scrollit, dir, err);
        }
        free(buf);
    }
}
```

⟨*enum* _anon_ (windows/rio/wctl.c) 183c⟩≡ (258b)

```
/* >= Top are disallowed if mouse button is pressed */
enum
{
    New,
    Resize,
```

```
    Move,
    Scroll,
    Noscroll,
    Set,
    Top,
    Bottom,
    Current,
    Hide,
    Unhide,
    Delete,
};
```

⟨*global* cmds 184a⟩≡ (258b)

```
static char *cmds[] = {
    [New] = "new",
    [Resize] = "resize",
    [Move] = "move",
    [Scroll] = "scroll",
    [Noscroll] = "noscroll",
    [Set] = "set",
    [Top] = "top",
    [Bottom] = "bottom",
    [Current] = "current",
    [Hide] = "hide",
    [Unhide] = "unhide",
    [Delete] = "delete",
    nil
};
```

⟨*enum* _anon_ (windows/rio/wctl.c)*2* 184b⟩≡ (258b)

```
enum
{
    Cd,
    Deltax,
    Deltay,
    Hidden,
    Id,

    Maxx,
    Maxy,
    Minx,
    Miny,

    PID,
    R,

    Scrolling,
    Noscrolling,
};
```

⟨*global* params 184c⟩≡ (258b)

```
static char *params[] = {
    [Cd]    = "-cd",
    [Deltax]   = "-dx",
    [Deltay]   = "-dy",
    [Hidden]   = "-hide",
    [Id]   = "-id",
    [Maxx]   = "-maxx",
    [Maxy]   = "-maxy",
    [Minx]   = "-minx",
```

```
    [Miny]   = "-miny",
    [PID]   = "-pid",
    [R]    = "-r",
    [Scrolling]   = "-scroll",
    [Noscrolling]  = "-noscroll",
    nil
};
```

⟨*function* goodrect 185a⟩≡ (246)

```
/*
 * Check that newly created window will be of manageable size
 */
int
goodrect(Rectangle r)
{
    if(!eqrect(canonrect(r), r))
        return 0;
    if(Dx(r)<100 || Dy(r)<3*font->height)
        return 0;
    /* must have some screen and border visible so we can move it out of the way */
    if(Dx(r) >= Dx(view->r) && Dy(r) >= Dy(view->r))
        return 0;
    /* reasonable sizes only please */
    if(Dx(r) > BIG*Dx(view->r))
        return 0;
    if(Dy(r) > BIG*Dx(view->r))
        return 0;
    return 1;
}
```

⟨*function* word 185b⟩≡ (258b)

```
static
int
word(char **sp, char *tab[])
{
    char *s, *t;
    int i;

    s = *sp;
    while(isspace(*s))
        s++;
    t = s;
    while(*s!='\0' && !isspace(*s))
        s++;
    for(i=0; tab[i]!=nil; i++)
        if(strncmp(tab[i], t, strlen(tab[i])) == 0){
            *sp = s;
            return i;
    }
    return -1;
}
```

⟨*function* set 185c⟩≡ (258b)

```
int
set(int sign, int neg, int abs, int pos)
{
    if(sign < 0)
        return neg;
    if(sign > 0)
        return pos;
```

```
    return abs;
}
```

⟨*function* newrect 186a⟩≡ (258b)

```
Rectangle
newrect(void)
{
    static int i = 0;
    int minx, miny, dx, dy;

    dx = min(600, Dx(view->r) - 2*Borderwidth);
    dy = min(400, Dy(view->r) - 2*Borderwidth);
    minx = 32 + 16*i;
    miny = 32 + 16*i;
    i++;
    i %= 10;

    return Rect(minx, miny, minx+dx, miny+dy);
}
```

⟨*function* shift 186b⟩≡ (258b)

```
void
shift(int *minp, int *maxp, int min, int max)
{
    if(*minp < min){
        *maxp += min-*minp;
        *minp = min;
    }
    if(*maxp > max){
        *minp += max-*maxp;
        *maxp = max;
    }
}
```

⟨*function* rectonscreen 186c⟩≡ (258b)

```
Rectangle
rectonscreen(Rectangle r)
{
    shift(&r.min.x, &r.max.x, view->r.min.x, view->r.max.x);
    shift(&r.min.y, &r.max.y, view->r.min.y, view->r.max.y);
    return r;
}
```

⟨*function* riostrtol 186d⟩≡ (258b)

```
/* permit square brackets, in the manner of %R */
int
riostrtol(char *s, char **t)
{
    int n;

    while(*s!='\0' && (*s==' ' || *s=='\t' || *s=='['))
        s++;
    if(*s == '[')
        s++;
    n = strtol(s, t, 10);
    if(*t != s)
        while((*t)[0] == ']')
            (*t)++;
    return n;
}
```

⟨*function* parsewctl 187⟩≡ (258b)

```
int
parsewctl(char **argp, Rectangle r, Rectangle *rp, int *pidp, int *idp, int *hiddenp, int *scrollingp, char **c
{
    int cmd, param, xy, sign;
    char *t;

    *pidp = 0;
    *hiddenp = 0;
    *scrollingp = scrolling;
    *cdp = nil;
    cmd = word(&s, cmds);
    if(cmd < 0){
        strcpy(err, "unrecognized wctl command");
        return -1;
    }
    if(cmd == New)
        r = newrect();

    strcpy(err, "missing or bad wctl parameter");

    while((param = word(&s, params)) >= 0){
        switch(param){ /* special cases */
        case Hidden:
            *hiddenp = 1;
            continue;
        case Scrolling:
            *scrollingp = 1;
            continue;
        case Noscrolling:
            *scrollingp = 0;
            continue;
        case R:
            r.min.x = riostrtol(s, &t);
            if(t == s)
                return -1;
            s = t;
            r.min.y = riostrtol(s, &t);
            if(t == s)
                return -1;
            s = t;
            r.max.x = riostrtol(s, &t);
            if(t == s)
                return -1;
            s = t;
            r.max.y = riostrtol(s, &t);
            if(t == s)
                return -1;
            s = t;
            continue;
        }
        while(isspace(*s))
            s++;
        if(param == Cd){
            *cdp = s;
            while(*s && !isspace(*s))
                s++;
            if(*s != '\0')
                *s++ = '\0';
            continue;
```

```
        }
        sign = 0;
        if(*s == '-'){
            sign = -1;
            s++;
        }else if(*s == '+'){
            sign = +1;
            s++;
        }
        if(!isdigit(*s))
            return -1;
        xy = riostrtol(s, &s);

        switch(param){
        case Minx:
            r.min.x = set(sign, r.min.x-xy, xy, r.min.x+xy);
            break;
        case Miny:
            r.min.y = set(sign, r.min.y-xy, xy, r.min.y+xy);
            break;
        case Maxx:
            r.max.x = set(sign, r.max.x-xy, xy, r.max.x+xy);
            break;
        case Maxy:
            r.max.y = set(sign, r.max.y-xy, xy, r.max.y+xy);
            break;
        case Deltax:
            r.max.x = set(sign, r.max.x-xy, r.min.x+xy, r.max.x+xy);
            break;
        case Deltay:
            r.max.y = set(sign, r.max.y-xy, r.min.y+xy, r.max.y+xy);
            break;
        case Id:
            if(idp != nil)
                *idp = xy;
            break;
        case PID:
            if(pidp != nil)
                *pidp = xy;
            break;
        case -1:
            strcpy(err, "unrecognized wctl parameter");
            return -1;
        }
    }

    *rp = rectonscreen(rectaddpt(r, view->r.min));

    while(isspace(*s))
        s++;
    if(cmd!=New && *s!='\0'){
        strcpy(err, "extraneous text in wctl message");
        return -1;
    }

    if(argp)
        *argp = s;

    return cmd;
}
```

⟨*function* wctlnew 189a⟩≡ (258b)

```
int
wctlnew(Rectangle rect, char *arg, int pid, int hideit, int scrollit, char *dir, char *err)
{
    char **argv;
    Image *i;

    if(!goodrect(rect)){
        strcpy(err, Ebadwr);
        return -1;
    }
    argv = emalloc(4*sizeof(char*));
    argv[0] = "rc";
    argv[1] = "-c";
    while(isspace(*arg))
        arg++;
    if(*arg == '\0'){
        argv[1] = "-i";
        argv[2] = nil;
    }else{
        argv[2] = arg;
        argv[3] = nil;
    }
    if(hideit)
        i = allocimage(display, rect, view->chan, false, DWhite);
    else
        i = allocwindow(desktop, rect, Refbackup, DWhite);
    if(i == nil){
        strcpy(err, Ewalloc);
        return -1;
    }
    border(i, rect, Selborder, red, ZP);

    new(i, hideit, scrollit, pid, dir, "/bin/rc", argv);

    free(argv); /* when new() returns, argv and args have been copied */
    return 1;
}
```

⟨*function* wtopme 189b⟩≡ (249)

```
void
wtopme(Window *w)
{
    if(w!=nil && w->i!=nil && !w->deleted && w->topped!=topped){
        topwindow(w->i);
        flushimage(display, 1);
        w->topped = ++topped;
    }
}
```

⟨*function* wbottomme 189c⟩≡ (249)

```
void
wbottomme(Window *w)
{
    if(w!=nil && w->i!=nil && !w->deleted){
        bottomwindow(w->i);
        flushimage(display, 1);
        w->topped = - ++topped;
    }
}
```

13.3 Recursive rio

⟨Mxxx *cases* 190a⟩≡ (60b)

```
MReshape,
```

⟨mousethread() *alts setup* 190b⟩+≡ (61b) ◁61c

```
alts[MReshape].c = mousectl->resizec;
alts[MReshape].v = nil;
alts[MReshape].op = CHANRCV;
```

⟨mousethread() *event loop cases* 190c⟩+≡ (61b) ◁61f

```
case MReshape:
    resized();
    break;
```

⟨*function* resized 190d⟩≡ (243b)

```
void
resized(void)
{
    Image *im;
    int i, j;
    bool ishidden;
    Rectangle r;
    Point o, n;
    Window *w;

    // updates view (and screen)
    if(getwindow(display, Refnone) < 0)
        error("failed to re-attach window");

    freescrtemps();
    freescreen(desktop);

    desktop = allocscreen(view, background, false);
    ⟨resized() sanity check desktop 191a⟩
    draw(view, view->r, background, nil, ZP);

    // old view rectangle
    o = subpt(viewr.max, viewr.min);
    n = subpt(view->clipr.max, view->clipr.min);

    for(i=0; i<nwindow; i++){
        w = windows[i];
        ⟨resized() continue if window was deleted 191b⟩
        r = rectsubpt(w->i->r, viewr.min);
        r.min.x = (r.min.x*n.x)/o.x;
        r.min.y = (r.min.y*n.y)/o.y;
        r.max.x = (r.max.x*n.x)/o.x;
        r.max.y = (r.max.y*n.y)/o.y;
        r = rectaddpt(r, view->clipr.min);

        ishidden = false;
        for(j=0; j<nhidden; j++)
            if(w == hidden[j]){
                ishidden = true;
                break;
            }
        if(ishidden){
            im = allocimage(display, r, view->chan, false, DWhite);
            r = ZR;
```

```
        }else
            im = allocwindow(desktop, r, Refbackup, DWhite);

        if(im)
            wsendctlmesg(w, Reshaped, r, im);
    }
    viewr = view->r;
    flushimage(display, true);
}
```

⟨resized() *sanity check desktop* 191a⟩≡ (190d)

```
if(desktop == nil)
    error("can't re-allocate desktop");
```

⟨resized() *continue if window was deleted* 191b⟩≡ (190d)

```
if(w->deleted)
    continue;
```

13.4 Advanced terminal editing

13.4.1 Snarf

⟨wkeyctl() *snarf and cut if not interrupt key* 191c⟩≡ (159b)

```
if(r != 0x7F){ // 0x7F = Interrupt key
    wsnarf(w);
    wcut(w);
}
```

Snarf menu

⟨button2menu() *cases* 191d⟩+≡ (167b) ◁168d 195a▷

```
case Cut:
    wsnarf(w);
    wcut(w);
    wscrdraw(w);
    break;

case Snarf:
    wsnarf(w);
    break;

case Paste:
    getsnarf();
    wpaste(w);
    wscrdraw(w);
    break;

case Send:
    getsnarf();
    wsnarf(w);
    if(nsnarf == 0)
        break;
    if(w->rawing){
        waddraw(w, snarf, nsnarf);
        if(snarf[nsnarf-1]!='\n' && snarf[nsnarf-1]!='\004')
                        waddraw(w, L"\n", 1);
    }else{
```

```
        winsert(w, snarf, nsnarf, w->nr);
        if(snarf[nsnarf-1]!='\n' && snarf[nsnarf-1]!='\004')
            winsert(w, L"\n", 1, w->nr);
    }
    wsetselect(w, w->nr, w->nr);
    wshow(w, w->nr);
    break;
```

⟨*function* wsnarf 192a⟩≡ (250b)

```
void
wsnarf(Window *w)
{
    if(w->q1 == w->q0)
        return;
    nsnarf = w->q1 - w->q0;
    snarf = runerealloc(snarf, nsnarf);
    snarfversion++; /* maybe modified by parent */
    runemove(snarf, w->r+w->q0, nsnarf);
    putsnarf();
}
```

⟨*function* wcut 192b⟩≡ (250b)

```
void
wcut(Window *w)
{
    if(w->q1 == w->q0)
        return;
    wdelete(w, w->q0, w->q1);
    wsetselect(w, w->q0, w->q0);
}
```

⟨*function* wpaste 192c⟩≡ (250b)

```
void
wpaste(Window *w)
{
    uint q0;

    if(nsnarf == 0)
        return;
    wcut(w);
    q0 = w->q0;
    if(w->rawing && q0==w->nr){
        waddraw(w, snarf, nsnarf);
        wsetselect(w, q0, q0);
    }else{
        q0 = winsert(w, snarf, nsnarf, w->q0);
        wsetselect(w, q0, q0+nsnarf);
    }
}
```

/mnt/wsys/snarf

⟨qid *cases* 192d⟩+≡ (100a) ◁176b 210d▷

```
Qsnarf,
```

⟨*dirtab array elements* 192e⟩+≡ (100f) ◁176c 210e▷

```
{ "snarf",  QTFILE, Qsnarf,  0600 },
```

⟨filsyswalk() *if snarf* 192f⟩≡ (103d)

```
if(snarffd>=0 && strcmp(x->req.wname[i], "snarf")==0)
    break; /* don't serve /dev/snarf if it's provided in the environment */
```

⟨xfidopen() *cases* 193a⟩+≡ (106f) ◁176d 216i▷

```
case Qsnarf:
    if(x->req.mode==ORDWR || x->req.mode==OWRITE){
        if(tsnarf)
            free(tsnarf); /* collision, but OK */
        ntsnarf = 0;
        tsnarf = malloc(1);
    }
    break;
```

⟨xfidclose() *other locals* 193b⟩≡ (107c)

```
int nb, nulls;
```

⟨xfidclose() *cases* 193c⟩+≡ (107c) ◁176e

```
/* odd behavior but really ok: replace snarf buffer when /dev/snarf is closed */
case Qsnarf:
    if(x->f->mode==ORDWR || x->f->mode==OWRITE){
        snarf = runerealloc(snarf, ntsnarf+1);
        cvttorunes(tsnarf, ntsnarf, snarf, &nb, &nsnarf, &nulls);
        free(tsnarf);
        tsnarf = nil;
        ntsnarf = 0;
    }
    break;
```

⟨xfidread() *cases* 193d⟩+≡ (109a) ◁177b 211a▷

```
/* The algorithm for snarf and text is expensive but easy and rarely used */
case Qsnarf:
    getsnarf();
    if(nsnarf)
        t = runetobyte(snarf, nsnarf, &n);
    else {
        t = nil;
        n = 0;
    }
    goto Text;
```

⟨*constant* MAXSNARF 193e⟩≡ (256)

```
#define MAXSNARF 100*1024
```

⟨xfidwrite() *cases* 193f⟩+≡ (109c) ◁178f 211b▷

```
case Qsnarf:
    /* always append only */
    if(ntsnarf > MAXSNARF){ /* avoid thrashing when people cut huge text */
        filsysrespond(x->fs, x, &fc, Elong);
        return;
    }
    tsnarf = erealloc(tsnarf, ntsnarf+cnt+1); /* room for NUL */
    memmove(tsnarf+ntsnarf, req->data, cnt);
    ntsnarf += cnt;
    snarfversion++;
    break;
```

⟨dostat() *adjust vers for snarf* 193g⟩≡ (110b)

```
if(dir->qid == Qsnarf)
    d.qid.vers = snarfversion;
```

⟨*global* tsnarf 193h⟩≡ (256)

```
static char *tsnarf;
```

⟨*global* ntsnarf 194a⟩≡ (256)
```
static int ntsnarf;
```

⟨*global* snarffd 194b⟩≡ (253a)
```
fdt  snarffd;
```

⟨main() *set some globals* 194c⟩+≡ (56a) ◁165g 217e▷
```
snarffd = open("/dev/snarf", OREAD|OCEXEC);
```

⟨*global* nsnarf 194d⟩≡ (253a)
```
int  nsnarf;
```

⟨*global* snarf 194e⟩≡ (253a)
```
Rune* snarf;
```

⟨*global* snarfversion 194f⟩≡ (242)
```
int  snarfversion; /* updated each time it is written */
```

⟨*function* putsnarf 194g⟩≡ (253a)
```
/*
 * /dev/snarf updates when the file is closed, so we must open our own
 * fd here rather than use snarffd
 */
void
putsnarf(void)
{
    int fd, i, n;

    if(snarffd<0 || nsnarf==0)
        return;
    fd = open("/dev/snarf", OWRITE);
    if(fd < 0)
        return;
    /* snarf buffer could be huge, so fprint will truncate; do it in blocks */
    for(i=0; i<nsnarf; i+=n){
        n = nsnarf-i;
        if(n >= 256)
            n = 256;
        if(fprint(fd, "%.*S", n, snarf+i) < 0)
            break;
    }
    close(fd);
}
```

⟨*function* getsnarf 194h⟩≡ (253a)
```
void
getsnarf(void)
{
    int i, n, nb, nulls;
    char *sn, buf[1024];

    if(snarffd < 0)
        return;
    sn = nil;
    i = 0;
    seek(snarffd, 0, 0);
    while((n = read(snarffd, buf, sizeof buf)) > 0){
        sn = erealloc(sn, i+n+1);
        memmove(sn+i, buf, n);
        i += n;
```

```
        sn[i] = 0;
    }
    if(i > 0){
        snarf = runerealloc(snarf, i+1);
        cvttorunes(sn, i, snarf, &nb, &nsnarf, &nulls);
        free(sn);
    }
}
```

13.4.2 Plumb

⟨button2menu() *cases* 195a⟩+≡ (167b) ◁191d

```
case Plumb:
    wplumb(w);
    break;
```

⟨*function* wplumb 195b⟩≡ (250b)

```
void
wplumb(Window *w)
{
    Plumbmsg *m;
    static int fd = -2;
    char buf[32];
    uint p0, p1;
    Cursor *c;

    if(fd == -2)
        fd = plumbopen("send", OWRITE|OCEXEC);
    if(fd < 0)
        return;
    m = emalloc(sizeof(Plumbmsg));
    m->src = estrdup("rio");
    m->dst = nil;
    m->wdir = estrdup(w->dir);
    m->type = estrdup("text");
    p0 = w->q0;
    p1 = w->q1;
    if(w->q1 > w->q0)
        m->attr = nil;
    else{
        while(p0>0 && w->r[p0-1]!=' ' && w->r[p0-1]!='\t' && w->r[p0-1]!='\n')
            p0--;
        while(p1<w->nr && w->r[p1]!=' ' && w->r[p1]!='\t' && w->r[p1]!='\n')
            p1++;
        sprint(buf, "click=%d", w->q0-p0);
        m->attr = plumbunpackattr(buf);
    }
    if(p1-p0 > messagesize-1024){
        plumbfree(m);
        return; /* too large for 9P */
    }
    m->data = runetobyte(w->r+p0, p1-p0, &m->ndata);
    if(plumbsend(fd, m) < 0){
        c = lastcursor;
        riosetcursor(&query, 1);
        sleep(300);
        riosetcursor(c, 1);
    }
    plumbfree(m);
```

```
}
```

⟨*struct* Plumbmsg 196a⟩≡ (236)

```
struct Plumbmsg
{
    char  *src;
    char  *dst;
    char  *wdir;
    char  *type;
    Plumbattr  *attr;
    int   ndata;
    char  *data;
};
```

⟨*struct* Plumbattr 196b⟩≡ (236)

```
struct Plumbattr
{
    char  *name;
    char  *value;
    Plumbattr  *next;
};
```

⟨*function* plumbsendtext 196c⟩≡ (284)

```
int
plumbsendtext(int fd, char *src, char *dst, char *wdir, char *data)
{
    Plumbmsg m;

    m.src = src;
    m.dst = dst;
    m.wdir = wdir;
    m.type = "text";
    m.attr = nil;
    m.ndata = strlen(data);
    m.data = data;
    return plumbsend(fd, &m);
}
```

Plumb messages

⟨*function* plumbopen 196d⟩≡ (283b)

```
int
plumbopen(char *name, int omode)
{
    int fd, f;
    char *s, *plumber;
    char buf[128], err[ERRMAX];

    if(name[0] == '/')
        return open(name, omode);

    /* find elusive plumber */
    if(access("/mnt/plumb/send", AWRITE) >= 0)
        plumber = "/mnt/plumb";
    else if(access("/mnt/term/mnt/plumb/send", AWRITE) >= 0)
        plumber = "/mnt/term/mnt/plumb";
    else{
        /* last resort: try mounting service */
        plumber = "/mnt/plumb";
```

```
        s = getenv("plumbsrv");
        if(s == nil)
            return -1;
        f = open(s, ORDWR);
        free(s);
        if(f < 0)
            return -1;
        if(mount(f, -1, "/mnt/plumb", MREPL, "") < 0){
            close(f);
            return -1;
        }
        if(access("/mnt/plumb/send", AWRITE) < 0)
            return -1;
    }

    snprint(buf, sizeof buf, "%s/%s", plumber, name);
    fd = open(buf, omode);
    if(fd >= 0)
        return fd;

    /* try creating port; used by non-standard plumb implementations */
    rerrstr(err, sizeof err);
    fd = create(buf, omode, 0600);
    if(fd >= 0)
        return fd;
    errstr(err, sizeof err);

    return -1;
}
```

⟨*function* Strlen 197a⟩≡ (283b)

```
static int
Strlen(char *s)
{
    if(s == nil)
        return 0;
    return strlen(s);
}
```

⟨*function* Strcpy 197b⟩≡ (283b)

```
static char*
Strcpy(char *s, char *t)
{
    if(t == nil)
        return s;
    return strcpy(s, t) + strlen(t);
}
```

⟨*function* quote 197c⟩≡ (283b)

```
/* quote attribute value, if necessary */
static char*
quote(char *s, char *buf, char *bufe)
{
    char *t;
    int c;

    if(s == nil){
        buf[0] = '\0';
        return buf;
    }
```

```
    if(strpbrk(s, " '=\t") == nil)
        return s;
    t = buf;
    *t++ = '\'';
    while(t < bufe-2){
        c = *s++;
        if(c == '\0')
            break;
        *t++ = c;
        if(c == '\'')
            *t++ = c;
    }
    *t++ = '\'';
    *t = '\0';
    return buf;
}
```

⟨*function* plumbpackattr 198a⟩≡ (283b)

```
char*
plumbpackattr(Plumbattr *attr)
{
    int n;
    Plumbattr *a;
    char *s, *t, *buf, *bufe;

    if(attr == nil)
        return nil;
    if((buf = malloc(4096)) == nil)
        return nil;
    bufe = buf + 4096;
    n = 0;
    for(a=attr; a!=nil; a=a->next)
        n += Strlen(a->name) + 1 + Strlen(quote(a->value, buf, bufe)) + 1;
    s = malloc(n);
    if(s == nil) {
        free(buf);
        return nil;
    }
    t = s;
    *t = '\0';
    for(a=attr; a!=nil; a=a->next){
        if(t != s)
            *t++ = ' ';
        strcpy(t, a->name);
        strcat(t, "=");
        strcat(t, quote(a->value, buf, bufe));
        t += strlen(t);
    }
    if(t > s+n)
        abort();
    free(buf);
    return s;
}
```

⟨*function* plumblookup 198b⟩≡ (283b)

```
char*
plumblookup(Plumbattr *attr, char *name)
{
    while(attr){
        if(strcmp(attr->name, name) == 0)
```

```
            return attr->value;
        attr = attr->next;
    }
    return nil;
}
```

⟨*function* plumbpack 199a⟩≡ (283b)

```
char*
plumbpack(Plumbmsg *m, int *np)
{
    int n, ndata;
    char *buf, *p, *attr;

    ndata = m->ndata;
    if(ndata < 0)
        ndata = Strlen(m->data);
    attr = plumbpackattr(m->attr);
    n = Strlen(m->src)+1 + Strlen(m->dst)+1 + Strlen(m->wdir)+1 +
        Strlen(m->type)+1 + Strlen(attr)+1 + 16 + ndata;
    buf = malloc(n+1); /* +1 for '\0' */
    if(buf == nil){
        free(attr);
        return nil;
    }
    p = Strcpy(buf, m->src);
    *p++ = '\n';
    p = Strcpy(p, m->dst);
    *p++ = '\n';
    p = Strcpy(p, m->wdir);
    *p++ = '\n';
    p = Strcpy(p, m->type);
    *p++ = '\n';
    p = Strcpy(p, attr);
    *p++ = '\n';
    p += sprint(p, "%d\n", ndata);
    memmove(p, m->data, ndata);
    *np = (p-buf)+ndata;
    buf[*np] = '\0'; /* null terminate just in case */
    if(*np >= n+1)
        abort();
    free(attr);
    return buf;
}
```

⟨*function* plumbsend 199b⟩≡ (283b)

```
int
plumbsend(int fd, Plumbmsg *m)
{
    char *buf;
    int n;

    buf = plumbpack(m, &n);
    if(buf == nil)
        return -1;
    n = write(fd, buf, n);
    free(buf);
    return n;
}
```

⟨*function* plumbline 200a⟩≡ (283b)

```
static int
plumbline(char **linep, char *buf, int i, int n, int *bad)
{
    int starti;
    char *p;

    starti = i;
    while(i<n && buf[i]!='\n')
        i++;
    if(i == n)
        *bad = 1;
    else{
        p = malloc((i-starti) + 1);
        if(p == nil)
            *bad = 1;
        else{
            memmove(p, buf+starti, i-starti);
            p[i-starti] = '\0';
        }
        *linep = p;
        i++;
    }
    return i;
}
```

⟨*function* plumbfree 200b⟩≡ (283b)

```
void
plumbfree(Plumbmsg *m)
{
    Plumbattr *a, *next;

    free(m->src);
    free(m->dst);
    free(m->wdir);
    free(m->type);
    for(a=m->attr; a!=nil; a=next){
        next = a->next;
        free(a->name);
        free(a->value);
        free(a);
    }
    free(m->data);
    free(m);
}
```

⟨*function* plumbunpackattr 200c⟩≡ (283b)

```
Plumbattr*
plumbunpackattr(char *p)
{
    Plumbattr *attr, *prev, *a;
    char *q, *v, *buf, *bufe;
    int c, quoting;

    buf = malloc(4096);
    if(buf == nil)
        return nil;
    bufe = buf + 4096;
    attr = prev = nil;
    while(*p!='\0' && *p!='\n'){
```

```
while(*p==' ' || *p=='\t')
    p++;
if(*p == '\0')
    break;
for(q=p; *q!='\0' && *q!='\n' && *q!=' ' && *q!='\t'; q++)
    if(*q == '=')
        break;
if(*q != '=')
    break; /* malformed attribute */
a = malloc(sizeof(Plumbattr));
if(a == nil)
    break;
a->name = malloc(q-p+1);
if(a->name == nil){
    free(a);
    break;
}
memmove(a->name, p, q-p);
a->name[q-p] = '\0';
/* process quotes in value */
q++; /* skip '=' */
v = buf;
quoting = 0;
while(*q!='\0' && *q!='\n'){
    if(v >= bufe)
        break;
    c = *q++;
    if(quoting){
        if(c == '\''){
            if(*q == '\'')
                q++;
            else{
                quoting = 0;
                continue;
            }
        }
    }else{
        if(c==' ' || c=='\t')
            break;
        if(c == '\''){
            quoting = 1;
            continue;
        }
    }
    *v++ = c;
}
a->value = malloc(v-buf+1);
if(a->value == nil){
    free(a->name);
    free(a);
    break;
}
memmove(a->value, buf, v-buf);
a->value[v-buf] = '\0';
a->next = nil;
if(prev == nil)
    attr = a;
else
    prev->next = a;
prev = a;
```

```
        p = q;
    }
    free(buf);
    return attr;
}
```

⟨*function* plumbaddattr 202a⟩≡ (283b)

```
Plumbattr*
plumbaddattr(Plumbattr *attr, Plumbattr *new)
{
    Plumbattr *l;

    l = attr;
    if(l == nil)
        return new;
    while(l->next != nil)
        l = l->next;
    l->next = new;
    return attr;
}
```

⟨*function* plumbdelattr 202b⟩≡ (283b)

```
Plumbattr*
plumbdelattr(Plumbattr *attr, char *name)
{
    Plumbattr *l, *prev;

    prev = nil;
    for(l=attr; l!=nil; l=l->next){
        if(strcmp(name, l->name) == 0)
            break;
        prev = l;
    }
    if(l == nil)
        return nil;
    if(prev)
        prev->next = l->next;
    else
        attr = l->next;
    free(l->name);
    free(l->value);
    free(l);
    return attr;
}
```

⟨*function* plumbunpackpartial 202c⟩≡ (283b)

```
Plumbmsg*
plumbunpackpartial(char *buf, int n, int *morep)
{
    Plumbmsg *m;
    int i, bad;
    char *ntext, *attr;

    m = malloc(sizeof(Plumbmsg));
    if(m == nil)
        return nil;
    memset(m, 0, sizeof(Plumbmsg));
    if(morep != nil)
        *morep = 0;
    bad = 0;
```

```
    i = plumbline(&m->src, buf, 0, n, &bad);
    i = plumbline(&m->dst, buf, i, n, &bad);
    i = plumbline(&m->wdir, buf, i, n, &bad);
    i = plumbline(&m->type, buf, i, n, &bad);
    i = plumbline(&attr, buf, i, n, &bad);
    i = plumbline(&ntext, buf, i, n, &bad);
    if(bad){
        plumbfree(m);
        return nil;
    }
    m->attr = plumbunpackattr(attr);
    free(attr);
    m->ndata = atoi(ntext);
    if(m->ndata != n-i){
        bad = 1;
        if(morep!=nil && m->ndata>n-i)
            *morep = m->ndata - (n-i);
    }
    free(ntext);
    if(!bad){
        m->data = malloc(n-i+1); /* +1 for '\0' */
        if(m->data == nil)
            bad = 1;
        else{
            memmove(m->data, buf+i, m->ndata);
            m->ndata = n-i;
            /* null-terminate in case it's text */
            m->data[m->ndata] = '\0';
        }
    }
    if(bad){
        plumbfree(m);
        m = nil;
    }
    return m;
}
```

⟨*function* plumbunpack 203a⟩≡ (283b)

```
Plumbmsg*
plumbunpack(char *buf, int n)
{
    return plumbunpackpartial(buf, n, nil);
}
```

⟨*function* plumbrecv 203b⟩≡ (283b)

```
Plumbmsg*
plumbrecv(int fd)
{
    char *buf;
    Plumbmsg *m;
    int n, more;

    buf = malloc(8192);
    if(buf == nil)
        return nil;
    n = read(fd, buf, 8192);
    m = nil;
    if(n > 0){
        m = plumbunpackpartial(buf, n, &more);
        if(m==nil && more>0){
```

```
            /* we now know how many more bytes to read for complete message */
            buf = realloc(buf, n+more);
            if(buf == nil)
                return nil;
            if(readn(fd, buf+n, more) == more)
                m = plumbunpackpartial(buf, n+more, nil);
        }
    }
    free(buf);
    return m;
}
```

Event-based plumbing

⟨*struct* EQueue 204a⟩≡ (283a)

```
struct EQueue
{
    int  id;
    char  *buf;
    int  nbuf;
    EQueue *next;
};
```

⟨*global* equeue 204b⟩≡ (283a)

```
static EQueue *equeue;
```

⟨*global* eqlock 204c⟩≡ (283a)

```
static Lock  eqlock;
```

⟨*function* partial 204d⟩≡ (283a)

```
static
int
partial(int id, Event *e, uchar *b, int n)
{
    EQueue *eq, *p;
    int nmore;

    lock(&eqlock);
    for(eq = equeue; eq != nil; eq = eq->next)
        if(eq->id == id)
            break;
    unlock(&eqlock);
    if(eq == nil)
        return 0;
    /* partial message exists for this id */
    eq->buf = realloc(eq->buf, eq->nbuf+n);
    if(eq->buf == nil)
        drawerror(display, "eplumb: cannot allocate buffer");
    memmove(eq->buf+eq->nbuf, b, n);
    eq->nbuf += n;
    e->v = plumbunpackpartial((char*)eq->buf, eq->nbuf, &nmore);
    if(nmore == 0){ /* no more to read in this message */
        lock(&eqlock);
        if(eq == equeue)
            equeue = eq->next;
        else{
            for(p = equeue; p!=nil && p->next!=eq; p = p->next)
                ;
            if(p == nil)
```

```
                drawerror(display, "eplumb: bad event queue");
            p->next = eq->next;
        }
        unlock(&eqlock);
        free(eq->buf);
        free(eq);
    }
    return 1;
}
```

⟨*function* addpartial 205a⟩≡ (283a)

```
static
void
addpartial(int id, char *b, int n)
{
    EQueue *eq;

    eq = malloc(sizeof(EQueue));
    if(eq == nil)
        return;
    eq->id = id;
    eq->nbuf = n;
    eq->buf = malloc(n);
    if(eq->buf == nil){
        free(eq);
        return;
    }
    memmove(eq->buf, b, n);
    lock(&eqlock);
    eq->next = equeue;
    equeue = eq;
    unlock(&eqlock);
}
```

⟨*function* plumbevent 205b⟩≡ (283a)

```
static
int
plumbevent(int id, Event *e, uchar *b, int n)
{
    int nmore;

    if(partial(id, e, b, n) == 0){
        /* no partial message already waiting for this id */
        e->v = plumbunpackpartial((char*)b, n, &nmore);
        if(nmore > 0) /* incomplete message */
            addpartial(id, (char*)b, n);
    }
    if(e->v == nil)
        return 0;
    return id;
}
```

⟨*function* eplumb 205c⟩≡ (283a)

```
int
eplumb(int key, char *port)
{
    int fd;

    fd = plumbopen(port, OREAD|OCEXEC);
    if(fd < 0)
```

```
        return -1;
    return estartfn(key, fd, 8192, plumbevent);
}
```

13.4.3 Auto complete

⟨wkeyctl() *special key cases and no special mode* 206a⟩+≡ (159b) ◁164d

```
case 0x06: /* ^F: file name completion */
case Kins:  /* Insert: file name completion */
    rp = namecomplete(w);
    if(rp == nil)
        return;
    nr = runestrlen(rp);
    q0 = w->q0;
    q0 = winsert(w, rp, nr, q0);
    wshow(w, q0+nr);
    free(rp);
    return;
```

⟨*struct* Completion 206b⟩≡ (235a)

```
struct Completion{
    uchar advance; /* whether forward progress has been made */
    uchar complete; /* whether the completion now represents a file or directory */
    char *string; /* the string to advance, suffixed " " or "/" for file or directory */
    int nmatch;  /* number of files that matched */
    int nfile;  /* number of files returned */
    char **filename; /* their names */
};
```

⟨*function* namecomplete 206c⟩≡ (250b)

```
Rune*
namecomplete(Window *w)
{
    int nstr, npath;
    Rune *rp, *path, *str;
    Completion *c;
    char *s, *dir, *root;

    /* control-f: filename completion; works back to white space or / */
    if(w->q0<w->nr && w->r[w->q0]>' ') /* must be at end of word */
        return nil;
    nstr = windfilewidth(w, w->q0, true);
    str = runemalloc(nstr);
    runemove(str, w->r+(w->q0-nstr), nstr);
    npath = windfilewidth(w, w->q0-nstr, false);
    path = runemalloc(npath);
    runemove(path, w->r+(w->q0-nstr-npath), npath);
    rp = nil;

    /* is path rooted? if not, we need to make it relative to window path */
    if(npath>0 && path[0]=='/'){
        dir = malloc(UTFmax*npath+1);
        sprint(dir, "%.*S", npath, path);
    }else{
        if(strcmp(w->dir, "") == 0)
            root = ".";
        else
            root = w->dir;
        dir = malloc(strlen(root)+1+UTFmax*npath+1);
```

```
        sprint(dir, "%s/%.*S", root, npath, path);
    }
    dir = cleanname(dir);

    s = smprint("%.*S", nstr, str);
    c = complete(dir, s);
    free(s);
    if(c == nil)
        goto Return;

    if(!c->advance)
        showcandidates(w, c);

    if(c->advance)
        rp = runesmprint("%s", c->string);

  Return:
    freecompletion(c);
    free(dir);
    free(path);
    free(str);
    return rp;
}
```

⟨*function* showcandidates 207⟩≡ (250b)

```
void
showcandidates(Window *w, Completion *c)
{
    int i;
    Fmt f;
    Rune *rp;
    uint nr, qline, q0;
    char *s;

    runefmtstrinit(&f);
    if (c->nmatch == 0)
        s = "[no matches in ";
    else
        s = "[";
    if(c->nfile > 32)
        fmtprint(&f, "%s%d files]\n", s, c->nfile);
    else{
        fmtprint(&f, "%s", s);
        for(i=0; i<c->nfile; i++){
            if(i > 0)
                fmtprint(&f, " ");
            fmtprint(&f, "%s", c->filename[i]);
        }
        fmtprint(&f, "]\n");
    }
    /* place text at beginning of line before host point */
    qline = w->qh;
    while(qline>0 && w->r[qline-1] != '\n')
        qline--;

    rp = runefmtstrflush(&f);
    nr = runestrlen(rp);

    q0 = w->q0;
    q0 += winsert(w, rp, runestrlen(rp), qline) - qline;
```

```
    free(rp);
    wsetselect(w, q0+nr, q0+nr);
}
```

⟨*function* windfilewidth 208a⟩≡ (250b)

```
int
windfilewidth(Window *w, uint q0, int oneelement)
{
    uint q;
    Rune r;

    q = q0;
    while(q > 0){
        r = w->r[q-1];
        if(r<=' ')
            break;
        if(oneelement && r=='/')
            break;
        --q;
    }
    return q0-q;
}
```

complete()

⟨*function* complete 208b⟩≡ (279a)

```
Completion*
complete(char *dir, char *s)
{
    long i, l, n, nfile, len, nbytes;
    int fd, minlen;
    Dir *dirp;
    char **name, *p;
    ulong* mode;
    Completion *c;

    if(strchr(s, '/') != nil){
        werrstr("slash character in name argument to complete()");
        return nil;
    }

    fd = open(dir, OREAD);
    if(fd < 0)
        return nil;

    n = dirreadall(fd, &dirp);
    if(n <= 0){
        close(fd);
        return nil;
    }

    /* find longest string, for allocation */
    len = 0;
    for(i=0; i<n; i++){
        l = strlen(dirp[i].name) + 1 + 1; /* +1 for /   +1 for \0 */
        if(l > len)
            len = l;
    }
```

```
name = malloc(n*sizeof(char*));
mode = malloc(n*sizeof(ulong));
c = malloc(sizeof(Completion) + len);
if(name == nil || mode == nil || c == nil)
    goto Return;
memset(c, 0, sizeof(Completion));

/* find the matches */
len = strlen(s);
nfile = 0;
minlen = 1000000;
for(i=0; i<n; i++)
    if(strncmp(s, dirp[i].name, len) == 0){
        name[nfile] = dirp[i].name;
        mode[nfile] = dirp[i].mode;
        if(minlen > strlen(dirp[i].name))
            minlen = strlen(dirp[i].name);
        nfile++;
    }

if(nfile > 0) {
    /* report interesting results */
    /* trim length back to longest common initial string */
    for(i=1; i<nfile; i++)
        minlen = longestprefixlength(name[0], name[i], minlen);

    /* build the answer */
    c->complete = (nfile == 1);
    c->advance = c->complete || (minlen > len);
    c->string = (char*)(c+1);
    memmove(c->string, name[0]+len, minlen-len);
    if(c->complete)
        c->string[minlen++ - len] = (mode[0]&DMDIR)? '/' : ' ';
    c->string[minlen - len] = '\0';
    c->nmatch = nfile;
} else {
    /* no match, so return all possible strings */
    for(i=0; i<n; i++){
        name[i] = dirp[i].name;
        mode[i] = dirp[i].mode;
    }
    nfile = n;
    c->nmatch = 0;
}

/* attach list of names */
nbytes = nfile * sizeof(char*);
for(i=0; i<nfile; i++)
    nbytes += strlen(name[i]) + 1 + 1;
c->filename = malloc(nbytes);
if(c->filename == nil)
    goto Return;
p = (char*)(c->filename + nfile);
for(i=0; i<nfile; i++){
    c->filename[i] = p;
    strcpy(p, name[i]);
    p += strlen(p);
    if(mode[i] & DMDIR)
        *p++ = '/';
    *p++ = '\0';
```

```
    }
    c->nfile = nfile;
    qsort(c->filename, c->nfile, sizeof(c->filename[0]), strpcmp);

  Return:
    free(name);
    free(mode);
    free(dirp);
    close(fd);
    return c;
}
```

⟨*function* longestprefixlength 210a⟩≡ (279a)

```
static int
longestprefixlength(char *a, char *b, int n)
{
    int i, w;
    Rune ra, rb;

    for(i=0; i<n; i+=w){
        w = chartorune(&ra, a);
        chartorune(&rb, b);
        if(ra != rb)
            break;
        a += w;
        b += w;
    }
    return i;
}
```

⟨*function* freecompletion 210b⟩≡ (279a)

```
void
freecompletion(Completion *c)
{
    if(c){
        free(c->filename);
        free(c);
    }
}
```

⟨*function* strpcmp 210c⟩≡ (279a)

```
static int
strpcmp(const void *va, const void *vb)
{
    char *a, *b;

    a = *(char**)va;
    b = *(char**)vb;
    return strcmp(a, b);
}
```

/mnt/wsys/wdir

⟨qid *cases* 210d⟩+≡ (100a) ◁192d 216g▷

```
Qwdir,
```

⟨*dirtab array elements* 210e⟩+≡ (100f) ◁192e 216h▷

```
{ "wdir",  QTFILE, Qwdir,  0600 },
```

⟨xfidread() *cases* 211a⟩+≡ (109a) ◁193d

```
case Qwdir:
    t = estrdup(w->dir);
    n = strlen(t);
    goto Text;
```

⟨xfidwrite() *cases* 211b⟩+≡ (109c) ◁193f 216j▷

```
case Qwdir:
    if(cnt == 0)
        break;
    if(req->data[cnt-1] == '\n'){
        if(cnt == 1)
            break;
        req->data[cnt-1] = '\0';
    }
    /* assume data comes in a single write */
    /*
      * Problem: programs like dossrv, ftp produce illegal UTF;
      * we must cope by converting it first.
      */
    snprint(buf, sizeof buf, "%.*s", cnt, req->data);
    if(buf[0] == '/'){
        free(w->dir);
        w->dir = estrdup(buf);
    }else{
        p = emalloc(strlen(w->dir) + 1 + strlen(buf) + 1);
        sprint(p, "%s/%s", w->dir, buf);
        free(w->dir);
        w->dir = cleanname(p);
    }
    break;
```

13.4.4 Word selection

⟨*global* clickwin 211c⟩≡ (250b)

```
static Window *clickwin;
```

⟨*global* clickmsec 211d⟩≡ (250b)

```
static uint clickmsec;
```

⟨*global* selectwin 211e⟩≡ (250b)

```
static Window *selectwin;
```

⟨*global* selectq 211f⟩≡ (250b)

```
static uint selectq;
```

⟨*function* wselect 211g⟩≡ (250b)

```
void
wselect(Window *w)
{
    uint q0, q1;
    int b, x, y, first;
    Frame *frm = &w->frm;

    first = 1;
    selectwin = w;
    /*
     * Double-click immediately if it might make sense.
     */
```

```
b = w->mc.buttons;
q0 = w->q0;
q1 = w->q1;
selectq = w->org + frcharofpt(frm, w->mc.xy);
if(clickwin==w && w->mc.msec-clickmsec<500)
if(q0==q1 && selectq==w->q0){
    wdoubleclick(w, &q0, &q1);
    wsetselect(w, q0, q1);
    flushimage(display, 1);
    x = w->mc.xy.x;
    y = w->mc.xy.y;
    /* stay here until something interesting happens */
    do
        readmouse(&w->mc);
    while(w->mc.buttons==b && abs(w->mc.xy.x-x)<3 && abs(w->mc.xy.y-y)<3);
    w->mc.xy.x = x; /* in case we're calling frselect */
    w->mc.xy.y = y;
    q0 = w->q0; /* may have changed */
    q1 = w->q1;
    selectq = q0;
}
if(w->mc.buttons == b){
    frm->scroll = framescroll;
    frselect(frm, &w->mc);
    /* horrible botch: while asleep, may have lost selection altogether */
    if(selectq > w->nr)
        selectq = w->org + frm->p0;
    frm->scroll = nil;
    if(selectq < w->org)
        q0 = selectq;
    else
        q0 = w->org + frm->p0;
    if(selectq > w->org + frm->nchars)
        q1 = selectq;
    else
        q1 = w->org + frm->p1;
}
if(q0 == q1){
    if(q0==w->q0 && clickwin==w && w->mc.msec-clickmsec<500){
        wdoubleclick(w, &q0, &q1);
        clickwin = nil;
    }else{
        clickwin = w;
        clickmsec = w->mc.msec;
    }
}else
    clickwin = nil;
wsetselect(w, q0, q1);
flushimage(display, 1);
while(w->mc.buttons){
    w->mc.msec = 0;
    b = w->mc.buttons;
    if(b & 6){
        if(b & 2){
            wsnarf(w);
            wcut(w);
        }else{
            if(first){
                first = 0;
                getsnarf();
```

```
                }
                wpaste(w);
            }
        }
        wscrdraw(w);
        flushimage(display, 1);
        while(w->mc.buttons == b)
            readmouse(&w->mc);
        clickwin = nil;
    }
}
```

⟨*global* `left1` 213a⟩≡ (250b)

```
static Rune left1[] =  { L'{', L'[', L'(', L'<', L'<<', 0 };
```

⟨*global* `right1` 213b⟩≡ (250b)

```
static Rune right1[] = { L'}', L']', L')', L'>', L'>>', 0 };
```

⟨*global* `left2` 213c⟩≡ (250b)

```
static Rune left2[] =  { L'\n', 0 };
```

⟨*global* `left3` 213d⟩≡ (250b)

```
static Rune left3[] =  { L'\'', L'"', L'`', 0 };
```

⟨*global* `left` 213e⟩≡ (250b)

```
Rune *left[] = {
    left1,
    left2,
    left3,
    nil
};
```

⟨*global* `right` 213f⟩≡ (250b)

```
Rune *right[] = {
    right1,
    left2,
    left3,
    nil
};
```

⟨*function* `wdoubleclick` 213g⟩≡ (250b)

```
void
wdoubleclick(Window *w, uint *q0, uint *q1)
{
    int c, i;
    Rune *r, *l, *p;
    uint q;

    for(i=0; left[i]!=nil; i++){
        q = *q0;
        l = left[i];
        r = right[i];
        /* try matching character to left, looking right */
        if(q == 0)
            c = '\n';
        else
            c = w->r[q-1];
        p = strrune(l, c);
        if(p != nil){
            if(wclickmatch(w, c, r[p-l], 1, &q))
```

```
                *q1 = q-(c!='\n');
            return;
        }
        /* try matching character to right, looking left */
        if(q == w->nr)
            c = '\n';
        else
            c = w->r[q];
        p = strrune(r, c);
        if(p != nil){
            if(wclickmatch(w, c, l[p-r], -1, &q)){
                *q1 = *q0+(*q0<w->nr && c=='\n');
                *q0 = q;
                if(c!='\n' || q!=0 || w->r[0]=='\n')
                    (*q0)++;
            }
            return;
        }
    }
    /* try filling out word to right */
    while(*q1<w->nr && isalnum(w->r[*q1]))
        (*q1)++;
    /* try filling out word to left */
    while(*q0>0 && isalnum(w->r[*q0-1]))
        (*q0)--;
}
```

⟨*function* `wclickmatch` 214a⟩≡ (250b)

```
int
wclickmatch(Window *w, int cl, int cr, int dir, uint *q)
{
    Rune c;
    int nest;

    nest = 1;
    for(;;){
        if(dir > 0){
            if(*q == w->nr)
                break;
            c = w->r[*q];
            (*q)++;
        }else{
            if(*q == 0)
                break;
            (*q)--;
            c = w->r[*q];
        }
        if(c == cr){
            if(--nest==0)
                return 1;
        }else if(c == cl)
            nest++;
    }
    return cl=='\n' && nest==1;
}
```

13.5 Automatic scrolling: `rio -s`

⟨*global* `scrolling` 214b⟩≡ (242)

```
bool  scrolling;
```

⟨main() *command line processing* 215a⟩≡ (56a) 215c ▷

```
case 's':
    scrolling = true;
    break;
```

13.6 Initial command: rio -i

⟨main() *locals* 215b⟩+≡ (56a) ◁165f 215f ▷

```
char *initstr = nil;
```

⟨main() *command line processing* 215c⟩+≡ (56a) ◁215a 215g ▷

```
case 'i':
    initstr = ARGF();
    if(initstr == nil)
        usage();
    break;
```

⟨main() *if initstr or kdbin* 215d⟩≡ (230i) 216b ▷

```
if(initstr)
    proccreate(initcmd, initstr, STACK);
```

⟨*function* initcmd 215e⟩≡ (243a)

```
void
initcmd(void *arg)
{
    char *cmd;

    cmd = arg;
    rfork(RFENVG|RFFDG|RFNOTEG|RFNAMEG);
    procexecl(nil, "/bin/rc", "rc", "-c", cmd, nil);
    fprint(STDERR, "rio: exec failed: %r\n");
    exits("exec");
}
```

13.7 Fake keyboard input: rio -k

13.7.1 rio -k

⟨main() *locals* 215f⟩+≡ (56a) ◁215b 216a ▷

```
char *kbdin = nil;
```

⟨main() *command line processing* 215g⟩+≡ (56a) ◁215c 217d ▷

```
case 'k':
    if(kbdin != nil)
        usage();
    kbdin = ARGF();
    if(kbdin == nil)
        usage();
    break;
```

⟨*global* wkeyboard 215h⟩≡ (242)

```
Window *wkeyboard; /* window of simulated keyboard */
```

⟨*global* kbdargv 215i⟩≡ (243a)

```
char *kbdargv[] = { "rc", "-c", nil, nil };
```

⟨main() *locals* 216a⟩+≡ (56a) ◁215f

```
Image *i;
Rectangle r;
```

⟨main() *if initstr or kdbin* 216b⟩+≡ (230i) ◁215d

```
if(kbdin){
    kbdargv[2] = kbdin;
    r = view->r;
    r.max.x = r.min.x+300;
    r.max.y = r.min.y+80;
    i = allocwindow(desktop, r, Refbackup, DWhite);
    wkeyboard = new(i, false, scrolling, 0, nil, "/bin/rc", kbdargv);
    if(wkeyboard == nil)
        error("can't create keyboard window");
}
```

13.7.2 wkeyboard

⟨mousethread() *if wkeyboard and button 6* 216c⟩≡ (61f)

```
if(wkeyboard!=nil && (mouse->buttons & (1<<5))){
    keyboardhide();
    break;
}
```

⟨mousethread() *if wkeyboard and ptinrect* 216d⟩≡ (61f)

```
/* override everything for the keyboard window */
if(wkeyboard!=nil && ptinrect(mouse->xy, wkeyboard->screenr)){
    /* make sure it's on top; this call is free if it is */
    wtopme(wkeyboard);
    winput = wkeyboard;
}
```

⟨wcurrent() *if wkeyboard* 216e⟩≡ (87c)

```
if(wkeyboard!=nil && w==wkeyboard)
    return;
```

⟨wclosewin() *if wkeyboard* 216f⟩≡ (88g)

```
if(w == wkeyboard)
    wkeyboard = nil;
```

13.7.3 /mnt/wsys/kdbin

⟨qid *cases* 216g⟩+≡ (100a) ◁210d

```
Qkbdin,
```

⟨*dirtab array elements* 216h⟩+≡ (100f) ◁210e

```
{ "kbdin",  QTFILE, Qkbdin,  0200 },
```

⟨xfidopen() *cases* 216i⟩+≡ (106f) ◁193a

```
case Qkbdin:
    if(w !=  wkeyboard){
        filsysrespond(x->fs, x, &fc, Eperm);
        return;
    }
    break;
```

⟨xfidwrite() *cases* 216j⟩+≡ (109c) ◁211b

```
case Qkbdin:
    keyboardsend(req->data, cnt);
    break;
```

⟨*function* keyboardsend 217a⟩≡ (256)

```
/*
 * Used by /dev/kbdin
 */
void
keyboardsend(char *s, int cnt)
{
    Rune *r;
    int i, nb, nr;

    r = runemalloc(cnt);
    /* BUGlet: partial runes will be converted to error runes */
    cvttorunes(s, cnt, r, &nb, &nr, nil);
    for(i=0; i<nr; i++)
        send(keyboardctl->c, &r[i]);
    free(r);
}
```

13.7.4 Keyboard hide

⟨*function* keyboardhide 217b⟩≡ (243b)

```
/*
 * Button 6 - keyboard toggle - has been pressed.
 * Send event to keyboard, wait for button up, send that.
 * Note: there is no coordinate translation done here; this
 * is just about getting button 6 to the keyboard simulator.
 */
void
keyboardhide(void)
{
    send(wkeyboard->mc.c, mouse);
    do
        readmouse(mousectl);
    while(mouse->buttons & (1<<5));
    send(wkeyboard->mc.c, mouse);
}
```

13.8 Font selection: `rio -f`

⟨*global* fontname 217c⟩≡ (243a)

```
char  *fontname;
```

⟨main() *command line processing* 217d⟩+≡ (56a) ◁215g

```
case 'f':
    fontname = ARGF();
    if(fontname == nil)
        usage();
    break;
```

⟨main() *set some globals* 217e⟩+≡ (56a) ◁194c

```
if(fontname == nil)
    fontname = getenv("font");
if(fontname == nil)
    fontname = "/lib/font/bit/lucm/unicode.9.font";

/* check font before barging ahead */
if(access(fontname, 0) < 0){
```

```
    fprint(STDERR, "rio: can't access %s: %r\n", fontname);
    exits("font open");
}

putenv("font", fontname);
```

13.9 Holding mode

⟨Window *config fields* 218a⟩+≡ (51) ◁119c

```
bool holding;
```

⟨winctl() *alts adjustments, if holding* 218b⟩≡ (123f)

```
if(w->holding)
    alts[WCread].op = CHANNOP;
```

⟨wsetcursor() *if holding* 218c⟩≡ (75d)

```
if(p==nil && w->holding)
    p = &whitearrow;
```

⟨wborder() *if holding* 218d⟩≡ (81b)

```
if(w->holding){
    if(type == Selborder)
        col = holdcol;
    else
        col = paleholdcol;
}
```

⟨wsetcols() *if holding* 218e⟩≡ (128g)

```
if(w->holding)
    if(w == input)
        w->frm.cols[TEXT] = w->frm.cols[HTEXT] = holdcol;
    else
        w->frm.cols[TEXT] = w->frm.cols[HTEXT] = lightholdcol;
```

⟨*global* holdcol 218f⟩≡ (249)

```
static Image *holdcol;
```

⟨*global* paleholdcol 218g⟩≡ (249)

```
static Image *paleholdcol;
```

⟨*global* lightholdcol 218h⟩≡ (249)

```
static Image *lightholdcol;
```

⟨wmk() *extra colors initialisation* 218i⟩+≡ (128e) ◁128i

```
holdcol      = allocimage(display, Rect(0,0,1,1), CMAP8, true, DMedblue);
lightholdcol = allocimage(display, Rect(0,0,1,1), CMAP8, true, DGreyblue);
paleholdcol  = allocimage(display, Rect(0,0,1,1), CMAP8, true, DPalegreyblue
```

⟨Wctlmesgkind *cases* 218j⟩+≡ (78a) ◁120b 225h▷

```
Holdon,
Holdoff,
```

⟨xfidwrite() *Qconsctl case* 218k⟩+≡ (117f) ◁119d

```
if(strncmp(req->data, "holdon", 6)==0){
    if(w->holding++ == 0)
        wsendctlmesg(w, Holdon, ZR, nil);
    break;
}
if(strncmp(req->data, "holdoff", 7)==0 && w->holding){
    if(--w->holding == false)
        wsendctlmesg(w, Holdoff, ZR, nil);
    break;
}
```

⟨xfidclose() *Qconsctl case, if holding* 219a⟩≡ (117e)

```
if(w->holding){
    w->holding = false;
    wsendctlmesg(w, Holdoff, ZR, nil);
}
```

⟨xfidwrite() *Qconsctl case, if rawon message and holding mode* 219b⟩≡ (119d)

```
if(w->holding){
    w->holding = false;
    wsendctlmesg(w, Holdoff, ZR, nil);
}
```

⟨wkeyctl() *if holding* 219c⟩≡ (65d)

```
if(r==0x1B || (w->holding && r==0x7F)){ /* toggle hold */
    if(w->holding)
        --w->holding;
    else
        w->holding++;
    wrepaint(w);
    if(r == 0x1B)
        return;
}
```

⟨wctlmesg() *cases* 219d⟩+≡ (78d) ◁120c 225i▷

```
case Holdon:
case Holdoff:
    ⟨wctlmesg() break if window was deleted 88f⟩
    wrepaint(w);
    flushimage(display, true);
    break;
```

13.10 Signals, notes

⟨*global* oknotes 219e⟩≡ (243a)

```
char *oknotes[] =
{
    "delete",
    "hangup",
    "kill",
    "exit",
    nil
};
```

⟨*function* shutdown 219f⟩≡ (243a)

```
int
shutdown(void *, char *msg)
{
    int i;
    static Lock shutdownlk;

    killprocs();
    for(i=0; oknotes[i]; i++)
        if(strncmp(oknotes[i], msg, strlen(oknotes[i])) == 0){
            lock(&shutdownlk); /* only one can threadexitsall */
            threadexitsall(msg);
        }
    fprint(STDERR, "rio %d: abort: %s\n", getpid(), msg);
    abort();
```

```
    exits(msg);
    return 0;
}
```

⟨*function* `killprocs` 220a⟩≡ (243a)

```
void
killprocs(void)
{
    int i;

    for(i=0; i<nwindow; i++)
        postnote(PNGROUP, windows[i]->pid, "hangup");
}
```

13.11 Timer

⟨*global* `clockfd` 220b⟩≡ (255)

```
fdt clockfd;
```

⟨`filsysinit()` *set clockfd* 220c⟩≡ (58b)

```
clockfd = open("/dev/time", OREAD|OCEXEC);
```

⟨*function* `getclock` 220d⟩≡ (255)

```
static
uint
getclock(void)
{
    char buf[32];

    seek(clockfd, 0, 0);
    read(clockfd, buf, sizeof buf);
    return atoi(buf);
}
```

⟨*struct* `Timer` 220e⟩≡ (237b)

```
struct Timer
{
    int  dt;
    int  cancel;
    Channel *c; /* chan(int) */
    Timer *next;
};
```

⟨`main()` *threads creation* 220f⟩+≡ (56a) ◁90a

```
timerinit();
```

⟨*function* `timerinit` 220g⟩≡ (258a)

```
void
timerinit(void)
{
    ctimer = chancreate(sizeof(Timer*), 100);
    proccreate(timerproc, nil, STACK);
}
```

⟨*function* timerproc 221⟩≡ (258a)

```
static
void
timerproc(void*)
{
    int i, nt, na, dt, del;
    Timer **t, *x;
    uint old, new;

    rfork(RFFDG);
    threadsetname("TIMERPROC");

    t = nil;
    na = 0;
    nt = 0;
    old = msec();
    for(;;){
        sleep(1); /* will sleep minimum incr */
        new = msec();
        dt = new-old;
        old = new;
        if(dt < 0) /* timer wrapped; go around, losing a tick */
            continue;
        for(i=0; i<nt; i++){
            x = t[i];
            x->dt -= dt;
            del = 0;
            if(x->cancel){
                timerstop(x);
                del = 1;
            }else if(x->dt <= 0){
                /*
                 * avoid possible deadlock if client is
                 * now sending on ctimer
                 */
                if(nbsendul(x->c, 0) > 0)
                    del = 1;
            }
            if(del){
                memmove(&t[i], &t[i+1], (nt-i-1)*sizeof t[0]);
                --nt;
                --i;
            }
        }
        if(nt == 0){
            x = recvp(ctimer);
    gotit:
            if(nt == na){
                na += 10;
                t = realloc(t, na*sizeof(Timer*));
                if(t == nil)
                    abort();
            }
            t[nt++] = x;
            old = msec();
        }
        if(nbrecv(ctimer, &x) > 0)
            goto gotit;
    }
}
```

⟨*global* ctimer 222a⟩≡ (258a)

```
// chan<?> (listener = ?, sender = ?)
static Channel* ctimer; /* chan(Timer*)[100] */
```

⟨*global* timer 222b⟩≡ (258a)

```
static Timer *timer;
```

⟨*function* msec 222c⟩≡ (258a)

```
static
uint
msec(void)
{
    return nsec()/1000000;
}
```

⟨*function* timerstop 222d⟩≡ (258a)

```
void
timerstop(Timer *t)
{
    t->next = timer;
    timer = t;
}
```

⟨*function* timercancel 222e⟩≡ (258a)

```
void
timercancel(Timer *t)
{
    t->cancel = true;
}
```

⟨*function* timerstart 222f⟩≡ (258a)

```
/*
 * timeralloc() and timerfree() don't lock, so can only be
 * called from the main proc.
 */
Timer*
timerstart(int dt)
{
    Timer *t;

    t = timer;
    if(t)
        timer = timer->next;
    else{
        t = emalloc(sizeof(Timer));
        t->c = chancreate(sizeof(int), 0);
    }
    t->next = nil;
    t->dt = dt;
    t->cancel = false;
    sendp(ctimer, t);
    return t;
}
```

13.12 Flushing

⟨Xfid *flushing fields* 222g⟩≡ (55c)

```
int flushtag; /* our tag, so flush can find us */
// chan<int> (listener = ?, sender = ?)
Channel *flushc;/* channel(int) to notify us we're being flushed */
bool flushing; /* another Xfid is trying to flush us */
```

⟨xfidallocthread() *create flushc channel* 223a⟩≡ (68g)

```
x->flushc = chancreate(sizeof(int), 0); /* notification only; nodata */
x->flushtag = -1;
```

⟨Xfid *other fields* 223b⟩≡ (55c)

```
QLock active;
```

⟨fcall *other methods* 223c⟩+≡ (55d) ◁111a 225f▷

```
[Tflush]   = filsysflush,
```

⟨*function* filsysflush 223d⟩≡ (255)

```
static
Xfid*
filsysflush(Filsys*, Xfid *x, Fid*)
{
    sendp(x->c, xfidflush);
    return nil;
}
```

⟨*function* xfidflush 223e⟩≡ (245a)

```
void
xfidflush(Xfid *x)
{
    Fcall fc;
    Xfid *xf;

    for(xf=xfid; xf; xf=xf->next)
        if(xf->flushtag == x->req.oldtag){
            xf->flushtag = -1;
            xf->flushing = true;
            incref(xf); /* to hold data structures up at tail of synchronization */
            if(xf->ref == 1)
                error("ref 1 in flush");
            if(canqlock(&xf->active)){
                qunlock(&xf->active);
                sendul(xf->flushc, 0);
            }else{
                qlock(&xf->active); /* wait for him to finish */
                qunlock(&xf->active);
            }
            xf->flushing = false;

            if(decref(xf) == 0)
                sendp(cxfidfree, xf);
            break;
        }
    filsysrespond(x->fs, x, &fc, nil);
}
```

⟨*function* filsyscancel 223f⟩≡ (254a)

```
void
filsyscancel(Xfid *x)
{
    if(x->buf){
        free(x->buf);
        x->buf = nil;
    }
}
```

⟨xfidxxx() *set flushtag* 223g⟩≡ (177b 115g 114i 112h)

```
x->flushtag = req->tag;
```

⟨xfidxxx() *unset flushtag* 224a⟩≡ (177b 115g 114i 112h)

```
x->flushtag = -1;
```

⟨xfidread() *when Qmouse, set alts for flush* 224b⟩≡ (112h)

```
alts[MRflush].c = x->flushc;
alts[MRflush].v = nil;
alts[MRflush].op = CHANRCV;
```

⟨xfidread() *when Qmouse, switch alt flush case* 224c⟩≡ (112h)

```
case MRflush:
    filsyscancel(x);
    return;
```

⟨xfidread() *when Qmouse, if flushing* 224d⟩≡ (112h)

```
if(x->flushing){
    recv(x->flushc, nil);  /* wake up flushing xfid */
    recv(mrm.cm, nil);   /* wake up window and toss data */
    filsyscancel(x);
    return;
}
```

⟨xfidread() *when Qcons, set alts for flush* 224e⟩≡ (114i)

```
alts[CRflush].c = x->flushc;
alts[CRflush].v = nil;
alts[CRflush].op = CHANRCV;
```

⟨xfidread() *when Qcons, switch alt flush case* 224f⟩≡ (114i)

```
case CRflush:
    filsyscancel(x);
    return;
```

⟨xfidread() *when Qcons, if flushing* 224g⟩≡ (114i)

```
if(x->flushing){
    recv(x->flushc, nil); /* wake up flushing xfid */
    recv(c2, nil);   /* wake up window and toss data */
    free(t);
    filsyscancel(x);
    return;
}
```

⟨xfidwrite() *when Qcons, set alts for flush* 224h⟩≡ (115g)

```
alts[CWflush].c = x->flushc;
alts[CWflush].v = nil;
alts[CWflush].op = CHANRCV;
```

⟨xfidwrite() *when Qcons, switch alt flush case* 224i⟩≡ (115g)

```
case CWflush:
    filsyscancel(x);
    return;
```

⟨xfidwrite() *when Qcons, if flushing* 224j⟩≡ (115g)

```
if(x->flushing){
    recv(x->flushc, nil); /* wake up flushing xfid */
    pair.s = runemalloc(1);
    pair.ns = 0;
    send(cwm.cw, &pair);  /* wake up window with empty data */
    filsyscancel(x);
    return;
}
```

⟨xfidread() *when Qwctl, set alts for flush* 225a⟩≡ (177b)

```
alts[WCRflush].c = x->flushc;
alts[WCRflush].v = nil;
alts[WCRflush].op = CHANRCV;
```

⟨xfidread() *when Qwctl, switch alt flush case* 225b⟩≡ (177b)

```
case WCRflush:
    filsyscancel(x);
    return;
```

⟨xfidread() *when Qwctl, if flushing* 225c⟩≡ (177b)

```
if(x->flushing){
    recv(x->flushc, nil); /* wake up flushing xfid */
    recv(c2, nil);   /* wake up window and toss data */
    free(t);
    filsyscancel(x);
    return;
}
```

13.13 Security

⟨filsysinit() *other locals* 225d⟩+≡ (58b) ◁182d

```
fdt fd;
char buf[128];
int n;
```

⟨filsysinit() *set fs user* 225e⟩≡ (58b)

```
fd = open("/dev/user", OREAD);
strcpy(buf, "Jean-Paul_Belmondo"); // lol
if(fd >= 0){
    n = read(fd, buf, sizeof buf-1);
    if(n > 0)
        buf[n] = '\0';
    close(fd);
}
fs->user = estrdup(buf);
```

⟨fcall *other methods* 225f⟩+≡ (55d) ◁223c

```
[Tauth]    = filsysauth,
```

⟨*function* filsysauth 225g⟩≡ (255)

```
static
Xfid*
filsysauth(Filsys *fs, Xfid *x, Fid*)
{
    Fcall fc;

    return filsysrespond(fs, x, &fc, "rio: authentication not required");
}
```

13.14 TODO Wakeup

⟨Wctlmesgkind *cases* 225h⟩+≡ (78a) ◁218j 226a▷

```
Wakeup,
```

⟨wctlmesg() *cases* 225i⟩+≡ (78d) ◁219d 226b▷

```
case Wakeup:
    break;
```

13.15 TODO Refresh

⟨Wctlmesgkind *cases* 226a⟩+≡ (78a) ◁225h

```
Refresh,
```

⟨wctlmesg() *cases* 226b⟩+≡ (78d) ◁225i

```
case Refresh:
    if(w->deleted || Dx(w->screenr)<=0 || !rectclip(&r, w->i->r))
        break;
    if(!w->mouseopen)
        wrefresh(w, r);
    flushimage(display, true);
    break;
```

⟨*function* wrefresh 226c⟩≡ (249)

```
void
wrefresh(Window *w, Rectangle)
{
    Frame *frm = &w->frm;

    /* BUG: rectangle is ignored */
    if(w == input)
        wborder(w, Selborder);
    else
        wborder(w, Unselborder);
    if(w->mouseopen)
        return;
    // else

    draw(w->i, insetrect(w->i->r, Borderwidth), frm->cols[BACK], nil,
         w->i->r.min);
    frm->ticked = false;
    if(frm->p0 > 0)
        frdrawsel(frm, frptofchar(frm, 0), 0, frm->p0, 0);
    if(frm->p1 < frm->nchars)
        frdrawsel(frm, frptofchar(frm, frm->p1), frm->p1, frm->nchars, 0);
    frdrawsel(frm, frptofchar(frm, frm->p0), frm->p0, frm->p1, 1);
    w->lastsr = ZR;
    wscrdraw(w);
}
```

Chapter 14

Conclusion

Appendix A

Debugging

⟨*constant* DEBUG 228a⟩≡ (237b)

```
#define DEBUG false
```

⟨filsysinit() *install dumper* 228b⟩≡ (58b)

```
fmtinstall('F', fcallfmt);
```

⟨filsysproc() *dump Fcall if debug* 228c⟩≡ (66f)

```
if(DEBUG)
    fprint(STDERR, "rio:<-%F\n", &x->req);
```

⟨filsysrespond() *dump Fcall t if debug* 228d⟩≡ (101b)

```
if(DEBUG)
    fprint(STDERR, "rio:->%F\n", fc);
```

Appendix B

Error Management

B.1 Error codes

⟨*global* Eperm 229a⟩≡ (259a)
```
char Eperm[] = "permission denied";
```

⟨*global* Eexist 229b⟩≡ (255)
```
char Eexist[] = "file does not exist";
```

⟨*global* Enotdir 229c⟩≡ (255)
```
char Enotdir[] = "not a directory";
```

⟨*global* Ebadfcall 229d⟩≡ (255)
```
char Ebadfcall[] = "bad fcall type";
```

⟨*global* Eoffset 229e⟩≡ (255)
```
char Eoffset[] = "illegal offset";
```

⟨*global* Ebadwr 229f⟩≡ (258b)
```
char Ebadwr[]  = "bad rectangle in wctl request";
```

⟨*global* Ewalloc 229g⟩≡ (258b)
```
char Ewalloc[]  = "window allocation failed in wctl request";
```

⟨*global* Einuse 229h⟩≡ (256)
```
char Einuse[] =  "file in use";
```

⟨*global* Edeleted 229i⟩≡ (256)
```
char Edeleted[] = "window deleted";
```

⟨*global* Ebadreq 229j⟩≡ (256)
```
char Ebadreq[] = "bad graphics request";
```

⟨*global* Etooshort 229k⟩≡ (256)
```
char Etooshort[] = "buffer too small";
```

⟨*global* Ebadtile 229l⟩≡ (256)
```
char Ebadtile[] = "unknown tile";
```

⟨*global* Eshort 229m⟩≡ (256)
```
char Eshort[] =  "short i/o request";
```

⟨*global* Elong 229n⟩≡ (256)
```
char Elong[] =   "snarf buffer too long";
```

⟨*global* Eunkid 230a⟩≡ (256)
```
char Eunkid[] =  "unknown id in attach";
```

⟨*global* Ebadrect 230b⟩≡ (256)
```
char Ebadrect[] =  "bad rectangle in attach";
```

⟨*global* Ewindow 230c⟩≡ (256)
```
char Ewindow[] =  "cannot make window";
```

⟨*global* Enowindow 230d⟩≡ (256)
```
char Enowindow[] =  "window has no image";
```

⟨*global* Ebadmouse 230e⟩≡ (256)
```
char Ebadmouse[] =  "bad format on /dev/mouse";
```

⟨*global* Ebadwrect 230f⟩≡ (256)
```
char Ebadwrect[] =  "rectangle outside screen";
```

⟨*global* Ebadoffset 230g⟩≡ (256)
```
char Ebadoffset[] =  "window read not on scan line boundary";
```

B.2 error(), derror()

⟨*global* errorshouldabort 230h⟩≡ (259a)
```
bool errorshouldabort = false;
```

⟨main() *error management after everything setup* 230i⟩≡ (56a)
```
errorshouldabort = true;/* suicide if there's trouble after this */
⟨main() if initstr or kdbin 215d⟩
threadnotify(shutdown, true);
```

⟨*function* error 230j⟩≡ (259a)
```
void
error(char *s)
{
    fprint(STDERR, "rio: %s: %r\n", s);
    if(errorshouldabort)
        abort();
    threadexitsall("error");
}
```

⟨*function* derror 230k⟩≡ (259a)
```
void
derror(Display*, char *errorstr)
{
    error(errorstr);
}
```

Appendix C

Utilities

⟨*function* min 231a⟩≡ (259b)

```
int
min(int a, int b)
{
    if(a < b)
        return a;
    return b;
}
```

⟨*function max (windows/rio/util.c)* 231b⟩≡ (259b)

```
int
max(int a, int b)
{
    if(a > b)
        return a;
    return b;
}
```

⟨*function* erealloc 231c⟩≡ (259b)

```
void*
erealloc(void *p, uint n)
{
    p = realloc(p, n);
    if(p == nil)
        error("realloc failed");
    return p;
}
```

⟨*function* emalloc 231d⟩≡ (259b)

```
void*
emalloc(uint n)
{
    void *p;

    p = malloc(n);
    if(p == nil)
        error("malloc failed");
    memset(p, 0, n);
    return p;
}
```

⟨*function* estrdup 231e⟩≡ (259b)

```
char*
estrdup(char *s)
{
```

```
    char *p;

    p = malloc(strlen(s)+1);
    if(p == nil)
        error("strdup failed");
    strcpy(p, s);
    return p;
}
```

⟨*function* isalnum 232a⟩≡ (259b)

```
//@Scheck: not dead, but conflict with the one in ctype.h
int isalnum(Rune c)
{
    /*
     * Hard to get absolutely right.  Use what we know about ASCII
     * and assume anything above the Latin control characters is
     * potentially an alphanumeric.
     */
    if(c <= ' ')
        return false;
    if(0x7F<=c && c<=0xA0)
        return false;
    if(utfrune("!\"#$%&'()*+,-./:;<=>?@[\\]^`{|}~", c))
        return false;
    return true;
}
```

⟨*function* runemalloc 232b⟩≡ (239)

```
#define runemalloc(n)  malloc((n)*sizeof(Rune))
```

⟨*function* runerealloc 232c⟩≡ (239)

```
#define runerealloc(a, n) realloc(a, (n)*sizeof(Rune))
```

⟨*function* runemove 232d⟩≡ (239)

```
#define runemove(a, b, n) memmove(a, b, (n)*sizeof(Rune))
```

⟨*function* cvttorunes 232e⟩≡ (259b)

```
void
cvttorunes(char *p, int n, Rune *r, int *nb, int *nr, int *nulls)
{
    uchar *q;
    Rune *s;
    int j, w;

    /*
     * Always guaranteed that n bytes may be interpreted
     * without worrying about partial runes.  This may mean
     * reading up to UTFmax-1 more bytes than n; the caller
     * knows this.  If n is a firm limit, the caller should
     * set p[n] = 0.
     */
    q = (uchar*)p;
    s = r;
    for(j=0; j<n; j+=w){
        if(*q < Runeself){
            w = 1;
            *s = *q++;
        }else{
            w = chartorune(s, (char*)q);
            q += w;
```

```
        }
        if(*s)
            s++;
        else if(nulls)
                *nulls = true;
    }
    *nb = (char*)q-p;
    *nr = s-r;
}
```

⟨*function* strrune 233a⟩≡ (259b)

```
Rune*
strrune(Rune *s, Rune c)
{
    Rune c1;

    if(c == 0) {
        while(*s++)
            ;
        return s-1;
    }

    while(c1 = *s++)
        if(c1 == c)
            return s-1;
    return nil;
}
```

⟨*function* runetobyte 233b⟩≡ (259b)

```
char*
runetobyte(Rune *r, int n, int *ip)
{
    char *s;
    int m;

    s = emalloc(n*UTFmax+1);
    m = snprint(s, n*UTFmax+1, "%.*S", n, r);
    *ip = m;
    return s;
}
```

Appendix D

Examples of Windowing System Applications TODO

Appendix E

Extra Code

E.1 include/

E.1.1 include/complete.h

⟨include/complete.h 235a⟩≡

```
#pragma lib "libcomplete.a"
#pragma src "/sys/src/libcomplete"

typedef struct Completion Completion;

⟨struct Completion 206b⟩

Completion*  complete(char *dir, char *s);
void   freecompletion(Completion*);
```

E.1.2 include/frame.h

⟨include/frame.h 235b⟩≡

```
#pragma src "/sys/src/libframe"
#pragma lib "libframe.a"

typedef struct Frbox Frbox;
typedef struct Frame Frame;

⟨enum _anon_ (include/frame.h) 128b⟩

⟨constant FRTICKW 129c⟩

⟨struct Frbox 130c⟩

⟨struct Frame 126g⟩

ulong frcharofpt(Frame*, Point);
Point frptofchar(Frame*, ulong);

int frdelete(Frame*, ulong, ulong);
void frinsert(Frame*, Rune*, Rune*, ulong);

void frselect(Frame*, Mousectl*);
void frselectpaint(Frame*, Point, Point, Image*);

void frdrawsel(Frame*, Point, ulong, ulong, int);
Point  frdrawsel0(Frame*, Point, ulong, ulong, Image*, Image*);
```

```
void frinit(Frame*, Rectangle, Font*, Image*, Image**);
void frsetrects(Frame*, Rectangle, Image*);
void frclear(Frame*, int);

// private??? frame_private.h?
uchar *_frallocstr(Frame*, unsigned);
void _frinsure(Frame*, int, unsigned);
Point _frdraw(Frame*, Point);
void _frgrowbox(Frame*, int);
void _frfreebox(Frame*, int, int);
void _frmergebox(Frame*, int);
void _frdelbox(Frame*, int, int);
void _frsplitbox(Frame*, int, int);
int _frfindbox(Frame*, int, ulong, ulong);
void _frclosebox(Frame*, int, int);
int _frcanfit(Frame*, Point, Frbox*);
void _frcklinewrap(Frame*, Point*, Frbox*);
void _frcklinewrap0(Frame*, Point*, Frbox*);
void _fradvance(Frame*, Point*, Frbox*);
int _frnewwid(Frame*, Point, Frbox*);
int _frnewwid0(Frame*, Point, Frbox*);
void _frclean(Frame*, Point, int, int);
void _frdrawtext(Frame*, Point, Image*, Image*);
void _fraddbox(Frame*, int, int);
Point _frptofcharptb(Frame*, ulong, Point, int);
Point _frptofcharnb(Frame*, ulong, int);
int _frstrlen(Frame*, int);

void frtick(Frame*, Point, int);
void frinittick(Frame*);

void frredraw(Frame*);

⟨function NRUNE 130d⟩
⟨function NBYTE 142f⟩
```

E.1.3 include/plumb.h

⟨include/plumb.h 236⟩≡

```
#pragma lib "libplumb.a"
#pragma src "/sys/src/libplumb"

/*
 * Message format:
 * source application\n
 * destination port\n
 * working directory\n
 * type\n
 * attributes\n
 * nbytes\n
 * n bytes of data
 */

typedef struct Plumbattr Plumbattr;
typedef struct Plumbmsg Plumbmsg;

⟨struct Plumbmsg 196a⟩
```

⟨*struct* Plumbattr 196b⟩

```
int      plumbsend(int, Plumbmsg*);
int      plumbsendtext(int, char*, char*, char*, char*);
Plumbmsg* plumbrecv(int);
char*  plumbpack(Plumbmsg*, int*);
Plumbmsg* plumbunpack(char*, int);
Plumbmsg* plumbunpackpartial(char*, int, int*);
char*  plumbpackattr(Plumbattr*);
Plumbattr* plumbunpackattr(char*);
Plumbattr* plumbaddattr(Plumbattr*, Plumbattr*);
Plumbattr* plumbdelattr(Plumbattr*, char*);
void  plumbfree(Plumbmsg*);
char*  plumblookup(Plumbattr*, char*);
int      plumbopen(char*, int);

int  eplumb(int, char*);
```

E.2 windows/rio/

E.2.1 windows/rio/dat.h

⟨*enum* _anon_ (windows/rio/dat.h)*3* 237a⟩≡ (237b)

```
enum
{
    ⟨constant Selborder 62a⟩
    ⟨constant Unselborder 87e⟩
    ⟨constants Scrollxxx 126c⟩
    ⟨constant BIG 151c⟩
};
```

⟨windows/rio/dat.h 237b⟩≡

```
// forward decls
typedef struct Window Window;
typedef struct Wctlmesg Wctlmesg;
typedef struct Filsys Filsys;
typedef struct Fid Fid;
typedef struct Xfid Xfid;
typedef struct Consreadmesg Consreadmesg;
typedef struct Conswritemesg Conswritemesg;
typedef struct Stringpair Stringpair;
typedef struct Dirtab Dirtab;
typedef struct Mouseinfo Mouseinfo;
typedef struct Mousereadmesg Mousereadmesg;
typedef struct Mousestate Mousestate;
typedef struct Ref Ref;
typedef struct Timer Timer;

//----------------------------------------------------------------------------
// Data structures and constants
//----------------------------------------------------------------------------

⟨enum qid 100a⟩

⟨function QID 100b⟩
⟨function WIN 100c⟩
⟨function FILE 100d⟩
```

⟨*enum* _anon_ (windows/rio/dat.h)*2* 170a⟩

⟨*constant* STACK 57g⟩

⟨*enum* _anon_ (windows/rio/dat.h)*3* 237a⟩

⟨*constant* DEBUG 228a⟩

⟨*enum* wctlmesgkind 78a⟩

⟨*struct* Wctlmesg 78b⟩

⟨*struct* Conswritemesg 115c⟩

⟨*struct* Consreadmesg 114e⟩

⟨*struct* Mousereadmesg 112e⟩

⟨*struct* Stringpair 114f⟩

⟨*struct* Mousestate 121a⟩

⟨*struct* Mouseinfo 120e⟩

⟨*struct* Window 51⟩

⟨*struct* Dirtab 100e⟩

⟨*struct* Fid 54c⟩

⟨*struct* Xfid 55c⟩

⟨*constant* Nhash 54a⟩

⟨*struct* Filsys 53i⟩

⟨*struct* Timer 220e⟩

```
//-----------------------------------------------------------------------------
// Globals
//-----------------------------------------------------------------------------

// see also draw.h globals: display, font, view

// globals.c

extern Screen *desktop;
extern Image *background;
extern Image *red;

extern Window **windows;
extern int  nwindow;
extern Window *input;

extern Window *hidden[100];
extern int  nhidden;

extern Filsys *filsys;
```

```
extern Keyboardctl *keyboardctl;
extern Mousectl *mousectl;
extern Mouse *mouse;

extern Channel *exitchan; // was static in rio.c
extern Channel* winclosechan;
extern Channel* deletechan;

extern int  snarfversion; /* updated each time it is written */

extern int  sweeping;
extern bool  menuing;
extern int  scrolling;

extern char  *startdir;

// misc
extern Window *wkeyboard; /* window of simulated keyboard */
extern QLock all;   /* BUG */
extern fdt  wctlfd;
extern int  maxtab;


// 9p.c
extern int  messagesize; /* negotiated in 9P version setup */


// thread_mouse.c
extern Rectangle viewr; // was static in rio.c

// data.c
extern Cursor boxcursor;
extern Cursor crosscursor;
extern Cursor sightcursor;
extern Cursor whitearrow;
extern Cursor query;
extern Cursor *corners[9];

// cursor.c
extern Cursor *lastcursor; // was static in wind.c

// snarf.c
extern fdt  snarffd;
extern Rune* snarf;
extern int  nsnarf;

// error.c
extern bool  errorshouldabort;
extern char Eperm[];
```

E.2.2 windows/rio/fns.h

⟨windows/rio/fns.h 239⟩≡

```
// thread_keyboard.c (for rio.c)
void keyboardthread(void*);
// thread_mouse.c    (for rio.c)
void mousethread(void*);
```

```
// threads_misc.c    (for rio.c)
void  winclosethread(void*);
void  deletethread(void*);
// threads_worker.c (for rio.c)
Channel* xfidinit(void);
void xfidflush(Xfid*);
// proc_fileserver (for rio.c and process_winshell.c)
Filsys* filsysinit(Channel*);
int  filsysmount(Filsys*, int);
// threads_window.c
void    winctl(void*);
// processes_winshell
void winshell(void*);

// data.c (for rio.c)
void iconinit(void);
// timer.c (for rio.c and scrl.c)
void timerinit(void);
void timerstop(Timer*);
void timercancel(Timer*);
Timer* timerstart(int);


// fsys.c (for rio.c and xfid.c and process_winshell.c)

// cursor.c
void riosetcursor(Cursor*, int);

// wm.c (for thread_mouse.c)
void    cornercursor(Window *w, Point p, bool force);
Image *bandsize(Window*);
Image* drag(Window*, Rectangle*);
void    button3menu(void);
Window *new(Image*, int, int, int, char*, char*, char**); // for wkeyboard
int whide(Window*); // for wctl
int wunhide(int);   // for wctl

// wind.c
void    wsendctlmesg(Window*, int, Rectangle, Image*);
Window* wmk(Image*, Mousectl*, Channel*, Channel*, int);
int  wclose(Window*);
void wclosewin(Window*);
Window* wpointto(Point);
void wcurrent(Window*);
Window* wlookid(int);
void wresize(Window*, Image*, int);
void wrefresh(Window*, Rectangle);
void wrepaint(Window*);
int  winborder(Window*, Point);
Window* wtop(Point);
void wtopme(Window*);
void wbottomme(Window*);
void wsetcursor(Window*, bool);
void wmovemouse(Window*, Point);
void wfill(Window*);
void wsetname(Window*);
void wsetpid(Window*, int, int);


// TODO
```

```
int     goodrect(Rectangle);

// graphical_window.c
void waddraw(Window*, Rune*, int);

// terminal.c
void    button2menu(Window *w);
void wkeyctl(Window*, Rune);
void wmousectl(Window*);
void wdelete(Window*, uint, uint);
uint winsert(Window*, Rune*, int, uint);
void wshow(Window*, uint);
void wsetselect(Window*, uint, uint);
void wsetorigin(Window*, uint, int);
uint wbacknl(Window*, uint, uint);
char* wcontents(Window*, int*);

// scroll.c
void freescrtemps(void);
void wscrdraw(Window*);
void  wscrsleep(Window*, uint);
void wscroll(Window*, int);

// snarf.c
void putsnarf(void);
void getsnarf(void);

// 9p.c (for fsys.c and xfid.c)
Xfid* filsysrespond(Filsys*, Xfid*, Fcall*, char*);
void filsyscancel(Xfid*);

// xfid.c  (for fsys.c)
void xfidattach(Xfid*);
void xfidopen(Xfid*);
void xfidclose(Xfid*);
void xfidread(Xfid*);
void xfidwrite(Xfid*);

// wctl.c (for fsys.c and xfid.c)
void wctlproc(void*);
void wctlthread(void*);
int     parsewctl(char**, Rectangle, Rectangle*, int*, int*, int*, int*, char**, char*, char*);
int     writewctl(Xfid*, char*);

// util.c
int     min(int, int);
int     max(int, int);
Rune* strrune(Rune*, Rune);
int     isalnum(Rune);
void cvttorunes(char*, int, Rune*, int*, int*, int*);
char*  runetobyte(Rune*, int, int*);/* was (byte*,int) runetobyte(Rune*,int);*/
void* erealloc(void*, uint);
void*  emalloc(uint);
char*  estrdup(char*);

⟨function runemalloc 232b⟩
⟨function runerealloc 232c⟩
⟨function runemove 232d⟩

// error.c
```

```
void    derror(Display*, char *); // for main.c
void error(char*);
```

E.2.3 windows/rio/globals.c

⟨windows/rio/globals.c 242⟩≡

```
#include <u.h>
#include <libc.h>

#include <draw.h>
#include <thread.h>
#include <cursor.h>
#include <mouse.h>
#include <keyboard.h>

#include <frame.h>

#include <fcall.h>

#include "dat.h"
#include "fns.h"

⟨global mousectl 50b⟩
⟨global mouse 50d⟩
⟨global keyboardctl 50c⟩

⟨global desktop 50e⟩
⟨global background 50f⟩
⟨global red 50g⟩

⟨global window 52h⟩
⟨global wkeyboard 215h⟩
⟨global nwindow 52i⟩


⟨global exitchan 57e⟩

⟨global winclosechan 89f⟩

⟨global deletechan 89b⟩


⟨global input 52l⟩
⟨global all 102c⟩
⟨global filsys 53j⟩
⟨global hidden 97c⟩
⟨global nhidden 97d⟩
⟨global scrolling 214b⟩


⟨global startdir 81i⟩
⟨global sweeping 76c⟩
⟨global wctlfd 182a⟩
⟨global menuing 85c⟩
⟨global snarfversion 194f⟩

⟨global maxtab 165e⟩
```

E.2.4 windows/rio/rio.c

⟨windows/rio/rio.c 243a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <window.h>
#include <thread.h>
#include <cursor.h>
#include <mouse.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>
#include <plumb.h>

#include "dat.h"
#include "fns.h"

/*
 *  WASHINGTON (AP) - The Food and Drug Administration warned
 * consumers Wednesday not to use ‘‘Rio’’ hair relaxer products
 * because they may cause severe hair loss or turn hair green....
 *    The FDA urged consumers who have experienced problems with Rio
 * to notify their local FDA office, local health department or the
 * company at 1-800-543-3002.
 */

⟨global fontname 217c⟩

⟨global kbdargv 215i⟩


⟨function usage 25⟩


⟨function initcmd 215e⟩

⟨global oknotes 219e⟩

⟨function killprocs 220a⟩


⟨function shutdown 219f⟩


⟨function threadmain 56a⟩
```

E.2.5 windows/rio/thread_mouse.c

⟨windows/rio/thread_mouse.c 243b⟩≡

```
#include <u.h>
#include <libc.h>

// for dat.h
#include <draw.h>
#include <mouse.h>
#include <cursor.h>
#include <keyboard.h>
#include <frame.h>
```

```
#include <fcall.h>
#include <thread.h>

#include <window.h>

#include "dat.h"
#include "fns.h"

⟨enum Mxxx 60b⟩

⟨function keyboardhide 217b⟩

⟨global viewr 50a⟩

⟨function resized 190d⟩


⟨function mousethread 61b⟩
```

E.2.6 windows/rio/thread_keyboard.c

⟨windows/rio/thread_keyboard.c 244a⟩≡

```
#include <u.h>
#include <libc.h>

// for dat.h
#include <draw.h>
#include <mouse.h>
#include <cursor.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>
#include <thread.h>

#include "dat.h"
#include "fns.h"

⟨function keyboardthread 60a⟩
```

E.2.7 windows/rio/threads_window.c

⟨windows/rio/threads_window.c 244b⟩≡

```
#include <u.h>
#include <libc.h>

// for dat.h
#include <draw.h>
#include <mouse.h>
#include <cursor.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>
#include <thread.h>

#include <window.h>

#include "dat.h"
```

```
#include "fns.h"

⟨enum Wxxx 63e⟩

⟨function deletetimeoutproc 89a⟩

⟨function wctlmesg 78d⟩

⟨function winctl 64a⟩
```

E.2.8 windows/rio/threads_worker.c

```
⟨windows/rio/threads_worker.c 245a⟩≡
  #include <u.h>
  #include <libc.h>

  // for dat.h
  #include <draw.h>
  #include <mouse.h>
  #include <cursor.h>
  #include <keyboard.h>
  #include <frame.h>
  #include <fcall.h>
  #include <thread.h>

  #include "dat.h"
  #include "fns.h"

  ⟨global xfidfree 68e⟩
  ⟨global xfid 68d⟩
  ⟨global cxfidalloc 59a⟩
  ⟨global cxfidfree 59b⟩

  ⟨enum Xxxx 68c⟩

  ⟨function xfidctl 69c⟩

  ⟨function xfidflush 223e⟩

  ⟨function xfidallocthread 68g⟩

  ⟨function xfidinit 59c⟩
```

E.2.9 windows/rio/threads_misc.c

```
⟨windows/rio/threads_misc.c 245b⟩≡
  #include <u.h>
  #include <libc.h>

  // for dat.h
  #include <draw.h>
  #include <mouse.h>
  #include <cursor.h>
  #include <keyboard.h>
  #include <frame.h>
  #include <fcall.h>
  #include <thread.h>
```

```
#include <window.h>

#include "dat.h"
#include "fns.h"

⟨function winclosethread 90b⟩

⟨function deletethread 89e⟩
```

E.2.10 windows/rio/wm.c

```
⟨windows/rio/wm.c 246⟩≡
  #include <u.h>
  #include <libc.h>

  // for dat.h
  #include <draw.h>
  #include <mouse.h>
  #include <cursor.h>
  #include <keyboard.h>
  #include <frame.h>
  #include <fcall.h>
  #include <thread.h>

  #include <window.h>

  #include "dat.h"
  #include "fns.h"

  // Most of the functions in this file are executed from
  // a threadmouse() context (via button3menu()). They send
  // a Wctlmesg to the window thread to get the actual modifications
  // done on the global windows state.

  //---------------------------------------------------------------------------
  // Menu
  //---------------------------------------------------------------------------

  ⟨enum _anon_ (windows/rio/rio.c) 77a⟩

  ⟨global menu3str 76e⟩

  ⟨global menu3 76d⟩

  //---------------------------------------------------------------------------
  // Helpers
  //---------------------------------------------------------------------------

  ⟨function goodrect 185a⟩


  ⟨function portion 75b⟩

  ⟨function whichcorner 75a⟩

  ⟨function cornercursor 74d⟩

  //---------------------------------------------------------------------------
  // Mouse actions
```

```
//-----------------------------------------------------------------------------
```

⟨*function* pointto 91a⟩

⟨*function* onscreen 86a⟩

⟨*function* sweep 85d⟩

⟨*function* drawedge 93c⟩

⟨*function* drawborder 93b⟩

⟨*function* drag 92⟩

⟨*function* cornerpt 96c⟩

⟨*function* whichrect 96b⟩

⟨*function* bandsize 95a⟩

```
//-----------------------------------------------------------------------------
// Window management
//-----------------------------------------------------------------------------
```

⟨*function* delete 88b⟩

⟨*function* resize 93e⟩

⟨*function* move 91d⟩

⟨*function* whide 97e⟩

⟨*function* wunhide 99⟩

⟨*function* hide 97b⟩

⟨*function* unhide 98e⟩

⟨*global* rcargv 81l⟩

⟨*function* new 79b⟩

```
//-----------------------------------------------------------------------------
// Entry point
//-----------------------------------------------------------------------------
```

⟨*function* button3menu 76b⟩

E.2.11 windows/rio/data.c

⟨windows/rio/data.c 247⟩≡

```
#include <u.h>
#include <libc.h>
```

```
#include <draw.h>
#include <thread.h>
#include <cursor.h>
#include <mouse.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>

#include "dat.h"
#include "fns.h"
```

⟨*global crosscursor (windows/rio/data.c)* 71a⟩

⟨*global boxcursor (windows/rio/data.c)* 71b⟩

⟨*global sightcursor (windows/rio/data.c)* 71c⟩

⟨*global whitearrow (windows/rio/data.c)* 72a⟩

⟨*global query (windows/rio/data.c)* 72b⟩

⟨*global* `tl` 72d⟩

⟨*global* `t` 72e⟩

⟨*global* `tr` 73a⟩

⟨*global* `r` 73b⟩

⟨*global* `br` 73c⟩

⟨*global* `b` 73d⟩

⟨*global* `bl` 73e⟩

⟨*global* `l` 74a⟩

⟨*global corners (windows/rio/data.c)* 72c⟩

⟨*function* `iconinit` 57b⟩

E.2.12 windows/rio/cursor.c

⟨`windows/rio/cursor.c` 248⟩≡

```
#include <u.h>
#include <libc.h>

// for dat.h
#include <draw.h>
#include <mouse.h>
#include <cursor.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>
#include <thread.h>

#include "dat.h"
#include "fns.h"
```

⟨*global* lastcursor 74b⟩

⟨*function* riosetcursor 74c⟩

E.2.13 windows/rio/wind.c

⟨windows/rio/wind.c 249⟩≡

```
#include <u.h>
#include <libc.h>

#include <draw.h>
#include <window.h>
#include <thread.h>
#include <cursor.h>
#include <mouse.h>
#include <keyboard.h>

#include <frame.h>
#include <fcall.h>

#include "dat.h"
#include "fns.h"
```

⟨*global* topped 52j⟩
⟨*global* id 52b⟩

⟨*global* cols 128c⟩
⟨*global* darkgrey 128h⟩

⟨*global* titlecol 81c⟩
⟨*global* lighttitlecol 81d⟩
⟨*global* holdcol 218f⟩
⟨*global* lightholdcol 218h⟩
⟨*global* paleholdcol 218g⟩

⟨*function* wsendctlmesg 78c⟩

⟨*function* wborder 81b⟩

⟨*function* wsetcols 128g⟩

⟨*function* wmk 80e⟩

⟨*function* wsetname 85a⟩

⟨*function* wresize 94c⟩

⟨*function* wrefresh 226c⟩

⟨*function* wclose 90c⟩

⟨*function* wrepaint 87d⟩

⟨*function* winborder 74e⟩

⟨*function* wmovemouse 96a⟩

⟨*function* wpointto 62h⟩

⟨*function* wcurrent 87c⟩

⟨*function* wsetcursor 75d⟩

⟨*function* wtop 87b⟩

⟨*function* wtopme 189b⟩

⟨*function* wbottomme 189c⟩

⟨*function* wlookid 103a⟩

⟨*function* wclosewin 88g⟩

⟨*function* wsetpid 83c⟩

E.2.14 windows/rio/processes_winshell.c

⟨windows/rio/processes_winshell.c 250a⟩≡

```
#include <u.h>
#include <libc.h>

// for dat.h
#include <draw.h>
#include <mouse.h>
#include <cursor.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>
#include <thread.h>

#include "dat.h"
#include "fns.h"
```

⟨*function* winshell 82d⟩

E.2.15 windows/rio/terminal.c

⟨windows/rio/terminal.c 250b⟩≡

```
#include <u.h>
#include <libc.h>

// for dat.h
#include <draw.h>
#include <mouse.h>
#include <cursor.h>
#include <keyboard.h>
```

```
#include <frame.h>
#include <fcall.h>
#include <thread.h>

#include <plumb.h>
#include <complete.h>

#include "dat.h"
#include "fns.h"
```

⟨*enum* `_anon_ (windows/rio/wind.c)` 137d⟩

⟨*enum* `_anon_ (windows/rio/rio.c)` *2* 167f⟩

⟨*global* `menu2str` 167e⟩

⟨*global* `menu2` 167d⟩

⟨*global* `clickwin` 211c⟩
⟨*global* `clickmsec` 211d⟩
⟨*global* `selectwin` 211e⟩
⟨*global* `selectq` 211f⟩

```
//-----------------------------------------------------------------------------
// Completion
//-----------------------------------------------------------------------------
```

⟨*function* `windfilewidth` 208a⟩

⟨*function* `showcandidates` 207⟩

⟨*function* `namecomplete` 206c⟩

```
//-----------------------------------------------------------------------------
// Editor
//-----------------------------------------------------------------------------
```

⟨*function* `wbswidth` 162d⟩

⟨*function* `wfill` 155c⟩

⟨*function* `wdelete` 163f⟩

⟨*function* `wbacknl` 154a⟩

⟨*function* `wsetorigin` 154b⟩

⟨*function* `wshow` 149a⟩

⟨*function* `wsetselect` 155d⟩

⟨*function* `winsert` 137a⟩

⟨*function* `wcontents` 172d⟩

```
//-----------------------------------------------------------------------------
```

```
// Cut/copy/paste
//-----------------------------------------------------------------------------
```

⟨*function* wsnarf 192a⟩

⟨*function* wcut 192b⟩

⟨*function* wpaste 192c⟩

```
//-----------------------------------------------------------------------------
// Scrolling
//-----------------------------------------------------------------------------
```

⟨*function* wframescroll 171b⟩

⟨*function* framescroll 171a⟩

```
//-----------------------------------------------------------------------------
// Selection
//-----------------------------------------------------------------------------
```

⟨*function* wclickmatch 214a⟩

⟨*global* left1 213a⟩
⟨*global* right1 213b⟩
⟨*global* left2 213c⟩
⟨*global* left3 213d⟩

⟨*global* left 213e⟩
⟨*global* right 213f⟩

⟨*function* wdoubleclick 213g⟩

⟨*function* wselect 211g⟩

```
//-----------------------------------------------------------------------------
// Clicking
//-----------------------------------------------------------------------------

//-----------------------------------------------------------------------------
// Plumb
//-----------------------------------------------------------------------------
```

⟨*function* wplumb 195b⟩

```
//-----------------------------------------------------------------------------
// Middle click
//-----------------------------------------------------------------------------
```

⟨*function* button2menu 167b⟩

```
//-----------------------------------------------------------------------------
// Mouse dispatch
//-----------------------------------------------------------------------------
```

⟨*function* wmousectl 168a⟩

```
//-----------------------------------------------------------------------------
```

```
// Key dispatch
//-----------------------------------------------------------------------------
```

⟨*function* interruptproc 164e⟩

⟨*function* wkeyctl 65d⟩

E.2.16 windows/rio/snarf.c

⟨windows/rio/snarf.c 253a⟩≡

```
#include <u.h>
#include <libc.h>

// for dat.h
#include <draw.h>
#include <mouse.h>
#include <cursor.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>
#include <thread.h>

#include "dat.h"
#include "fns.h"
```

⟨*global* snarffd 194b⟩

⟨*global* snarf 194e⟩
⟨*global* nsnarf 194d⟩

⟨*function* putsnarf 194g⟩

⟨*function* getsnarf 194h⟩

E.2.17 windows/rio/graphical_window.c

⟨windows/rio/graphical_window.c 253b⟩≡

```
#include <u.h>
#include <libc.h>

// for dat.h
#include <draw.h>
#include <mouse.h>
#include <cursor.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>
#include <thread.h>

#include "dat.h"
#include "fns.h"
```

⟨*function* waddraw 122e⟩

E.2.18 windows/rio/9p.c

⟨windows/rio/9p.c 254a⟩≡

```
#include <u.h>
#include <libc.h>

// for dat.h
#include <draw.h>
#include <mouse.h>
#include <cursor.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>
#include <thread.h>

#include "dat.h"
#include "fns.h"

⟨global messagesize 66e⟩

⟨function filsysrespond 101b⟩

⟨function filsyscancel 223f⟩
```

E.2.19 windows/rio/proc_fileserver.c

⟨windows/rio/proc_fileserver.c 254b⟩≡

```
#include <u.h>
#include <libc.h>

// for dat.h
#include <draw.h>
#include <mouse.h>
#include <cursor.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>
#include <thread.h>

#include "dat.h"
#include "fns.h"

extern Xfid*  (*fcall[])(Filsys*, Xfid*, Fid*);
extern char Ebadfcall[];
extern bool firstmessage;
extern Fid*  newfid(Filsys*, int);
extern fdt clockfd;

⟨global srvpipe (windows/rio/fsys.c) 181a⟩
⟨global srvwctl (windows/rio/fsys.c) 182b⟩

⟨function filsysproc 66f⟩

⟨function cexecpipe 58c⟩

⟨function post 181c⟩


⟨function filsysinit 58b⟩
```

⟨*function* filsysmount 83e⟩

E.2.20 windows/rio/fsys.c

⟨windows/rio/fsys.c 255⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <thread.h>
#include <cursor.h>
#include <mouse.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>

#include "dat.h"
#include "fns.h"

⟨global Eexist 229b⟩
⟨global Enotdir 229c⟩
⟨global Ebadfcall 229d⟩
⟨global Eoffset 229e⟩


⟨global dirtab 100f⟩

static uint  getclock(void);
Fid*  newfid(Filsys*, int);
static int  dostat(Filsys*, int, Dirtab*, uchar*, int, uint);

⟨global clockfd 220b⟩
⟨global firstmessage 67g⟩

static Xfid* filsysflush(Filsys*, Xfid*, Fid*);
static Xfid* filsysversion(Filsys*, Xfid*, Fid*);
static Xfid* filsysauth(Filsys*, Xfid*, Fid*);
static Xfid* filsysnop(Filsys*, Xfid*, Fid*);
static Xfid* filsysattach(Filsys*, Xfid*, Fid*);
static Xfid* filsyswalk(Filsys*, Xfid*, Fid*);
static Xfid* filsysopen(Filsys*, Xfid*, Fid*);
static Xfid* filsyscreate(Filsys*, Xfid*, Fid*);
static Xfid* filsysread(Filsys*, Xfid*, Fid*);
static Xfid* filsyswrite(Filsys*, Xfid*, Fid*);
static Xfid* filsysclunk(Filsys*, Xfid*, Fid*);
static Xfid* filsysremove(Filsys*, Xfid*, Fid*);
static Xfid* filsysstat(Filsys*, Xfid*, Fid*);
static Xfid* filsyswstat(Filsys*, Xfid*, Fid*);

⟨global fcall 55d⟩


⟨function filsysversion 67f⟩

⟨function filsysauth 225g⟩

⟨function filsysflush 223d⟩

⟨function filsysattach 101f⟩
```

⟨*function* numeric 175a⟩

⟨*function* filsyswalk 103d⟩

⟨*function* filsysopen 105g⟩

⟨*function* filsyscreate 111b⟩

⟨*function* idcmp 174f⟩

⟨*function* filsysread 107d⟩

⟨*function* filsyswrite 109b⟩

⟨*function* filsysclunk 107b⟩

⟨*function* filsysremove 111c⟩

⟨*function* filsysstat 110a⟩

⟨*function* filsyswstat 111d⟩

⟨*function* newfid 54f⟩

⟨*function* getclock 220d⟩

⟨*function* dostat 110b⟩

E.2.21 windows/rio/xfid.c

⟨windows/rio/xfid.c 256⟩≡

```
#include <u.h>
#include <libc.h>

#include <draw.h>
#include <marshal.h>
#include <window.h>
#include <thread.h>
#include <cursor.h>
#include <mouse.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>
#include <plumb.h>

#include "dat.h"
#include "fns.h"
```

⟨*constant* MAXSNARF 193e⟩

⟨*global* Einuse 229h⟩
⟨*global* Edeleted 229i⟩
⟨*global* Ebadreq 229j⟩
⟨*global* Etooshort 229k⟩
⟨*global* Ebadtile 229l⟩
⟨*global* Eshort 229m⟩
⟨*global* Elong 229n⟩
⟨*global* Eunkid 230a⟩

⟨*global* Ebadrect 230b⟩
⟨*global* Ewindow 230c⟩
⟨*global* Enowindow 230d⟩
⟨*global* Ebadmouse 230e⟩
⟨*global* Ebadwrect 230f⟩
⟨*global* Ebadoffset 230g⟩

⟨*global* tsnarf 193h⟩
⟨*global* ntsnarf 194a⟩

⟨*function* xfidattach 102d⟩

⟨*function* xfidopen 106f⟩

⟨*function* xfidclose 107c⟩

⟨*function* keyboardsend 217a⟩

⟨*enum* _anon_ (windows/rio/xfid.c)*2* 115d⟩

⟨*function* xfidwrite 109c⟩

⟨*function* readwindow 125⟩

⟨*enum* _anon_ (windows/rio/xfid.c)*3* 114g⟩
⟨*enum* _anon_ (windows/rio/xfid.c)*4* 112f⟩
⟨*enum* _anon_ (windows/rio/xfid.c)*5* 176g⟩

⟨*function* xfidread 109a⟩

E.2.22 windows/rio/scrl.c

⟨windows/rio/scrl.c 257⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <thread.h>
#include <cursor.h>
#include <mouse.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>

#include "dat.h"
#include "fns.h"
```

⟨*global* scrtmp 151b⟩

⟨*function* scrtemps 151d⟩

⟨*function* freescrtemps 170b⟩

⟨*function* scrpos 150a⟩

⟨*function* wscrdraw 149c⟩

⟨*function* wscrsleep 170c⟩

⟨*function* wscroll 169d⟩

E.2.23 windows/rio/time.c

⟨windows/rio/time.c 258a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <thread.h>
#include <cursor.h>
#include <mouse.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>

#include "dat.h"
#include "fns.h"
```

⟨*global* ctimer 222a⟩
⟨*global* timer 222b⟩

⟨*function* msec 222c⟩

⟨*function* timerstop 222d⟩

⟨*function* timercancel 222e⟩

⟨*function* timerproc 221⟩

⟨*function* timerinit 220g⟩

⟨*function* timerstart 222f⟩

E.2.24 windows/rio/wctl.c

⟨windows/rio/wctl.c 258b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <window.h>
#include <thread.h>
#include <cursor.h>
#include <mouse.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>
#include <plumb.h>

#include "dat.h"
#include "fns.h"

#include <ctype.h>
```

⟨*global* Ebadwr 229f⟩
⟨*global* Ewalloc 229g⟩

⟨*enum* _anon_ (windows/rio/wctl.c) 183c⟩

⟨*global* cmds 184a⟩

⟨*enum* _anon_ (windows/rio/wctl.c)*2* 184b⟩

⟨*global* params 184c⟩

⟨*function* word 185b⟩

⟨*function* set 185c⟩

⟨*function* newrect 186a⟩

⟨*function* shift 186b⟩

⟨*function* rectonscreen 186c⟩

⟨*function* riostrtol 186d⟩

⟨*function* parsewctl 187⟩

⟨*function* wctlnew 189a⟩

⟨*function* writewctl 178g⟩

⟨*function* wctlthread 183b⟩

⟨*function* wctlproc 183a⟩

E.2.25 windows/rio/error.c

⟨windows/rio/error.c 259a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <thread.h>

⟨global errorshouldabort 230h⟩

// could be in 9p.c too
⟨global Eperm 229a⟩

⟨function error 230j⟩

⟨function derror 230k⟩
```

E.2.26 windows/rio/util.c

⟨windows/rio/util.c 259b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <thread.h>
#include <cursor.h>
```

```
#include <mouse.h>
#include <keyboard.h>
#include <frame.h>
#include <fcall.h>

#include "dat.h"
#include "fns.h"
```

⟨*function* cvttorunes 232e⟩

⟨*function* erealloc 231c⟩

⟨*function* emalloc 231d⟩

⟨*function* estrdup 231e⟩

⟨*function* isalnum 232a⟩

⟨*function* strrune 233a⟩

⟨*function* min 231a⟩

⟨*function max (windows/rio/util.c)* 231b⟩

⟨*function* runetobyte 233b⟩

E.3 windows/apps/

E.3.1 windows/apps/lens.c

⟨*enum* _anon_ (windows/apps/lens.c) 260a⟩≡ (265)

```
enum {
    Edge = 5,
    Maxmag = 16
};
```

⟨*enum* _anon_ (windows/apps/lens.c)*2* 260b⟩≡ (265)

```
enum {
    Mzoom,
    Munzoom,
    Mgrid,
    Mredraw,
    Mexit
};
```

⟨*global* menustr 260c⟩≡ (265)

```
static char *menustr[] = {
    "zoom",
    "unzoom",
    "grid",
    "redraw",
    "exit",
    nil
};
```

⟨*global menu (windows/apps/lens.c)* 261a⟩≡ (265)

```
static Menu menu = {
    menustr,
    nil,
    -1
};
```

⟨*global* lastp 261b⟩≡ (265)

```
static Point lastp;
```

⟨*global red (windows/apps/lens.c)* 261c⟩≡ (265)

```
static Image *red;
```

⟨*global tmp (windows/apps/lens.c)* 261d⟩≡ (265)

```
static Image *tmp;
```

⟨*global* grid 261e⟩≡ (265)

```
static Image *grid;
```

⟨*global* chequer 261f⟩≡ (265)

```
static Image *chequer;
```

⟨*global* screenfd 261g⟩≡ (265)

```
static int screenfd;
```

⟨*global* mag 261h⟩≡ (265)

```
static int mag = 4;
```

⟨*global* showgrid 261i⟩≡ (265)

```
static int showgrid = 0;
```

⟨*global* screenr 261j⟩≡ (265)

```
static Rectangle screenr;
```

⟨*global* screenbuf 261k⟩≡ (265)

```
static uchar *screenbuf;
```

⟨*function* drawit 261l⟩≡ (265)

```
void
drawit(void)
{
    Rectangle r;
    border(view, view->r, Edge, red, ZP);
    magnify();
    r = insetrect(view->r, Edge);
    draw(view, r, tmp, nil, tmp->r.min);
    flushimage(display, true);
}
```

⟨*global* bypp 261m⟩≡ (265)

```
static int bypp;
```

⟨*function main (windows/apps/lens.c)* 262⟩≡ (265)

```
void
main(int argc, char *argv[])
{
    Event e;
    char buf[5*12];
    ulong chan;
    int d;

    USED(argc, argv);

    if(initdraw(nil, nil, "lens") < 0){
        fprint(2, "lens: initdraw failed: %r\n");
        exits("initdraw");
    }
    einit(Emouse|Ekeyboard);

    red = allocimage(display, Rect(0, 0, 1, 1), CMAP8, 1, DRed);
    chequer = allocimage(display, Rect(0, 0, 2, 2), GREY1, 1, DBlack);

    draw(chequer, Rect(0, 0, 1, 1), display->white, nil, ZP);
    draw(chequer, Rect(1, 1, 2, 2), display->white, nil, ZP);
    lastp = divpt(addpt(view->r.min, view->r.max), 2);

    screenfd = open("/dev/screen", OREAD);
    if(screenfd < 0){
        fprint(2, "lens: can't open /dev/screen: %r\n");
        exits("screen");
    }
    if(read(screenfd, buf, sizeof buf) != sizeof buf){
        fprint(2, "lens: can't read /dev/screen: %r\n");
        exits("screen");
    }
    chan = strtochan(buf);
    d = chantodepth(chan);
    if(d < 8){
        fprint(2, "lens: can't handle screen format %11.11s\n", buf);
        exits("screen");
    }
    bypp = d/8;
    screenr.min.x = atoi(buf+1*12);
    screenr.min.y = atoi(buf+2*12);
    screenr.max.x = atoi(buf+3*12);
    screenr.max.y = atoi(buf+4*12);
    screenbuf = malloc(bypp*Dx(screenr)*Dy(screenr));
    if(screenbuf == nil){
        fprint(2, "lens: buffer malloc failed: %r\n");
        exits("malloc");
    }
    eresized(0);

    for(;;)
        switch(event(&e)){
        case Ekeyboard:
            switch(e.kbdc){
            case 'q':
            case 0x7f:
            case '\04':
            caseexit:
                exits(nil);
```

```
            case '=':
            case '+':
            casezoom:
                if(mag < Maxmag){
                    mag++;
                    makegrid();
                    drawit();
                }
                break;
            case 'g':
            casegrid:
                showgrid = !showgrid;
                makegrid();
                drawit();
                break;
            case '-':
            case '_':
            caseunzoom:
                if(mag > 1){
                    mag--;
                    makegrid();
                    drawit();
                }
                break;
            case '.':
            case ' ':
            caseredraw:
                drawit();
                break;
            case '1': case '2': case '3': case '4': case '5': case '6': case '7': case '8': case '9': case'0':
                mag = e.kbdc-'0';
                if(mag == 0)
                    mag = 10;
                makegrid();
                drawit();
                break;
            }
            break;
        case Emouse:
            if(e.mouse.buttons & 1){
                lastp = e.mouse.xy;
                drawit();
            }
            if(e.mouse.buttons & 4)
                switch(emenuhit(3, &e.mouse, &menu)){
                case Mzoom:
                    goto casezoom;
                case Munzoom:
                    goto caseunzoom;
                case Mgrid:
                    goto casegrid;
                case Mredraw:
                    goto caseredraw;
                case Mexit:
                    goto caseexit;
                }
            break;
        }
}
```

⟨*function* makegrid 264a⟩≡ (265)

```
void
makegrid(void)
{
    int m;
    if (grid != nil) {
        freeimage(grid);
        grid = nil;
    }
    if (showgrid) {
        m = mag;
        if (m < 5)
            m *= 10;
        grid = allocimage(display, Rect(0, 0, m, m),
            CHAN2(CGrey, 8, CAlpha, 8), 1, DTransparent);
        if (grid != nil){
            draw(grid, Rect(0, 0, m, 1), chequer, nil, ZP);
            draw(grid, Rect(0, 1, 1, m), chequer, nil, ZP);
        }
    }
}
```

⟨*function eresized (windows/apps/lens.c)* 264b⟩≡ (265)

```
void
eresized(int new)
{
    if(new && getwindow(display, Refnone) < 0){
        fprint(2, "lens: can't reattach to window: %r\n");
        exits("attach");
    }
    freeimage(tmp);
    tmp = allocimage(display, Rect(0, 0, Dx(view->r)-Edge, Dy(view->r)-Edge+Maxmag), view->chan, 0, DNofill);
    if(tmp == nil){
        fprint(2, "lens: allocimage failed: %r\n");
        exits("allocimage");
    }
    drawit();
}
```

⟨*function* magnify 264c⟩≡ (265)

```
void
magnify(void)
{
    int x, y, xx, yy, dd, i;
    int dx, dy;
    int xoff, yoff;
    uchar out[8192];
    uchar sp[4];

    dx = (Dx(tmp->r)+mag-1)/mag;
    dy = (Dy(tmp->r)+mag-1)/mag;
    xoff = lastp.x-Dx(tmp->r)/(mag*2);
    yoff  = lastp.y-Dy(tmp->r)/(mag*2);

    yy = yoff;
    dd = dy;
    if(yy < 0){
        dd += dy;
        yy = 0;
    }
```

```
    if(yy+dd > Dy(screenr))
        dd = Dy(screenr)-yy;
    seek(screenfd, 5*12+bypp*yy*Dx(screenr), 0);
    if(readn(screenfd, screenbuf+bypp*yy*Dx(screenr), bypp*Dx(screenr)*dd) != bypp*Dx(screenr)*dd){
        fprint(2, "lens: can't read screen: %r\n");
        return;
    }

    for(y=0; y<dy; y++){
        yy = yoff+y;
        if(yy>=0 && yy<Dy(screenr))
            for(x=0; x<dx; x++){
                xx = xoff+x;
                if(xx>=0 && xx<Dx(screenr)) /* snarf pixel at xx, yy */
                    for(i=0; i<bypp; i++)
                        sp[i] = screenbuf[bypp*(yy*Dx(screenr)+xx)+i];
                else
                    sp[0] = sp[1] = sp[2] = sp[3] = 0;

                for(xx=0; xx<mag; xx++)
                    if(x*mag+xx < tmp->r.max.x)
                        for(i=0; i<bypp; i++)
                            out[(x*mag+xx)*bypp+i] = sp[i];
            }
        else
            memset(out, 0, bypp*Dx(tmp->r));
        for(yy=0; yy<mag && y*mag+yy<Dy(tmp->r); yy++){
            werrstr("no error");
            if(loadimage(tmp, Rect(0, y*mag+yy, Dx(tmp->r), y*mag+yy+1), out, bypp*Dx(tmp->r)) != bypp*Dx(tmp->
                exits("load");
            }
        }
    }
    if (showgrid && mag && grid)
        draw(tmp, tmp->r, grid, nil, mulpt(Pt(xoff, yoff), mag));
}
```

⟨windows/apps/lens.c 265⟩≡

```
#include <u.h>
#include <libc.h>

#include <draw.h>
#include <window.h>
#include <event.h>
```

⟨*enum* _anon_ (windows/apps/lens.c) 260a⟩

⟨*enum* _anon_ (windows/apps/lens.c)*2* 260b⟩

⟨*global* menustr 260c⟩

⟨*global menu (windows/apps/lens.c)* 261a⟩

⟨*global* lastp 261b⟩
⟨*global red (windows/apps/lens.c)* 261c⟩
⟨*global tmp (windows/apps/lens.c)* 261d⟩
⟨*global* grid 261e⟩
⟨*global* chequer 261f⟩
⟨*global* screenfd 261g⟩
⟨*global* mag 261h⟩

⟨*global* showgrid 261i⟩
⟨*global* screenr 261j⟩
⟨*global* screenbuf 261k⟩

```
void magnify(void);
void makegrid(void);
```

⟨*function* drawit 261l⟩

⟨*global* bypp 261m⟩

⟨*function main (windows/apps/lens.c)* 262⟩

⟨*function* makegrid 264a⟩

⟨*function eresized (windows/apps/lens.c)* 264b⟩

⟨*function* magnify 264c⟩

E.3.2 windows/apps/statusbar.c

⟨*enum* _anon_ (windows/apps/statusbar.c) 266a⟩≡ (272b)

```
enum {PNCTL=3};
```

⟨*global* nokill 266b⟩≡ (272b)

```
static int nokill;
```

⟨*global* textmode 266c⟩≡ (272b)

```
static int textmode;
```

⟨*global* title 266d⟩≡ (272b)

```
static char *title;
```

⟨*global* light 266e⟩≡ (272b)

```
static Image *light;
```

⟨*global* dark 266f⟩≡ (272b)

```
static Image *dark;
```

⟨*global text (windows/apps/statusbar.c)* 266g⟩≡ (272b)

```
static Image *text;
```

⟨*function* initcolor 266h⟩≡ (272b)

```
static void
initcolor(void)
{
    text = display->black;
    light = allocimagemix(display, DPalegreen, DWhite);
    dark = allocimage(display, Rect(0,0,1,1), CMAP8, 1, DDarkgreen);
}
```

⟨*global* rbar 266i⟩≡ (272b)

```
static Rectangle rbar;
```

⟨*global* ptext 266j⟩≡ (272b)

```
static Point ptext;
```

⟨*global* last 266k⟩≡ (272b)

```
static int last;
```

⟨*global lastp (windows/apps/statusbar.c)* 267a⟩≡ (272b)

```
static int lastp = -1;
```

⟨*global* backup 267b⟩≡ (272b)

```
static char backup[80];
```

⟨*function* drawbar 267c⟩≡ (272b)

```
void
drawbar(void)
{
    int i, j;
    int p;
    char buf[400], bar[200];
    static char lastbar[200];

    if(n > d || n < 0 || d <= 0)
        return;

    i = (Dx(rbar)*n)/d;
    p = (n*100LL)/d;

    if(textmode){
        if(Dx(rbar) > 150){
            rbar.min.x = 0;
            rbar.max.x = 150;
            return;
        }
        bar[0] = '|';
        for(j=0; j<i; j++)
            bar[j+1] = '#';
        for(; j<Dx(rbar); j++)
            bar[j+1] = '-';
        bar[j++] = '|';
        bar[j++] = ' ';
        sprint(bar+j, "%3d%% ", p);
        for(i=0; bar[i]==lastbar[i] && bar[i]; i++)
            ;
        memset(buf, '\b', strlen(lastbar)-i);
        strcpy(buf+strlen(lastbar)-i, bar+i);
        if(buf[0])
            write(1, buf, strlen(buf));
        strcpy(lastbar, bar);
        return;
    }

    if(lastp == p && last == i)
        return;

    if(lastp != p){
        sprint(buf, "%d%%", p);

        stringbg(view, addpt(view->r.min, Pt(Dx(rbar)-30, 4)), text, ZP, display->defaultfont, buf, light, ZP);
        lastp = p;
    }

    if(last != i){
        if(i > last)
            draw(view, Rect(rbar.min.x+last, rbar.min.y, rbar.min.x+i, rbar.max.y),
                dark, nil, ZP);
        else
```

```
            draw(view, Rect(rbar.min.x+i, rbar.min.y, rbar.min.x+last, rbar.max.y),
                light, nil, ZP);
        last = i;
    }
    flushimage(display, 1);
}
```

⟨*function eresized (windows/apps/statusbar.c)* 268a⟩≡ (272b)

```
void
eresized(int new)
{
    Point p, q;
    Rectangle r;

    if(new && getwindow(display, Refnone) < 0)
        fprint(2,"can't reattach to window");

    r = view->r;
    draw(view, r, light, nil, ZP);
    p = string(view, addpt(r.min, Pt(4,4)), text, ZP,
        display->defaultfont, title);

    p.x = r.min.x+4;
    p.y += display->defaultfont->height+4;

    q = subpt(r.max, Pt(4,4));
    rbar = Rpt(p, q);

    ptext = Pt(r.max.x-4-stringwidth(display->defaultfont, "100%"), r.min.x+4);
    border(view, rbar, -2, dark, ZP);
    last = 0;
    lastp = -1;

    drawbar();
}
```

⟨*function* bar 268b⟩≡ (272b)

```
void
bar(Biobuf *b)
{
    char *p, *f[2];
    Event e;
    int k, die, parent, child;

    parent = getpid();

    die = 0;
    if(textmode)
        child = -1;
    else
    switch(child = rfork(RFMEM|RFPROC)) {
    case 0:
        sleep(1000);
        while(!die && (k = eread(Ekeyboard|Emouse, &e))) {
            if(nokill==0 && k == Ekeyboard && (e.kbdc == 0x7F || e.kbdc == 0x03)) { /* del, ctl-c */
                die = 1;
                postnote(PNPROC, parent, "interrupt");
                _exits("interrupt");
            }
        }
```

```
        _exits(0);
    }

    while(!die && (p = Brdline(b, '\n'))) {
        p[Blinelen(b)-1] = '\0';
        if(tokenize(p, f, 2) != 2)
            continue;
        n = strtoll(f[0], 0, 0);
        d = strtoll(f[1], 0, 0);
        drawbar();
    }
    postnote(PNCTL, child, "kill");
}
```

⟨*function usage (windows/apps/statusbar.c)* 269a⟩≡ (272b)

```
static void
usage(void)
{
    fprint(2, "usage: aux/statusbar [-kt] [-w minx,miny,maxx,maxy] 'title'\n");
    exits("usage");
}
```

⟨*function main (windows/apps/statusbar.c)* 269b⟩≡ (272b)

```
void
main(int argc, char **argv)
{
    Biobuf b;
    char *p, *q;
    int lfd;

    p = "0,0,200,60";

    ARGBEGIN{
    case 'w':
        p = ARGF();
        break;
    case 't':
        textmode = 1;
        break;
    case 'k':
        nokill = 1;
        break;
    default:
        usage();
    }ARGEND;

    if(argc != 1)
        usage();

    title = argv[0];

    lfd = dup(0, -1);

    while(q = strchr(p, ','))
        *q = ' ';
    Binit(&b, lfd, OREAD);
    if(textmode || newwin(p) < 0){
        textmode = 1;
        rbar = Rect(0, 0, 60, 1);
    }else{
```

```
        if(initdraw(0, 0, "bar") < 0)
            exits("initdraw");
        initcolor();
        einit(Emouse|Ekeyboard);
        eresized(0);
    }
    bar(&b);

    exits(0);
}
```

⟨*function* rdenv 270a⟩≡ (272b)

```
/* all code below this line should be in the library, but is stolen from colors instead */
static char*
rdenv(char *name)
{
    char *v;
    int fd, size;

    fd = open(name, OREAD);
    if(fd < 0)
        return 0;
    size = seek(fd, 0, 2);
    v = malloc(size+1);
    if(v == 0){
        fprint(2, "%s: can't malloc: %r\n", argv0);
        exits("no mem");
    }
    seek(fd, 0, 0);
    read(fd, v, size);
    v[size] = 0;
    close(fd);
    return v;
}
```

⟨*function* newwin 270b⟩≡ (272b)

```
int
newwin(char *win)
{
    char *srv, *mntsrv;
    char spec[100];
    int srvfd, cons, pid;

    switch(rfork(RFFDG|RFPROC|RFNAMEG|RFENVG|RFNOTEG|RFNOWAIT)){
    case -1:
        fprint(2, "statusbar: can't fork: %r\n");
        return -1;
    case 0:
        break;
    default:
        exits(0);
    }

    srv = rdenv("/env/wsys");
    if(srv == 0){
        mntsrv = rdenv("/mnt/term/env/wsys");
        if(mntsrv == 0){
            fprint(2, "statusbar: can't find $wsys\n"); //$
            return -1;
        }
```

```
        srv = malloc(strlen(mntsrv)+10);
        sprint(srv, "/mnt/term%s", mntsrv);
        free(mntsrv);
        pid  = 0;   /* can't send notes to remote processes! */
    }else
        pid = getpid();
    USED(pid);
    srvfd = open(srv, ORDWR);
    free(srv);
    if(srvfd == -1){
        fprint(2, "statusbar: can't open %s: %r\n", srv);
        return -1;
    }
    sprint(spec, "new -r %s", win);
    if(mount(srvfd, -1, "/mnt/wsys", 0, spec) == -1){
        fprint(2, "statusbar: can't mount /mnt/wsys: %r (spec=%s)\n", spec);
        return -1;
    }
    close(srvfd);
    unmount("/mnt/acme", "/dev");
    bind("/mnt/wsys", "/dev", MBEFORE);
    cons = open("/dev/cons", OREAD);
    if(cons==-1){
    NoCons:
        fprint(2, "statusbar: can't open /dev/cons: %r");
        return -1;
    }
    dup(cons, 0);
    close(cons);
    cons = open("/dev/cons", OWRITE);
    if(cons==-1)
        goto NoCons;
    dup(cons, 1);
    dup(cons, 2);
    close(cons);
// wctlfd = open("/dev/wctl", OWRITE);
    return 0;
}
```

⟨*function* screenrect 271⟩≡ (272b)

```
Rectangle
screenrect(void)
{
    int fd;
    char buf[12*5];

    fd = open("/dev/screen", OREAD);
    if(fd == -1)
        fd=open("/mnt/term/dev/screen", OREAD);
    if(fd == -1){
        fprint(2, "%s: can't open /dev/screen: %r\n", argv0);
        exits("window read");
    }
    if(read(fd, buf, sizeof buf) != sizeof buf){
        fprint(2, "%s: can't read /dev/screen: %r\n", argv0);
        exits("screen read");
    }
    close(fd);
    return Rect(atoi(buf+12), atoi(buf+24), atoi(buf+36), atoi(buf+48));
}
```

⟨*function* postnote 272a⟩≡ (272b)

```
int
postnote(int group, int pid, char *note)
{
    char file[128];
    int f, r;

    switch(group) {
    case PNPROC:
        sprint(file, "/proc/%d/note", pid);
        break;
    case PNGROUP:
        sprint(file, "/proc/%d/notepg", pid);
        break;
    case PNCTL:
        sprint(file, "/proc/%d/ctl", pid);
        break;
    default:
        return -1;
    }

    f = open(file, OWRITE);
    if(f < 0)
        return -1;

    r = strlen(note);
    if(write(f, note, r) != r) {
        close(f);
        return -1;
    }
    close(f);
    return 0;
}
```

⟨windows/apps/statusbar.c 272b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <window.h>
#include <bio.h>
#include <event.h>

⟨enum _anon_ (windows/apps/statusbar.c) 266a⟩

static char* rdenv(char*);
int newwin(char*);
Rectangle screenrect(void);

⟨global nokill 266b⟩
⟨global textmode 266c⟩
⟨global title 266d⟩

⟨global light 266e⟩
⟨global dark 266f⟩
⟨global text (windows/apps/statusbar.c) 266g⟩

⟨function initcolor 266h⟩

⟨global rbar 266i⟩
⟨global ptext 266j⟩
```

```
static vlong n, d;
⟨global last 266k⟩
⟨global lastp (windows/apps/statusbar.c) 267a⟩

⟨global backup 267b⟩

⟨function drawbar 267c⟩

⟨function eresized (windows/apps/statusbar.c) 268a⟩

⟨function bar 268b⟩


⟨function usage (windows/apps/statusbar.c) 269a⟩

⟨function main (windows/apps/statusbar.c) 269b⟩


⟨function rdenv 270a⟩

⟨function newwin 270b⟩

⟨function screenrect 271⟩

⟨function postnote 272a⟩
```

E.3.3 windows/apps/winwatch.c

⟨*struct* Win 273a⟩≡ (278)

```
struct Win {
    int n;
    int dirty;
    char *label;
    Rectangle r;
};
```

⟨*global* exclude 273b⟩≡ (278)

```
static Reprog  *exclude  = nil;
```

⟨*global* win 273c⟩≡ (278)

```
static Win *win;
```

⟨*global* nwin 273d⟩≡ (278)

```
static int nwin;
```

⟨*global* mwin 273e⟩≡ (278)

```
static int mwin;
```

⟨*global* onwin 273f⟩≡ (278)

```
static int onwin;
```

⟨*global* lightblue 273g⟩≡ (278)

```
static Image *lightblue;
```

⟨*enum* _anon_ (windows/apps/winwatch.c) 273h⟩≡ (278)

```
enum {
    PAD = 3,
    MARGIN = 5
};
```

⟨*function erealloc (windows/apps/winwatch.c)* 274a⟩≡ (278)

```
static void*
erealloc(void *v, ulong n)
{
    v = realloc(v, n);
    if(v == nil)
        sysfatal("out of memory reallocating %lud", n);
    return v;
}
```

⟨*function emalloc (windows/apps/winwatch.c)* 274b⟩≡ (278)

```
static void*
emalloc(ulong n)
{
    void *v;

    v = malloc(n);
    if(v == nil)
        sysfatal("out of memory allocating %lud", n);
    memset(v, 0, n);
    return v;
}
```

⟨*function estrdup (windows/apps/winwatch.c)* 274c⟩≡ (278)

```
static char*
estrdup(char *s)
{
    int l;
    char *t;

    if (s == nil)
        return nil;
    l = strlen(s)+1;
    t = emalloc(l);
    memcpy(t, s, l);

    return t;
}
```

⟨*function* `refreshwin` 274d⟩≡ (278)

```
static void
refreshwin(void)
{
    char label[128];
    int i, fd, lfd, n, nr, nw, m;
    Dir *pd;

    if((fd = open("/dev/wsys", OREAD)) < 0)
        return;

    nw = 0;
/* i'd rather read one at a time but rio won't let me */
    while((nr=dirread(fd, &pd)) > 0){
        for(i=0; i<nr; i++){
            n = atoi(pd[i].name);
            sprint(label, "/dev/wsys/%d/label", n);
            if((lfd = open(label, OREAD)) < 0)
                continue;
            m = read(lfd, label, sizeof(label)-1);
            close(lfd);
```

```
            if(m < 0)
                continue;
            label[m] = '\0';
            if(exclude != nil && regexec(exclude,label,nil,0))
                continue;

            if(nw < nwin && win[nw].n == n && strcmp(win[nw].label, label)==0){
                nw++;
                continue;
            }

            if(nw < nwin){
                free(win[nw].label);
                win[nw].label = nil;
            }

            if(nw >= mwin){
                mwin += 8;
                win = erealloc(win, mwin*sizeof(win[0]));
            }
            win[nw].n = n;
            win[nw].label = estrdup(label);
            win[nw].dirty = 1;
            win[nw].r = Rect(0,0,0,0);
            nw++;
        }
        free(pd);
    }
    while(nwin > nw)
        free(win[--nwin].label);
    nwin = nw;
    close(fd);
}
```

⟨*function* drawnowin 275a⟩≡ (278)

```
static void
drawnowin(int i)
{
    Rectangle r;

    r = Rect(0,0,(Dx(view->r)-2*MARGIN+PAD)/cols-PAD, font->height);
    r = rectaddpt(rectaddpt(r, Pt(MARGIN+(PAD+Dx(r))*(i/rows),
                MARGIN+(PAD+Dy(r))*(i%rows))), view->r.min);
    draw(view, insetrect(r, -1), lightblue, nil, ZP);
}
```

⟨*function* drawwin 275b⟩≡ (278)

```
static void
drawwin(int i)
{
    draw(view, win[i].r, lightblue, nil, ZP);
    _string(view, addpt(win[i].r.min, Pt(2,0)), display->black, ZP,
        font, win[i].label, nil, strlen(win[i].label),
        win[i].r, nil, ZP, SoverD);
    border(view, win[i].r, 1, display->black, ZP);
    win[i].dirty = 0;
}
```

⟨*function* geometry 275c⟩≡ (278)

```
static int
```

```
geometry(void)
{
    int i, ncols, z;
    Rectangle r;

    z = 0;
    rows = (Dy(view->r)-2*MARGIN+PAD)/(font->height+PAD);
    if(rows*cols < nwin || rows*cols >= nwin*2){
        ncols = nwin <= 0 ? 1 : (nwin+rows-1)/rows;
        if(ncols != cols){
            cols = ncols;
            z = 1;
        }
    }

    r = Rect(0,0,(Dx(view->r)-2*MARGIN+PAD)/cols-PAD, font->height);
    for(i=0; i<nwin; i++)
        win[i].r = rectaddpt(rectaddpt(r, Pt(MARGIN+(PAD+Dx(r))*(i/rows),
                    MARGIN+(PAD+Dy(r))*(i%rows))), view->r.min);

    return z;
}
```

⟨*function redraw (windows/apps/winwatch.c)* 276a⟩≡ (278)

```
static void
redraw(Image *view, int all)
{
    int i;

    all |= geometry();
    if(all)
        draw(view, view->r, lightblue, nil, ZP);
    for(i=0; i<nwin; i++)
        if(all || win[i].dirty)
            drawwin(i);
    if(!all)
        for(; i<onwin; i++)
            drawnowin(i);

    onwin = nwin;
}
```

⟨*function eresized (windows/apps/winwatch.c)* 276b⟩≡ (278)

```
void
eresized(int new)
{
    if(new && getwindow(display, Refmesg) < 0)
        fprint(2,"can't reattach to window");
    geometry();
    redraw(view, 1);
}
```

⟨*function* `click` 276c⟩≡ (278)

```
static void
click(Mouse m)
{
    int fd, i, j;
    char buf[128];

    if(m.buttons == 0 || (m.buttons & ~4))
```

```
        return;

    for(i=0; i<nwin; i++)
        if(ptinrect(m.xy, win[i].r))
            break;
    if(i == nwin)
        return;

    do
        m = emouse();
    while(m.buttons == 4);

    if(m.buttons != 0){
        do
            m = emouse();
        while(m.buttons);
        return;
    }

    for(j=0; j<nwin; j++)
        if(ptinrect(m.xy, win[j].r))
            break;
    if(j != i)
        return;

    sprint(buf, "/dev/wsys/%d/wctl", win[i].n);
    if((fd = open(buf, OWRITE)) < 0)
        return;
    write(fd, "unhide\n", 7);
    write(fd, "top\n", 4);
    write(fd, "current\n", 8);
    close(fd);
}
```

⟨*function usage (windows/apps/winwatch.c)* 277a⟩≡ (278)

```
static void
usage(void)
{
    fprint(2, "usage: winwatch [-e exclude] [-f font]\n");
    exits("usage");
}
```

⟨*function main (windows/apps/winwatch.c)* 277b⟩≡ (278)

```
void
main(int argc, char **argv)
{
    char *fontname;
    int Etimer;
    Event e;

    fontname = "/lib/font/bit/lucidasans/unicode.8.font";
    ARGBEGIN{
    case 'f':
        fontname = EARGF(usage());
        break;
    case 'e':
        exclude = regcomp(EARGF(usage()));
        if(exclude == nil)
            sysfatal("Bad regexp");
        break;
```

```
    default:
        usage();
    }ARGEND

    if(argc)
        usage();

    initdraw(0, 0, "winwatch");
    lightblue = allocimagemix(display, DPalebluegreen, DWhite);
    if(lightblue == nil)
        sysfatal("allocimagemix: %r");
    if((font = openfont(display, fontname)) == nil)
        sysfatal("font '%s' not found", fontname);

    refreshwin();
    redraw(view, 1);
    einit(Emouse|Ekeyboard);
    Etimer = etimer(0, 2500);

    for(;;){
        switch(eread(Emouse|Ekeyboard|Etimer, &e)){
        case Ekeyboard:
            if(e.kbdc==0x7F || e.kbdc=='q')
                exits(0);
            break;
        case Emouse:
            if(e.mouse.buttons)
                click(e.mouse);
            /* fall through  */
        default: /* Etimer */
            refreshwin();
            redraw(view, 0);
            break;
        }
    }
}
```

⟨windows/apps/winwatch.c 278⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
//#include <draw_private.h> // for _string, but should not use it
#include <window.h>
#include <event.h>
#include <regexp.h>

extern Point _string(Image *dst, Point pt, Image *src, Point sp, Font *f, char *s, Rune *r, int len, Rectangle

typedef struct Win Win;
⟨struct Win 273a⟩



⟨global exclude 273b⟩
⟨global win 273c⟩
⟨global nwin 273d⟩
⟨global mwin 273e⟩
⟨global onwin 273f⟩
static int rows, cols;
⟨global lightblue 273g⟩
```

```
extern Font *font;
```

⟨*enum* _anon_ (windows/apps/winwatch.c) 273h⟩

⟨*function erealloc (windows/apps/winwatch.c)* 274a⟩

⟨*function emalloc (windows/apps/winwatch.c)* 274b⟩

⟨*function estrdup (windows/apps/winwatch.c)* 274c⟩

⟨*function* refreshwin 274d⟩

⟨*function* drawnowin 275a⟩

⟨*function* drawwin 275b⟩

⟨*function* geometry 275c⟩

⟨*function redraw (windows/apps/winwatch.c)* 276a⟩

⟨*function eresized (windows/apps/winwatch.c)* 276b⟩

⟨*function* click 276c⟩

⟨*function usage (windows/apps/winwatch.c)* 277a⟩

⟨*function main (windows/apps/winwatch.c)* 277b⟩

E.4 windows/libcomplete/

E.4.1 windows/libcomplete/complete.c

⟨windows/libcomplete/complete.c 279a⟩≡

```
#include <u.h>
#include <libc.h>
#include "complete.h"
```

⟨*function* longestprefixlength 210a⟩

⟨*function* freecompletion 210b⟩

⟨*function* strpcmp 210c⟩

⟨*function* complete 208b⟩

E.5 windows/libframe/

E.5.1 windows/libframe/frbox.c

⟨windows/libframe/frbox.c 279b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <thread.h>
```

```
#include <mouse.h>
#include <frame.h>

⟨constant SLOP 130e⟩

⟨function _fraddbox 130f⟩

⟨function _frclosebox 131g⟩

⟨function _frdelbox 131e⟩

⟨function _frfreebox 131f⟩

⟨function _frgrowbox 131a⟩

⟨function dupbox 142e⟩

⟨function runeindex 143c⟩

⟨function truncatebox 143a⟩

⟨function chopbox 143b⟩

⟨function _frsplitbox 142d⟩

⟨function _frmergebox 147a⟩

⟨function _frfindbox 142c⟩
```

E.5.2 windows/libframe/frdelete.c

```
⟨windows/libframe/frdelete.c 280a⟩≡
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <thread.h>
  #include <mouse.h>
  #include <frame.h>

  ⟨function frdelete 147b⟩
```

E.5.3 windows/libframe/frdraw.c

```
⟨windows/libframe/frdraw.c 280b⟩≡
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <thread.h>
  #include <mouse.h>
  #include <frame.h>

  ⟨function _frdrawtext 136c⟩

  ⟨function nbytes 153a⟩

  ⟨function frdrawsel 152a⟩

  ⟨function frdrawsel0 152b⟩
```

⟨*function* frredraw 159a⟩

⟨*function* frtick 151h⟩

⟨*function* _frdraw 144⟩

⟨*function* _frstrlen 145c⟩

E.5.4 windows/libframe/frinit.c

⟨windows/libframe/frinit.c 281a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <thread.h>
#include <mouse.h>
#include <frame.h>
```

⟨*function* frinit 127c⟩

⟨*function* frinittick 129d⟩

⟨*function* frsetrects 127d⟩

⟨*function* frclear 127g⟩

E.5.5 windows/libframe/frinsert.c

⟨windows/libframe/frinsert.c 281b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <thread.h>
#include <mouse.h>
#include <frame.h>
```

⟨*constant* DELTA 138f⟩
⟨*constant* TMPSIZE 143d⟩
⟨*global* frame 138c⟩

⟨*function* bxscan 143e⟩

⟨*function* chopframe 146a⟩

⟨*struct* points_frinsert 138d⟩

⟨*function* frinsert 139⟩

E.5.6 windows/libframe/frptofchar.c

⟨windows/libframe/frptofchar.c 281c⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <thread.h>
#include <mouse.h>
```

```
#include <frame.h>

⟨function _frptofcharptb 133b⟩

⟨function frptofchar 133a⟩

⟨function _frptofcharnb 134d⟩

⟨function _frgrid 136a⟩

⟨function frcharofpt 135⟩
```

E.5.7 windows/libframe/frselect.c

```
⟨windows/libframe/frselect.c 282a⟩≡
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <thread.h>
  #include <mouse.h>
  #include <frame.h>

  ⟨function region 158b⟩

  ⟨function frselect 156⟩

  ⟨function frselectpaint 158a⟩
```

E.5.8 windows/libframe/frstr.c

```
⟨windows/libframe/frstr.c 282b⟩≡
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <thread.h>
  #include <mouse.h>
  #include <frame.h>

  ⟨constant CHUNK (windows/libframe/frstr.c) 132d⟩
  ⟨function ROUNDUP 132e⟩

  ⟨function _frallocstr 132f⟩

  ⟨function _frinsure 132g⟩
```

E.5.9 windows/libframe/frutil.c

```
⟨windows/libframe/frutil.c 282c⟩≡
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <thread.h>
  #include <mouse.h>
  #include <frame.h>

  ⟨function _frcanfit 134c⟩
```

⟨*function* _frcklinewrap 134a⟩

⟨*function* _frcklinewrap0 134b⟩

⟨*function* _fradvance 133c⟩

⟨*function* _frnewwid 145a⟩

⟨*function* _frnewwid0 145b⟩

⟨*function* _frclean 146b⟩

E.6 windows/libplumb/

E.6.1 windows/libplumb/event.c

⟨windows/libplumb/event.c 283a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <event.h>
#include "plumb.h"

typedef struct EQueue EQueue;
```

⟨*struct* EQueue 204a⟩

⟨*global* equeue 204b⟩
⟨*global* eqlock 204c⟩

⟨*function* partial 204d⟩

⟨*function* addpartial 205a⟩

⟨*function* plumbevent 205b⟩

⟨*function* eplumb 205c⟩

E.6.2 windows/libplumb/mesg.c

⟨windows/libplumb/mesg.c 283b⟩≡

```
#include <u.h>
#include <libc.h>
#include "plumb.h"
```

⟨*function* plumbopen 196d⟩

⟨*function* Strlen 197a⟩

⟨*function* Strcpy 197b⟩

⟨*function* quote 197c⟩

⟨*function* plumbpackattr 198a⟩

⟨*function* plumblookup 198b⟩

⟨*function* plumbpack 199a⟩

⟨*function* plumbsend 199b⟩

⟨*function* plumbline 200a⟩

⟨*function* plumbfree 200b⟩

⟨*function* plumbunpackattr 200c⟩

⟨*function* plumbaddattr 202a⟩

⟨*function* plumbdelattr 202b⟩

⟨*function* plumbunpackpartial 202c⟩

⟨*function* plumbunpack 203a⟩

⟨*function* plumbrecv 203b⟩

E.6.3 windows/libplumb/plumbsendtext.c

⟨windows/libplumb/plumbsendtext.c 284⟩≡

```
#include <u.h>
#include <libc.h>
#include "plumb.h"
```

⟨*function* plumbsendtext 196c⟩

Glossary

```
API = Application Programming Interface
GUI = Graphical User Interface
IDE = Integrated Development Environment
IPC = Inter Process Communication
RPC = Remote Procedure Call
LOC = Lines Of Code
PTY = Pseudo TTY
TTY = Teletype
WIMP = Window Icon Menu Pointer
MIME = Multipurpose Internet Mail Extensions
PDA = Personal Digital Assistant
```

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

Bibliography

[FDFH90] James Foley, Andries Van Dam, Steven Feiner, and John Hughes. *Computer Graphics, Principles and Practice*. Addison-Wesley, 1990. cited page(s) 9, 14

[GRA89] James Gosling, David S.H. Rosenthal, and Michelle J. Arden. *The NeWS Book, an Introduction to the Network/Extensible Window System*. Springer-Verlag, 1989. cited page(s) 9, 14

[HDF+86] F.R.A. Hopgood, D.A. Duce, E.V.C Fielding, K. Robinson, and A.S. Williams. *Methodology of Window Management*. Springer-Verlag, 1986. Available at http://www.chilton-computing.org.uk/inf/literature/books/wm/overview.htm. cited page(s) 14

[Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. cited page(s) 15

[KR88] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 1988. cited page(s) 13

[Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. https://github.com/aryx/syncweb. cited page(s) 15

[Pad14] Yoann Padioleau. *Principia Softwarica: The Plan 9 Kernel 9*. 2014. cited page(s) 11, 13, 21, 27, 29, 31, 33, 36, 41, 44

[Pad16a] Yoann Padioleau. *Principia Softwarica: The Plan 9 Core Libraries*. 2016. cited page(s) 13, 29, 33, 34, 35, 36, 38, 40

[Pad16b] Yoann Padioleau. *Principia Softwarica: The Plan 9 Graphics System draw*. 2016. cited page(s) 10, 11, 13, 19, 21, 23, 24, 27, 28, 29, 30, 31, 32, 33, 34, 36, 37, 46, 49

[Pad16c] Yoann Padioleau. *Principia Softwarica: The Plan 9 Network Stack /net*. 2016. cited page(s) 13, 37, 41, 42

[Pad18] Yoann Padioleau. *Principia Softwarica: The Plan 9 Shell rc*. 2018. cited page(s) 29, 41

[Pik83a] Rob Pike. The blit: A multiplexed graphics terminal. Technical report, Bell Labs, 1983. Also available at lib_graphics/docs/blit-1983.pdf. cited page(s) 10, 15

[Pik83b] Rob Pike. Graphics in overlapping bitmap layers. In *SIGGRAPH*, pages 331–356, 1983. Also available at lib_graphics/docs/pike-bitmap-1983.ps. cited page(s) 18, 24

[Pik88] Rob Pike. Window systems should be transparent. In *Computing Systems*, pages 279–296, 1988. Also available at windows/docs/transparent_wsys.pdf. cited page(s) 11, 15, 21

[Pik89] Rob Pike. A concurrent window system. In *Computing Systems*, pages 133–154, 1989. Also available at windows/docs/concurrent_window_system.pdf. cited page(s) 10, 15

[Pik91] Rob Pike. 8 1/2, the plan 9 window system. In *USENIX Summer conference*, pages 257–265, 1991. Also available at windows/docs/81/2.pdf. cited page(s) 10, 11, 15, 27, 45

[Pik00a] Rob Pike. Plumbing and other utilities. Technical report, Bell Labs, 2000. Also available at windows/docs/plumb.ps. cited page(s) 36

[Pik00b] Rob Pike. Rio: Design of a concurrent window system. Technical report, Bell Labs, 2000. Also available at windows/docs/rio_slides.pdf. cited page(s) 10, 15

[PPD+95] Rob Pike, Dave Presotto, Sean Dorward, Bob Flandrena, Ken Thompson, Howard Trickey, and Phil Winterbottom. Plan 9 from bell labs. In *Computing Systems*, pages 221–254, 1995. Also available at docs/articles/9.ps. cited page(s) 11, 14

[PPT+93] Rob Pike, Dave Presotto, Ken Thompson, Howard Trickey, and Phil Winterbottom. The use of name spaces in plan 9. In *Operating Systems Review*, pages 72–76, 1993. Also available at docs/articles/names.ps. cited page(s) 11, 14

[SG86] Robert W. Scheifler and Jim Gettys. The X Window system. In *ACM Transaction of Graphics*, pages 79–109, 1986. cited page(s) 11

[SIKH82] David Canfield Smith, Charles Irby, Ralph Kimball, and Eric Harslem. The star user interface: an overview. In *AFIPS*, pages 515–528, 1982. cited page(s) 16

[TML+79] C. P. Thacker, E. M. McCreight, B. W. Lampson, R. F. Sproull, and D. R. Boggs. Alto: A personal computer. Technical report, Xerox PARC, 1979. CSL-79-11. cited page(s) 9